

The SECR book

A handbook of spatially explicit capture–recapture methods
Version 1.0.2



Murray Efford

2025-07-09

Table of contents

Foreword	9
Why SECR?	9
Why this book?	10
Organisation	10
Software	10
Recommended citation	11
Feedback	11
Acknowledgments	11
I Basics	13
1 Concepts and terminology	14
1.1 Motivation	14
1.2 State and observation models	15
1.3 Spatial population	15
1.4 Detectors	16
1.5 Area search	16
1.6 Sampling across time	17
1.7 Data structure	17
1.8 Model fitting	18
2 Simple example	23
2.1 Input data	23
2.2 Check data	25
2.3 Fit a simple model	27
2.4 Output	28
2.5 Revisiting buffer width	30
2.6 Overall probability of detection	31
II Theory	34
3 Likelihood-based SECR	35
3.1 Notation	35
3.2 Likelihood	36
3.3 Distance-dependent detection	37
3.4 Detector types	38
3.5 Fixed N	41

3.6	Confidence intervals	41
3.7	Varying effort	42
4	Area search	44
4.1	Detector types for area search	44
4.2	Transect search	47
5	Special topics	48
5.1	Multi-session likelihood	48
5.2	Groups	48
5.3	Effective sampling area	49
5.4	Conditional likelihood	49
5.5	Relative density	50
5.6	Adjustment for spatially selective prior marking	51
5.7	Population size N	51
5.8	Finite mixture models	52
5.9	Hybrid finite mixture models	52
5.10	Alternative parameterizations	53
5.11	Model-based location of AC	54
III	Performance	55
6	Assumptions and robustness	56
6.1	Robustness	56
	Assumption 1: Population closed	57
	Assumption 2: Identification without error	59
	Assumption 3: Detection declines radially from randomly located, fixed AC	63
	Assumption 4: Probability of detection constant	64
	Assumption 5: Independence	71
6.2	Summary	75
7	Empirical validation	77
7.1	Brushtail possums in New Zealand	77
7.2	Red squirrels in the Yukon	77
7.3	Rodents in New Mexico	78
7.4	Chimpanzees in Ivory Coast	79
7.5	Red-backed salamanders in Massachusetts	79
7.6	Summary	80
8	Study design	81
8.1	Studies focussing on design	81
8.2	Data requirements for SECR	81
8.3	Precision and power	83
8.4	Pilot parameter values	84
8.5	Expected sample size	86
8.6	A useful approximation	87

8.7	Array size versus home range	87
8.8	The $n - r$ tradeoff	88
8.9	Spacing and precision	89
8.10	Detectors required for uniform array	90
8.11	Number of detectors in fixed region	91
8.12	Clustered designs	91
8.13	How many clusters?	97
8.14	Designing for spatial variation in parameters	97
8.15	Summary	99
IV	Practice	101
9	R package secr	102
9.1	History	102
9.2	Object classes	102
9.3	Functions	104
9.4	Detector types	104
9.5	Input	105
9.6	Output	105
9.7	Documentation and support	105
9.8	Using <code>secr.fit</code>	106
10	Detection model	111
10.1	Distance-detection functions	111
10.2	Detection submodels	117
10.3	Varying effort	123
11	Density model	128
11.1	Absolute vs relative density	129
11.2	Brush-tail possum example	129
11.3	Using the ‘model’ argument	132
11.4	Prediction and plotting	140
11.5	Scaling	140
11.6	This is not a density surface	141
11.7	Relative density	143
12	Habitat mask	146
12.1	Background	146
12.2	What is a mask for?	147
12.3	Masks in <code>secr</code>	147
12.4	How wide should the buffer be?	150
12.5	Grid cell size	154
12.6	Excluding non-habitat	156
12.7	Mask covariates	158
12.8	Regional population size	160

12.9	Plotting masks	162
12.10	More on creating and manipulating masks	163
13	Working with fitted models	165
13.1	Bundling fitted models	165
13.2	Recognizing failure-to-fit	165
13.3	Prediction	165
13.4	Tabulation of estimates	166
13.5	Population size	167
13.6	Model selection	168
13.7	Assessing model fit	170
13.8	Model re-fit	172
14	Multiple sessions	173
14.1	Input	173
14.2	Manipulation	176
14.3	Fitting	177
14.4	Simulation	180
14.5	Problems	180
14.6	Speed	181
14.7	Caveats	183
15	Individual heterogeneity	184
15.1	Finite mixture models in secr	185
15.2	Mitigating factors	189
15.3	Hybrid ‘hcov’ model	190
16	Sex differences	191
16.1	Models	191
16.2	Demonstration	192
16.3	Choosing a model	195
16.4	Final comments	197
17	Simulation	198
17.1	Simulation step by step	198
17.2	Simulation in secrdesign	203
17.3	Advanced simulations	204
	Appendices	208
A	Troubleshooting secr	208
A.1	Bias limit exceeded	208
A.2	Initial log likelihood NA	208
A.3	Variance calculation failed	210
A.4	Log likelihood becomes NA or improbably large after a few evaluations	212
A.5	Possible maximization error: nlm code 3	212

A.6	Possible maximization error: nlm code 4	213
A.7	<code>secur.fit</code> requests more memory than is available	215
A.8	Covariate factor levels differ between sessions	215
A.9	Estimates depend on starting values	216
B	Faster is better	217
B.1	Mask tuning	217
B.2	Conditional likelihood	217
B.3	Mashing	218
B.4	Parallel fitting	219
B.5	Reducing complexity (session- or group-specific models)	220
B.6	Reducing number of levels of detection covariates	220
B.7	Collapsing occasions	220
B.8	Individual mask subsets	221
B.9	Collapsing detectors	223
B.10	“multi” detector type instead of “proximity”	223
B.11	Some models are just slower than others	223
C	Spatial data	224
C.1	Spatial data in R	224
C.2	Spatial data in <code>secur</code>	225
C.3	Limits of the Cartesian model in <code>secur</code>	230
D	Area and transect search	231
D.1	Parameterisation	231
D.2	Example data: flat-tailed horned lizards	232
D.3	Data input	233
D.4	Fitting	233
D.5	Cue data	234
D.6	Discretizing polygon data	236
D.7	Transect search	237
D.8	More on polygons	239
D.9	Technical notes	241
E	Mark-resight models	242
E.1	How is mark-resight different?	242
E.2	Distribution of marked animals	242
E.3	Overview of implementation in <code>secur</code>	243
E.4	Data preparation	246
E.5	Model fitting	250
E.6	Limitations	255
E.7	Step-by-step guide	256
E.8	Miscellaneous	259
F	Non-Euclidean distances	265
F.1	Basics	265

F.2	Static userdist	265
F.3	Dynamic userdist	269
F.4	Examples	270
F.5	And the winner is...	276
F.6	Notes	277
F.7	Simulation after Sutherland et al. (2015)	279
G	Telemetry	285
G.1	Example	285
G.2	Standalone telemetry data	288
G.3	Combining telemetry and capture–recapture	291
G.4	Model fitting	293
G.5	Simulation	296
G.6	Technical notes	301
G.7	Limitations	302
G.8	Telemetry-related functions.	303
H	Parameterizations	305
H.1	Theory	305
H.2	Implementation	307
H.3	Example	309
H.4	Limitations	310
H.5	Other notes	310
H.6	Abundance	312
I	Density revisited	313
I.1	User-provided model functions	313
I.2	Sigmoid density function	315
I.3	More on link functions	317
J	Trend revisited	319
J.1	‘Dlambda’ parameterization	319
J.2	Covariate and other trend models	320
J.3	Fixing coefficients	321
J.4	Technical notes and tips	322
K	Expected counts	323
K.1	Number of individuals n	323
K.2	Number of detections C	323
K.3	Number of recaptures r	324
K.4	Number of movements m	324
K.5	Individuals detected at two or more detectors n_2	325
K.6	Single-catch traps	326
L	Datasets	327
	References	328

Foreword

This book is about the methods for describing animal populations that have come to be called ‘spatially explicit capture–recapture’ or simply ‘spatial capture–recapture’. We use ‘SECR’ as a general label for these data and models.

SECR data are observations of marked animals at known locations. The observations are from a well-defined regime of spatial sampling, most often with traps, cameras, or some other type of passive detector. SECR models are used to estimate parameters of the animal population, particularly the population density. We focus on ‘closed’ populations whose composition does not change during sampling.

Why SECR?

Non-spatial capture–recapture methods are highly developed and powerful (Cooch & White, 2023; Otis et al., 1978; Williams et al., 2002). SECR plugs some gaps in non-spatial methods (particularly with respect to density estimation), and has some unexpected benefits:

1. *Freedom from edge effects*

Estimation of density with non-spatial capture–recapture is dogged by uncertain edge effects. SECR explicitly accounts for edge effects so density estimates are unbiased.

2. *Reduced individual heterogeneity*

Unmodelled individual heterogeneity of detection is a universal source of bias in capture–recapture (e.g., Laake & Collier, 2024). Spatial sampling is a potent source of heterogeneity, due to differential access to detectors. SECR models this component of heterogeneity, which then ceases to be a problem.

3. *Scalable detection model*

The detection model in SECR is built from components describing the interaction between a single individual and a single detector. Parameter estimates can therefore be used to simulate sampling with novel detector configurations, which is key to designing better studies.

4. *Coherent adjustment for effort*

Known variation in effort, including incomplete use of a detector array, is included directly in the model, without additional covariates or parameters.

5. *Spatial pattern (covariates)*

SECR allows density to be modelled as a function of continuous spatial covariates.

Why this book?

The literature of SECR has grown beyond the attention spans and time budgets of most users. Major SECR publications are Efford (2004), Borchers & Efford (2008), Royle et al. (2014), Borchers & Fewster (2016), Sutherland et al. (2019) and Turek et al. (2021).

This book provides both a gentle introduction, in the spirit of Cooch & White (2023), and more in-depth treatment of important topics. It is software oriented and therefore unashamedly partial and incomplete. Much of the content is drawn from earlier papers and the documentation of R (R Core Team, 2024) packages. Some topics are yet to be included (e.g., acoustic data) but documentation may be found on the [DENSITY](#) website. Others such as partial identity models (Augustine et al., 2018) have yet to be considered at all.

SECR has become popular for the potential benefits noted above. But are the results reliable? Understanding the real-world performance of SECR is an active research area with its own questions. Are particular field data adequate? Are results robust when assumptions are not strictly met? How can we design better studies? We assemble the evidence in a form that we hope will be useful to practitioners.

Organisation

Part I introduces the concepts of SECR and walks the reader through a simple example. Part II establishes the necessary theory. Part III provides substantial new material on the performance of SECR: Which assumptions really matter? and How should studies be designed? Part IV is a practical guide to SECR modelling with the R package **secr**. Appendices provide detail on specialised topics such as area and transect searches, spatial mark-resight and non-Euclidean distances.

We expect that most readers will start with Part I and thereafter jump to topics of interest. Cross-references are provided to fill in relevant detail that may have been missed.

Software

The R package **secr** (Efford, 2025a) provides most of the functionality we will need. It performs maximum likelihood estimation for a range of closed-population SECR models. The Windows application [DENSITY](#) (Efford et al., 2004) has been superseded, although its graphical interface can still come in handy.

Bayesian approaches using Markov chain Monte Carlo (MCMC) methods are a flexible, but generally slower, alternative to maximum likelihood (Section [1.8.4](#)). The R package [nimbleSCR](#) promises to make MCMC methods for SECR more accessible and faster.

Add-on packages extend the capability of **secr**:

- [seclinear](#) enables the estimation of linear density (e.g., animals per km) for populations in linear habitats such as stream networks ([seclinear-vignette.pdf](#)).
- [ipsecl](#) fits models by simulation and inverse prediction, rather than maximum likelihood; this is a rigorous way to analyse data from single-catch traps ([ipsecl-vignette.pdf](#)).
- [secdesign](#) enables the assessment of alternative study designs by Monte Carlo simulation; scenarios may differ in detector (trap) layout, sampling intensity, and other characteristics ([secdesign-vignette.pdf](#)).
- [openCR](#) implements the open-population models of Efford & Schofield (2020).

These packages are available from the [CRAN](#) repository – just open R and type `install.packages('xxxx')` where `xxxx` is the package name.

Other R packages for SECR may be found outside CRAN. A distinct maximum-likelihood implementation by Sutherland et al. (2019) is available on GitHub (<https://github.com/jaroylo/oSCR>).

Recommended citation

Efford, M. G. (2025) The SECR book. A handbook of spatially explicit capture–recapture methods. Version 1.0.0. *Zenodo* <https://doi.org/10.5281/zenodo.15109938>.

The book is available online at <https://murrayefford.github.io/SECRbook/>. The Quarto source files, including R code, are at <https://github.com/murrayefford/SECRbook>. See the [docs folder](#) for the most recent pdf.

Feedback

This is work in progress. If you find an error or would like to make a suggestion, please raise an issue on [GitHub](#) or contact the author directly.

Acknowledgments

Ian Durbach, Brian Gerber, Dan Linden, Cyril Milleret, Joanne Potts, and Rahel Sollmann gave helpful comments on earlier versions of some chapters. For help with particular chapters I especially thank Rahel (Chapter 6) and Ian (Chapter 8). Errors remain my own. Matt Schofield provided encouragement, answered some theoretical questions, and reviewed several chapters. Thanks to John Boulanger for his collaboration on study design and analysis over many projects, and to Alice Kenney for her snowshoe hare photograph.

The simulations in Chapter 6 used the New Zealand eScience Infrastructure (NeSI) high performance computing facilities URL <https://www.nesi.org.nz/>.

Thanks for data -

- Ken Burnham: snowshoe hares Chapter [2](#)
- Jared Laufenburg et al. & Great Smoky Mountains NP: black bears Chapter [12](#)
- Kevin Young: horned lizards Appendix [D](#)
- Garth Mowat: Selkirk grizzly bears Appendix [F](#)
- Chris Sutherland: non-Euclidean simulation Appendix [F](#)

See also the links in Appendix [L](#).



This book is licensed under the [Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International](#) license.

Murray Efford
Dunedin, April 2025

Part I

Basics

1 Concepts and terminology

This chapter briefly introduces the concepts and terminology of SECR. The following chapter gives a [simple example](#). Technical details are provided in the [Theory](#) chapters.

1.1 Motivation

“Measurement of the size of animal populations is a full-time job and should be treated as a worthy end in itself.”

S. Charles Kendeigh (1944)



Photo: L.L. Getz

Only a brave ecologist would make this claim today. Measuring animal populations is now justified in more practical terms - the urgent need for evidence to track biodiversity decline or to manage endangered or pest species or those we wish to harvest sustainably. But we rather like Kendeigh’s formulation, to which could be added the lure of the statistical challenges. This book has no more to say about the diverse reasons for measuring populations: we will focus instead on a particular toolkit.

Populations of some animal species can be censused by direct observation, but many species are elusive or cryptic. Surveying these species requires indirect methods, often using passive devices such as traps or cameras that accumulate records over time. Passive devices encounter animals as they move around. The scale of that movement is unknown, which creates uncertainty regarding the population that is sampled. Furthermore, the number of observed individuals increases indefinitely as more and more peripheral individuals are encountered¹.

SECR cuts through this problem by modelling the spatial scale of detection to obtain an unbiased estimate of the stationary density of individuals.

¹For a bounded population the number of observed individuals has a natural limit, the true population size. Even when it exists, this is not usually the number that would be estimated by non-spatial capture–recapture (Otis et al., 1978).

1.2 State and observation models

It helps to think of SECR in two parts: the state model and the observation model (Borchers et al., 2002). The state model describes the biological reality we wish to describe, the animal population. The observation model represents the sampling process by which we learn about the population. The observation model matters only as the lens through which we can achieve a clear view of the population. We treat the state model (spatial population) and observation model (detection process) independently, with rare exceptions (Section 5.10).

1.3 Spatial population

An animal population in SECR is a spatial point pattern. Each point represents the location of an animal's activity centre, often abbreviated AC.

Statistically, we think of a particular point pattern, a particular distribution of AC, as just one possible outcome of a random process. Run the process again and AC will land in different places and even differ in number. We equate the *intensity* of the random process with the ecological parameter 'population density'. This formulation is more challenging than the usual "density = number divided by area", but it opens the door to rigorous statistical treatment.

The usual process model is a 2-dimensional Poisson process (Fig. 1.1). By fitting the SECR model we can estimate the intensity surface of the Poisson process that we represent by $D(\mathbf{x})$ where $\mathbf{x}$ stands for the x, y coordinates of a point. Density may be *homogeneous* (a flat surface, with constant $D(\mathbf{x})$) or *inhomogeneous* (spatially varying intensity). Inhomogeneous density may differ discretely, between habitats, or continuously in response to mapped covariates.

An inhomogeneous intensity surface is considered to depend on a vector of parameters ϕ , hence $D(\mathbf{x}; \phi)$. For constant density ϕ is a single value. If there are two habitats ϕ provides a value for each one. If density varies linearly with a covariate then ϕ comprises the intercept and slope.

i Activity centre vs home range centre

'Activity centre' is often used in preference to 'home range centre' because it appears more neutral. 'Home range' implies a particular pattern of behaviour: spatial familiarity and repeated use in contrast to nomadism. However, SECR assumes the very pattern of behaviour (persistent use) that distinguishes a home range, and it is safe to use 'activity centre' and 'home range centre' interchangeably in this context.

1.4 Detectors

Basic SECR uses sampling devices (‘detectors’) placed at known locations. We need to recognise individuals whenever they are detected. At least some of the individuals should be detected at multiple locations. The accumulated detections of each known individual are its ‘detection history’. Device types differ according to how they affect animal behaviour and the data they collect (Table 1.1). The term ‘detector type’ groups devices according to the required probability model (Section 3.4).

Detection may be entirely passive and non-invasive if individuals carry unique natural marks (e.g., pelage patterns) or their DNA can be sampled. Devices that record detections passively are “proximity detectors”. Proximity detectors may be split according to the distribution of the number of detections per animal per occasion (binary, Poisson, or binomial), with binary being the most common².

Animals without natural marks must be marked individually on their first detection. This implies capture and handling. Only devices that hold an animal until it is released can truly be called ‘traps’.³ The probability model for trapping must allow for exclusivity: an animal can be found at only one trap on any occasion, and some traps (‘single-catch’ traps) also exclude other individuals after the first.

Table 1.1: Examples of SECR sampling devices

Device	Marks	Detector type	Example
Automatic camera	natural marks: stripes and spots	proximity	tiger, Royle, Karanth, et al. (2009)
Hair snag	microsatellite DNA	proximity	grizzly bear, Mowat & Strobeck (2000)
Cage trap	numbered ear tag	single-catch trap	brushtail possum, Efford et al. (2005)
Ugglan trap	numbered ear tag	multi-catch trap	field vole, Ergon & Gardner (2013)
Mist net	numbered leg band	multi-catch trap	red-eyed vireo, Borchers & Efford (2008)

1.5 Area search

The preceding description assumes sampling at discrete points in space. Searches may also be conducted across space, yielding a possibly unique location for each detection. There are parallel SECR models for such data (Chapter 4, Appendix D). Searched areas are somewhat analogous to detectors, and may or may not allow only one detection per occasion, like true

²In the **secr** software, type ‘proximity’ refers specifically to binary proximity detectors.

³Confusingly, **secr** uses ‘traps’ as a generic name for R objects holding detector coordinates and other information. This software-specific jargon should be avoided in publications.

traps. Data from area searches may be easier to analyse if they are [discretized](#) and treated as coming from many point detectors.

1.6 Sampling across time

For most vertebrates we expect population turnover (births, deaths, AC movement) on a time scale of months or years. Population change is often negligible over shorter spans (days or weeks, depending on the species, time of year etc.). Sampling over shorter spans can therefore treat the size and composition of a population as fixed: it is said to be ‘closed’. This greatly simplifies analysis. We assume closure except when considering [breaches of assumptions](#).

A set of samples from a closed population comprises a sampling ‘session’. For trap-type detectors there must be multiple ‘occasions’ within a sampling session to obtain recaptures. For proximity-type detectors the role of occasions is more subtle, and data may usually be collapsed to animal- and detector-specific counts. The spatial pattern of binary detections alone is sufficient to obtain an estimate of density (Efford, Dawson, et al., 2009).

i Continuous time

Some devices such as automatic cameras record data in continuous time. Detection events are stamped with the clock time and date, rather than assigned to discrete occasions. SECR models may be written for continuous time data, and these models have mathematical elegance. They find practical application in some niche cases (e.g. Distiller & Borchers, 2015).

It is not necessary for all detectors to be used on all occasions. Incomplete usage (and other variation in effort per occasion – Efford et al., 2013) may be recorded for each detector and allowed for in the analysis.

Data collected across multiple sessions potentially include the loss of some individuals and recruitment of others. An open population model is the natural way to go (e.g., Efford & Schofield, 2020). However, the complexity of open-population models can be avoided if sessions are treated as independent in a ‘multi-session’ closed population analysis (Chapter [14](#)).

1.7 Data structure

Data for a single SECR session comprise a 3-dimensional rectangular array with dimensions corresponding to known animals, sampling occasions, and detectors. Data in each cell of the array are usually binary (0/1) but may be integer counts > 1 (e.g., if binary data have been collapsed by occasion). In **secr**, an R object of class ‘capthist’ holds data in this form, along with the coordinates of the detectors in its ‘traps’ attribute. The user constructs a capthist object from text or spreadsheet input using data entry functions described in Chapter [9](#).

1.8 Model fitting

A SECR model combines a model for the point process (the state model) and a model for distance-dependent detection (the observation model). Unbiased estimates of population density (and other parameters) are obtained by jointly fitting the state and observation models.

1.8.1 Distance-dependent detection

In order to estimate density from a sample we must account for the sampling process. The process is inherently spatial: each animal is more likely to be detected near its AC, and less likely to be detected far away. Sampling filters the geographic locations of animals as indicated in Fig. 1.1.

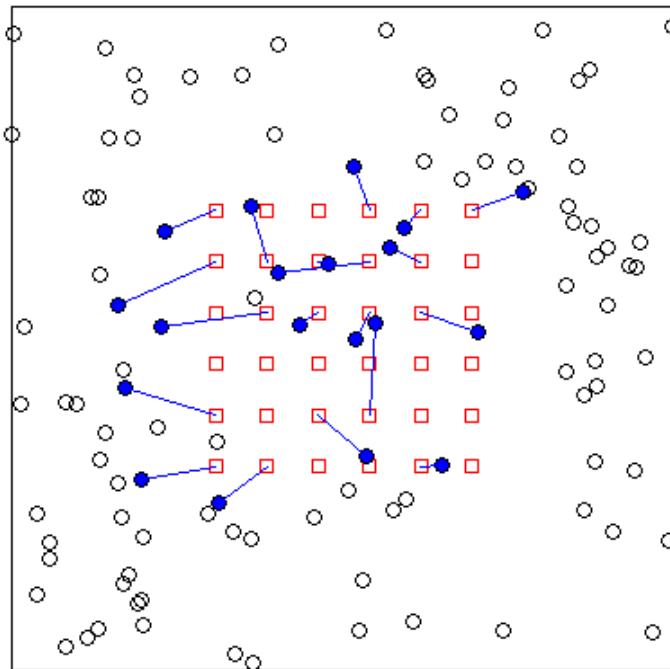


Figure 1.1: Distance-dependent detection of a uniformly distributed population (circles represent activity centres) at single-catch traps on a square grid (squares). Lines indicate random captures on a single occasion - each detected individual (filled circle) is linked to the trap in which it was caught. For this detector type SECR requires multiple sampling occasions.

The true locations of animals are not known, and therefore the distance-dependent probabilities cannot be calculated directly. The model is fitted by marginalising (integrating over)

animal locations.

Distance-dependent detection is represented by a ‘detection function’ with intercept, scale, and possibly shape, determined by parameters to be estimated⁴.

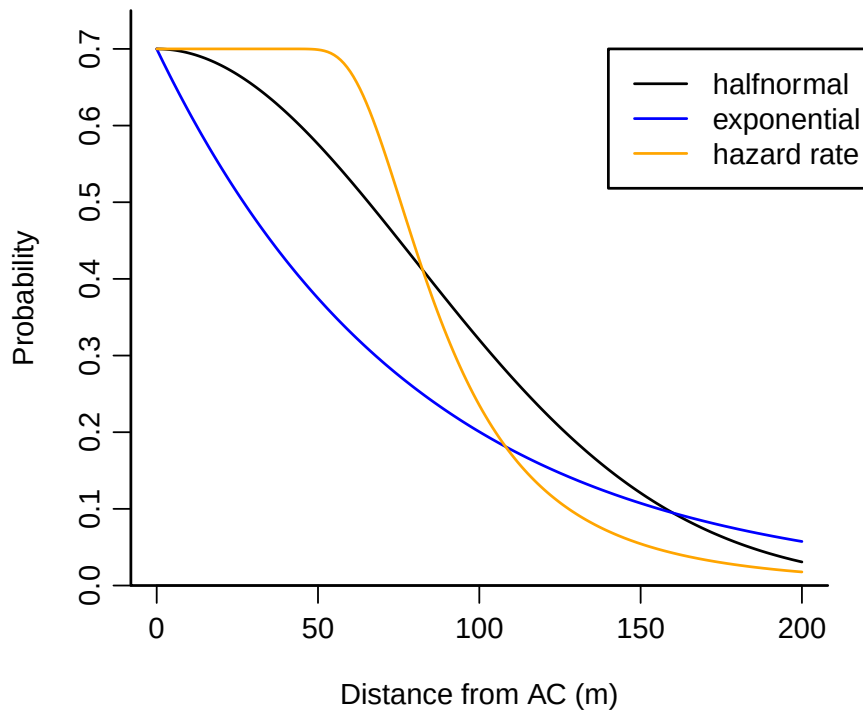


Figure 1.2: Some detection functions

1.8.2 Habitat

SECR models include a map of potential habitat near the detectors. Here ‘potential’ means ‘feasible locations for the AC of detected animals’. Excluded are sites that are known *a priori* to be unoccupied, and sites that are so distant that an animal centred there has negligible chance of detection.

The habitat map is called a ‘habitat mask’ in **secr** and a ‘state space’ in the various Bayesian implementations. It is commonly formed by adding a constant-width buffer around the detectors. For computational convenience the map is discretized as many small pixels. Spatial covariates (vegetation type, elevation, etc.) may be attached to each pixel for use in density models. Buffer width and pixel size are considered in Chapter 12. The scale at which spatial covariates affect density is considered [separately](#).

⁴All detection functions have intercept (g_0 , λ_0) and scale (σ) parameters; some such as the hazard rate function have a further parameter that controls some aspect of shape.

1.8.3 Link functions

A simple SECR model has three parameters: density D , and the intercept g_0 and spatial scale σ of the detection function. Valid values of each parameter are restricted to part of the real number line (positive values for D and σ , values between zero and one for g_0). A straightforward way to constrain estimates to valid values is to conduct maximization of the likelihood on a transformed (‘link’) scale: at each evaluation the parameter value is back transformed to the natural scale. The link function for all commonly used parameters defaults to ‘log’ (for positive values) except for g_0 which defaults to ‘logit’ (for values between zero and one).

Table 1.2: Link functions

Name	Function	Inverse
log	$y = \log(x)$	$\exp(y)$
logit	$y = \log[p/(1 - p)]$	$1/[1 + \exp(-y)]$
identity	$y = x$	y
cloglog	$y = \log(-\log(1 - p))$	$1 - \exp(-\exp(y))$

Working on a link scale is especially useful when the parameter is itself a function of co-variates. For example, $\log(D) = \beta_0 + \beta_1 c$ for a log-linear function of a spatially varying covariate c . The coefficients β_0 and β_1 are estimated in place of D per se.

We sometimes follow MARK (e.g., Cooch & White, 2023) and use ‘beta parameters’ for coefficients on the link scale and ‘real parameters’ for the core parameters ($D, g_0, \lambda_0, \sigma$) on the natural scale.

1.8.4 Estimation

There are several ways to estimate the parameters of the SECR probability model, all of them computer-intensive. We focus on numerical maximization of the log likelihood (Borchers & Efford (2008) and Chapter 3). The likelihood integrates over the unknown locations of the animals’ activity centres. This is achieved in practice by summing over cells in a discretized map of the [habitat](#).

In outline, a function to compute the log likelihood from a vector of beta parameters is passed, along with the data, to an optimization function. Optimization is iterative. For illustration, Fig. 1.3 shows the sequence of likelihood evaluations with two maximization algorithms when the parameter vector consists of only the intercept and spatial scale of detection. Optimization returns the maximized log likelihood, a vector of parameter values at the maximum, and the Hessian matrix from which the variance-covariance matrix of the estimates may be obtained.

Bayesian methods make use of algorithms that sample from a Markov chain (MCMC) to approximate the posterior distribution of the parameters. MCMC for abundance estimation

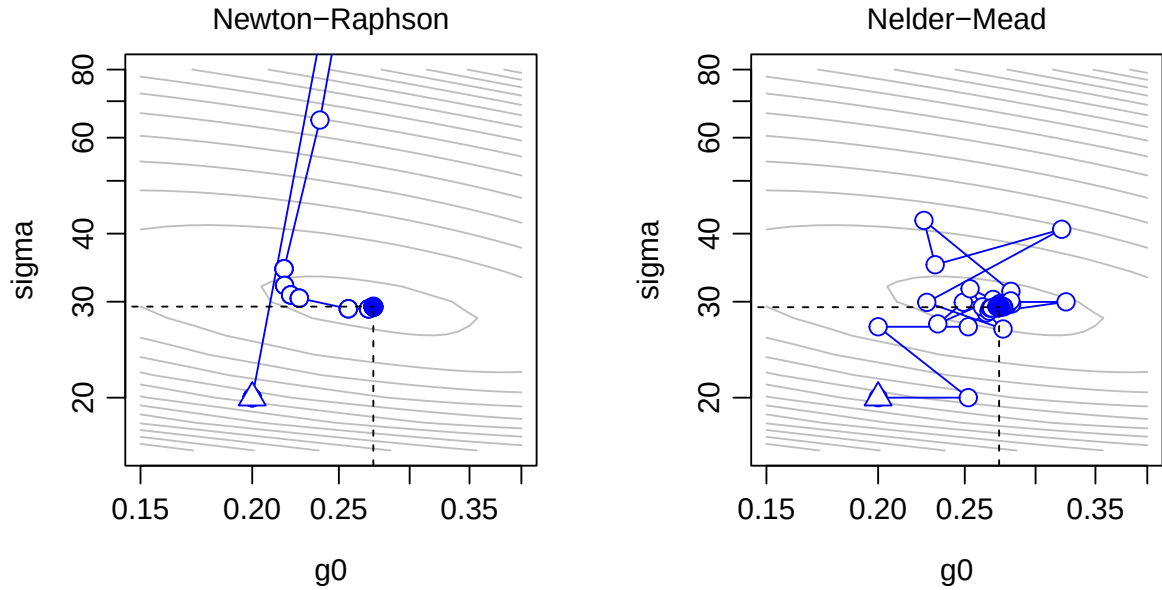


Figure 1.3: Numerical maximization of conditional likelihood by two methods; triangle – initial values; filled circle – estimates. Both converge on the same estimates (dashed lines). Newton-Raphson is the default method in *secr*.

faces the special problem that an unknown number of individuals, at unknown locations, remain undetected. The issue is addressed by data augmentation (Royle & Young, 2008) or using a semi-complete data likelihood (King et al., 2016).

Early anxiety about the suitability of MLE and asymptotic variances for SECR with small samples appears to have been mistaken. Royle, Karanth, et al. (2009) believed that “... the practical validity of these procedures cannot be asserted in most situations involving small samples”. This has not been borne out by subsequent simulations. The final discussion of Gerber & Parmenter (2015) is pertinent: they give an even-handed summary of the philosophical and practical issues, and suggest that “the asymptotic argument against likelihood methods for SCR is moot.” Palmero et al. (2023) reported that Bayesian methods provide more precise estimates of density, but this appears to be an artefact: it is a mistake to compare estimates from MLE models with random $N(A)$ with estimates from Bayesian models with fixed $N(A)$ (see Section 3.5 for background on $N(A)$, the population in area A).

The choice between Bayesian and frequentist (MLE) methods is now an issue of convenience for most users:

- MLE provides fast and repeatable results for established models with a small to medium number of parameters.
- Bayesian methods have an advantage for novel models and possibly for those with many parameters.

Bayesian priors are usually chosen to be ‘uninformative’, and under these conditions the mode of the posterior distribution is expected to be a close match to the maximum likelihood estimate. Informative priors, perhaps based on estimates of detection parameters from past

studies, are a way to make something out of very sparse datasets. Their use must be fully reported and explained.

A further method, simulation and inverse prediction, has a niche use for data from single-catch traps (Efford et al., 2004; Efford, 2023).

2 Simple example

In this chapter we use the R package **secr** to fit an SECR model to the Alaskan snowshoe hare data of Burnham and Cushwa (Otis et al., 1978).



Photo: Alice Kenney

“In 1972, Burnham and Cushwa (pers. comm.) laid out a livetrapping grid in a black spruce forest 30 miles (48.3 km) north of Fairbanks, Alaska. The basic grid was 10 x 10, with traps spaced 200 feet (61 m) apart. Trapping for snowshoe hares *Lepus americanus* was carried out for 9 consecutive days in early winter. Traps were not baited for the first 3 days, and therefore we have chosen to analyze the data from the last 6 days of trapping.”

Otis et al. (1978, p. 36)

2.1 Input data

The raw data are in two text files, the capture file and the trap layout file. Data from Otis et al. (1978) have been transformed for **secr** (code in [secr-tutorial.pdf](#)).

```
fnames <- c("hareCH6capt.txt", "hareCH6trap.txt")
url <- paste0('https://www.otago.ac.nz/density/examples/', fnames)
download.file(url, fnames, method = "libcurl")
```

The capture file “hareCH6capt.txt” has one line per capture and four columns (header lines are commented out and are not needed). Here we display the first 6 lines. The first column is a session label derived from the original study name; here there is only one session. The second column is an identifier unique to an individual (e.g., tag number). Sampling occasions are numbered 1, 2 etc. (in some studies there is only one).

```
# Burnham and Cushwa snowshoe hare captures
# Session ID Occasion Detector
wickershamunburne 1 2 0201
wickershamunburne 19 1 0501
wickershamunburne 72 5 0601
wickershamunburne 73 6 0601
...
```

The trap layout file “hareCH6trap.txt” has one row per trap and columns for the detector label and x- and y-coordinates. We display the first 6 lines. The detector label is used to link captures to trap sites. Coordinates can relate to any rectangular coordinate system; **secr** will assume distances are in metres. These coordinates simply describe a 10×10 square grid with spacing 61 m.

Tip

Do not use unprojected geographic coordinates (latitude and longitude). Section [C.2.1](#) shows how to transform geographic coordinates to rectangular coordinates (e.g., UTM).

```
# Burnham and Cushwa snowshoe hare trap layout
# Detector x y
0101 0 0
0201 60.96 0
0301 121.92 0
0401 182.88 0
...
```

After opening R, we load **secr** and read the data files to construct a capthist object. The detectors are single-catch traps (maximum of one capture per animal per occasion and one capture per trap per occasion).

```
library(secr)
hareCH6 <- read.capthist("hareCH6capt.txt", "hareCH6trap.txt", detector = "single")
```

No errors found :-)

The capthist object `hareCH6` now contains almost all the information needed to fit a model.

2.2 Check data

First review a summary of the data. See [?summary.capthist](#) for definitions of the summary statistics `n`, `u`, `f` etc.

```
summary(hareCH6)
```

```
Object class      capthist
Detector type     single
Detector number   100
Average spacing   60.96 m
x-range           0 548.64 m
y-range           0 548.64 m
```

Counts by occasion

	1	2	3	4	5	6	Total
<code>n</code>	16	28	20	26	23	32	145
<code>u</code>	16	24	9	9	6	4	68
<code>f</code>	25	22	13	5	1	2	68
<code>M(t+1)</code>	16	40	49	58	64	68	68
<code>losses</code>	0	0	0	0	0	0	0
<code>detections</code>	16	28	20	26	23	32	145
<code>detectors visited</code>	16	28	20	26	23	32	145
<code>detectors used</code>	100	100	100	100	100	100	600

These are *spatial* data so we learn a lot by mapping them. The `plot` method for `capthist` objects has some handy arguments; set `tracks = TRUE` to join consecutive captures of each individual.

```
par(mar = c(1,1,3,1)) # reduce margins
plot(hareCH6, tracks = TRUE)
```

wickershamunburne
6 occasions, 145 detections, 68 animals

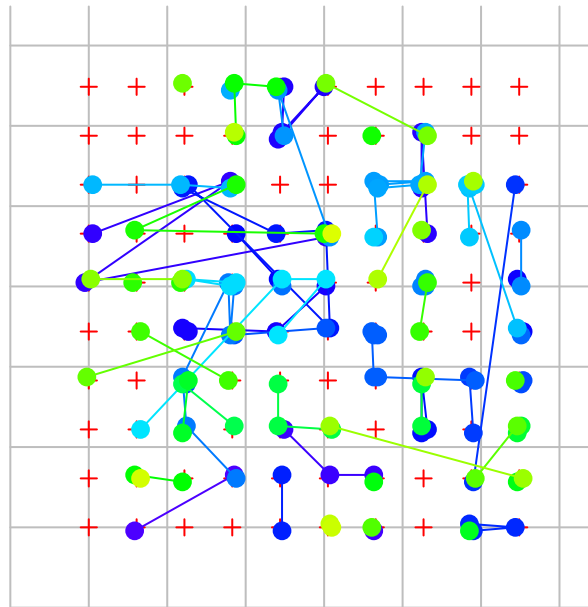


Figure 2.1: Snowshoe hare spatial capture data. Trap sites (red crosses) are 61 m apart. Grid lines (grey) are 100 m apart (use arguments `gridlines` and `gridspace` to suppress the grid or vary its spacing). Colours help distinguish individuals, but some are recycled.

The most important insight is that individuals tend to be recaptured near their site of first capture. This is expected when the individuals of a species occupy home ranges. In SECR models the tendency for detections to be localised is reflected in the spatial scale parameter σ . Good estimation of σ and density D requires spatial recaptures (i.e. captures at sites other than the site of first capture).

Successive trap-revealed movements can be extracted with the `moves` function and summarised with `hist`:

```
m <- unlist(moves(hareCH6))
par(mar = c(3.2,4,2,1), mgp = c(2.1,0.6,0)) # reduce margins
hist(m, breaks = seq(0,500,61), xlab = "Movement m", main = "")
```

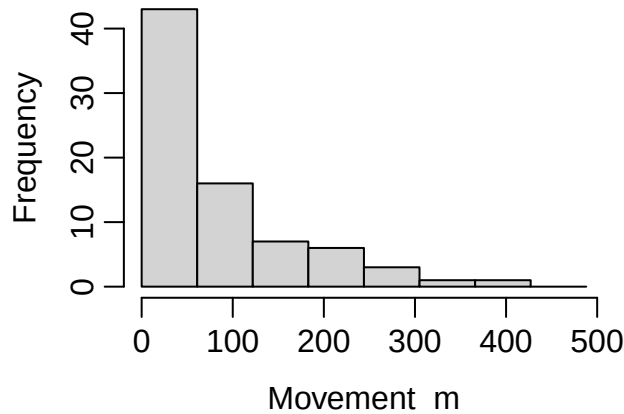


Figure 2.2: Trap-revealed movements of snowshoe hares

About 30% of trap-revealed movements were of > 100 m (Fig. 2.2; try also `plot(ecdf(m))`), so we can be sure that peripheral hares stood a good chance of being trapped even if their home ranges were centred well outside the area plotted in Fig. 2.1.

2.3 Fit a simple model

Next we fit the simplest possible SECR model with function `secr.fit`. The `buffer` argument determines the habitat extent - we take a stab at this and check it later. Setting `trace = FALSE` suppresses printing of intermediate likelihood evaluations; it doesn't hurt to leave it out. We save the fitted model with the name 'fit'. Fitting is much faster if we use parallel processing in multiple threads - the number will depend on your machine, but `ncores = 7` is OK in Windows with a quad-core processor.

```
fit <- secr.fit (hareCH6, buffer = 250, trace = FALSE, ncores = 7)
```

Warning: multi-catch likelihood used for single-catch traps

A warning is generated. The data are from single-catch traps, but there is no usable theory for likelihood-based estimation from single-catch traps. This is not the obstacle it might seem, because simulations seem to show that the alternative likelihood for multi-catch traps may be used without biasing the density estimates (Efford, Borchers, et al., 2009). It is safe to ignore the warning for now. The issue arises later as a breach of the [independence assumption](#).

2.4 Output

The output from `secr.fit` is an R object of class 'secr'. If you investigate the structure of `fit` with `str(fit)` it will seem to be a mess: it is a list with more than 25 components, none of which contains the final estimates you are looking for.

To examine model output or extract particular results you should use one of the functions defined for the purpose. Technically, these are S3 methods for the class 'secr'. The key methods are `print`, `plot`, `AIC`, `coef`, `vcov`, and `predict`. Append 'secr' when seeking help e.g. `?print.secr`.

Typing the name of the fitted model at the R prompt invokes the `print` method for `secr` objects and displays a more useful report.

```
fit
```

```
secr.fit(capthist = hareCH6, buffer = 250, trace = FALSE, ncores = 7)
secr 5.2.1, 17:45:36 04 Feb 2025
```

```
Detector type      single
Detector number    100
Average spacing    60.96 m
x-range            0 548.64 m
y-range            0 548.64 m
```

```
N animals      : 68
N detections    : 145
N occasions     : 6
Mask area      : 104.595 ha
```

```
Model           : D~1 g0~1 sigma~1
Fixed (real)    : none
Detection fn     : halfnormal
Distribution     : poisson
N parameters    : 3
Log likelihood   : -607.988
AIC              : 1221.98
AICc             : 1222.35
```

```
Beta parameters (coefficients)
      beta SE.beta    lcl    ucl
D      0.382529 0.129950 0.127831 0.637227
g0     -2.723792 0.160932 -3.039213 -2.408372
sigma   4.224580 0.065323 4.096549 4.352610
```


Variance-covariance matrix of beta parameters

	D	g0	sigma
D	0.01688706	-0.00173068	-0.00162336
g0	-0.00173068	0.02589902	-0.00737561
sigma	-0.00162336	-0.00737561	0.00426709

Fitted (real) parameters evaluated at base levels of covariates

	link	estimate	SE.estimate	lcl	ucl
D	log	1.4659870	0.19131245	1.1363609	1.8912283
g0	logit	0.0615839	0.00930046	0.0456855	0.0825365
sigma	log	68.3457767	4.46931482	60.1324204	77.6809774

The report comprises these sections that you should locate in the preceding output:

- function call and time stamp
- summary of the data
- description of the model, including the maximized log likelihood, Akaike's Information Criterion AIC
- estimates of model coefficients ('beta' parameters)
- estimates of variance-covariance matrix of the coefficients
- estimates of the 'real' parameters

The last three items are generated by the `coef`, `vcov` and `predict` methods respectively. The final table of estimates is the most interesting, but it is derived from the other two. For our simple model there is one beta parameter for each real parameter¹. The estimated density is 1.47 hares per hectare, 95% confidence interval 1.14–1.89 hares per hectare².

The other two real parameters jointly determine the detection function that you can easily plot with 95% confidence limits:

```
par(mar = c(5,4.5,2,1)) # adjust white margins
plot(fit, limits = TRUE)
```

¹We can get from beta parameter estimates to real parameter estimates by applying the inverse of the link function e.g. $\hat{D} = \exp(\hat{\beta}_D)$, and similarly for confidence limits; standard errors require a delta-method approximation (Lebreton et al. 1992).

²One hectare (ha) is 10000 m² or 0.01 km².

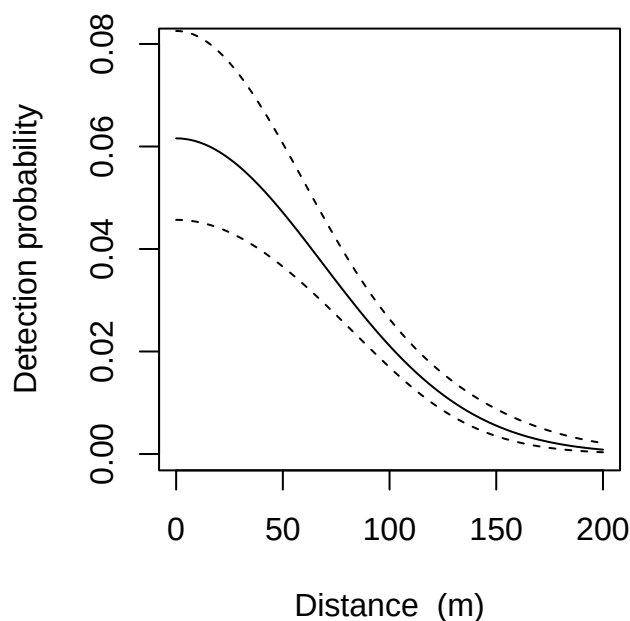


Figure 2.3: Fitted halfnormal detection function, with 95% confidence limits

2.5 Revisiting buffer width

Choosing a buffer width is a common stumbling block. We used `buffer = 250` without any explanation. Here it is. As far as we know, the snowshoe hare traps were surrounded by suitable habitat. We limit our attention to the area immediately around the traps by specifying a habitat buffer. The `buffer` argument is a short-cut for defining the potential habitat (area of integration); the alternative is to provide a habitat mask in the `mask` argument of `secr.fit`. Buffers and habitat masks are covered at length in Chapter 12.

Buffer width is not critical as long as it is wide enough that animals at the edge have effectively zero chance of appearing in our sample, so that increasing the buffer has negligible effect on estimates. For half-normal detection (the default) a buffer of 4σ is usually enough³. We check the present model with the function `esaPlot`. The estimated density⁴ has easily reached a plateau at the chosen buffer width (dashed red line):

```
par(mar = c(5,4,2,1)) # adjust white margins
esaPlot(fit)
abline(v = 250, lty = 2, col = 'red')
```

³This is not just the tail probability of a normal deviate; think about how the probability of an individual being detected at least once changes with (i) the duration of sampling (ii) the density of detector array.

⁴These are Horvitz-Thompson-like estimates of density obtained by dividing the observed number of individuals n by [effective sampling areas](#) (Borchers and Efford 2008) computed as the cumulative sum over mask cells ordered by distance from the traps. The algorithm treats the detection parameters as known and fixed.

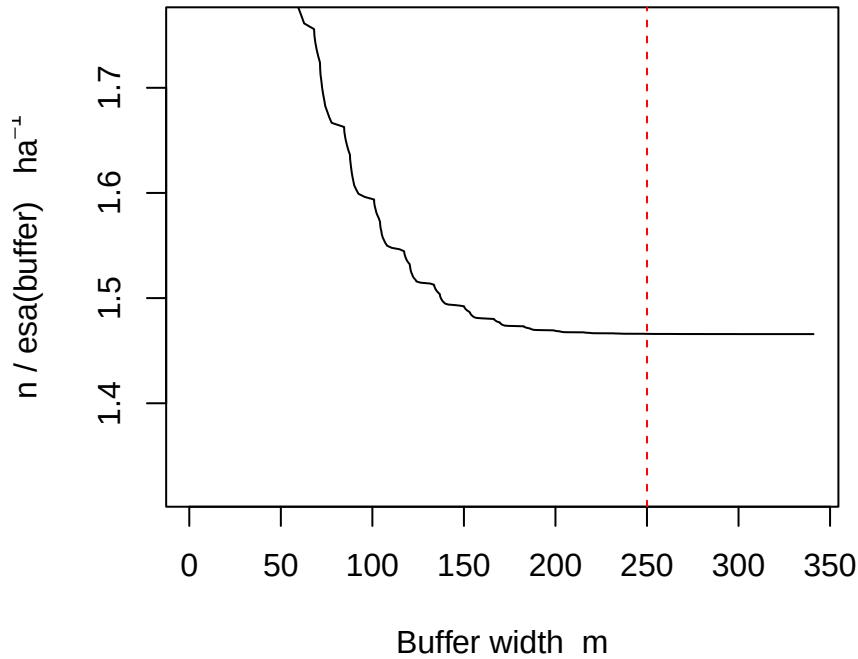


Figure 2.4: Post hoc evaluation of buffer width using `esaPlot()`

2.6 Overall probability of detection

As a final flourish, in Fig. 2.5 we plot contours of the overall probability of detection $p(\mathbf{x}; \theta)$ as a function of AC location $\mathbf{x}$, given the fitted model. The white line is the outer edge of the automatic mask generated by `secur.fit` with a 250-m buffer.

```
tr <- traps(hareCH6) # just the traps
dp <- detectpar(fit) # extract detection parameters from simple model
mask300 <- make.mask(tr, nx = 128, buffer = 300)
covariates(mask300)$pd <- pdot(mask300, tr, detectpar = dp,
  nooccasions = 6)
par(mar = c(1,1,1,5)) # adjust white margin
plot(mask300, cov = 'pd', dots = FALSE, border = 1, inset = 0.1,
  title = 'p.(x)')
plot(tr, add = TRUE) # over plot trap locations
pdotContour(tr, nx = 128, detectfn = 'HN', detectpar = dp,
  nooccasions = 6, add = TRUE)
plotMaskEdge(make.mask(tr, 250, type = 'trapbuffer'), add = TRUE,
  col = 'white')
```

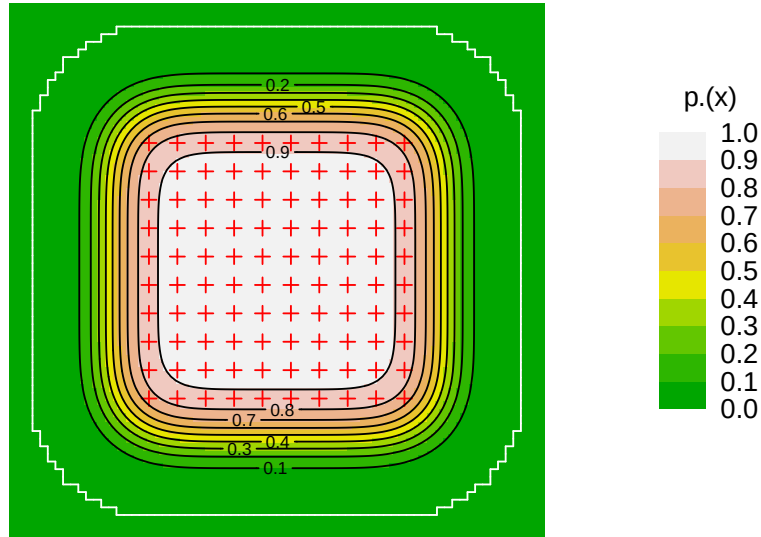


Figure 2.5: Contour plot of overall detection probability $p.(x; \theta)$.

```

covariates(mask300)$pd <- pdot(mask300, tr, detectpar = dp,
  nooccasions = 6)
covariates(mask300)$d <- distancetotrap(mask300, tr)
covariates(mask300)$dclass <- cut(distancetotrap(mask300, tr),
  c(0,100,200,400))
par(mar = c(4,4,2,2))
f <- c(0,0.1,0.2,0.3,0.4,0.5)
nm <- nrow(mask300)
hist(covariates(mask300)$pd, xlim = c(0,1), ylim = c(0,nm/2),
  breaks = seq(0,1,0.05), col = 'forestgreen', axes=F,
  ylab = "Fraction of mask population",
  xlab = "Overall probability of detection", main = "")
axis(1)
axis(2, at = nm * f, labels = f)
for (i in 0:9)
hist(covariates(mask300)$pd[covariates(mask300)$pd>i/10], breaks =
  seq(0,1,0.05), add = T, col = terrain.colors(10)[i+1])

```

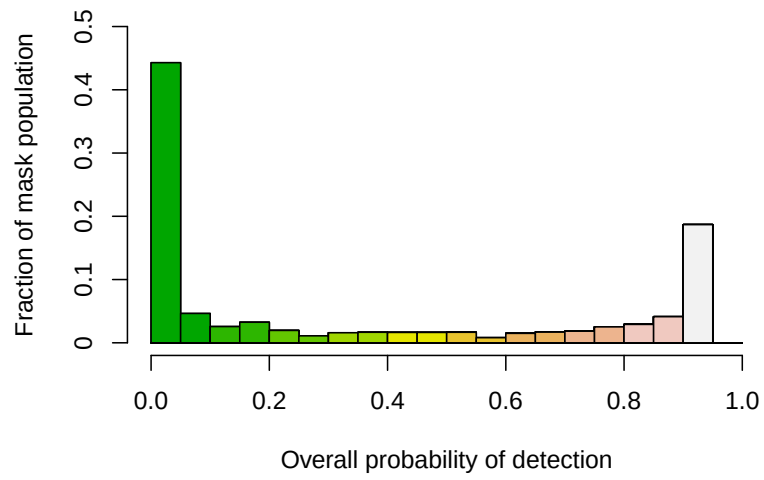


Figure 2.6: Distribution of overall detection probability $p(\mathbf{x}; \theta)$ for AC within the area of Fig. 2.5.

Part II

Theory

3 Likelihood-based SECR

Part II provides background on the theory of SECR that is mostly for reference. This chapter covers general notation and theory for point detectors. Chapter 4 describes theory for area and transect searches. Chapter 5 has extensions for multiple sessions, groups, effective sampling area, conditional estimation, relative density, population size, finite mixtures, hybrid mixture models, alternative parameterizations and model-based location of activity centres (‘fxi’). These chapters aim to be independent of any particular implementation, although some cross-references to **secr** creep in.

3.1 Notation

We use notation and terminology from Borchers & Efford (2008), with minor variations and extensions from Efford, Borchers, et al. (2009), Efford (2011) and elsewhere (Table 3.1).

Table 3.1: Mathematical notation for SECR

Category	Symbol	Meaning
General	AC	activity centre
	$\mathbf{x}$	point (x, y) in the plane
Data	n	number of individuals detected
	S	number of sampling occasions
	K	number of detectors
	ω_i	spatial detection history of the i -th animal
	Ω	set of all detection histories $\omega_i, i = 1..n$
Parameters	$D(\mathbf{x}; \phi)$ ¹	intensity at $\mathbf{x}$ of AC Poisson point process
	ϕ	parameter vector for AC point process
	θ	vector of detection parameters (minimally (g_0, σ) or (λ_0, σ))
	g_0	intercept of distance-probability-of-detection function
	λ_0	intercept of distance-hazard-of-detection function
	σ	spatial scale parameter of distance-detection function
Model	$d_k(\mathbf{x})$	distance of point $\mathbf{x}$ from detector k

¹We use $D(\mathbf{x})$ in preference to $\lambda(\mathbf{x})$ because λ has multiple meanings.

Category	Symbol	Meaning
	$\lambda(d)$	hazard of detection at distance d (distance-hazard-of-detection function)
	$g(d)$	probability of detection at distance d (distance-probability-of-detection function)
	$p(\mathbf{x}; \theta)$	probability that an individual with AC at $\mathbf{x}$ is detected at least once
	$h_{isk}(\mathbf{x}; \theta)$	hazard of detection at detector k for animal i on occasion s
	$p_{isk}(\mathbf{x}; \theta)$	probability of detection corresponding to $h_{isk}(\mathbf{x}; \theta)$
	$p_k(\mathbf{x}; \theta)$	$p_{isk}(\mathbf{x}; \theta)$ constant across individuals i and occasions s
	$\Lambda(\phi, \theta)$	expected number of detected individuals n
Habitat	A	potential habitat ('habitat mask', 'state space') $A \subset R^2$
	$ A $	area of A
	$N(A)$	number of AC in A

3.2 Likelihood

Parameters of the state model (ϕ) and the detection model (θ) are estimated jointly by maximizing the logarithm of the likelihood:

$$L(\phi, \theta | \Omega) = \Pr(n | \phi, \theta) \Pr(\Omega | n, \phi, \theta). \quad (3.1)$$

When density is constant across space, ϕ drops out of the rightmost term, which then relates to the detection (observation) model alone, and maximization of this component gives unbiased estimates of θ (see [Conditional likelihood](#)).

3.2.1 Number of individuals

If AC follow an inhomogeneous Poisson process then n has a Poisson distribution with parameter

$$\Lambda(\phi, \theta) = \int_{R^2} D(\mathbf{x}; \phi) p(\mathbf{x}; \theta) d\mathbf{x}, \quad (3.2)$$

where $D(\mathbf{x}; \phi)$ is the density at $\mathbf{x}$ and $p(\mathbf{x}; \theta)$ is the overall probability of detecting an AC at $\mathbf{x}$ (see Section 3.4 and Chapter 4). Thus $\Pr(n | \phi, \theta) = \Lambda^n \exp(-\Lambda) / n!$.

If the population size in a defined area A is considered to be [fixed](#) rather than Poisson then the distribution of n is binomial with size $N(A)$.

i Why R^2 ?

The region of integration R^2 refers to the entire plane, an infinite extent in two dimensions. Why use this when we know animal populations do not go on forever, and we later restrict integration to just part of the plane for computational reasons? The reason is that the integration over R^2 is a clear, model-based benchmark; restriction to a finite region A is arbitrary and must be done deliberately, with awareness of the consequences. Restriction intrudes distractions such as whether $N(A)$ is fixed or Poisson.

3.2.2 Detection histories

In general we have

$$\Pr(\Omega|n, \phi, \theta) = \binom{n}{n_1, \dots, n_C} \prod_{i=1}^n \Pr(\omega_i | \omega_i > 0, \phi, \theta), \quad (3.3)$$

where $\omega_i > 0$ indicates a non-empty detection history. The multinomial coefficient uses the frequencies $n_1, \dots, n_C$ of each of the C observed histories. The coefficient is a constant not involving parameters and it can be omitted without changing the model fit (consistent inclusion or exclusion is needed for valid likelihood-based comparisons such as those using AIC).

We do not know the true AC locations, but they can be integrated² out of the likelihood using an expression for their spatial distribution, i.e.

$$\Pr(\omega_i | \omega_i > 0, \phi, \theta) = \int_{R^2} \Pr(\omega_i | \omega_i > 0, \theta, \mathbf{x}) f(\mathbf{x} | \omega_i > 0, \phi, \theta) d\mathbf{x} \quad (3.4)$$

where $f(\mathbf{x} | \omega_i > 0, \phi, \theta)$ is the conditional density of an AC given that the individual was detected. The conditional density is given by

$$f(\mathbf{x} | \omega_i > 0, \phi, \theta) = \frac{D(\mathbf{x}; \phi) p(\mathbf{x}; \theta)}{\Lambda(\phi, \theta)}. \quad (3.5)$$

3.3 Distance-dependent detection

The key idea of SECR is that the probability of detecting a particular animal at a particular detector on one occasion can be represented as a function of the distance between its AC and the detector. The function should decline effectively to zero at large distances. Distances are not observed directly, and we rely on functions of somewhat arbitrary shape. Fortunately, the estimates are not very sensitive to the choice. Detection functions are covered in detail

²Integration is commonly performed by summing over many small cells for a finite region near the detectors, as both $\Pr(\omega_i)$ and $f(\mathbf{x})$ decline to zero at greater distances. We state the model in terms of the real plane and defer discussion of the region of integration to Chapter 12.

in Chapter 10. Either probability $g(d)$ or hazard $\lambda(d)$ may be modelled as a function of distance. A halfnormal form is commonly used (e.g., $g(d) = g_0 \exp(-d^2/2/\sigma^2)$). The shapes of, e.g., halfnormal $g(d)$ and halfnormal $\lambda(d)$ are only subtly different, but $\lambda(d)$ is preferred because it lends itself to mathematical manipulation and occurs more widely in the literature (often with different notation).

The function $\lambda(d)$ may be transformed into a probability with $g(d) = 1 - \exp[-\lambda(d)]$ and the reverse transformation is $\lambda(d) = -\log[1 - g(d)]$. The intercept parameter g_0 has been replaced by λ_0 ; although the name σ is retained for the spatial scale parameter this is not exactly interchangeable between the models.

3.4 Detector types

The SECR data ω_i for each detected individual comprise a matrix with dimensions S (occasions) and K (detectors). The matrix entries ω_{isk} may be binary (0/1) or integer (0, 1, 2, ...). Various probability models exist for ω_{isk} . The appropriate probability model follows in most cases directly from the type of detector device; we therefore classify probability models according to device type. Table 3.2 matches this classification to that of Royle et al. (2014). This section covers passive detection at a point; [area searches](#) are considered later.

Table 3.2: Detector types based on Efford & Boulanger (2019, Table 1) with cross references to Royle et al. (2014)

Detector type	Royle et al.	Data
Binary proximity	Bernoulli ¹	binary animal $\times$ occasion $\times$ detector
Count proximity		
Poisson	Poisson	integer animal $\times$ occasion $\times$ detector
Binomial	Binomial	integer animal $\times$ occasion $\times$ detector
Multi-catch trap	Multinomial	binary animal $\times$ occasion
Single-catch trap	—	binary animal $\times$ occasion, exclusive
Area search		integer animal $\times$ occasion $\times$ detector
Transect search		integer animal $\times$ occasion $\times$ detector
Exclusive area search ²		binary animal $\times$ occasion
Exclusive transect search ²		binary animal $\times$ occasion

1. Also ‘Binomial’ in Royle & Gardner (2011)

2. ‘Exclusive’ here means that an individual can be detected at no more than one detector (polygon or transect) per occasion.

For each type of detector we require $\Pr(\omega_{isk}|\mathbf{x})$ and the overall probability of detection $p(\mathbf{x})$. For some detector types it is more natural to express the probability in terms of the occasion- and trap-specific hazard $h_{sk} = \lambda[d_k(\mathbf{x}); \theta'] = -\log(1 - g[d_k(\mathbf{x}); \theta])$ than the probability $p_{sk}(\mathbf{x}) = g[d_k(\mathbf{x}); \theta] = 1 - \exp\{-\lambda[d_k(\mathbf{x}); \theta']\}$ ³.

³The parameter vectors θ and θ' differ for detection functions expressed in terms of probability ($g()$) and hazard ($\lambda()$).

We summarise the probability models in Table 3.3, with comments on each point detector type below.

Table 3.3: Summary of point detector types (conditioning on θ omitted to save space)

Detector type	$\Pr(\omega_{isk} \mathbf{x})$	$p_s(\mathbf{x})$
Binary proximity	$p_{sk}(\mathbf{x})^{\omega_{isk}} [1 - p_{sk}(\mathbf{x})]^{(1-\omega_{isk})}$	$1 - \prod_s \prod_k 1 - p_{sk}(\mathbf{x})$
Count proximity		
Poisson	$\{h_{sk}(\mathbf{x})^{\omega_{isk}} \exp[-h_{sk}(\mathbf{x})]\} / \omega_{isk}!$	$1 - \exp[-\sum_s \sum_k h_{sk}(\mathbf{x})]$
Binomial ¹	$\binom{B_s}{\omega_{isk}} p_{sk}(\mathbf{x})^{\omega_{isk}} [1 - p_{sk}(\mathbf{x})]^{(B_s - \omega_{isk})}$	$1 - \prod_s \prod_k [1 - p_{sk}(\mathbf{x})]^{B_s}$
Multi-catch trap ²	$\{1 - \exp[-H_s(\mathbf{x})]\} \frac{h_{sk}(\mathbf{x})}{H_s(\mathbf{x})}$	$1 - \exp[-\sum_s H_s(\mathbf{x})]$

1. B_s is the size of the binomial distribution, the number of opportunities for detection, assumed constant across detectors
2. $H_s = \sum_k h_{sk}(\mathbf{x})$ is the hazard summed over traps

3.4.1 Binary proximity detector

A proximity detector records the presence of an individual at or near a point without restricting its movement. The data are binary when there can be no more than one detection of an individual at a particular detector on a particular occasion, or repeat detections are deliberately discarded.

i Note

Discarding hard-won data may seem perverse, but it may be justified if the fraction is small (i.e. repeat detections rare). Binary data are less prone to problems due to non-independence of repeated visits to a detector, especially when a few individuals generate many repeat detections. Binary models also tend to be faster to fit and dodge the question of what discrete distribution should be used for the counts.

Assuming independence among detectors, the distance-detection model applies directly as the probability of detection in a particular detector, and the overall probability of detection is the complement of the product of probabilities of non-detection in all detectors.

3.4.2 Count proximity detectors

Some devices count visits by each individual: each datum is then a non-negative integer rather than binary. Automatic cameras (‘camera traps’) are a common example. Modelling depends on the distribution of the counts, for which the primary options are Poisson and binomial.

3.4.2.1 Poisson count proximity detector

If each visit by an individual is independent of other visits in a given interval (occasion) then the number of visits has a Poisson distribution. The sole parameter of the discrete Poisson distribution is the expected number of visits.

Independence may seem implausible, but it is commonly indistinguishable from non-independence when the expected number is small. However, it is not uncommon for one or two individuals to be photographed many times by one camera, and the Poisson is then unsuitable. The negative binomial is an alternative that models the overdispersion that results from non-independence, while requiring an additional parameter. There do not yet seem to be convincing applications of the negative binomial in SECR, and collapsing data to binary is recommended.

3.4.2.2 Binomial count proximity detector

Binomial counts arise when there is a known, finite number of opportunities for detection within each occasion that we denote B_s . This is the result when binary proximity data over many occasions are collapsed to a count on single occasion (e.g., Efford, Dawson, et al., 2009). The initial number of occasions is known ($B_s = S$) and places an upper limit on the count. Collapsing is often efficient. It precludes modelling parameter variation or learned responses across occasions.

Each count is binomial with size B_s and probability equal to the per-occasion detection probability.

3.4.3 Multi-catch trap

A trap is a device that detains an animal until it is released, allowing only one detection of that animal per occasion. The single-detector, single-AC probability from a distance-dependent detection function (preceding section) must be modified to allow for prior capture in a different trap: traps effectively “compete” for animals. If the trap remains open for captures of further animals then the solution is a straightforward competing risk model (Borchers & Efford, 2008).

The competing risk model uses the occasion- and trap-specific hazard h_{sk} .

3.4.4 Single-catch trap

A single-catch trap can catch only one animal at a time. This entails competition both among traps for animals and among animals for traps. No simple likelihood is available. Simulation-based methods (Efford, 2004, 2023) must be used for unbiased estimation of θ and trend in density unless the time of detection has been recorded (Distiller & Borchers, 2015). The multi-catch likelihood provides nearly unbiased estimates of uniform density.

3.5 Fixed N

The number of individuals of the target species (population size or N) is the parameter of primary interest in non-spatial closed population capture–recapture. This is seldom true in spatial capture–recapture. It is nevertheless the source of considerable confusion for two related reasons:

1. The population size depends on the extent of the modelled habitat (‘habitat mask’ or ‘state space’, area A) which is usually arbitrary, and
2. Having defined an arbitrary extent for numerical convenience (Chapter 12) the results, particularly the variance of $\hat{D}$ or $\hat{N}(A)$, depend on whether the enclosed population is assumed to be a random variable or fixed.

We expand here on the consequences of treating N as fixed.

The formulation of the state model as a Poisson process in two dimensions (Chapter 1) does not refer to population size. Under that model, the number of AC in an arbitrary area A is a poisson-distributed random variable regardless of whether the 2-D process is homogeneous or inhomogeneous.

The state model may also be cast as a ‘conditional’ or ‘binomial’ Poisson process’ (Illian et al., 2008). For an arbitrary area A the number of AC $N(A)$ is then considered fixed rather than Poisson. This is implicit in the Bayesian formulation of Royle et al. (2014). The distribution of n in the conditional model is binomial with size $N(A)$ and probability $p_c(\phi, \theta) = \int_A p(\mathbf{x}; \theta) f(\mathbf{x}; \phi) d\mathbf{x}$, where $f(\mathbf{x}; \phi) = D(\mathbf{x}; \phi) / \int_A D(\mathbf{x}; \phi) d\mathbf{x}$.

The form conditional on $N(A)$ leads to narrower confidence intervals for density owing to the exclusion of variation in $N(A)$ among realisations of the Poisson process for AC. This makes sense when A contains an isolated population with a natural boundary, but most applications do not meet this criterion.

3.6 Confidence intervals

Maximizing the log likelihood leads to a straightforward estimate of the asymptotic covariance matrix $\mathbf{V}$ of the beta parameters. If $\hat{\theta}$ is the vector of estimates and $\mathbf{H}(\hat{\theta})$ is the Hessian matrix evaluated at $\hat{\theta}$ then an estimate of the covariance matrix is $\hat{\mathbf{V}} = \mathbf{H}(\hat{\theta})^{-1}$.

i Hessian

The Hessian matrix is the square matrix of second-order partial derivatives of the log likelihood. For more on asymptotic variances of MLE see Seber (1982), Borchers et al. (2002), Cooch & White (2023) 1.3.2, and many statistics texts.

The sampling error of MLE is asymptotically normal, and symmetric (Wald) intervals for SECR parameters appear to work well on the link scale i.e. $\hat{\theta}_j \pm z_{\alpha/2} \hat{\sigma}_j$ is a $100(1 - \alpha)\%$

interval for $\hat{\theta}_j$ where $-z_{\alpha/2}$ is the $\alpha/2$ quantile of the standard normal deviate ($z_{0.025} = 1.96$) and $\hat{\sigma}_j^2$ is the estimated variance from $\hat{\mathbf{V}}$.

On back transformation to the natural ('real') scale these intervals become asymmetrical and generally have good coverage properties.

The method of profile likelihood is also available (e.g., Evans et al. (1996); `secr::confint.secr`), but it is seldom used as no problem has been shown with intervals based on asymptotic variances. Similarly, the additional computation required by parametric bootstrap methods is not usually warranted.

3.7 Varying effort

When sampling effort varies between detectors or over time in a capture–recapture study we expect a commensurate change in the number of detections. Allowing for known variation in effort when modelling detections has these benefits:

- detection parameters are related to a consistent unit of effort (e.g., one trap set for one day)
- the fit of the detection model is improved
- trends in the estimates may be modelled without confounding.

Borchers & Efford (2008) allowed the duration of exposure to vary between sampling occasions in their competing-hazard model for multi-catch traps. Efford et al. (2013) generalised the method to allow joint variation in effort over detectors and over time (occasions), and considered other detector types.

We use T_{sk} for the effort on occasion s at detector k . At its simplest, T_{sk} can be a binary indicator taking the values 0 (detector not used) or 1 (detector used) (when $T_{sk} = 0$, no detections are possible and $\omega_{isk} = 0$). For small, continuously varying, T_{sk} we expect the number of detections to increase linearly with T_{sk} ; saturation may occur at higher effort, depending on the detector type. Examples of possible effort variables are the number of days that each automatic camera was operated in a tiger study, or the number of rub trees sampled for DNA in each grid cell of a grizzly bear study.

Following convention in non-spatial capture–recapture (Cooch & White, 2023) we could model g_0 or λ_0 on the link scale (logit or log) as a linear function of T_{sk} (a time covariate if constant across detectors, a detector covariate if constant across occasions, or a time-varying detector-level covariate). However, this is suboptimal because varying effort has a linear effect only on λ_0 for Poisson count detectors, and the estimation of additional parameters is an unnecessary burden. T_{sk} is like an offset in a generalised linear model: it can be included in the SECR model without estimating an additional coefficient.

The SECR models for various detectors (Table 3.3) are expressed in terms of either the probability p_{sk} or the hazard h_{sk} . Each of these scales differently with T_{sk} as shown in Table 3.4. Only in the Poisson case is the expected number of detections linear on effort.

Table 3.4: Including effort in SECR models for various detector types. $p'_{sk}(\mathbf{x})$ and $h'_{sk}(\mathbf{x})$ replace the matching quantities in Table 3.3.

Detector type	Adjusted probability or hazard
Multi-catch trap	$h'_{sk}(\mathbf{x}) = h_{sk}(\mathbf{x})T_{sk}; H'_s(\mathbf{x}) = \sum_k h'_{sk}(\mathbf{x})$
Binary proximity	$p'_{sk}(\mathbf{x}) = 1 - (1 - p_{sk}(\mathbf{x}))^{T_{sk}}$
Poisson count proximity	$h'_{sk}(\mathbf{x}) = h_{sk}(\mathbf{x})T_{sk}$
Binomial count proximity	see below

For binomial count detectors we use a formulation not based directly on instantaneous hazard, as explained by Efford et al. (2013). For these detectors T_{sk} (assumed integer) is taken as the size of the binomial (maximum possible detections) and $p_{sk}(\mathbf{x})$ is unchanged.

4 Area search

Area searches differ from other modes of detection in that each detection may have different coordinates, and the coordinates are continuously distributed rather than constrained to fixed points by the field design. Searched areas may comprise one or more polygons, each of which can be considered a ‘detector’. Efford (2011) gave technical background on the fitting of polygon models to spatially explicit capture–recapture data by maximum likelihood. Royle & Young (2008) and Royle et al. (2014) provide a Bayesian solution.

Before launching into some rather heavy theory, we note that this can all be avoided by treating polygon data as if they were collected at many point detectors (pixel centres) obtained by discretizing the polygon(s).

4.1 Detector types for area search

Area-search analogues exist for each of the [point detector types](#).

- The ‘Poisson-count polygon’ type is suited to individually identifiable cues (e.g., faeces sampled for DNA).
- ‘Exclusive polygons’ are an analogue of multi-catch traps - they provide at most one detection of an individual per occasion, most likely as a result of a direct search for the animal itself. The horned lizard dataset of Royle & Young (2008) is a good example .
- The ‘binomial-count polygon’ type may result from collapsing exclusive polygon data to a single occasion.

We do not consider the area-search analogue of a binary proximity point detector because it seems improbable that binary data would be collected from each of several areas on one occasion.

4.1.1 Detection model for area search

The distance-dependent detection model for point detectors is replaced for area searches by an overlap model. The hazard of detection of an individual within an irregular searched area is modelled as a function of the quantitative overlap between its home range¹ (assumed circular) and the area searched (Fig. 4.1).

¹‘Home range’ is used here loosely - a more nuanced explanation would distinguish between the stationary distribution of activity (the home-range utilisation distribution) and the spatial distribution of cues (opportunities for detection) generated by an individual.

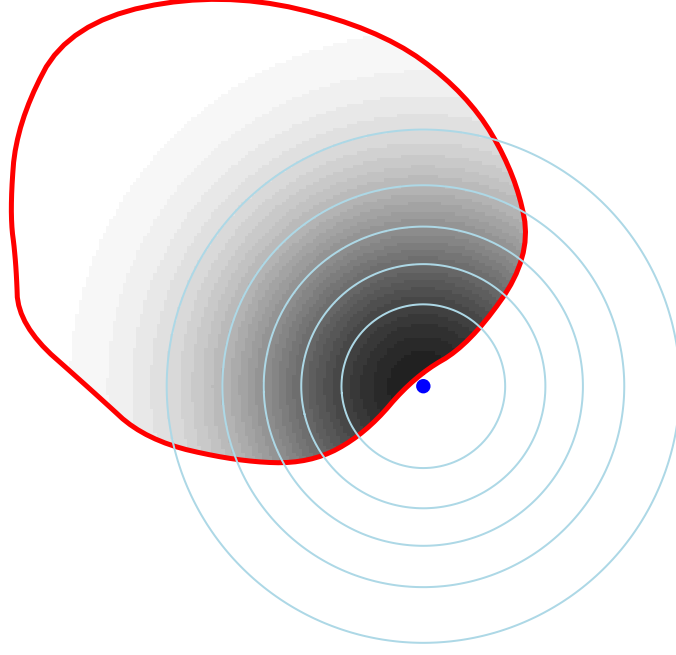


Figure 4.1: Hazard of detection for an animal centered at the blue dot on an irregular searched polygon (red outline). The cumulative hazard is modelled by the quantitative overlap (grey shading) between a radially symmetrical probability density (circular contours) and the searched area.

We use λ_0 for the expected number of detections (detected cues) of an animal whose home range lies completely within the area κ , and $h(\mathbf{u}|\mathbf{x})$ for the probability density of activity at point $\mathbf{u}$ for an animal centred at $\mathbf{x}$ (i.e. $\int_{R^2} h(\mathbf{u}|\mathbf{x}) d\mathbf{u} = 1$). Then the expected number of detected cues from an individual on occasion s in polygon k is

$$h_{sk}(\mathbf{x}; \theta) = \lambda_0 \int_{\kappa_k} h(\mathbf{u}|\mathbf{x}; \theta^-) du, \quad (4.1)$$

where $\theta = (\lambda_0, \theta^-)$ is the vector of detection parameters and κ_k refers to the k -th polygon. The probability of at least one detected cue is $p_{sk}(\mathbf{x}) = 1 - \exp[-h_{sk}(\mathbf{x})]$ as before. The detector-level probabilities conditional on AC location $\mathbf{x}$ follow directly from Table 3.3 (repeated in Table 4.1).

Table 4.1: Detector-level probabilities for area-search detector types

Detector type	$\Pr(\omega_{isk} \mathbf{x})$	$p.(\mathbf{x})$
Count polygon		
Poisson	$\{h_{sk}(\mathbf{x})^{\omega_{isk}} \exp[-h_{sk}(\mathbf{x})]\} / \omega_{isk}!$	$1 - \exp[-\sum_s \sum_k h_{sk}(\mathbf{x})]$
Binomial	$\binom{B_s}{\omega_{isk}} p_{sk}(\mathbf{x})^{\omega_{isk}} [1 - p_{sk}(\mathbf{x})]^{(B_s - \omega_{isk})}$	$1 - \prod_s \prod_k [1 - p_{sk}(\mathbf{x})]^{B_s}$
Exclusive polygon ¹	$\{1 - \exp[-H_s(\mathbf{x})]\}^{\frac{h_{sk}(\mathbf{x})}{H_s(\mathbf{x})}}$	$1 - \exp[-\sum_s H_s(\mathbf{x})]$

1. $H_s = \sum_k h_{sk}(\mathbf{x})$ is the hazard summed over areas. If a single polygon is searched then $h_{sk}(\mathbf{x}) = H_s(\mathbf{x})$, simplifying the expression for $\Pr(\omega_{isk}|\mathbf{x})$.

4.1.2 Location within searched polygon

The only data we have considered to this point are the occasion- and detector-specific binary or integer values ω_{isk} that record detections at the level of polygons. Each spatial detection history ω_i also includes within-polygon locations. These provide important information on detection scale σ and spatial variation in density ϕ , and we need to include them in the likelihood.

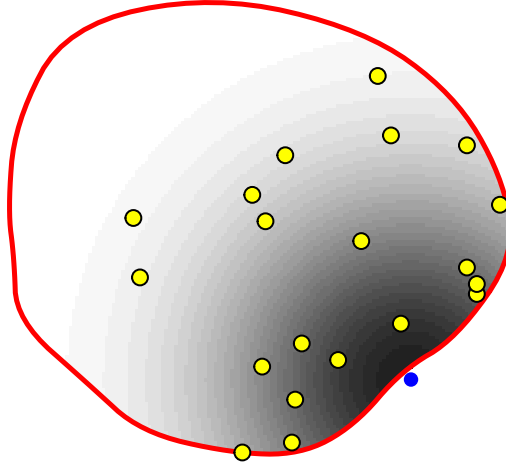


Figure 4.2: Cues of one animal (yellow) within a searched polygon (red outline).

For each individual i there are ω_{isk} locations of detected cues on occasion s at detector k . We use $\mathbf{Y}_{isk}$ for the collection and $\mathbf{y}_{iskj}$ ($j = 1, \dots, \omega_{isk}$) for each separate location. Then

$$\Pr(\mathbf{Y}_{isk}|\mathbf{x}, \theta^-) \propto \prod_{j=1}^{\omega_{isk}} \frac{h(\mathbf{y}_{iskj}|\mathbf{x}; \theta^-)}{\int_{\kappa_k} h(\mathbf{u}|\mathbf{x}; \theta^-) du}. \quad (4.2)$$

4.1.3 Likelihood

The likelihood component associated with ω_i , conditional on $\mathbf{x}$, is a product of the probability of observing ω_{isk} cues, and the probability of the within-polygon locations:

$$\Pr(\omega_i|\omega_i > 0, \mathbf{x}; \phi, \theta) \propto \frac{1}{p(\mathbf{x}; \theta)} \prod_s \prod_k \Pr(\omega_{isk}|\mathbf{x}; \theta) \Pr(\mathbf{Y}_{isk}|\mathbf{x}, \theta^-).$$

Inserted in Eq. 3.3, along with $f(\mathbf{x}|\omega_i > 0; \phi, \theta)$ from Eq. 3.5, this provides the likelihood component $\Pr(\Omega|n, \phi, \theta)$ of Eq. 3.1.

4.2 Transect search

Transect detectors are the linear equivalent of polygons. Cues may be observed only along the searched zero-width transect. Transect detectors, like polygon detectors, may be independent or exclusive. See [Appendix D](#) for more.

5 Special topics

The utility of SECR is enhanced by several extensions to the basic theory in the preceding chapters.

The most important of these are the linear sub-models for [detection](#) and [density](#) that we describe in their respective chapters. Others we list here.

5.1 Multi-session likelihood

The data may comprise multiple independent datasets to be analysed together. We call these ‘sessions’, following the terminology of **secr**. They may be synchronous spatial samples from non-overlapping populations or samples of the same population widely separated in time. If there are J such datasets we can denote them $\Omega_j, j = 1, \dots, J$. The likelihood to be maximized is then $\prod_j L(\phi, \theta | \Omega_j)$. While the parameter vector (ϕ, θ) is common to all sessions, the mechanism of [linear submodels](#) allows ‘real’ parameter values to be specific to a session, common to multiple sessions, or modelled as a function of session-level covariates.

Failure of the independence assumption in a multi-session analysis results in underestimation of the sampling variance.

5.2 Groups

Users of MARK (Cooch & White, 2023) will be familiar with the stratification of a population into ‘groups’, each with potentially different values for parameters such as survival probability. Grouping requires that each animal is assigned to a group on first detection, and that the assignment is permanent. Then density and any detection parameter may be modelled as a function of the grouping factor. It seems natural to treat females and males as two groups as their movement behaviour and hence detection parameters may differ greatly, but in Chapter [16](#) we suggest other ways to model sex differences. Group models are available in **secr** only when maximizing the full likelihood. When maximizing the [conditional likelihood](#) the model may specify an individual factor covariate directly, with the same effect as grouping.

5.3 Effective sampling area

The effective sampling area is defined (Borchers & Efford, 2008) as

$$a(\theta) \equiv \int_{R^2} p(\mathbf{x}; \theta) d\mathbf{x}. \quad (5.1)$$

This is a scalar *effective* area for which $\hat{D} = n/a(\hat{\theta})$ is an unbiased estimate of density as developed in Section 5.4. It does not correspond to a geographic region or delimited polygon on the ground. It bears *no* relation to the traditional ‘effective trapping area’ A_W (e.g., Otis et al., 1978) for which $\hat{D} = \hat{N}/A_W$, given a non-spatial population estimate $\hat{N}$ and boundary strip width W . Variation in $a(\theta)$ depends not only on obviously spatial quantities such as the extent of the detector array and the spatial scale of detection σ , but also on non-spatial quantities such as sampling effort (e.g., the number of days of trapping) and the intercept of the distance-dependent detection function (g_0, λ_0) .

Gardner et al. (2009) and Royle, Nichols, et al. (2009) defined an ‘effective trapping area’ or ‘effective sample area’ A_e somewhat differently, omitting the intercept of the detection function. To our knowledge their definition has not found further use. A version closer to ours appears in Royle et al. (2014, Section 5.12).

Efford & Mowat (2014) defined a ‘single-detector sampling area’ $a_0(\theta) = 2\pi\lambda_0\sigma^2$ that is equal to Eq. 5.1 for an isolated detector with [detection functions](#) HHN or HEX. For K isolated detectors $a(\theta) = Ka_0(\theta)$, but overlap of the ‘catchment areas’ of adjacent detectors leads to $a(\theta) < Ka_0(\theta)$.

5.4 Conditional likelihood

The detection parameters (θ) may be estimated by maximizing the likelihood conditional on n (Eq. 3.3). When density is constant this reduces to

$$L_n(\theta|\Omega) \propto \prod_{i=1}^n \frac{\int_{R^2} \Pr(\omega_i|\mathbf{x}; \theta) d\mathbf{x}}{a(\theta)}.$$

Conditioning on n allows individual covariates $\mathbf{z}_i$ to be included in the detection model, so that the parameter vector takes a potentially unique value $\theta_i = f(\mathbf{z}_i)$ for each individual. The effective sampling area then varies among individuals as a function of their covariates. A corresponding Horvitz-Thompson-like derived estimate of density is $\hat{D}_{HT} = n / \sum_{i=1}^n a(\hat{\theta}_i)$ (Borchers & Efford, 2008).

When the conditional likelihood is maximized, the inverse Hessian provides variances for θ . The variance of the derived estimate of density depends also on the distribution of n . Following Huggins (1989),

$$\text{var}(\hat{D}_{HT}) = s^2 + \hat{\mathbf{G}}_{\theta}^T \hat{\mathbf{I}} \hat{\mathbf{G}}_{\theta} \quad (5.2)$$

where s^2 is the variance of $\hat{D}$ when θ is known, $\hat{\mathbf{I}}$ is the estimated information matrix (inverse Hessian), and $\hat{\mathbf{G}}$ is a vector containing the gradients of $\hat{D}$ with respect to the elements of θ ,

evaluated at the maximum likelihood estimates. Numerical evaluation of the second term is straightforward.

When the distribution of AC is inhomogeneous Poisson and detections are independent we expect n to have a Poisson distribution. If n is Poisson then $s^2 = \sum_{i=1}^n a(\hat{\theta}_i)^{-2}$. This simplifies to $n/a(\hat{\theta})^2$ in the absence of individual covariates.

When $N(A)$ is fixed, n is binomial and $s^2 = \sum_{i=1}^n [1 - a(\hat{\theta}_i)/|A|]/a(\hat{\theta}_i)^2$.

The variance from Eq. 5.2 is on the natural scale, and the Wald confidence interval computed on this scale is symmetrical. Intervals that are symmetric on the log scale and asymmetric on the natural scale have better coverage properties. These are obtained as $(\hat{D}_{HT}/C, \hat{D}_{HT}C)$ where $C = \exp\{z_{\alpha/2} \sqrt{\log[1 + \frac{\text{var}(\hat{D}_{HT})}{\hat{D}_{HT}^2}]}\}$ (Burnham et al., 1987; Chao, 1989).

5.5 Relative density

Conditioning on n *without* requiring uniform density as in the previous section leads to another result that is useful. We can estimate relative density, the distribution of AC in relation to habitat covariates, rather than absolute density. Conditioning allows individual covariates of detection $\mathbf{z}_i$ as before, and fitting is faster than for absolute density (Efford, 2025c).

We define relative density by $D'(\mathbf{x}|\phi^-) \equiv k^{-1}D(\mathbf{x}|\phi)$ where ϕ^- is a reduced vector of coefficients for the density sub-model and k is a constant of proportionality. Then we can discard the first factor in Eq. 3.1 and maximize a likelihood based on the second factor alone

$$L_r(\phi^-, \theta|\Omega, n) \propto \prod_{i=1}^n \frac{\int \Pr(\omega_i|\theta, \mathbf{x}) D'(\mathbf{x}|\phi^-) p(\mathbf{x}|\theta, \mathbf{z}_i) d\mathbf{x}}{\int D'(\mathbf{x}|\phi^-) p(\mathbf{x}|\theta, \mathbf{z}_i) d\mathbf{x}}. \quad (5.3)$$

For a log link the new parameter vector ϕ^- corresponds to the original ϕ with one coefficient, the intercept, fixed at zero. For an identity link the intercept is fixed at 1 and other coefficients are scaled. A derived estimate of the constant of proportionality is

$$\hat{k} = \sum_{i=1}^n 1 / \int D'(\mathbf{x}|\hat{\phi}^-) p(\mathbf{x}|\hat{\theta}, \mathbf{z}_i) d\mathbf{x}.$$

A derived estimate of the absolute density at point $\mathbf{x}$ is

$$\hat{D}(\mathbf{x}|n, \hat{\phi}^-, \hat{\theta}) = \sum_{i=1}^n \frac{D'(\mathbf{x}|\hat{\phi}^-)}{\int D'(\mathbf{x}|\hat{\phi}^-) p(\mathbf{x}|\hat{\theta}, \mathbf{z}_i) d\mathbf{x}}.$$

This approach makes the same sampling assumptions as the full model, including that individuals enter the sample by a spatial detection process whose parameter vector θ we estimate from the data. For poisson-distributed AC, parameter estimates from maximizing Eq. 5.3 are identical to those of the full likelihood except for the missing density intercept.

5.6 Adjustment for spatially selective prior marking

In some scenarios, the *only* individuals at risk of detection are those that were marked in an earlier phase of the study. This is the case for the automated detection of animals with implanted acoustic telemetry tags or passive integrated transponders (PIT) (e.g., Whoriskey et al., 2019). The data resemble mark–resight data (Appendix E) except that they do not include counts of unmarked animals. Estimates of ϕ^- using Eq. 5.3 then describe only the distribution of the individuals marked previously. Even if we can assume that AC are stationary between the two phases, ϕ^- is biased as a model for population distribution as it incorporates the spatial selectivity of marking.

Lack of information on the marking phase also afflicts mark-resight analyses (e.g., Efford & Hunter, 2018). The possible ‘solutions’ are all ad hoc. We can assume that the probability of becoming marked was independent of location, or we can scale $p(\mathbf{x}; \phi^-)$ by an externally computed variable $q(\mathbf{x})$ that is proportional to the spatial probability of marking. Essentially, $q(\mathbf{x})$ is an offset in the model for relative density.

An example is shown in Section 11.7. A model with flat (constant) probability of prior marking is inevitably a poor fit because in reality tagged individuals are concentrated near the detectors.

5.7 Population size N

Population size is the number of individual AC in a particular region; we denote this $N(A)$ for region A . For a flat density surface $E[N(A)] = D \cdot |A|$ where $|A|$ is the area of A .

D and $N(A)$ are interchangeable for specified A . In most applications of SECR there is no naturally defined region A , and therefore $\hat{N}(A)$ depends on the arbitrary choice of A . This weakness is not shared by $\hat{D}$.

i Abundance

Population size is sometimes termed ‘abundance’. We avoid this usage because ‘abundance’ has also been used as an umbrella term for density and population size, and its overtones are vague and biblical rather than scientific.

The population size of any region B may be predicted *post hoc* from a fitted density model using

$$\hat{N}(B) = \int_B \hat{D}(\mathbf{x}) d\mathbf{x}.$$

The prediction variance of $\hat{N}(B)$ follows from Poisson assumptions regarding $D(\mathbf{x})$ (Efford & Fewster, 2013).

5.8 Finite mixture models

Finite mixture models for individual heterogeneity of capture probability were formalised for non-spatial capture–recapture by Pledger (2000) and remain widely used (e.g., Cooch & White, 2023). These are essentially random-effect models in which the distribution of capture probability comprises two or more latent classes, each with a capture probability and probability of membership.

Borchers & Efford (2008, p. 381) gave the likelihood for a Poisson SECR model with U latent classes in proportions $\psi = (\psi_1, \dots, \psi_U)$. In all examples we have tried U is 2 or 3. For each class u there is an associated vector of detection parameters θ_u (collectively θ). Omitting the constant multinomial term,

$$\Pr(\Omega|n, \phi, \theta, \psi) \propto \prod_{i=1}^n \sum_{u=1}^U \int \frac{\Pr\{\omega_i|\mathbf{x}; \theta_u\}}{p(\mathbf{x}; \theta_u)} f(\mathbf{x}, u|\omega_i > 0) d\mathbf{x}$$

where

$$f(\mathbf{x}, u|\omega_i > 0) = \frac{D(\mathbf{x}; \phi) p(\mathbf{x}; \theta_u) \psi_u}{\sum_{u=1}^U \int D(\mathbf{x}; \phi) p(\mathbf{x}; \theta_u) \psi_u d\mathbf{x}}.$$

The expected number of detected animals n , replacing Eq. 3.2, is a weighted sum over latent classes:

$$\Lambda(\phi, \theta, \psi) = \sum_{u=1}^U \psi_u \int D(\mathbf{x}; \phi) p(\mathbf{x}; \theta_u) d\mathbf{x}. \quad (5.4)$$

Integration is over points in potential habitat, as usual.

5.9 Hybrid finite mixture models

We can modify the finite mixture likelihood for data in which the class membership of some or all individuals is known. Indicate the class membership of the i -th individual by a variable u_i that may take values 0, 1, ..., U , where $u_i = 0$ indicates an individual of unknown class, and the class frequencies are $n_0, n_1, \dots, n_U$ (not to be confused with $n_1, \dots, n_C$ in Eq. 3.3). We assume here that detection histories are sorted by class membership, starting with the unknowns.

The expression for λ in Eq. 5.4 is unchanged, but we must split $\Pr(\Omega|n, \phi, \theta, \psi)$ and include a multinomial term for the observed distribution over classes:

$$\begin{aligned} \Pr(\Omega|n, \phi, \theta, \psi) &\propto \prod_{i=1}^{n_0} \sum_{u=1}^U \int \frac{\Pr\{\omega_i|\mathbf{x}; \theta_u\}}{p(\mathbf{x}; \theta_u)} f(\mathbf{x}, u|\omega_i > 0) d\mathbf{x} \\ &\times \prod_{i=n_0+1}^n \int \frac{\Pr\{\omega_i|\mathbf{x}; \theta_{u_i}\}}{p(\mathbf{x}; \theta_{u_i})} f'(\mathbf{x}|\omega_i > 0; u_i) d\mathbf{x} \\ &\times \binom{n - n_0}{n_1, \dots, n_U} \prod_{u=1}^U \left[\frac{\lambda_u}{\lambda} \right]^{n_u}, \end{aligned} \quad (5.5)$$

where $\lambda_u = \psi_u \int D(\mathbf{x})p(\mathbf{x}; \theta_u) d\mathbf{x}$, and the multinomial coefficient $\binom{n-n_0}{n_1, \dots, n_U}$ is a constant that can be omitted. Rather than representing the joint probability density of $\mathbf{x}$ and u_i as in $f(\cdot)$ previously, $f'(\cdot)$ is the probability density of $\mathbf{x}$ for given u_i :

$$f'(\mathbf{x}|\omega_i > 0; u_i) = \frac{D(\mathbf{x})p(\mathbf{x}; \theta_{u_i})}{\int D(\mathbf{x})p(\mathbf{x}; \theta_{u_i})d\mathbf{x}}.$$

The likelihood conditions on the number of known-class animals detected ($n - n_0$), rather than modelling class identification as a random process. It assumes that the probability that class will be recorded does not depend on class, and that such recording when it happens is without error.

For homogeneous density the likelihood simplifies to

$$\begin{aligned} \Pr(\Omega|n, \phi, \theta, \psi) &\propto \prod_{i=1}^{n_0} \sum_{u=1}^U \int \frac{\Pr\{\omega_i|\mathbf{x}; \theta_u\}\psi_u}{\sum_u a(\theta_u)\psi_u} d\mathbf{x} \\ &\times \prod_{i=n_0+1}^n \int \frac{\Pr\{\omega_i|\mathbf{x}; \theta_{u_i}\}}{a(\theta_{u_i})} d\mathbf{x} \prod_{u=1}^U \left[\frac{a(\theta_u)\psi_u}{\sum_u a(\theta_u)\psi_u} \right]^{n_u}, \end{aligned}$$

where $a(\theta_u) = \int p(\mathbf{x}; \theta_u) d\mathbf{x}$.

5.10 Alternative parameterizations

The ‘real’ parameters in SECR are typically assumed to be independent (orthogonal). However, some parameter pairs co-vary in predictable ways owing to constraints on animal behaviour. Here it is more straightforward to work with the hazard detection functions. The intercept of the detection function λ_0 declines with increasing σ , all else being equal (Efford & Mowat, 2014). This is inevitable if detection is strictly proportional to time spent near a point, given a bivariate home range utilisation model (pdf for activity). Also, home-range size and the SECR parameter σ decline with population density (Efford et al., 2016). Allowing for covariation may improve biological insight and lead to more parsimonious models.

Covariation may be ‘hard-wired’ into SECR models by reparameterization. In each case a new ‘surrogate’ parameter is proportional to a combination of the co-varying parameters. One of the co-varying parameters is seen as driving variation, while the other is inferred from the surrogate and the driver. Deviations from the expected covariation are implied when the surrogate is found to vary (i.e. a model with varying surrogate is superior to a model with constant surrogate).

For concreteness, consider a difference in home range size over the seasons, causing variation in σ . A reasonable null hypothesis is that there will be reciprocal seasonal variation in λ_0 such that $a_0 = 2\pi\lambda_0\sigma^2$ is constant. Here variation in σ is the driver, a_0 is the surrogate, and λ_0 is derived.

Table 5.1: Parameterizations implementing three covariation models. The third option combines the first two. In **secr**, a_0 is called ‘a0’ and k is called ‘sigmak’.

Parameters	Driver	Surrogate	Derived	Effect
(D, a_0, σ)	σ	a_0	$\lambda_0 = a_0/(2\pi\sigma^2)$	Reciprocal σ^2 , λ_0
(D, λ_0, k)	D	k	$\sigma = k/\sqrt{D}$	Density-dependent σ
(D, a_0, k)	D	k, a_0	$\sigma = k/\sqrt{D}$ $\lambda_0 = a_0/(2\pi\sigma^2)$	both

See Appendix H for further detail on alternative parameterizations and their implementation in **secr**.

5.11 Model-based location of AC

Assume we have fitted a spatial model by integrating over the unknown locations of AC for a given SECR dataset $\Omega = \omega_1, \omega_2, \dots, \omega_n$. We may retrospectively infer the probability density of the AC corresponding to each detection history, using the model and the estimated parameters:

$$f(\mathbf{x}|\omega_i; \hat{\phi}, \hat{\theta}) = \frac{\Pr(\omega_i|\mathbf{x}; \hat{\theta})D(\mathbf{x}; \hat{\phi})}{\int_{R^2} \Pr(\omega_i|\mathbf{x}; \hat{\theta})D(\mathbf{x}; \hat{\phi}) d\mathbf{x}}. \quad (5.6)$$

This is equivalent to the posterior distribution of each latent AC in Bayesian applications. See [fxi](#) and related functions in **secr**. For known θ and known ϕ (unless $D(\mathbf{x}; \phi)$ uniform) the modal location of the AC for animal i may be estimated by maximizing $\Pr(\omega_i|\mathbf{x}; \theta)D(\mathbf{x}; \phi)$ with respect to $\mathbf{x}$. The distribution often has more than one mode for animals at the edge of a detector array or searched area. It should not be confused with the home range utilisation pdf.

Section 11.6 discusses the use and interpretation of $f(\mathbf{x}|\omega_i; \hat{\phi}, \hat{\theta})$ (see also Durbach et al., 2024).

Part III

Performance

6 Assumptions and robustness

The three chapters in this part address questions we group under the heading of ‘Performance’:

- What is assumed in the analysis and how robust are the estimates? This chapter
- Is there empirical evidence that the methods work? Chapter [7](#)
- How should studies be designed? Chapter [8](#)

The SECR probability models of Chapter [3](#) that underlie the methods in this book incorporate certain assumptions. The validity and reliability of the methods therefore depend to an extent on how well the field data meet the assumptions. The assumptions include those of non-spatial capture–recapture (Otis et al., 1978). In addition, SECR rests on quite specific spatial models for the population and for detection (Chapter [3](#)).

We list the assumptions before assessing their impact in practice - the topic of [robustness](#).

Assumption 1.	The population is closed (no animals die, emigrate or recruit during sampling).
Assumption 2.	Individuals are identified without error.
Assumption 3.	The probability of detecting an animal at a particular detector declines radially from a fixed point, its activity centre (AC).
Assumption 4.	The probability of detection is constant across individuals, detectors and times, conditional on the AC.
Assumption 5.	AC locations are independent, as are detections of different individuals and sequential detections of a single individual.

We omit one further assumption: that the probability of detection declines to zero for AC at the edge of the habitat mask unless it is naturally bounded. This is addressed in the analysis by correct choice of [detection function](#) and buffer or mask (Chapter [12](#)).

Restrictive assumptions often may be relaxed under specific extensions to the SECR model, including the various ‘sub-models’ of Chapter [10](#).

6.1 Robustness

For assumptions that cannot be met by better design or customized modelling, we rely on the robustness of SECR estimators. A robust estimator gives estimates that are close to the truth even when assumptions have been breached. We care most about estimates

of population density, which may be robust even when estimates of other parameters are not.

Our main criterion will be the relative bias of an estimator, abbreviated RB and defined for an estimator $\hat{\theta}$ of any arbitrary parameter θ as

$$\text{RB}(\hat{\theta}) = \frac{\text{E}(\hat{\theta} - \theta)}{\theta}.$$

Robustness is estimated in practice by obtaining $\hat{\theta}$ for a large sample of simulated datasets. Simulation allows the statistician to control precisely any deviation from the assumptions. There have now been several simulation studies that we review below. We supplement these with new simulations that we outline below and describe in full in the GitHub repo [MurrayEfford/secr-simulations](https://github.com/MurrayEfford/secr-simulations).

Breaches of assumptions may also impair estimates of sampling variance, leading to poor coverage of confidence intervals even when an estimator is unbiased. ‘Poor coverage’ here means that the true value lies outside the computed confidence interval more often (or sometimes less often) than expected by chance given the nominal level (e.g. 95%). Coverage of simulated intervals is therefore a further criterion.

Simulations cannot span the full range of scenarios, so the results are only indicative. Ideally we would supplement simulations with field validations of the method, but as we see in Chapter 7 these are difficult to execute and interpret.

Another approach is to test whether a particular dataset is consistent with each assumption. This was popular in the past (Otis et al., 1978), but has lost ground along with the declining credibility of null hypothesis testing, the growth of modelling frameworks, and reliance on the inherent robustness of the spatial methods.

Our simulations use a base scenario of sampling with a square grid of 64 binary proximity detectors operated for 10 occasions; detector spacing is 2σ for a hazard half-normal detection function with $\lambda_0 = 0.1$. A population with density $0.5\sigma^{-2}$ is distributed uniformly at random in a region extending 4σ beyond the detectors. We report relative bias of density estimates and detection parameters under a null model – one with uniform density and no additional effects on detection – except where stated. The range $|\text{RB}| < 10\%$ is shaded on the graphs, and bars indicate 95% confidence limits, although these are often obscured by the plotted points. Off-scale values are flagged with an asterisk (*).

Assumption 1: Population closed

Births, deaths and dispersal result in population turnover. Over an extended period of turnover, more animals may be observed than were present at any instant, and density estimates will be biased upwards. Studies using automatic cameras often accumulate data slowly over many days, and study duration has triggered much angst (e.g., Harihar et al., 2017). How much does a little turnover matter? Conversely, How long can the sampling period be? Results for non-spatial models (Kendall, 1999) cannot be transferred. We

distinguish turnover due to movement of AC from *in situ* births and deaths, and consider movement of AC separately under [Assumption 3b](#).

P. Dupont et al. (2019) investigated the effect of increasing study duration on the precision and bias of population size estimates in the presence of population turnover. Their results are complicated by an artifact that caused their Bayesian estimator to be positively biased for short durations (P. Dupont et al., 2019, p. 669).

Additional simulations are shown in Fig. 6.1. Turnover in a constant population resulted in positive bias equal to about 70% of the mortality over the duration of the sampling. Thus 50% annual mortality ($\sim 16\%$ over 3 months) resulted in about $+11\%$ relative bias in a 3-month study, and coverage of nominal 95% confidence intervals dropped to about 82%. The new results are broadly consistent with those from the more complex scenarios of P. Dupont et al. (2019) for ‘slow’ and ‘intermediate’ life histories.

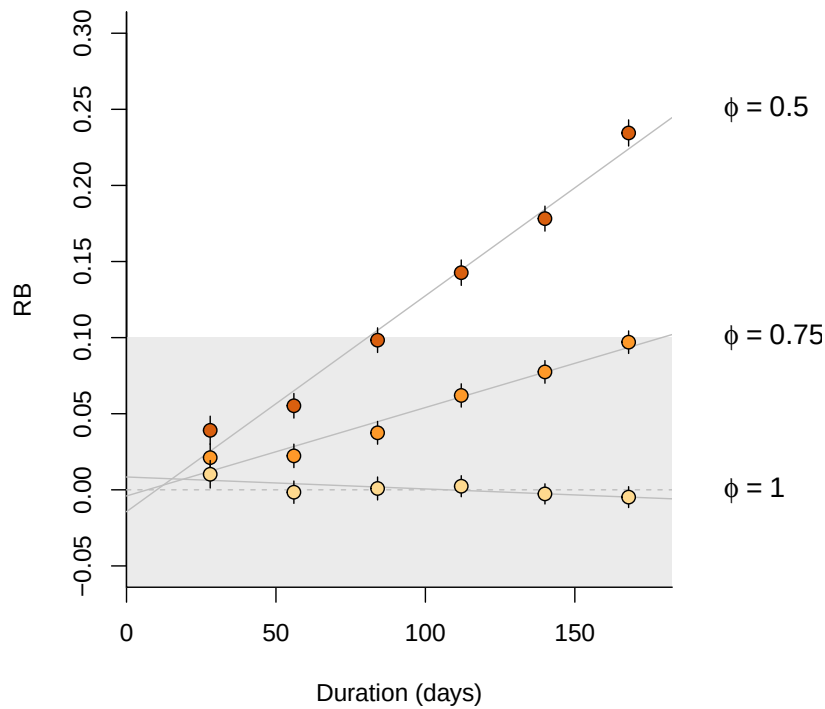


Figure 6.1: Simulated effect of study duration on relative bias of density estimates for three levels of annual survival ϕ in a population with constant density (recruitment = mortality). $\phi = 1$ corresponds to a closed population.

Assumption 1b: Population closed to immigration and emigration

This overlaps with Assumption 3 and is covered [later](#).

Assumption 2: Identification without error

We can generally assume accurate identification on recapture of animals trapped and marked by conventional methods such as numbered leg bands or ear tags so long as these are not [lost](#). However, identification may be unreliable with modern methods for passive sampling using natural marks (DNA from hair or faecal samples, and images from motion-sensitive cameras). This is a major limitation.

In genetic sampling there are two possible reasons for mis-identification: (i) there is too little variation at the chosen loci to distinguish all individuals in the sampled population, and (ii) genotyping is subject to error. Mills et al. (2000), Lukacs & Burnham (2005), Waits & Paetkau (2005), and Lampa et al. (2013) reviewed the early literature, and citations of those reviews are a good entry point to the voluminous recent literature. Sethi et al. (2014) provide technical advice. Augustine et al. (2020) provide a useful summary and a model framework that encompasses the various errors. Kodi et al. (2024) is a recent SECR study.

Natural marks such as coat patterns are prone to identification problems that parallel those from genotyping: individuals may not be distinguishable or some may be mis-recorded, leading to spurious ‘ghost’ individuals. The robustness of estimates will depend on the likely magnitude of each effect. We address the bias for varying frequencies of each effect below, with particular reference to genotyping.

A single camera may photograph only the left or right flank of a passing animal. Owing to the asymmetry of patterns, identity cannot be inferred conclusively from a single photograph. This identification problem can be addressed in the field by using paired cameras, so that both flanks are recorded (Karanth & Nichols, 1998). There may still be a minority of single-sided photographs, and probabilistic models have been suggested to incorporate these (Augustine et al., 2018).

Assumption 2a. Natural marks sufficiently diverse

The ability to distinguish individuals is measured by the probability of identity (PI). This is the probability that two individuals drawn at random from the population will appear the same, i.e. have the same genotype at the loci examined. Identity of genotypes was considered by Mills et al. (2000) to cause a “shadow effect”, as the existence of some individuals is concealed. The problem for SECR is even greater than in non-spatial capture–recapture. A group of two or more indistinguishable individuals becomes a ‘super individual’ whose detections spread over a larger area than each occupies individually.

Simulations in [Fig. 6.2](#) and [Fig. 6.3](#) illustrate the potential impact of shadow effects on SECR estimates. In a scenario with about 110 detected individuals, $PI = 10^{-3}$ resulted in relative bias of -20% in density estimates. This is largely due to the inflated spatial footprint of each super individual, which causes positive bias in estimates of the spatial scale of detection σ . Bias was negligible for $PI < 10^{-4}$ and that level was achieved in most of the field studies tabulated by Lampa et al. (2013).

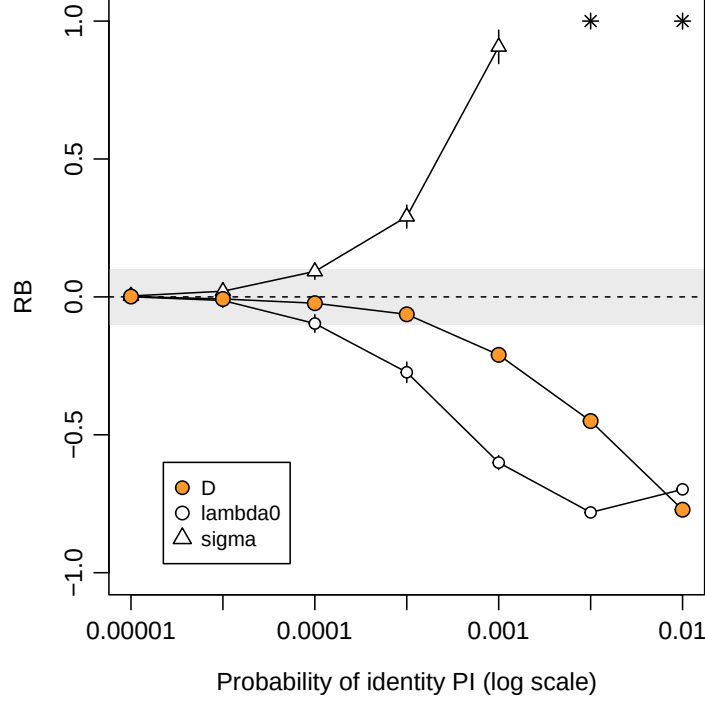


Figure 6.2: Relative bias of parameter estimates from null model when some individuals cannot be distinguished (‘shadow effects’).

Assumption 2b. Natural marks not corrupted

DNA samples degrade over time exposed to heat, moisture and UV light (e.g., Woodruff et al., 2014). This results in both a lower rate of successful DNA amplification, and increasing frequency of genotyping errors. A key genotyping error is the phenomenon of allelic dropout, when one allele at a heterozygous locus fails to amplify, resulting in an apparent homozygote. False alleles may also appear in the laboratory and give the appearance of a distinct genotype and individual. Either error is likely to result in a spurious so-called ‘ghost’ individual that is never recaptured. Mis-identification of photographs may also be common for some species. For example, Johansson et al. (2020) found 12.5% of photographs were mis-classified in a controlled study of snow leopards *Panthera uncia*, with most errors leading to ghost individuals.

We measure the ghost effect by the probability a detection results in a ghost individual. Unlike shadow effects, ghost individuals have almost no effect on $\hat{\sigma}$. However, they do cause negative bias in $\hat{\lambda}_0$ and a reciprocal (positive) bias in density estimates $\hat{D}$ (Fig. 6.5).

More extensive simulations were published by Kodi et al. (2024) for scenarios intended to mimic automatic camera surveys of snow leopards (*Panthera uncia*). They expressed the ghost effect in terms of the proportion of ghost individuals in the dataset. The highest level of ghosting in our simulations (0.2 per occasion) resulted in 28% of ghost individuals overall, which was close to the maximum considered by Kodi et al. (2024) (30%), and their low-encounter-rate scenarios gave simulated bias similar to that in Fig. 6.5.

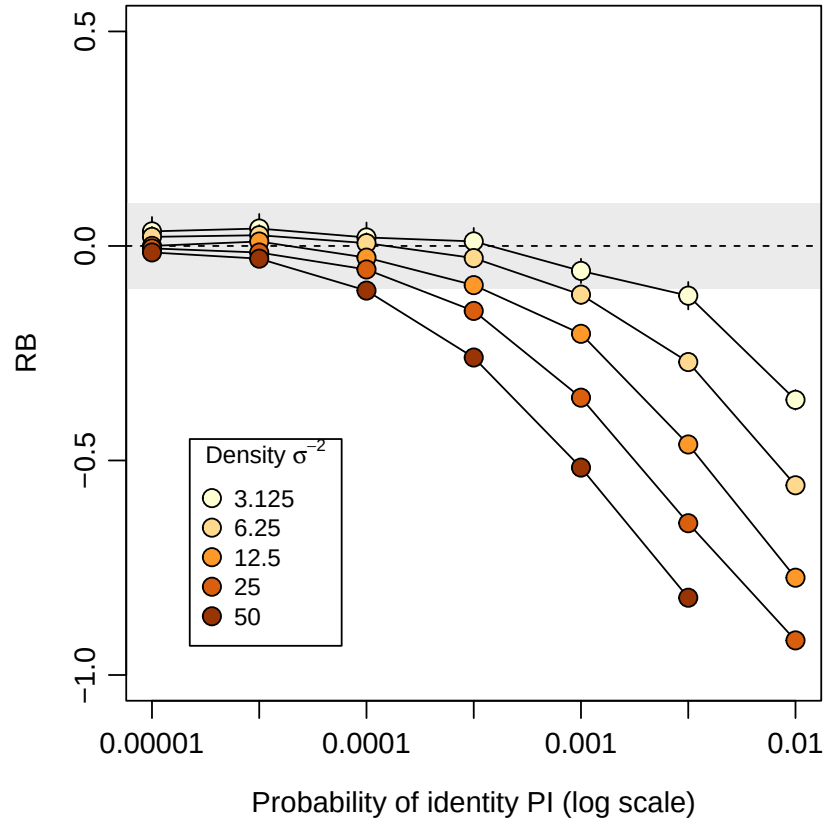


Figure 6.3: Relative bias of density estimates as function of PI for varying density. Density is expressed in units of animals per σ^2 . At low density few individuals are detected, reducing the chance of mis-identification for given PI.

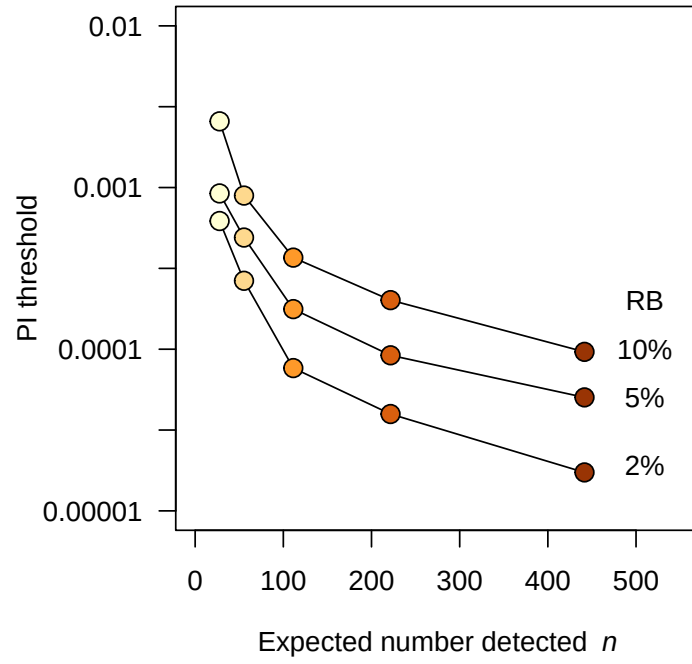


Figure 6.4: Threshold of PI at which absolute relative bias of density estimates exceeded the threshold shown, as a function of the expected number of individuals detected.

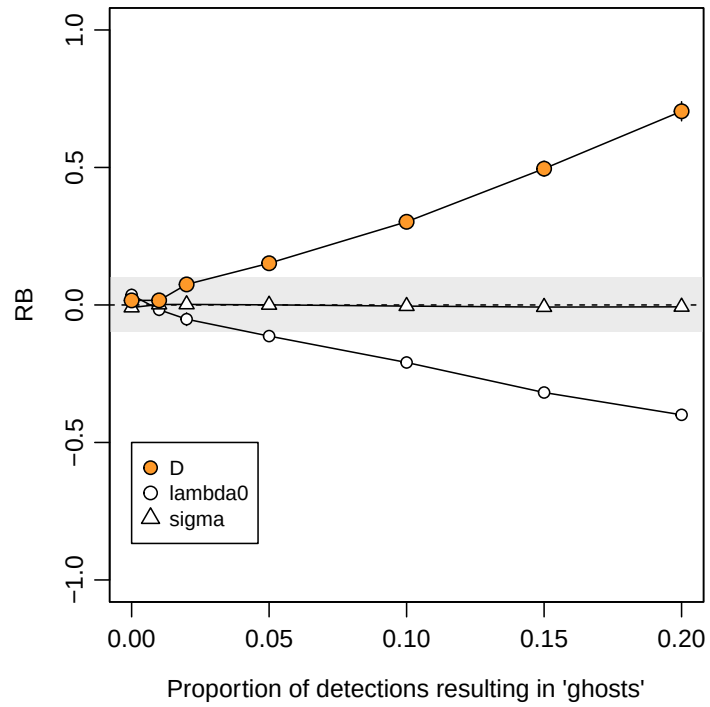


Figure 6.5: Relative bias of parameter estimates from null model when genotyping errors generate 'ghost' individuals.

Given sufficient diversity at the chosen loci, culling of inadequate samples and intensive checking of doubtful genotypes is generally sufficient to ensure adequate data (Paetkau, 2003). Lukacs & Burnham (2005) raised doubts about possible biases due to sample culling, but these have not been confirmed. Elimination of poor samples for which identity is uncertain tends to reduce sample size and precision. Precision can be improved by judiciously including samples with fewer loci in models that allow for uncertain identity (Augustine et al., 2020).

Unknown identity can be handled in SECR-like probability models. These generally lack power unless supplemented by detections of known individuals or telemetry (references in Appendix G). Bias due to ghost individuals may be reduced in some circumstances by modelling only detection histories with more than one detection by a straightforward modification to the likelihood (Kodi et al., 2024) or by inverse prediction (see vignette for `ipsecr` $\geq 1.4.3$).

Assumption 2c: Tags not lost

Tag loss is a well-documented source of bias in non-spatial capture–recapture (e.g., Malcolm-White et al., 2020), particularly in open population studies (Seber & Schofield, 2019), but it has not to our knowledge been addressed specifically for SECR.

Loss of a tag may leave a trace such as a ripped ear, generating a class of ‘previously tagged, identity unknown’ individuals. Probability models may be developed in the future to cover this case (cf Augustine et al., 2018).

We simulated the simpler case that tag loss leads to an apparently new ‘recycled’ individual. Unsurprisingly, this causes positive bias in $\hat{D}$ that increases with the rate of loss and the duration of the study (Github and Fig. 6.6).

Assumption 3: Detection declines radially from randomly located, fixed AC

SECR treats detection as a declining function of distance from the activity centre. This follows from a biological model in which detection hazard is proportional to each animal’s utilisation distribution (its home range conceived as a stationary 2-dimensional probability density function (van Winkle, 1975)), and the utilisation distribution is unimodal.

Assumption 3a: Home ranges circular

Concerns were expressed by Ivan et al. (2013) about the effect of non-circularity of home ranges. Simulations by Efford (2019) generally defused those concerns, with an important caveat: estimates of the spatial scale of detection σ and density are unreliable when elongated home ranges are sampled with a linear array of detectors. Simulations of randomly oriented elliptical ranges with an aspect ratio of 3:1 resulted in bias on the order of +13% for

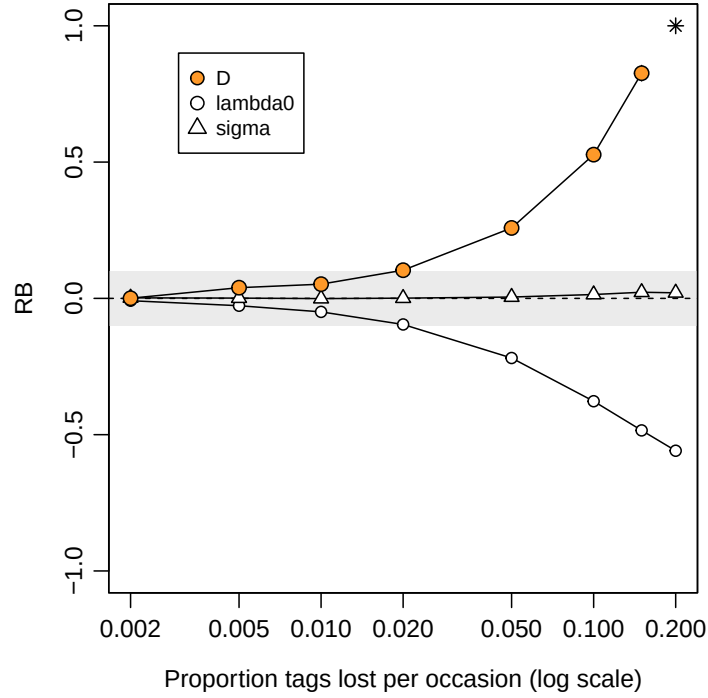


Figure 6.6: Relative bias of parameter estimates from null model when tags are lost at a constant per-occasion rate. 10 occasions, multi-catch traps.

a linear array (Efford, 2019: Fig. 2). Bias is extreme (often $>50\%$, Efford, 2019: Table 1) when home ranges are both elongated and have a common alignment to the array (Fig. 6.7 c). Modelling the anisotropy can be beneficial when the alignment is predictable from the landscape (Efford, 2019; Murphy et al., 2016; Murphy & Luja, 2025) and the array is not exactly linear, but the method is not universally applicable.

Assumption 3b. Home ranges stationary

Robustness of SECR to movement of home ranges (dispersal) was demonstrated by Royle et al. (2015). Harihar et al. (2017) expressed a further concern that movement over a long sampling duration would breach Assumption 1 (closure).

We simulated detections over 100 ‘days’ during which each AC underwent a random walk governed by a Gaussian kernel with scale σ_m . Details are given on [GitHub](#).

Assumption 4: Probability of detection constant

Otis et al. (1978) classified non-spatial capture–recapture models according to three possible sources of variation in detection probability: time (t – sampling occasion), behaviour (b – learned response to capture) and heterogeneity (h – persistent individual differences). Models may accommodate any one of these sources (t, b, h) or their combinations (tb, th,

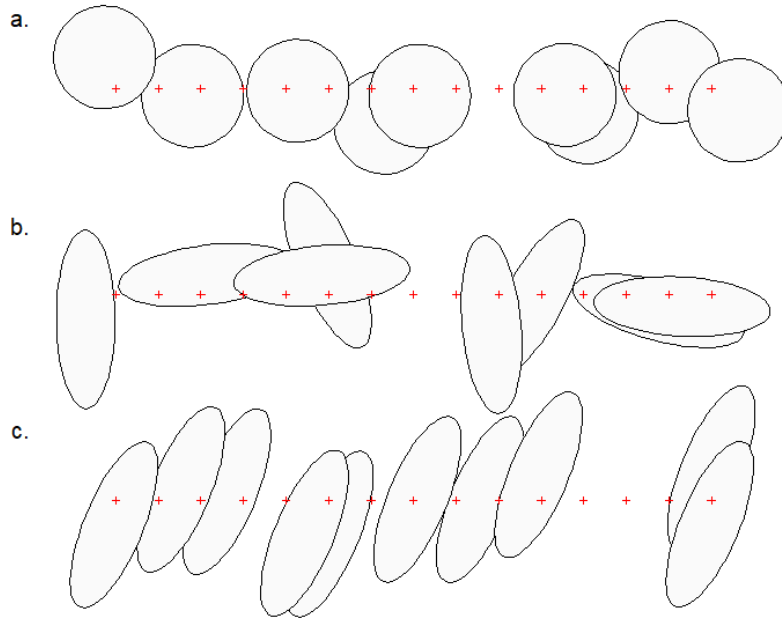


Figure 6.7: SECR models typically assume that home ranges are circular (a). Noncircularity of home ranges causes limited bias in density estimates when ranges are oriented at random with respect to a linear detector array (b). Systematic alignment of ranges to the array (c) causes large bias in $\hat{\sigma}$ and $\hat{D}$.

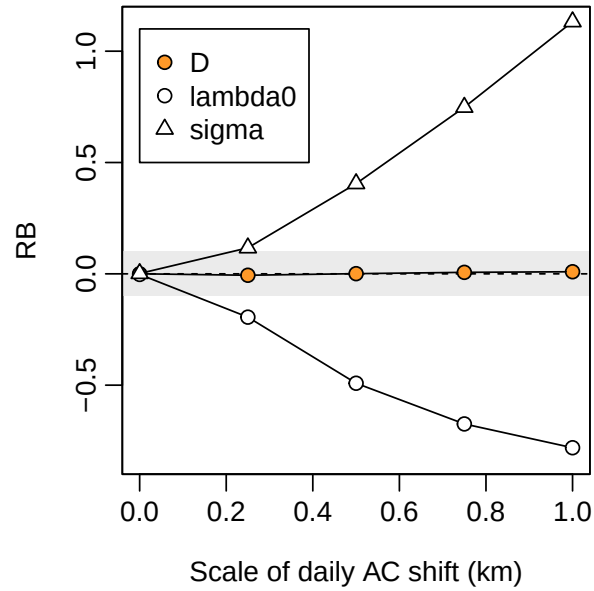


Figure 6.8: Relative bias of density estimates from studies with varying between-occasion movement. Both $\hat{\lambda}_0$ and $\hat{\sigma}$ are biased by movement, but the net effect on $\hat{D}$ is negligible.

bh, tbh). Space adds other sources of variation – most simply, detection probability may also vary between detectors (d) – and more complex potential interactions.

For SECR we have at least two parameters that control detection (g_0 or λ_0 , and σ).

Detection parameters may also be considered a function of AC location, but for (relative) simplicity we stick to t, b, h, and d.

4a. Temporal variation

Sollmann (2024) reached the conclusion from simulations that temporal variation in the baseline detection probability g_0 may safely be ignored when fitting SECR models. Density estimates from null and temporal models are usually the same or nearly so.

Further [simulations](#) confirm the lack of bias in $\hat{D}$, but show that temporal variation in σ causes significant bias in estimates of both λ_0 and σ under the null model (Fig. 6.9).

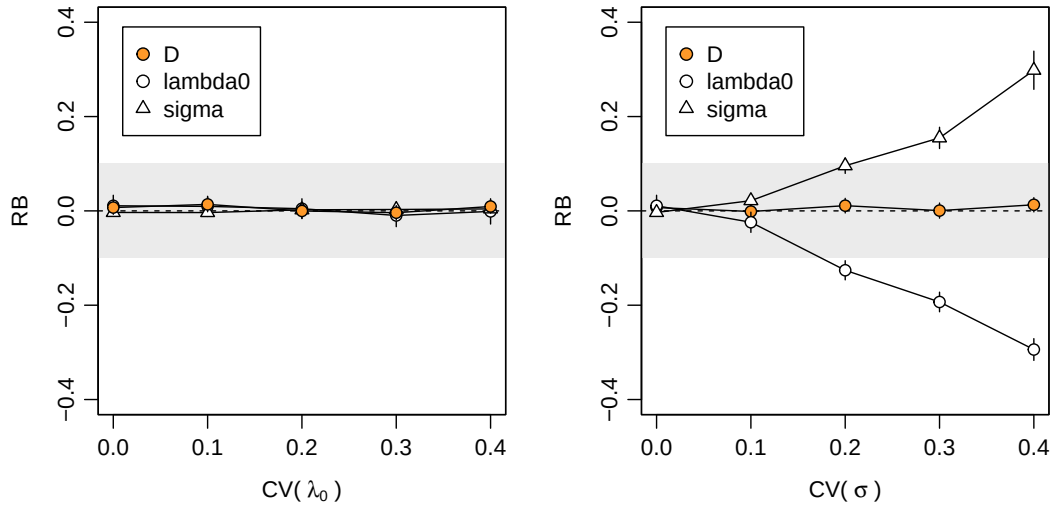


Figure 6.9: Relative bias of parameter estimates from null model given temporal variation in λ_0 or σ .

4b. Behavioural responses

We introduced behavioural responses in Chapter 10. Behavioural responses may also be treated as a breach of the [assumption of serial independence](#), but we keep them here for consistency with the non-spatial literature.

The discovery of a detector and experience of capture may result in either a positive (trap-happy) or negative (trap-shy) change in detection probability. The response may persist for the duration of sampling or be transient (Markovian) and apply only at the next sampling occasion. In SECR there is the further complication that the response may be general, applying across all detectors, or localised to the initial detector.

Each of these responses is readily modelled if the sequence of detections is known. This is generally not the case with proximity detectors, for which an animal may visit multiple detectors on one sampling occasion¹. We therefore restrict consideration of learned responses to trapping data (single-catch and multi-catch traps). This is not a great loss, as proximity detectors are typically non-invasive and less likely than traps to affect behaviour, except when they are baited (the Great Smoky Mountains National Park [black bear dataset](#) is a good example).

If a behavioural response is not modelled then it may cause a heavy bias in density estimates, positive for trap shyness and negative for trap happiness (Fig. 6.10). In [simulations](#) we describe the magnitude of the behavioural response by a ‘recapture factor’ by which the initial (naive) hazard is multiplied after first capture. We simulated recapture factors between 0.25 and 2.0, where 1.0 represents no behavioural response. The bias is much reduced if the effect is detector-specific, but then $\hat{\sigma}$ is also biased. For the scenario used in these [simulations](#), the relative bias of density estimates fell between -10% and +10% for recapture factors between 0.5 and 2. In Chapter 10 we estimated a much larger response by [deermice](#) (about 10-fold, but with poor precision), although there was only a small difference in estimated density between the null and bk models. Behavioural responses are potentially important for SECR and deserve closer investigation.

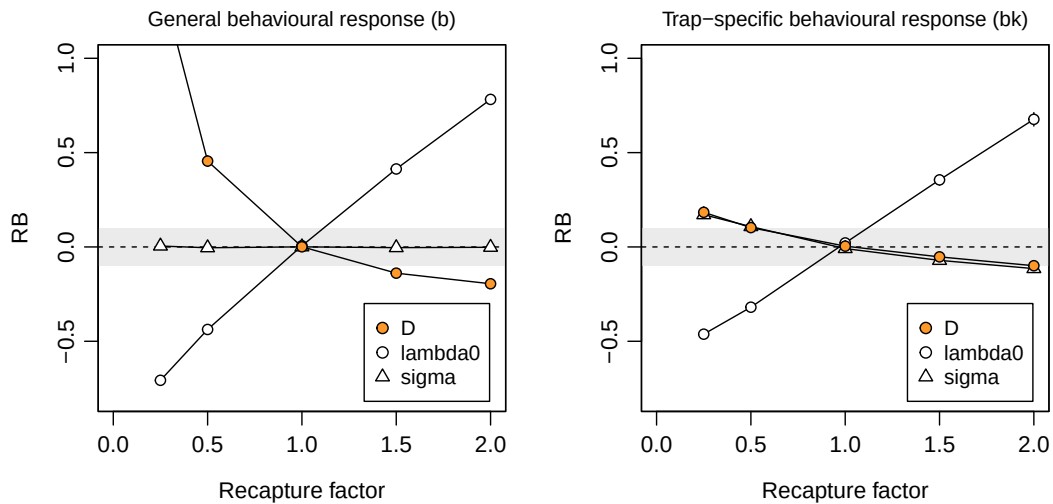


Figure 6.10: Relative bias of parameter estimates from null model given behavioural variation in λ_0 . Relative bias of density was off-scale (+1.41) for recapture factor 0.25 with a general behavioural response.

i Note

Animals may encounter a detector without being detected. For example, a camera flash may be triggered when an animal is out of frame, or a trap door may fall without a clean capture. If such events cause avoidance then the behavioural response leaves

¹A reviewer points out that the data are sufficient to model a detector-specific response at binary proximity detectors.

no trace in the histories of detected animals and cannot be fully modelled. We would nevertheless expect some evidence for the aversive effect within detected histories.

4c. Individual heterogeneity

Individual heterogeneity has long been the bane of capture–recapture. Some of the variation is due to proximity to detectors, and does not bias SECR estimates because it is included in the model. A modest level of additional variation has little effect on null-model estimates: Efford & Mowat (2014) found that the relative bias of $\hat{D}$ did not exceed -0.05 when $CV(a_0) < 0.3$ for $a_0 = 2\pi\lambda_0\sigma^2$ (based on hazard halfnormal or exponential detection function). We confirmed this in [simulations](#) that varied each of the components λ_0 and σ^2 separately (Fig. 6.11). Biologically we expect an inverse relationship between λ_0 and σ^2 , so in general $CV(a_0)$ will be less than either $CV(\lambda_0)$ or $CV(\sigma^2)$ on its own (Efford & Mowat, 2014).

Despite these appeals to robustness, Fig. 6.11 indicates a risk of significant bias in estimates of both density and detection parameters when individual heterogeneity is large.

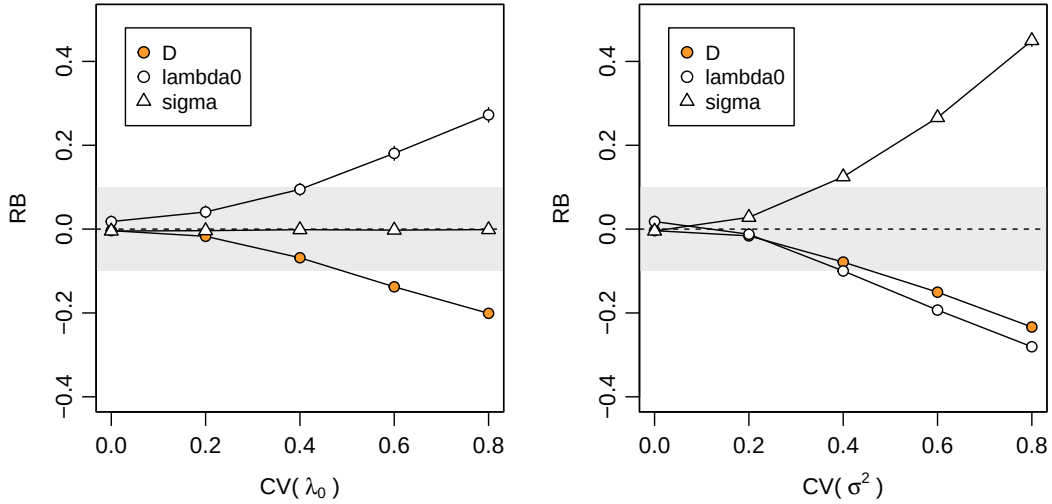


Figure 6.11: Relative bias of parameter estimates from null model given individual variation in λ_0 and σ^2 .

4d. Homogeneity across detectors

Spatial (detector-specific) variation in detection parameters is conceptually linked to individual heterogeneity. The location of each individual determines its detection probability via a particular set of AC-to-detector distances, as modelled automatically in SECR. Variation among detectors in efficiency (λ_{0_k}) adds between-animal heterogeneity because the detectors near a particular AC differ in efficiency from the average (Fig. 6.12), and the variation is not modelled automatically.

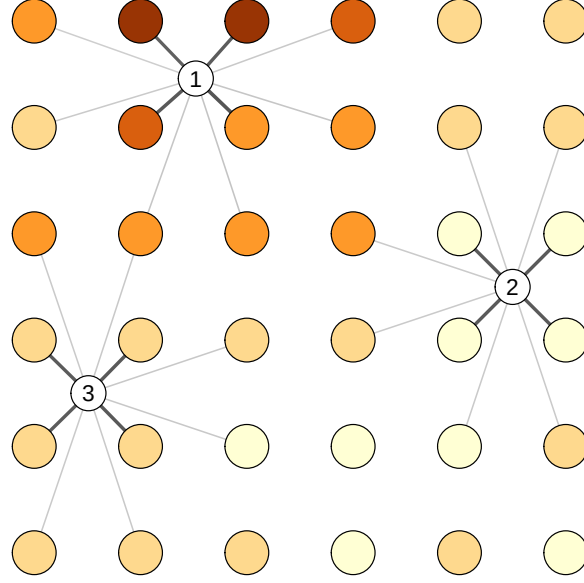


Figure 6.12: Variation in detector efficiency induces individual heterogeneity in overall detection probability $p(\vec{x})$. AC 1–3 have similar exposure to detectors but experience respectively high, low and medium detection probability because of the neighbourhoods in which they are located.

The individual heterogeneity induced by between-detector variation is increased by spatial autocorrelation on the scale of home ranges, because this makes it more likely that all detectors in the neighborhood of an AC will be above average or below average. Variable λ_{0_k} may also affect null-model estimates indirectly owing to bias in $\hat{\sigma}$ and perhaps $\hat{\lambda}_0$, but these interactions are poorly understood and we do not consider them further.

A Gaussian random field (GRF) is a convenient source of random spatial variation. Spatial structure of a GRF is controlled by its covariance function, which is typically exponential with a single scale parameter. Several authors have used a GRF to generate detector-specific variation in detection probability on the link scale (logit or cloglog). Reported bias in null-model density estimates from these simulations was typically in the range 0 to 20% Dey et al. (2023).

Detector-level variation has several possible origins –

1. Habitat-determined variation in detector efficiency,
2. Selective use of habitats (third-order selection of D. H. Johnson (1980)), or
3. Unrecorded variation in sampling intensity (Dey et al., 2023).

Selective use implies a modification to each animal’s utilisation distribution. Time spent in preferred habitats allows less time in other habitats, and the time spent near any detector depends on the mix of habitats in the neighbourhood. Thus (1) and (2) imply different models of detector-level variation that we characterise as ‘unnormalised’ and ‘normalised’ (Efford, 2014).

We illustrate the effect of (1) and (2) with the scenarios simulated by Royle, Chandler, Sun, et al. (2013) and Efford (2014). The spatial scale parameter of the exponential covariance function was fixed at 2.5σ , the spacing of a 7 x 7 grid of binary proximity detectors.

The original model specified the hazard of detection as

$$\lambda_k(\mathbf{x}) = \exp(\alpha_0 - \alpha_1 d^2 + \alpha_2 X_k), \quad (6.1)$$

where d is the distance between an AC at $\mathbf{x}$ and detector k and X_k is a spatially autocorrelated variable with marginal distribution $N(0, 1)$, evaluated at detector k . The first two parameters are transformed versions of the parameters of the **secr** hazard-halfnormal detection function ($\alpha_0 = \log(\lambda_0)$, $\alpha_1 = 1/(2\sigma^2)$; Chapter 10). The magnitude of detector-level variation was controlled by parameter α_2 .

Bias in density estimates was small ($\text{RB}(\hat{D}) < 5\%$) for $\alpha_2 < 0.5$, but could become worrying for large α_2 (Fig. 6.13 a). But what value of α_2 is realistic? Royle, Chandler, Sun, et al. (2013) estimated $\alpha_2 = 0.2$ for black bear data from DNA hair snares, or $\alpha_2 = -0.3$ for a combined hair-snare and telemetry model.

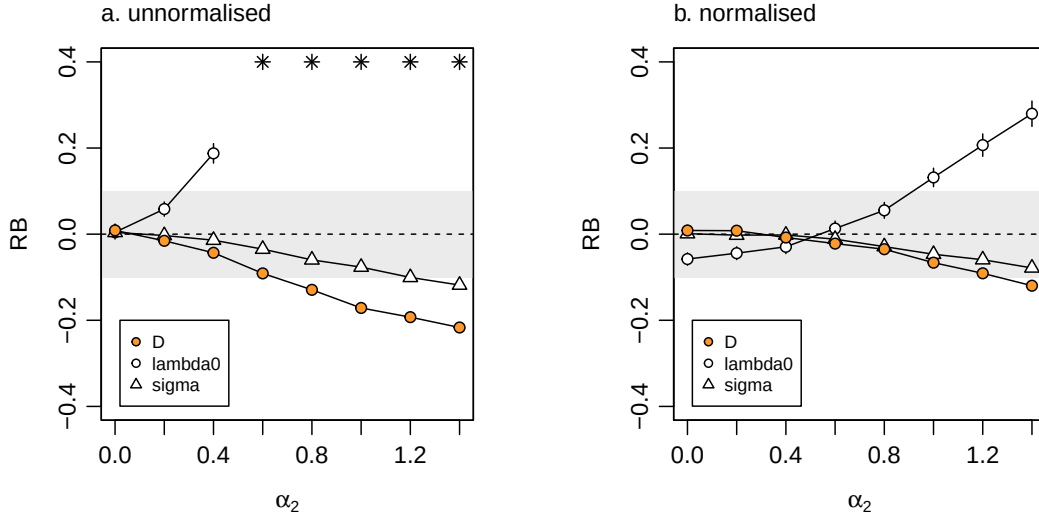


Figure 6.13: Relative bias of parameter estimates from null model when λ_0 varies among detectors according to (a) unnormalised model of Royle, Chandler, Sun, et al. (2013), and (b) normalised model of Efford (2014). The magnitude of variation depends on the coefficient α_2 (Eq. 6.1).

Normalisation, as implied by third-order habitat selection, attenuates the effect on the bias of density estimates (Fig. 6.13 b). We note also that detector-level covariate effects measured at each detector are readily included in the model; estimates are then nearly unbiased (e.g., Efford, 2014).

Large bias can arise when the sample is assumed to come from a detector array larger than that actually sampled (Dey et al., 2023; Moqanaki et al., 2021). This would seem to come under the heading ‘Data requirements’ (Chapter 8.2) rather than model robustness.

Further complexity is possible. Stevenson et al. (2021) suggested modelling a latent Gaussian random field for each individual and showed how the parameters of the GRF could be estimated, assuming these to be shared across individuals. It is unclear how the latent GRF should be interpreted biologically. Activity is not normalised, so the combination of detection function and latent GRF does not describe a utilisation distribution.

Assumption 5: Independence

‘Non-independence’ covers a diverse set of possible deviations from the basic SECR model. Broadly, these may be due either to interactions among individuals (5a, 5b below) or to serial correlation in the detection histories of individuals considered separately (5c, 5d below).

Assumption 5a: AC locations independent

The SECR model of Borchers & Efford (2008) treats activity centres as distributed independently according to an inhomogeneous Poisson point process. This allows local density to vary according to habitat or other persistent effects. It does not allow for the spacing behaviour of the animals themselves, which can lead to either contagion or repulsion of AC. Several authors have ventured into this area (Bischof et al., 2020; López-Bao et al., 2018; McLaughlin & Bar, 2020; Reich & Gardner, 2014; Russell et al., 2012; **Efford2024?**). The general conclusion is that point estimates of density from SECR are robust to clustering of AC due to social behaviour, but the implied overdispersion leads to confidence limits that are too narrow.

5b. Individuals are detected independently of each other

Animals that move in groups are unlikely to be detected independently. Bischof et al. (2020) labelled the phenomenon “cohesion”. The primary result is overdispersion and underestimation of the sampling variance. Confidence intervals will be too narrow, resulting in less-than-nominal coverage. Earlier optimism (Bischof et al., 2020) that a measure of overdispersion would correct for the underestimation appears to be unwarranted (**Efford2024?**).

McLaughlin & Bar (2020) modelled random associations between adjacent neighbours; the method has yet to find general application.

Animals are not detected independently in single-catch traps because capture of one closes a trap to others. Maximizing the multi-catch likelihood provides unbiased estimates of density and spatial scale in most cases (see Distiller & Borchers (2015) for an exception), but estimates of the intercept of the detection function (g_0 , λ_0) may be strongly biased. The coverage of confidence intervals based on the multi-catch likelihood has yet to be assessed. An algorithm combining simulation and inverse prediction allows unbiased estimation of all parameters (Efford, 2004, 2023), but lacks the flexibility of MLE.

5c. Detections of an individual are independent

Trapping by definition causes non-independence of detection events: an animal that is captured cannot be caught anywhere else until released. The non-independence of detections in multi-catch traps is readily modelled as competing risk, as we saw in Chapter 3.

Behavioural response to detection is a further form of non-independence that we have covered [already](#).

A more problematic form of dependence arises when a single visit to a passive detector results in multiple observations. This can happen when DNA is amplified from multiple hair samples left at a hair snag, or when a camera takes multiple photographs of an animal on a single visit. Analysing such data as if they were independent events results in overdispersion and accompanying underestimation of sampling variance. A simple solution is to collapse each individual $\times$ occasion $\times$ detector count to a binary observation. An alternative is to fit an overdispersed (negative binomial) model for the counts; this method has yet to be critically evaluated.

Assumption 5d: Locations independent within stationary home range

In the basic SECR model of Chapter 3 the next detection of an individual may happen anywhere in its home range, regardless of where it was last detected. More biologically realistic patterns of movement lead to sequential dependence of locations. It is entirely likely that animals use their home ranges in a nested fashion so that activity during any sub-interval is localised in part of the range. Animals also may shift their AC during sampling, with similar consequences for data and models.

Royle et al. (2015) modelled movement patterns they called “transience” (a Gaussian random walk) and “dispersal” (discrete shift of AC). They concluded

... while estimators of density are extremely robust, even to pathological levels of movement (e.g., complete transience), the estimator of the spatial scale parameter of the encounter probability model is confounded with the dispersal/transience scale parameter.

We extend their simulations by considering a model in which the overall home range is stationary but locations within the home range are autocorrelated. The bivariate Ornstein-Uhlenbeck (OU) distribution is a convenient model with these properties (Dunn & Gipson, 1977; Hooten et al., 2017; Johnson et al., 2008). The home range as a whole is bivariate normal; we simulate the uncorrelated, circular case with equal variance on both axes. R code and details of the simulations are provided [on GitHub](#).

Fig. 6.14 illustrates OU movement tracks for individuals with increasing autocorrelation parameter τ . Detection is assumed here to happen when an individual is within some small threshold distance ϵ of a detector at the end of a time step. Other detection models are possible.

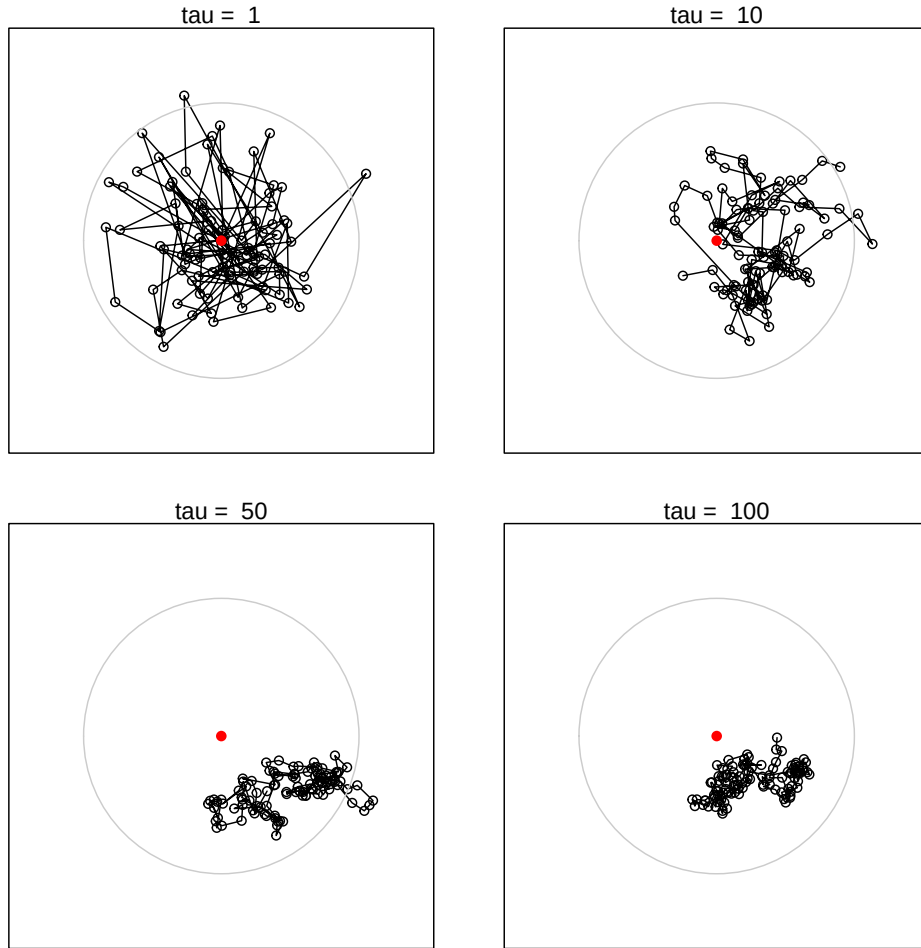


Figure 6.14: Examples of correlated movement paths over 100 time steps ($t = 100$). Parameter ‘tau’ (τ) controls autocorrelation. Red dot indicates activity centre; grey circle is 95% probability contour of distribution as $t \rightarrow \infty$ or $\tau \rightarrow 0$.

One consequence of autocorrelated movement is that the overall spatial scale increases with the duration and decreases with autocorrelation. With respect to telemetry, Otis & White (1999)² concluded that autocorrelation *per se* did not bias inference so long as summary statistics were not generalized beyond the temporal sampling frame:

Sampling designs that predefine a time frame of interest, and that generate representative samples of an animal's movement during this time frame, should not be affected by length of the sampling interval and autocorrelation.

The analogy with SECR is close: we are concerned with unbiased estimation of the detection parameters λ_0, σ that describe the detection process *over the time frame of sampling*. Extrapolation to other time frames would require knowledge of the correlation structure of the data, expressed in a model such as the bivariate Ornstein-Uhlenbeck distribution (e.g., Dunn & Gipson, 1977; Hooten et al., 2017), but that is not relevant to inference for the chosen time frame.

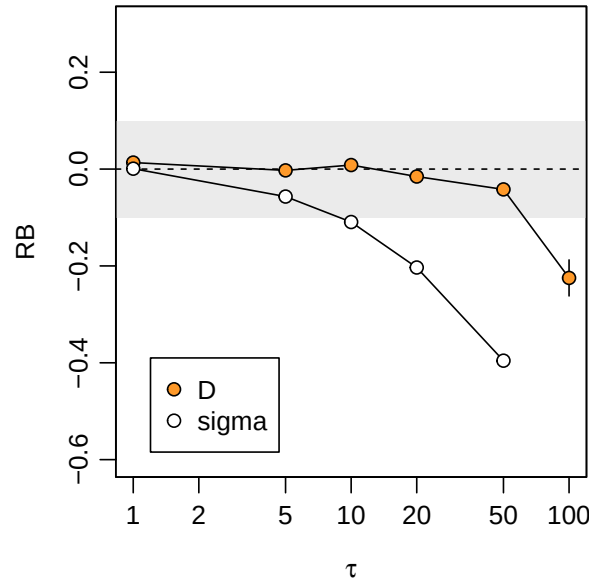


Figure 6.15: Relative bias RB of null-model density estimates of density D and spatial scale σ from Ornstein-Uhlenbeck model with autocorrelation parameter τ .

Fig. 6.15 confirms the robustness of SECR null-model density estimates to serial correlation of location over a broad range of values ($0 \leq \tau \leq 50$). Bias is apparent in estimates of the global σ , as expected from the reduced extent of movements in a given time frame. Note here that autocorrelation induces negative bias in $\hat{\sigma}$. There is no direct analogue of λ_0 in the OU generating model, so we cannot determine the bias in $\hat{\lambda}_0$.

²This interpretation was endorsed by Fieberg (2007), but disputed by Noonan et al. (2019).

6.2 Summary

Most breaches of assumption can be managed to have only minor effects on density estimates, as we summarise in Table 6.2. Simulations only tell part of the story: to evaluate robustness we also need to know the real-life magnitude of any breach. The magnitude of some breaches may be assessed from external evidence (e.g. telemetry), or from a more general model that incorporates the effect (e.g., finite mixtures for individual heterogeneity; spatial random effect models Dey et al. (2023)). More work is needed on particular sampling systems and species.

Special care is needed when these effects may apply:

- mis-identification leads to the appearance of super individuals (the shadow effect)
- a learned (behavioural) response is global rather than specific to detector locations
- large individual heterogeneity
- alignment of home ranges with detector array

We have not considered in detail the potential for correlated variation in overall detection (measured as the [effective sampling area](#)) and density. Failure to model such correlated variation led to poor estimates in the simulations of McLellan et al. (2023) (see also Section 8.14).

Several breaches lead to underestimation of sampling variance and impaired coverage of confidence intervals. If the magnitude of the breach can be estimated then better confidence intervals may be achieved by applying a variance inflation factor ($\hat{c}$) as in MARK (Cooch & White, 2023). Computing $\hat{c}$ is not straightforward and simulation may help. Further research is needed to quantify overdispersion, and consequently $\hat{c}$, under the various non-independence scenarios. (Efford2024?) suggested a route for density estimates when the AC have known clustering. Efford & Hunter (2018) applied simulation to the special case of mark-resight counts (Appendix E).

Particular detection parameters may be estimated poorly even when density estimates are robust. A common example arises when a multi-catch model is used for single-catch data: the estimated intercept of the detection function is strongly biased while density estimates are generally unbiased.

Table 6.2: Robustness of SECR models to breaches of assumptions. Split colours indicate that the effect is generally weak or ignorable, but may be damaging in some circumstances if not modelled.

Breach of assumption	Effect	Mitigating models
1. Closure		
1a. Births and deaths	 RB($\hat{D}$) proportional turnover	open population
1b. Dispersal		open population
2. Identification		

Breach of assumption		Effect	Mitigating models		
2a. Inadequate marks		-ve $RB(\hat{D})$ when high probability of identity	Augustine et al. (2020)		
2b. Marks mis-read		+ve $RB(\hat{D})$ increases with proportion ghosts	Kodi et al. (2024)		
2c. Tags lost		+ve $RB(\hat{D})$ when high rate or long study			
3. Home range HR					
3a. HR non-circular		severe $RB(\hat{D})$ when HR align with detectors	anisotropic model		
3b. HR non-stationary					
4. Constant parameters					
4a. Temporal variation			covariate, time-specific		
4b. Behavioural responses			learned response		
4c. Individual heterogeneity		-ve $RB(\hat{D})$ nonlinear on $CV(a)$	covariate, finite mixture		
4d. Detector heterogeneity			covariate, GRF		
5. Independence					
5a. AC not independent					
5b. Individuals interact			Efford (2023)		
5c. Serial dependence					
5d. Location autocorrelated					
Legend		ignorable	 minor	 major	 severe

7 Empirical validation

There have been few attempts to validate empirically the estimates of density from SECR: critical evaluations have relied overwhelmingly on simulation. Empirical validation requires a reliable estimate of the true density, usually from counting the individuals with activity centres in a known area. Enumeration may be by intensive observation of conspicuous diurnal species or by sampling until no new individual is found. Delineation of the relevant area is straightforward when the habitat is a natural island or fenced area. We exclude studies such as Twining et al. (2022) that evaluated SECR by comparing density estimates among methods, as these are vulnerable to shared biases.

We describe four attempts at empirical validation of density estimates, each with a different approach. The evidence from empirical validation studies tends to be weak, owing to the wide confidence intervals and methodological constraints. None is entirely convincing, as we shall see.

A fifth study suggests that detection parameters may vary between detector layouts. This raises a caveat for simulation (Section 8.4).

7.1 Brushtail possums in New Zealand

Efford et al. (2005) live-trapped brushtail possums (*Trichosurus vulpecula*) on a coastal peninsula and compared the results to an attempted total removal by leg-hold trapping and acute poisoning. SECR estimates of density from five hollow grids (36 traps around the perimeter of a square) were consistent with the removal estimate.

Although broadly reassuring, the brushtail possum study had weaknesses. The live-trapping data were compromised by tag loss (documented in **secr**), the landward boundary of the removal area was somewhat arbitrary, and sampling may have been inadequate to represent density variation across the peninsula (see also [Rodents in New Mexico](#)).

7.2 Red squirrels in the Yukon

Van Katwyk (2014) analysed data from a multi-year behavioural study of red squirrels (*Tamiasciurus hudsonicus*) in the Yukon. Squirrels were marked and followed across six study areas, each about 36-ha in area. Intensive trapping at defended food middens maintained nearly 100% marking coverage. Capture-recapture data came from 50 cage traps operated periodically in the centre of each study area.

Calculation of ‘true’ density focussed on the area within the perimeter of the trapping grid. Behavioural observations of all squirrels whose territories potentially overlapped the grid (i.e. within one territory width) were scored for time spent inside the grid. The sum of these proportions is the number of ‘animal equivalents’ (Boutin, 1984) that gives the population density when divided by the grid area.

SECR density estimates averaged only 5% less than the ‘true’ density calculated by this method, and the estimates showed a strong correlation between sessions with the varying ‘true’ density. It is hard to fault this validation. The ‘true’ density was limited to the interior of each grid, whereas each SECR estimate includes data from a somewhat wider area and sample sizes were modest.

7.3 Rodents in New Mexico

One major evaluation has questioned the reliability of SECR. Gerber & Parmenter (2015) re-analysed live-trapping data for rodents of several taxa confined to pens in New Mexico (Parmenter et al., 2003). Traps were arranged in either a central square grid or a star-shaped pattern, a ‘trapping web’. The main study was followed by intensive trapping across the full extent of each pen to enumerate each taxon.

SECR provided instances of both over- and under-estimation, and its performance was “sometimes underwhelming”. The authors focussed on heterogeneity of detection and asymmetry of home ranges as possible explanations, although direct evidence of these effects was lacking. Asymmetry is not a significant source of bias in itself (Efford (2019)). Heterogeneity must be large to cause significant bias, and the bias is negative (e.g., Efford & Mowat (2014)) see also 6.

We suggest an alternative explanation. The main trap layouts did not sample the full extent of each pen, and the probability of detecting an individual in the periphery was quite low for species with small home ranges such as *Perognathus flavus* (Fig. 7.1). Perfect correspondence with pen-wide exhaustive trapping is not to be expected if the density in the under-sampled peripheral zone differs from the central zone. This applies whether the difference is random or systematic. The peripheral zone ($p(\mathbf{x}) < 0.25$) was a larger fraction of the pen for the grid ($\approx 54\%$) than for the web ($\approx 28\%$), which could explain why the grid estimates of *P. flavus* density were particularly poor (Gerber & Parmenter, 2015, Figures 2a, 3a).

The final exhaustive trapping might be used to test the hypothesis of spatial heterogeneity, but these data are not available. Some evidence of within-pen density variation may be gleaned from density models fitted to the capture–recapture data: in 50% of the populations a density model that included distance-to-wall as a covariate provided better fit than a null model in 50% of cases (likelihood ratio test $P < 0.05$; unpubl. results).

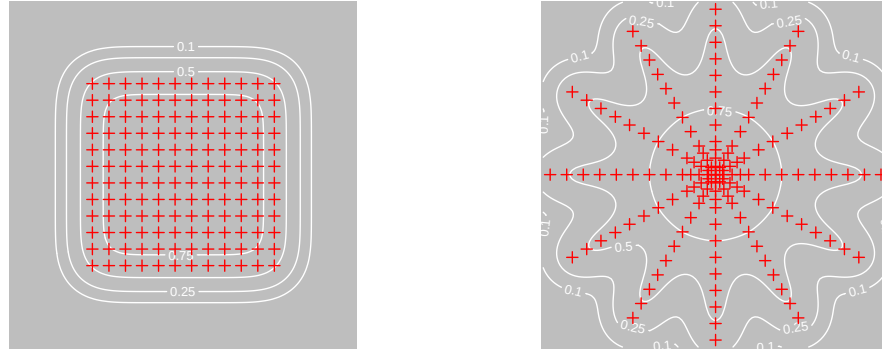


Figure 7.1: *Peroganthus flavus* contours of overall 5-night detection probability $p(\mathbf{x})$ (0.1, 0.25, 0.5, 0.75) within a ca. 4.2-ha pen (grey shading). a. Grid, b. Trapping web. Detection parameters from first *P. flavus* population in each case.

7.4 Chimpanzees in Ivory Coast

Després-Einspenner et al. (2017) applied SECR to automatic camera records of chimpanzees *Pan troglodytes* in a single group territory. Group size (27 excluding unweaned young) was known from intensive observations. The authors reported a good match between the estimated and known populations. However, σ could not be estimated because individuals moved throughout the territory and cameras were not placed outside it (see also comments on array size under [Study design](#)). This required them to treat the extent of the population as known (confined within the independently determined territory boundary). The resulting analysis is in essence non-spatial, and does not validate SECR.

7.5 Red-backed salamanders in Massachusetts

Fleming et al. (2021) examined the effect of varying the configuration of cover boards used to survey red-backed salamanders (*Plethodon cinereus*). Cover boards are artificial refuges placed on the ground. Data on salamanders found under cover boards are analysed as if cover boards were multi-catch traps. The authors' concern was that SECR estimates of salamander density might depend on the layout and interfere with comparisons and aggregation of data across studies. Estimates of detection parameters (half-normal intercept g_0 and spatial scale σ) were sensitive to the extent and spacing of cover board arrays, whereas density estimates were mostly robust. Salamanders appeared more mobile (larger $\hat{\sigma}$) when cover boards were further apart. Fleming et al. (2021) suggested salamanders might move directly to the next cover board, presumably within some distance limit.

7.6 Summary

None of these field studies fully validates SECR methods, but there is at least no strong counter evidence. We encourage field researchers to test SECR further. Where discrepancies are found it is important to investigate the cause. The assumption of independence among detectors underpins the compounding of single-detector distance-detection functions to predict array-level detection probability. Evidence supporting this assumption might relieve the need for elaborate models such as Stevenson et al. (2021).

8 Study design

Study design for SECR brings together many disparate considerations, some external to SECR itself (goals, cost, logistics). In this chapter we hope to build an understanding of the working properties of SECR and integrate the external considerations into a strategy for effective study design.

Defining the **area of interest** is a necessary first step towards rational study design. By ‘area of interest’ we mean the geographic region for which an estimate of population density or population size is required. When the goal is to characterise the density of a species in a particular habitat it is tempting to think that “any patch will do”, but defining the area, and hence the scope of inference, is an important discipline. Even when the primary goal is to infer the relationship between density and habitat covariates, rather than to estimate average density or population size, defining a region ties the model to particular patterns of variation in the covariates. Goals, cost, logistics, and tradeoffs among these factors, will determine the area of interest. These are study-specific, so we cannot say much more here.

The key design variables are the choice of methods for detection and individual identification, the number and placement of detectors, and the duration of sampling. The available detection and identification methods vary from taxon to taxon, and we do not consider them in detail. Cost is an ever-present constraint on the remaining variables, but we leave it to the end.

Tools for study design are provided in the R package **secrdesign** and its online interface [secrdesignapp](#). Simulation of candidate designs is a central method, but for clarity we defer instruction on that until Chapter 17.

8.1 Studies focussing on design

General issues of study design for SECR were considered by Sollmann et al. (2012), Tobler & Powell (2013), Sun et al. (2014), Clark (2019), and Efford & Boulanger (2019). Algorithmic optimisation of detector locations was described by G. Dupont et al. (2021) and Durbach et al. (2021). Design issues appeared incidentally in Efford et al. (2005), Efford & Fewster (2013), Noss et al. (2012), and Palmero et al. (2023), among other papers.

8.2 Data requirements for SECR

We list here some minimum data requirements that might be labelled “assumptions” but are more fundamental. They should be addressed by appropriate study design.

Requirement 1. Sampling representative of the area of interest

This requirement is important and easy to overlook. The problem takes care of itself when detectors are placed evenly throughout the area of interest, exposing all individuals to a similar probability of detection. This is usually too costly when the area of interest is large. Sub-sampling is then called for, and we consider the options later.

Deliberate, statistically non-representative, placement of detectors may be justified in certain studies, specifically when the goal is to infer the relationship between density and habitat variables. It is then optimal to sample along the environmental gradients of interest. We do not address study design for this case.

i More on modelling as an alternative to representative sampling

Our emphasis on representative and spatially balanced sampling may seem over-the-top. SECR is model-based and the variance estimates make no assumptions regarding the sampling design. Sampling deliberately across environmental gradients may well provide the best information on particular covariate effects, and a fitted model with these covariates may be extrapolated across the area of interest to estimate average density. Nevertheless, we advocate representative sampling because it protects against vulnerabilities of a purely model-based approach:

- density may be non-uniform for reasons not related to any observable covariate (e.g., historical disease outbreaks)
- spatial variation in detection parameters (λ_0, σ) is often omitted from models
- covariates are not causal, and different correlations may apply in unsampled parts of the region
- extrapolation to values of the covariate(s) that are rare or absent in the data is risky, especially with the log-linear models commonly used.

Requirement 2. Many individuals detected more than once

This requirement is common to all capture–recapture methods. Estimation of detection parameters conditions on the first detection, and detections after the first are needed to estimate a non-zero detection rate. The question ‘How many is enough?’ is addressed later.

Requirement 3. Spread adequate to estimate spatial scale of detection

By ‘spread’ we mean the spatial extent of the *detections of an individual*. The spread of detections must be adequate in two respects:

Requirement 3a. Some individuals are detected at more than one detector, and

Requirement 3b. Detections of an individual should be localised to part of the detector array.

Adequate spread is achieved by matching the size and spacing of a detector array to the scale of movement, as shown schematically in Fig. 8.1.

Array too small

Adequate

Spacing too large

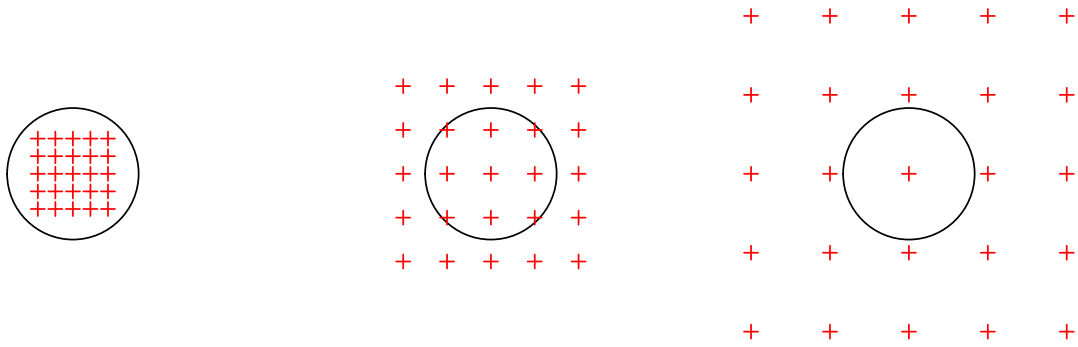


Figure 8.1: Three scenarios for array size in relation to home range size. For intuition we use a circle to indicate a hard-edged uniform home range. When the array is too small, recaptures are equally likely anywhere in the array. When the spacing is too large, recaptures must be at the same detector. In either case, the recaptures carry almost no information on the spatial scale of detection σ .

Some qualification is needed here. We require only that *some* individuals, a representative sample, are detected at more than one detector, and that the detector array is large enough to differentiate points in the middle and edge of *some* home ranges. Failure to meet the ‘spread’ requirement may be addressed, at least in principle, by combining SECR and [telemetry](#), but improved study design is a better solution.

A poorly designed detector array may give data that cannot be analysed by SECR, or provide highly biased estimates of low precision. Such designs were described by Efford & Boulanger (2019) as ‘pathological’. We seek a non-pathological and representative design that gives the most precise estimates possible for a given cost.

8.3 Precision and power

Precise estimates have narrow confidence intervals and high power to detect change. Precision and bias jointly determine the accuracy of estimates, measured by the root-mean-square error (the difference between estimates and the true value). Bias is typically a minor component, and for now we assume it is negligible.

Precision is conveniently measured by its inverse, the relative standard error (RSE) where $\text{RSE}(\hat{\theta}) = \widehat{\text{SE}}(\hat{\theta})/\hat{\theta}$ for estimate $\hat{\theta}$ of parameter θ . In statistics, the standard error is the square root of the sampling variance, estimated for MLE as described in Section 3.6. Wildlife papers often use ‘CV’ for the RSE of estimates, but this confuses relative standard error and relative standard deviation.

The question ‘What precision do I need?’ requires clarity on the purpose of the study. If the purpose is to compare density estimates from different times or places with the same effort then the power to detect change of a given magnitude is a direct function of the initial

$\text{RSE}(\hat{D})$ (Fig. 8.2). Although $\text{RSE} = 0.2$ is often touted as adequate “for management purposes”, estimates with $\text{RSE} = 0.2$ have low ($\leq 50\%$) power to detect a change in density of even $\pm 50\%$ at the usual $\alpha = 0.05$ (Fig. 8.2). If reliable estimates are vital for population management, as claimed routinely in methodological papers, then surely greater precision is required. $\text{RSE} = 0.1$ is a better general target.

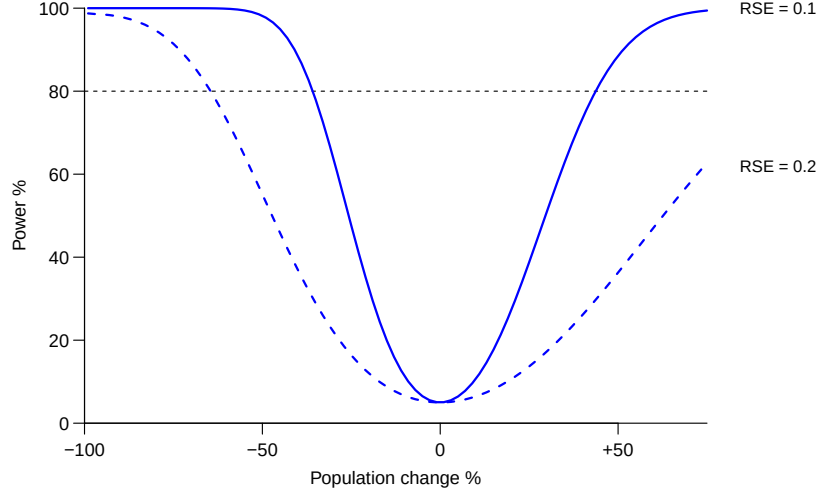


Figure 8.2: Power of a 2-sided test ($\alpha = 0.05$) to detect change in density between two surveys as a function of effect size ($(D_2/D_1 - 1) \times 100$ on the x-axis) for two levels of $\text{RSE}(\hat{D})$ in the initial survey. For initial $\text{RSE} = 0.1$, only changes greater than -36% and $+44\%$ are detected with power greater than 80% (dashed line). Reproduced from Efford & Boulanger (2019) Fig. 1.

We lack a clear guide to required RSE in other studies, for which the effect size may be ill-defined. We have observed that fitted models with $\text{RSE}(\hat{D}) \gg 0.2$ behave erratically with respect to AIC model selection, presumably because of sampling variance in the AIC values themselves.

8.4 Pilot parameter values

To evaluate a potential study design we must know something about the target population. Here we describe the population and the behaviour of individuals by a simple SECR model and its parameters: uniform density D and the parameters λ_0 and σ of a hazard half-normal [detection function](#). Predictions regarding the suitability and performance of any design then depend on the values of D , λ_0 and σ . This seems like a catch-22 – impossible until we have estimates – but for design purposes we can call on approximate values from multiple sources

- published estimates from studies of similar species,
- a low-precision pilot study, or
- indirect inference.

Reviews of SECR studies are a useful source of pilot estimates (e.g., Palmero et al., 2023). The hardest parameter to pin down is λ_0 , as this is very study-specific. The good news is that it has only a secondary effect on the relative merits of different array designs.

i hazard λ_0 vs probability g_0

We emphasise *hazard* detection functions (intercept λ_0) over *probability* detection functions (intercept g_0) because the formulae for expected counts are straightforward (Appendix K). However, pilot values of the probability parameters are more commonly available. Note that the fitted σ also differs. For design purposes, the parameters of the probability detection function (g_0, σ) may be substituted for the corresponding parameters of the hazard detection functions (λ_0, σ). If desired, the internal function `dfcast` of **secrdesign** provides a more precise match e.g., `secrdesign::dfcast(detectfn = 'HN', detectpar=list(g0 = 0.2, sigma = 25))`

Indirect inference is a murky option, but better than nothing, and there are some constraints. The quantity $k = \sigma\sqrt{D}$ (loosely described by Efford et al. (2016) as an index of home range overlap) usually falls in the range 0.3–1.3 for solitary species (M. Efford unpubl.). **secr** expresses D in animals / ha and σ in metres, so a factor of 100 is needed, and $\sigma = \frac{100}{\sqrt{2D}}$ is a good start ($k \approx 0.707$).

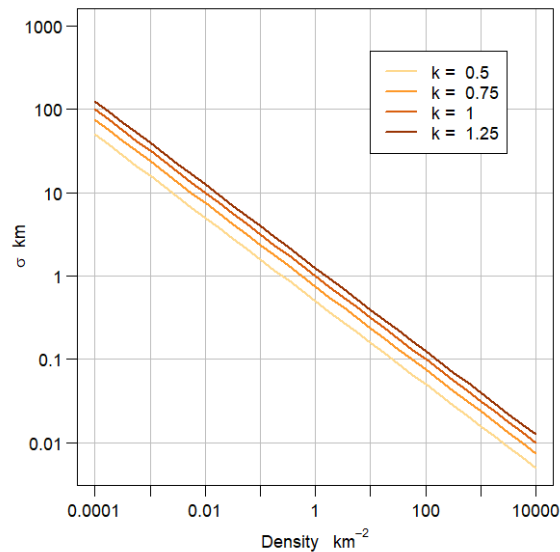


Figure 8.3: Relationship between density and the spatial scale parameter σ for different values of the overlap index k .

We can use the close analogy between the detection function and a home-range utilisation distribution to extract a pilot value of σ from home range data. Most directly, a circular bivariate normal (BVN) model can be fitted to telemetry data; we then use the dispersion parameter directly as a pilot value of σ for hazard half-normal detection. If the home range data have been summarised as the area a within a notional 95% activity contour then the

spatial scale parameter of the hazard half-normal is close to $\sigma = \sqrt{\frac{a}{6\pi}}$ (Jennrich & Turner, 1969 Eq. 13)¹.

⚠ Scalability

Simulation to compare study designs commonly rests on the assumption that the underlying detection process is constant. If parameters vary between layouts in a way not captured by the model (see [salamander](#) example) then extrapolation to novel layouts is unreliable. There is an analogous problem when locations are [serially correlated](#): estimates of detection parameters (especially σ) are then specific to the duration of the pilot study and cannot safely be extrapolated to other durations.

8.5 Expected sample size

Given some pilot parameter values, we might proceed directly to simulating data from different designs and computing the resulting SECR estimates. This is effective, but slow. A useful preliminary step is to check the expected sample size of potential designs.

The sample size for SECR is more than a single number. We suggest calculating the expected values of these count statistics:

- n the number of distinct individuals detected at least once,
- r the total number of re-detections (any detection after an individual is first detected), and
- m the total number of movements (re-detections at a detector different to the preceding one).

The counts depend on the detector configuration, the extent of habitat (buffer width), and the type of detector (trap, proximity detector etc.) in addition to the parameter values. Formulae are given in Appendix K. Calculation is fast and the counts give insight on whether the data generated by a design are likely to be adequate. $E(r)$ is a direct measure of Requirement 2 above. $E(m)$ addresses Requirement 3a ('Spacing too large' in Fig. 8.1).

i Spatial recaptures

The term 'spatial recaptures' corresponds loosely with our m . Spatial recaptures are a *sine qua non* of SECR, but their role in determining precision can be overstated. They are a consequence of Requirements 2 and 3, not an independent effect. Furthermore:

- Movements are ill-defined when detections are aggregated by time, as then we cannot distinguish the capture histories ABABA and AAABB at detectors A and B (both are (3A, 2B), but one implies 4 movements and the other one).
- Spatial and non-spatial recaptures have barely distinguishable effects when detectors are very close; a profusion of close spatial recaptures is not very useful (e.g., Durbach

¹More generally, $\sigma = \sqrt{\frac{a}{\pi \log[(1-p)^{-2}]}}$ where p is the probability contour (Jennrich & Turner, 1969 Eq. 12).

8.6 A useful approximation

Efford & Boulanger (2019) found that $E(n)$ and $E(r)$ alone were often sufficient to predict the precision of SECR density estimates. The relationship is

$$\text{RSE}(\hat{D}) \approx 1/\sqrt{\min\{E(n), E(r)\}}.$$

This formula supports intuition regarding the effect of particular designs on precision, especially the ‘*n-r tradeoff*’ described later. Computation is much faster than estimating $\hat{D}$. The approximation is used in the interactive Shiny application [secrdesignapp](#) and for [algorithmic optimisation](#) (Durbach et al., 2021). Limitations and refinements are discussed in the papers cited.

8.7 Array size versus home range

We lack a count statistic matching the second part of Requirement 3 (‘Array too small’ in Fig. 8.1). We therefore define an *ad hoc* measure of sampling scale that we call the ‘extent ratio’: this is the diameter of the array (the distance between the most extreme detectors) divided by the nominal diameter of a 95% home range. The 95% home-range radius is $r_{0.95} \approx 2.45\sigma$ for a BVN utilisation distribution (from the previous formula).

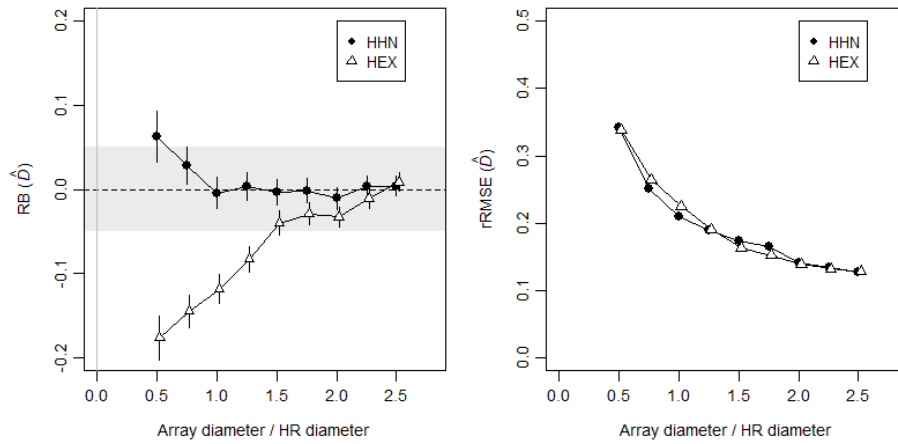


Figure 8.4: Effect of array size on relative bias (RB) and relative root-mean-square error (rRMSE) of density estimates. Simulations of a square array with size (diagonal length) divided by the diameter of a 95% BVN home range. Density was estimated with both the detection function used to generate the data (HHN) and a mis-specified detection function (HEX).

Simulations described on [GitHub](#) show that SECR performs poorly when the extent ratio is less than 1: the relative bias and root-mean-square error of $\hat{D}$ increase abruptly (Fig. 8.4). We earlier touted the robustness of $\hat{D}$ to misspecification of the [detection function](#), but this desirable property evaporates when the array is small, as shown by estimates from a hazard exponential function applied to hazard half-normal data in Fig. 8.4.

Efford (2011) simulated area-search data for a range of area sizes. Plotting those results with the extent ratio as the x-axis shows a similar pattern to Fig. 8.4 (see [GitHub](#)).

The extent ratio does not provide a precise criterion because it combines two somewhat arbitrary measures (array diameter, 95% home range diameter). What the simulations reveal about array size can be summed up in a simple rule: the grid or searched area should be at least the size of the home range, and larger arrays provide greater robustness and accuracy.

8.8 The $n - r$ tradeoff

The performance of the SECR density estimator depends on both the number of individuals n and the number of re-detections r . However, we cannot maximise their expected values simultaneously. $E(n)$ is greatest when detectors are far apart and each detector samples a different set of individuals (AC). For binary proximity detectors $E(r)$ is greatest when detectors are clumped together and declines monotonically with spacing; for multi-catch traps, $E(r)$ increases and then declines (Fig. 8.5).

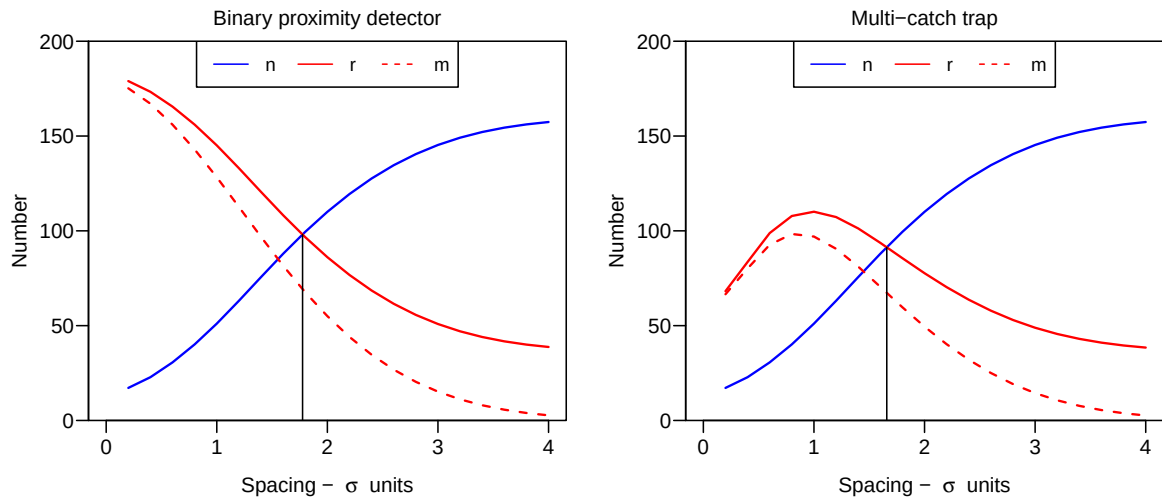


Figure 8.5: Sample size as a function of detector spacing for two detector types. Square grid of 64 detectors operated for 10 occasions with $D = 0.5\sigma^{-2}$ and $\lambda_0 = 0.1$.

To maximise the sample size we seek a satisfactory compromise, an intermediate spacing of detectors that yields both high $E(n)$ and high $E(r)$. There is so far no evidence for weighting one more than the other; later simulations suggest aiming for $E(n) \approx E(r)$, as indicated by

the vertical lines in Fig. 8.5. The number of movement recaptures m follows the pattern of total recaptures r and need not be considered separately.

8.9 Spacing and precision

We expect precision to improve with sample size. From the last section we understand that sample size depends somewhat subtly on detector spacing: wider spacing increases n because a larger area is sampled, but it ultimately reduces r . We next use simulation to demonstrate the effect on the precision. We measure precision by the relative standard error of density estimates $\text{RSE}(\hat{D})$. In the following examples we assume a Poisson distribution for n rather than the binomial distribution that results when $N(A)$ is fixed.

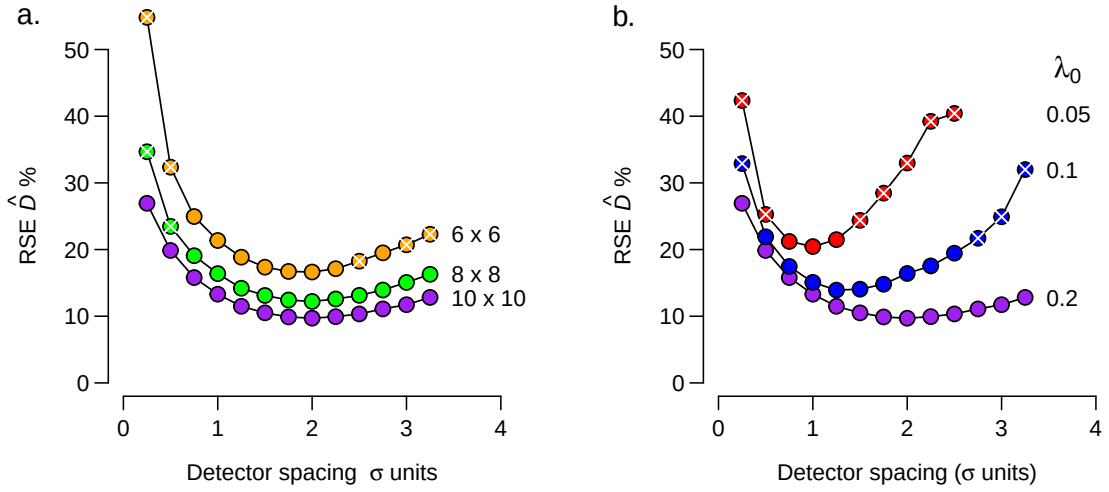


Figure 8.6: Relationship between precision and detector spacing for scenarios of differing grid size and detection rate λ_0 (5 sampling occasions, reproduced from Fig. 2, Efford & Boulanger, 2019).

The simulations in Fig. 8.6 are representative: there is an optimal spacing (often around 2σ for a hazard half-normal detection function) and a broad range of spacings yielding similar precision. The optimum is close, but not identical, to the spacing that gives $E(n) = E(r)$ in Fig. 8.5. The variability of the simulations increases away from the optimum spacing: if we are in the right ballpark then very few replicates are needed to capture it.

Although 2σ is often mentioned as an optimum, the optimal spacing is smaller when sampling intensity is low. This happens when there are few sampling occasions or λ_0 is small, as illustrated in Fig. 8.6.

The curves in Fig. 8.7 are given by $R_{opt} = 2\sqrt{\lambda_0 S}$ where R_{opt} is the optimal spacing in σ units and S is the number of occasions. The reason for this apparently tight relationship has yet to be determined, and it is unreliable for intensive sampling (large $S\lambda_0$), as indicated by the simulations for $\lambda_0 = 0.2$ and $S \geq 10$.

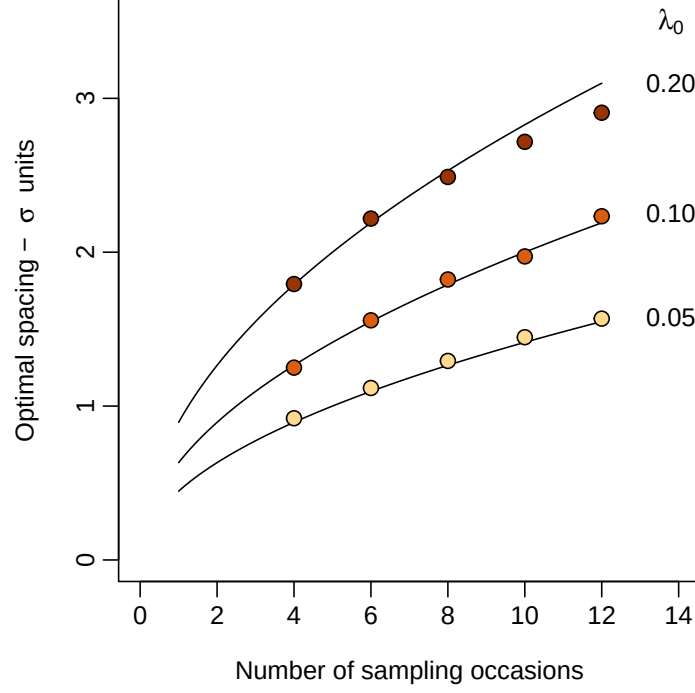


Figure 8.7: Effect of sampling intensity (number of occasions and λ_0) on optimal detector spacing from simulations. Base scenario as for binary proximity detectors in Fig. 8.5.

8.10 Detectors required for uniform array

Increasing imprecision of $\hat{D}$ and failure of estimation in many cases (indicated by white crosses in Fig. 8.6) place a limit on the size of region A that can be sampled with a uniform grid of K detectors. At the optimum spacing (i.e. about $2\sigma\sqrt{\lambda_0 S}$) we require $K > A/(4\sigma^2\lambda_0 S)$.

A region of interest with area $A \gg 4K\sigma^2\lambda_0 S$ cannot be covered adequately with a uniform grid of K detectors. Thus for binary proximity detectors and hazard half-normal detection with $\lambda_0 = 0.1$,

- when $\sigma = 50$ m, sampling with 100 detectors for $S = 5$ occasions covers 50 ha at 71-m spacing
- when $\sigma = 1$ km (implying larger home range and lower density), then sampling with 100 detectors for $S = 5$ occasions covers 200 km² at 1.4-km spacing.

The spacing may be stretched somewhat from the optimum if we are willing to accept suboptimal precision, but increasing spacing by more than 50% also runs the risk of bias (Fig. 8.6).

The ratio $R_K = 4\sigma^2 K/A$ is a useful guide for planning, where K is the number of detectors that can be deployed. $R_K \geq 1.0$ allows for a uniform grid, whereas for $R_K \ll 1.0$ there are too few detectors for an efficient uniform grid. Here we assume $\sqrt{\lambda_0 S} \approx 1.0$.

8.11 Number of detectors in fixed region

We have referred to $2\sigma\sqrt{\lambda_0 S}$ as an ‘optimum’ spacing for the scenario that the number of detectors is fixed. Increasing the number of detectors in an area of interest can be expected to increase sample sizes, particularly $E(r)$, and hence to improve on precision beyond the ‘optimum’. For a non-arbitrary area $R_K > 1.0$ is entirely appropriate, and cost becomes the decider. The benefit from increasing the density of detectors may be small; for example, trebling the number of detectors in a 10-ha area from $K_0 = A/(4\sigma^2\lambda_0 S) = 62.5$ resulted in only a 27% reduction in $\text{RSE}(\hat{D})$ for the base parameter values of Fig. 8.5.

8.12 Clustered designs

We have assumed a regular grid of detectors with constant spacing. Regular grids have an almost mystical status in small mammal trapping (Otis et al., 1978) that can be justified, for non-spatial capture–recapture, by the need to expose every individual to the same probability of capture. Checking traps laid in straight lines is also easier than navigating on foot to random points.

SECR removes the strict requirement for equal exposure, and large-scale studies often use vehicular transport and navigation by GPS, removing the advantage of straight lines. We use the term ‘clustered design’ for any layout of detectors that departs from a simple grid. Clustering may be geometrical, as in a systematic layout of regular subgrids, or incidental, as when detectors are placed at a simple random sample of sites or according to some other non-geometrical rule such as [algorithmic optimisation](#).

8.12.1 Rationale

The primary use for a clustered design is to survey a large region of interest with a limited number of detectors, while ensuring some close spacings to allow recaptures (r, m). It is impossible to meet Requirement 3a in this scenario without clustering. The threshold of area for a region to be considered ‘large’ was addressed [above](#).

Local density almost certainly varies across any large region of interest. Clustering of detectors implies patchy sampling, with the risk that selective placement of detectors in either high- or low-density areas leads to biased estimates. Spatial variation in detection from excessive clustering also adds to the variance of estimates. The risks are minimised by selecting a widely distributed and spatially representative sample. This can be achieved with a randomly located systematic array of small subgrids (e.g., Clark, 2019), and some other methods to be discussed.

A secondary application of clustered designs is to encompass heterogeneous σ (e.g., sex differences) by providing a range of spacings. G. Dupont et al. (2021, p. 8) supposed a general benefit of irregularity “to gain better resolution of movement distances for estimating σ ”, but this has yet to be demonstrated.

Clustering of detectors can reduce the total distance that must be traveled to visit all detectors. If travel is a major cost then clustering may allow more detectors to be used. This applies regardless of the size of region.

8.12.2 Systematic clustered designs

Subgrids

Random systematic designs usually provide a representative sample with lower sampling variance than a simple random sample (Cochran, 1977; Thompson, 2012). Any systematic design should have a random origin. For SECR each point on the systematic grid is the location of a subgrid. Subgrids may be any shape. We focus on square and hollow grids (Fig. 8.8), but circles, hexagons, and straight lines are also possible.

If subgrids are far apart then few individuals will be caught on more than one subgrid, and they can be designed and analysed as independent units (e.g., Clark, 2019). For design this means that the spacing of detectors on each subgrid should be optimised according to the considerations already raised in Section 8.5 to Section 8.9. For analysis it means that a homogeneous density model may be fitted to aggregated (‘mashed’) data from j subgrids as if they were collected from a single subgrid with the shared geometry and j times the density.

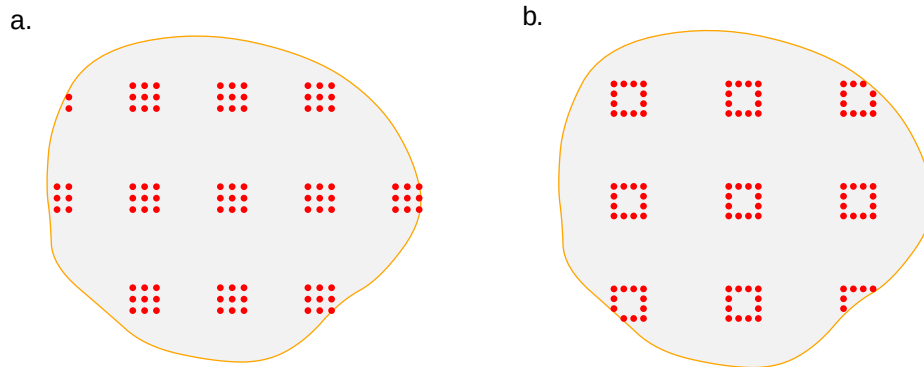


Figure 8.8: Examples of systematic clustered layouts with (a) 98 detectors at 12-m spacing, and (b) 99 detectors at 10-m spacing, in a 10-ha region of interest (grey).

Truncation of subgrids at the edge of the region of interest reduces the size and value of marginal clusters. Two other geometrical designs avoid this problem: a lacework design (available in Efford (2025a)) and a spatial coverage design, following Walvoort et al. (2010).

Lacework

In a lacework design, detectors are placed along two sets of equally spaced lines that cross at right angles to form a lattice (Fig. 8.9). By making the spacing of the lattice lines (lattice

spacing a) an integer multiple of the spacing of detectors along lines (detector spacing b) we avoid odd spacing at the intersections. The expected number of points on a lacework design is given by $E(K) = A/a^2 (2a/b - 1)$.

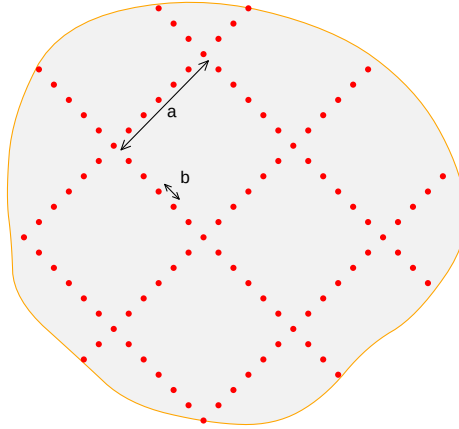


Figure 8.9: Lacework with 102 detectors at 17.4-m spacing in a 10-ha region of interest.

As with any systematic layout, it is desirable to randomize the origin of the lacework. The orientation is arbitrary. Lacework designs have the advantage of requiring only two design variables (a, b) when subgrid methods involve three (number or spacing of subgrids, plus spacing and extent of each subgrid).

Spatial coverage

Spatial coverage may be achieved by first dividing the region of interest into equal-sized strata and then placing a subgrid in each stratum. A k -means algorithm is used to form the strata as compact clusters of pixels (Walvoort et al., 2010). A detector cluster may be centred or placed at random within each stratum (Fig. 8.10). We include the spatial coverage approach here because it uses nested subgrids, although the result is not strictly systematic.

8.12.3 Non-systematic designs

Non-systematic designs result from algorithms for the placement of detectors that do not impose a particular geometry. One candidate is a simple random sample of points in the region of interest (SRS). Various algorithms improve on SRS with respect to spatial balance and efficiency. We focus on the Generalized Random Tessellation Stratified algorithm implemented by Dumelle et al. (2023) in the R package **spsurvey**, but there are other options (Robertson et al., 2018).

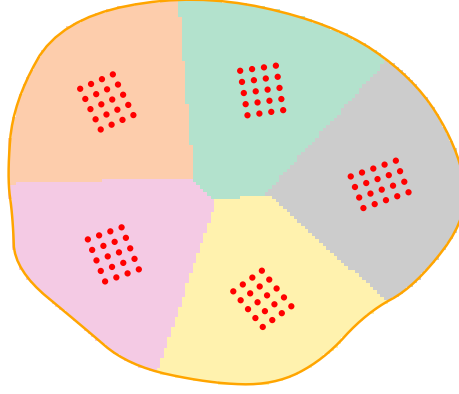


Figure 8.10: Spatial coverage design with 5 randomly oriented subgrids of 20 detectors centred in equal-area strata.

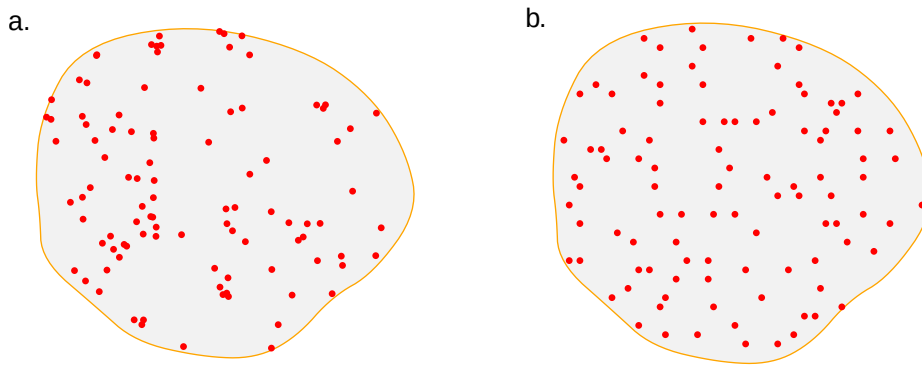


Figure 8.11: Non-systematic clustered layouts (a) simple random sample, and (b) Generalized Random Tessellation Stratified sample, 100 detectors in a 10-ha region of interest (grey).

Algorithmic optimisation

Choosing a set of detector locations may be reduced to finding a subset of potential locations that maximises some criterion (benefit function). A genetic algorithm is a robust way to search the vast set of possible layouts. Wolters (2015) implemented a genetic algorithm in R that was subsequently applied to the optimisation of SECR designs by G. Dupont et al. (2021) and Durbach et al. (2021). The set of potential locations may simply be a fine grid of points, excluding points that are deemed inaccessible or fall in non-habitat. Results depend on the choice of criterion: Durbach et al. (2021) used the minimum of $E(n)$ and $E(r)$, following the suggestion of Efford & Boulanger (2019) that this often leads to near-maximal precision of $\hat{D}$. Optimisation may take many iterations and results are not unique.

A larger criticism is that the resulting layouts are not spatially representative. The example in Fig. 8.12 shows 100 detectors placed “optimally” when $R_K = 0.16$. Detectors are clumped to maximise the precision criterion $\min(E(n), E(r))$ without regard to spatial balance. We

view this to be a major weakness and discourage general use of the method except for exploratory purposes.

Algorithmic optimisation has revealed one point that makes intuitive sense and is otherwise hidden. AC near the edge tend to have access to fewer detectors than central AC, and increased density of detectors near the edge is beneficial for increasing n , but may reduce r (G. Dupont et al., 2021).

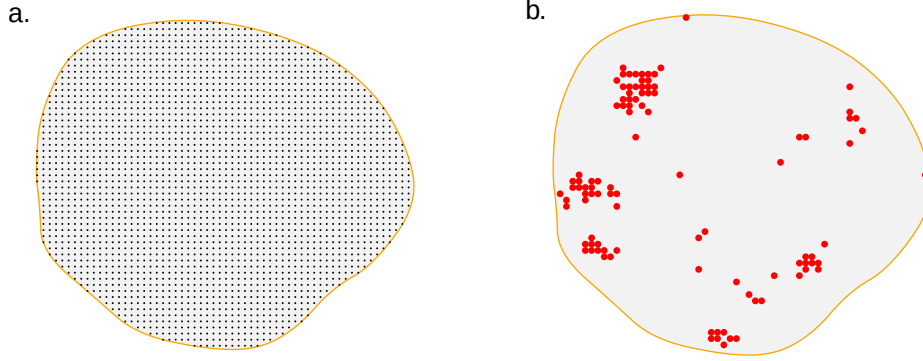


Figure 8.12: Algorithmic optimisation of layout. (a) 2498 potential detector locations in a 100-ha (1-km²) region, and (b) Layout comprising a subset of 100 points that maximises $\min(E(n), E(r))$ after 1000 iterations with the base parameters in Fig. 8.5.

8.12.4 Comparison of clustered layouts

There is no general way to navigate the multiplicity of clustered designs. As an example, we compare the preceding clustered designs for a particular scenario: 100 binary proximity detectors in a 1-km² region of interest for a species with $\sigma = 0.02$ km ($R_K = 0.16$) keeping other parameters as in Fig. 8.5.

As before, we expected precision to be driven by sample size, n and r , with $E(n)$ maximised by scattering detectors widely. Differences among the designs with respect to the usual measures of performance (RB, RSE, rRMSE) were mostly minor (Table 8.1). The exceptions were SRS (RB($\hat{D}$) +4%) and GRTS (RB($\hat{D}$) +5%); these designs detected many individuals, but generated few re-detections. The lacework design gave similar results to SRS in this instance, but results can be improved with closer spacing along fewer lines.

The algorithmically optimised design was one of several with $RSE(\hat{D}) \approx 11\%$. Tight clustering of detectors resulted in the shortest travel distance of the designs considered. However, ‘GA optimised’ clearly lacked spatial balance and is therefore likely to provide biased estimates of average density if density varies systematically across the region.

Costing is complex and study-specific. Travel is an important component (time or mileage) and so we computed the length of the shortest route that included all detectors. Other components are the cost of detectors and initial setup (both proportional to K), and laboratory processing of DNA samples (proportional to $n+r$). Optimal route length is an example of

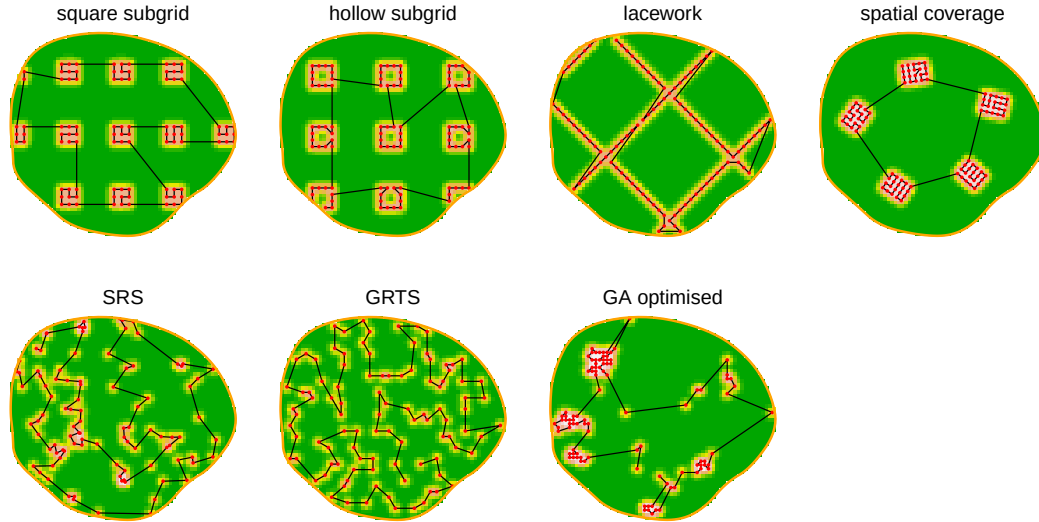


Figure 8.13: Layouts with about 100 detectors in a region 1-km². Colour codes the probability of detection for an individual centred in each pixel (dark green <0.05, white >0.95). Detectors are joined by a minimum-length route.

the ‘traveling salesman problem’ (TSP). Heuristic methods in R package **TSP** (Hahsler & Hornik, 2007) do not produce a single minimum and perform poorly with regular grids. We therefore used the [Concorde](#) software [outside R](#) to obtain optimal routes.

Table 8.1: Performance of seven sampling designs ($K \approx 100$ detectors) with respect to sample size (expected numbers of individuals $E(n)$, re-detections $E(r)$, and movements $E(m)$), relative bias RB, relative standard error RSE, root-mean-square error rRMSE, and route length L . RB, RSE and rRMSE are percentages based on 100 simulations for each design.

Design	K	$E(n)$	$E(r)$	$E(m)$	RB	RSE	rRMSE	L km
square subgrid	98	191	101	54	0.6	10.6	9.7	6.2
hollow subgrid	99	196	99	52	0.2	11.2	9.5	5.5
lacework	103	217	93	41	1.2	11.7	11.9	5.4
spatial coverage	100	182	124	74	1.0	9.4	10.0	5.9
SRS	100	215	92	39	1.5	14.2	14.9	7.2
GRTS	100	239	68	12	2.2	17.1	23.3	8.1
GA optimised	100	153	153	112	1.0	10.6	10.3	5.2

8.13 How many clusters?

For the preceding comparison we fixed the number of detectors in advance. More realistically, we might set a target $\text{RSE}(\hat{D})$ and ask how many clusters (i.e. subgrids) are required. Efford & Boulanger (2019) observed that if clusters are independent (widely spaced) then $E(n)$ and $E(r)$ both scale with the number of clusters c , and $\text{RSE}(\hat{D})$ scales with $1/\sqrt{c}$. Thus we can predict the precision for a single cluster and extrapolate to assemblages of clusters (Fig. 8.14). Having determined the required number of clusters we can decide on a systematic spacing or apply the [spatial coverage](#) method.

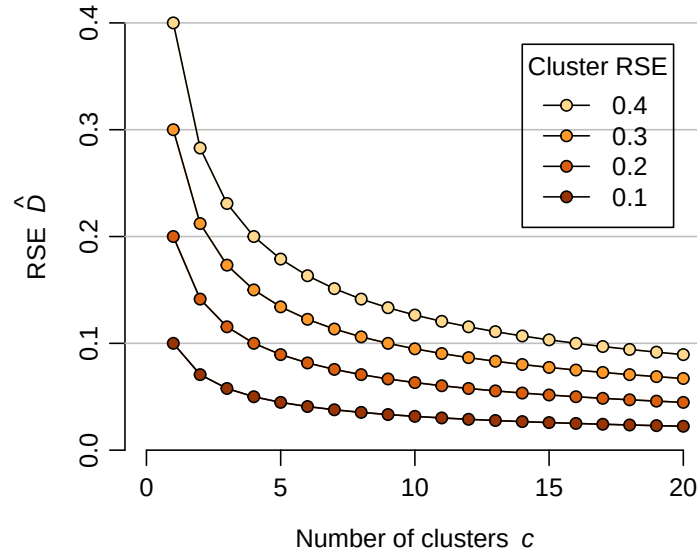


Figure 8.14: Overall $\text{RSE}(\hat{D})$ from cluster assemblages varying in size and single-cluster RSE.

8.14 Designing for spatial variation in parameters

We have so far treated the parameters of the SECR model as constant across space. How should designs be modified to allow for spatial variation?

We must first understand the interaction between spatial variation in density and sampling intensity. Variation in density alone is not a source of significant bias in SECR. However, when sampling is more intensive in areas of high density, or areas of low density, a model that assumes homogeneous density and sampling produces biased estimates.

Some aspects of sampling intensity are under the control of the experimenter: these are the type, number and spacing of detectors, and the duration of sampling. We group these under the heading ‘sampling effort’. They will usually be known and included explicitly in the SECR model.

Other variation in sampling intensity may be due to behavioural or habitat factors that drive spatial variation in detection parameters (e.g., λ_0, σ) unknown to the experimenter.

We illustrate six scenarios for the interaction between spatial variation in density and effort, and between density and detection in Fig. 8.15. The estimate of overall density ($\hat{D}$ from the null model $D \sim 1$) is unbiased when effort and detection are homogeneous (Fig. 8.15 a,d). Uncorrelated variation in effort does not bias $\hat{D}$ (Fig. 8.15 b).

Selectively reducing effort in low-density areas (Fig. 8.15 c) causes significant bias that may be reduced or eliminated by modelling the variation in density. In the simple scenario of Fig. 8.15 c, a binary variable ‘habitat’ distinguishing between known low-density and high-density areas could be used in a model ($D \sim \text{habitat}$).

Variation in detection parameters causes bias even when not correlated with density (Fig. 8.15 e). This is the effect of **inhomogeneous detection** noted previously. The effect is large when density and detection are correlated (Fig. 8.15 f and McLellan et al. (2023)).

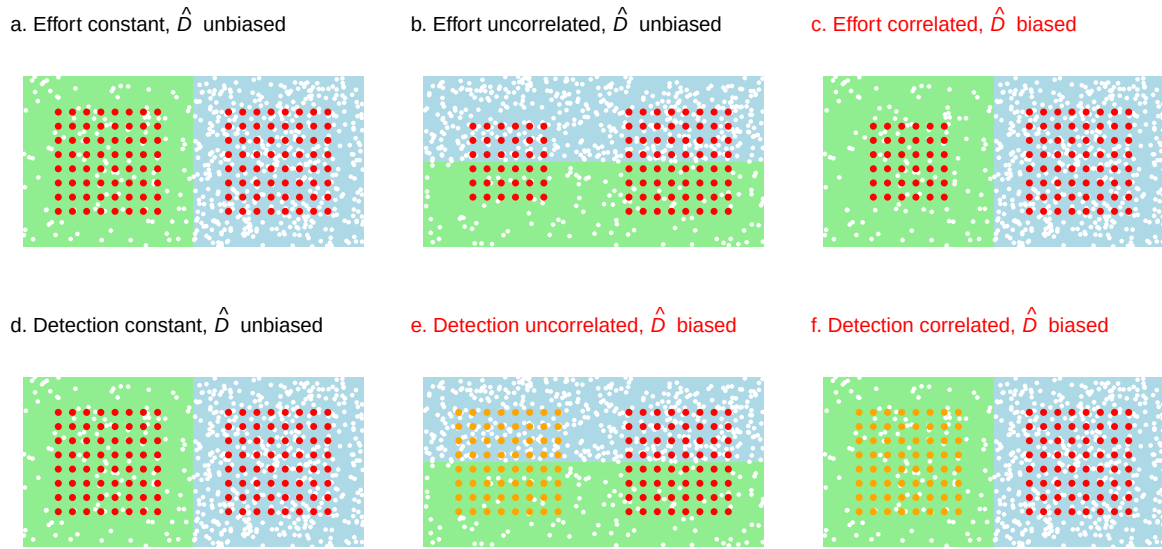


Figure 8.15: Effect of spatially varying effort (grid size 6 x 6 vs 8 x 8) and detection (λ_0 orange low vs red high) on null-model estimate of overall density when density of AC (white dots) varies spatially between low (light green) and high (light blue). See [GitHub](#) for simulation results.

Spatially representative sampling aims to avoid correlation between density and effort (Fig. 8.15 c). A strong spatial model for density overcomes the problem, but it cannot usually be guaranteed that suitable covariates will be found, and representative sampling insures against model failure.

Stratified sampling

Deliberate stratification of effort is a special case. Stratification is used in conventional sampling to increase the precision of estimates for a given effort (e.g., Cochran, 1977; Thompson,

2012). We note that in SECR only detected individuals and their re-detections contribute to the estimation of detection parameters and, in this respect, effort in low-density areas is mostly wasted. Greater precision may therefore be achieved in some studies by concentrating effort where high density is expected. This requires care. A stratified analysis combines stratum-specific density estimates as follows.

Suppose a region of interest with area A may be divided into H subregions with area A_h where $A = \sum_h A_h$. Subregions have densities $D_1, D_2, \dots, D_H$. Independent SECR sampling followed by separate model fitting in each region provides estimates $\hat{D}_1, \hat{D}_2, \dots, \hat{D}_H$. The weight for each region is based on its area: $W_h = A_h/A$ and $\sum_h W_h = 1$.

Then an estimate of overall density is $\hat{D} = \sum_h W_h \hat{D}_h$ and an estimate of the sampling variance is $\widehat{\text{var}}(\hat{D}) = \sum_{h=1}^H W_h^2 \widehat{\text{var}}(D_h)$.

The benefits of stratification for SECR have yet to be fully analysed. They are likely to be greatest when strata of contrasting density are easily distinguished, and when uncertainty regarding detection parameters (and hence the effective sampling area) dominates $\text{var}(n) = s^2$ in the [combined variance](#).

We conducted a simulation experiment with two density strata (40% and 160% of the overall average) and a constant total effort allocated differentially (Fig. 8.16). Details are on [GitHub](#). For this example precision was best when about 70% of detectors were placed in the high-density stratum, but the improvement over equal allocation was tiny ($\Delta\text{RSE} \approx 0.005$) for 10 occasions and small ($\Delta\text{RSE} \approx 0.013$) for 4 occasions. Effort may also be stratified by varying the number of sampling occasions.

It may help to assume that detection parameters are uniform throughout (i.e. shared among strata); a formal test of detection homogeneity is likely to have low power.

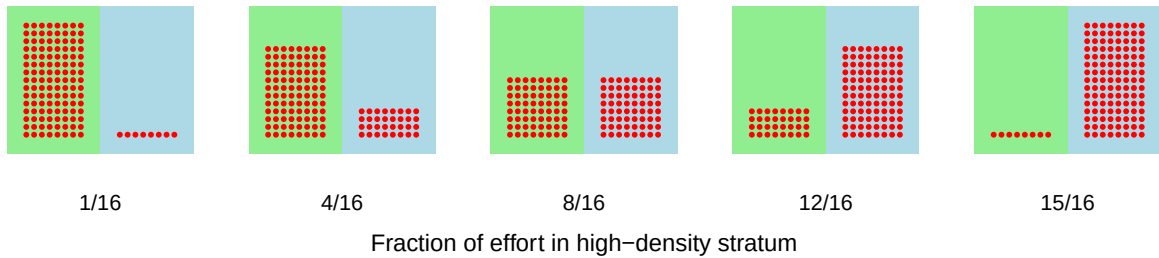


Figure 8.16: Design for evaluation of stratified effort.

8.15 Summary

The SECR method is very flexible, within limits. Our approach to study design for SECR is not prescriptive. Users should bear in mind these principles:

- sampling should be representative of the area of interest
- without large samples a study may lack the statistical power to answer real-world questions (this is not unique to SECR)

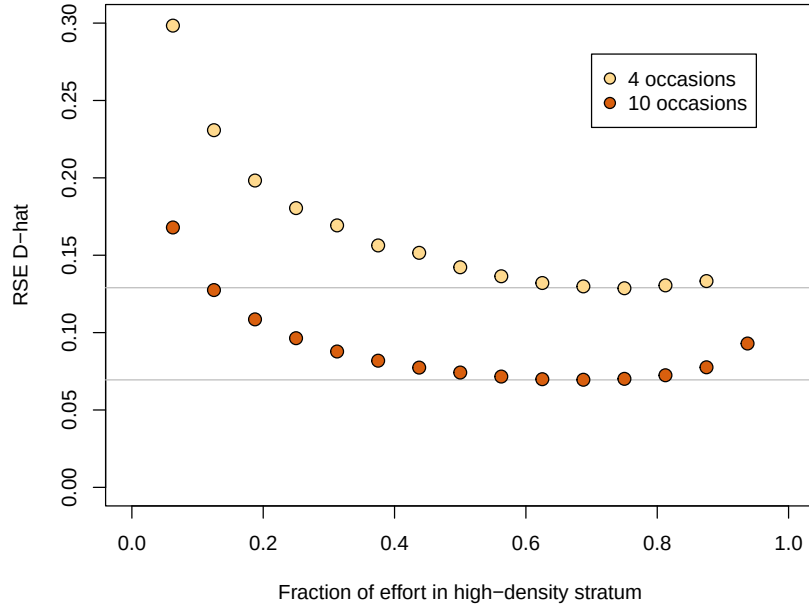


Figure 8.17: Precision of overall density estimated with stratified effort. Grey line indicates minimum RSE when 88 of 128 detectors were placed in the high-density stratum (4 occasions, 0.129; 10 occasions, 0.069).

- two scales are important - the size of the area of interest and the scale of movement/detection
- robustness to model misspecification is greatest when the detector array spans several home ranges (Fig. 8.4)
- precise estimation of density requires both many individuals (n) and many recaptures (r) and these aspects of sample size often entail a design tradeoff
- large areas may be sampled with clustered detectors in various configurations
- logistic feasibility and cost considerations may override power in deciding among designs
- unmodelled spatial heterogeneity of density is not problematic in itself, but becomes so when confounded with spatially heterogeneous detection, due either to known variation in effort or to unseen factors
- stratified sampling (concentrating sampling effort where density is predicted to be greater) has the potential to increase precision, but in our exploratory simulation the gains were minimal.

It should be stressed that our simulation results (e.g., Fig. 8.4, Fig. 8.6, Fig. 8.7, Fig. 8.17, and Table 8.1) relate to particular scenarios, and the conclusions we draw may be reversed in other scenarios.

Part IV

Practice

9 R package secr

This chapter provides an overview of **secr** (Efford, 2025a). Following chapters expand on key topics.

To reproduce examples in the book you will need a recent version of **secr** (5.2.0 or later). Text in **this font** refers to R objects that are documented in online help for the **secr** package, or in base R. R code often generates warnings. Some warnings are merely reminders (e.g., that a default value is used for a key argument). For clarity, we do not display routine warnings for examples in the text.

In the examples we often use the function `list.secr.fit` to fit several competing models while holding constant other arguments of `secr.fit`. The result is a single ‘seclist’ that may be passed to `AIC`, `predict` etc.

9.1 History

secr supercedes the Windows program DENSITY, an earlier graphical interface to SECR methods (Efford et al., 2004; Efford, 2012). The package was first released in March 2010 and continues to be developed. It implements almost all the methods described by Borchers & Efford (2008), Efford, Borchers, et al. (2009), Efford (2011), Efford & Fewster (2013), Efford et al. (2013), Efford & Mowat (2014), Efford et al. (2016), Efford & Hunter (2018) and Efford (2025c). External C++ code (Eddelbuettel et al., 2023) is used for computationally intensive operations. Multi-threading on multiple CPUs with RcppParallel (Allaire et al., 2023) gives major speed gains. The package is available from [CRAN](#); the development version is on [GitHub](#).

An interactive graphic user interface to many features of **secr** is provided by the Shiny app ‘secrapp’ available on [GitHub](#) and, currently, on a server at the [University of Otago](#).

9.2 Object classes

R operates on *objects* each of which has a *class*. **secr** defines a set of R classes and methods for spatial capture–recapture data and models fitted to those data. To perform an SECR analysis you construct each of these objects in turn. Fig. 9.1 indicates the relationships among the classes.

i R classes

A ‘class’ in R specifies a particular type of data object and the functions (methods) by which it is manipulated (computed, printed, plotted etc). Technically, **secr** uses old-fashioned ‘S3’ classes. See the R documentation `?class` for further explanation.

Table 9.1: Essential classes in **secr**

Class	Data
traps	locations of detectors; detector type (‘proximity’, ‘multi’, etc.)
capthist	spatial detection histories, including a ‘traps’ object
mask	raster map of habitat near the detectors
secr	fitted SECR model

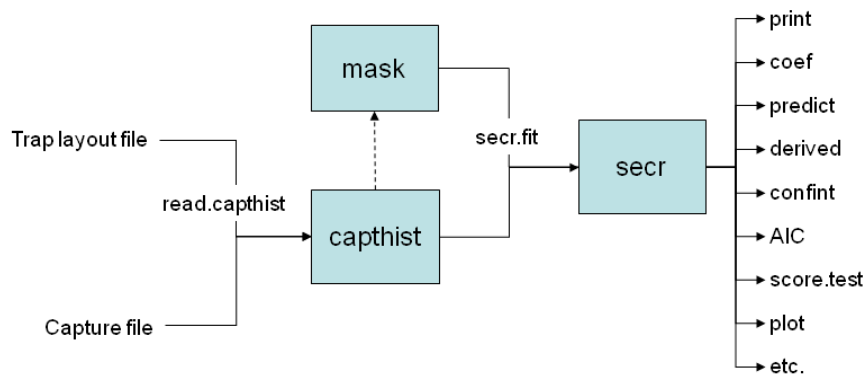


Figure 9.1: Workflow in **secr**

- Each object class (shaded box) comes with methods to display and manipulate the data it contains (e.g. `print`, `summary`, `plot`, `rbind`, `subset`).
- The function `read.capthist` forms a ‘capthist’ object from input in two files, one the detector layout (saved as attribute ‘traps’) and the other the capture data.
- By default, a habitat mask is generated automatically by `secr.fit` using a specified buffer around the detectors (traps) (dashed arrow). The function `make.mask` gives greater control over this step.
- Any of the objects input to `secr.fit` (traps, capthist, mask) may include a dataframe of covariates saved as an attribute. Covariate names may be used in model formulae; the `covariates` method is used to extract or replace covariates. Use `addCovariates` for trap or mask covariates from spatial data sources (e.g., shapefile or ‘sf’ object)
- Fitted secr models may be manipulated with the methods shown on the right.

💡 Tip

Avoid using '[' to extract subsets from capthist, traps, mask and other **secr** objects. Use the provided **subset** methods. It is generally safe to use '[' to extract one session from a multi-session capthist object.

9.3 Functions

For details of how to use each **secr** function see the help pages and vignettes.

Table 9.2: Core functions of secr. S3 methods are marked with an asterisk

Function	Purpose
addCovariates	add spatial covariates to traps or mask
AIC *	model selection, model weights
covariates	extract or replace covariates of traps, capthist or mask
derived *	compute density from conditional likelihood models
make.mask	construct habitat mask (= mesh)
plot *	plot capthist, traps or mask
predict *	compute 'real' parameters for arbitrary levels of predictor variables
predictDsurface	evaluate density surface at each point of a mask
read.capthist	input captures and trap layout from Density format, one call
region.N *	compute expected and realised population size in specified region
secr.fit	maximum likelihood fit; result is a fitted 'secr' object
summary *	summarise capthist, traps, mask, or fitted model
traps	extract or replace traps object in capthist

9.4 Detector types

Each 'traps' object has a detector type attribute that is a character value.

Table 9.3: Basic detector types in secr. See the appendices for [area-search](#) and [telemetry](#) types.

Name	Type	Description
"single"	single-catch trap	catch one animal at a time
"multi"	multi-catch trap	may catch more than one animal at a time
"proximity"	binary proximity	records presence at a point without restricting movement
"count" ¹	Poisson count proximity	[binomN = 0] allows >1 detection per animal per time

Name	Type	Description
	Binomial count proximity	[<code>binomN > 0</code>] up to <code>binomN</code> detections per animal per time

1. The “count” detector type is generic for integer data; the actual type depends on the `secr.fit` argument ‘`binomN`’.

There is limited support in **secr** for the analysis of locational data from telemetry (‘telemetry’ detector type). Telemetry data are used to augment capture–recapture data (Appendix G).

9.5 Input

Data input is covered in the data input vignette [secr-datainput.pdf](#). One option is to use text files in the formats used by DENSITY; these accommodate most types of data. Two files are required, one of detector (trap) coordinates and one of the detections (captures) themselves; the function `read.caphist` reads both files and constructs a `capthist` object. It is also possible to construct the `capthist` object in two stages, first making a `traps` object (with `read.traps`) and a captures dataframe, and then combining these with `make.caphist`. This more general route may be needed for unusual datasets.

9.6 Output

Function `secr.fit` returns an object of class **secr**. This is an R list with many components. Assigning the output to a named object saves both the fit and the data for further manipulation. Typing the object name at the R prompt invokes `print.secr` which formats the key results. These include the dataframe of estimates from the `predict` method for **secr** objects. Functions are provided for further computations on **secr** objects (e.g., AIC model selection, model averaging, profile-likelihood confidence intervals, and likelihood-ratio tests). Several of these were listed in Table 9.2.

One system of units is used throughout. Distances are in metres and areas are in hectares (ha). The unit of density for 2-dimensional habitat is animals per hectare. $1 \text{ ha} = 10000 \text{ m}^2 = 0.01 \text{ km}^2$. To convert density to animals per km^2 , multiply by 100. Density in linear habitats (see package [secrlinear](#)) is expressed in animals per km.

9.7 Documentation and support

The primary documentation for **secr** is in the help pages that accompany the package. Help for a function is obtained in the usual way by typing a question mark at the R prompt, followed by the function name. Note the ‘Index’ link at the bottom of each help page –

you will probably need to scroll down to find it. The index may also be accessed with `help(package = secr)`. Static and potentially outdated versions of the help pages are available [here](#).

The consolidated help pages are in the [manual](#). Searching this pdf is a powerful way to locate a function for a particular task.

Other documentation has traditionally been in the form of pdf vignettes built with **knitr** and available at <https://otago.ac.nz/density/SECRinR.html>. That content has been included in this book.

The [GitHub repository](#) holds the development version, and bugs may be reported there by raising an Issue. New versions will be posted on [CRAN](#) and noted on <https://www.otago.ac.nz/density/>, but there may be a delay. For information on changes in each version, type at the R prompt:

```
news (package = "secr")
```

Help may be sought in online forums such as [phidot](#) and [secrgroup](#).

9.8 Using `secr.fit`

We saw `secr.fit` in action in Chapter 2. Here we expand on particular arguments.

9.8.1 `buffer` – buffer width

Specifying the buffer width is an alternative to specifying a habitat mask. The choice of [buffer width](#) is discussed at length in Chapter 12.

9.8.2 `start` – starting values

[Numerical maximization](#) of the likelihood requires a starting value for each coefficient in the model. `secr.fit` relieves the user of this chore by applying an algorithm that works in most cases. The core of the algorithm is exported in function [autoini](#).

1. Compute an approximate bivariate normal σ from the 2-D dispersion of individual locations:

$$\sigma = \sqrt{\frac{\sum_{i=1}^n \sum_{j=1}^{n_i} [(x_{i,j} - \bar{x}_i)^2 + (y_{i,j} - \bar{y}_i)^2]}{2 \sum_{i=1}^n (n_i - 1)}},$$

where $(x_{i,j}, y_{i,j})$ is the location of the j -th detection of individual i . This is implemented in the function [RPSV](#) with `CC = TRUE`. The value is approximate because it ignores that detections are constrained by the locations of the detectors.

2. Find by numerical search the value of g_0 that with σ predicts the observed mean number of captures per individual (Efford, Dawson, et al., 2009, Appendix B).
3. Compute the [effective sampling area](#) $a(g_0, \sigma)$.
4. Compute $D = n/a(g_0, \sigma)$, where n is the number of individuals detected.

After transformation this provides intercepts on the link scale for the core parameters D , g_0 and σ . For hazard models λ_0 is first set to $-\log(1 - g_0)$. The starting values of further coefficients are set to zero on the link scale.

Users may provide their own starting values. These may be a vector of coefficients on the link scale, a named list of values for ‘real’ parameters, or a previously fitted model that includes some or all of the required coefficients.

9.8.3 model – detection and density sub-models

The core parameters are ‘real’ parameters in the terminology of MARK (Cooch & White, 2023). Three real parameters are commonly modelled in **secr**: ‘D’ (for density), and ‘g0’ and ‘sigma’ (for the detection function). Other ‘real’ parameters appear in particular contexts. ‘z’ is a shape parameter that is used only when the [detection function](#) has three parameters. Some detection functions primarily model the cumulative hazard of detection, rather than the probability of detection; these use the real parameter ‘lambda0’ in place of ‘g0’. A further ‘real’ parameter is the mixing proportion ‘pmix’, used in [finite mixture models](#) and [hybrid mixture models](#).

By default, each ‘real’ parameter is assumed to be constant over time, space and individual. We specify more interesting, and often better-fitting, models with the ‘model’ argument of **secr.fit**. Here ‘model’ refers to variation in a parameter that may be explained by known factors and covariates, perhaps better designated a ‘sub-model’ of the overarching SECR model (Chapter 3).

Sub-models are defined symbolically using a subset of the R formula notation. A separate linear predictor is used for each core parameter. The model argument of **secr.fit** is a list of formulae, one for each ‘real’ parameter. Null formulae such as $D \sim 1$ may be omitted, and a single non-null formula may be presented on its own rather than in list() form.

Sub-models are constructed differently for detection and density parameters as explained in Chapter 10 and Chapter 11.

9.8.4 CL – conditional vs full likelihood

‘CL’ switches between maximizing the likelihood conditional on n (TRUE) or the full likelihood (FALSE). The conditional option is faster because it does not estimate absolute density. Uniform absolute density may be estimated from the conditional fit, or indeed any fit, with the **derived** method. For Poisson n (the default), the estimate is identical within numerical error to that from the full likelihood. The alternative (binomial n) is obtained by setting the [details argument](#) ‘distribution = “binomial”’. Relative density (density modelled

as a function of covariates, without an intercept) may be modelled with `CL = TRUE` in recent versions (see sections on the [theory](#) and [implementation](#) of relative density).

9.8.5 method – maximization algorithm

Models are fitted in `secr.fit` by numerically maximizing the log-likelihood with functions from the **stats** package (R Core Team, 2024). The default method is ‘Newton-Raphson’ in the function `stats::nlm`. By default, all reported variances, covariances, standard errors and confidence limits are asymptotic and based on a numerical estimate of the information matrix, as described [here](#).

The Newton-Raphson algorithm is fast, but it sometimes fails to compute the information matrix correctly, causing some standard errors to be set to NA; see the ‘method’ argument of `secr.fit` for alternatives. Use `confint.secr` for profile likelihood intervals and `sim.secr` for parametric bootstrap intervals (both are slow).

Numerical maximization has some implications for the user. Computation may be slow, especially if there are many points in the mask, and estimates may be sensitive to the particular choice of mask (either explicitly in `make.mask` or implicitly via the ‘buffer’ argument).

9.8.6 ncores – multi-threading

On processors with multiple cores it is possible to speed up computation by using cores in parallel. This happens automatically in `secr.fit` and a few other functions using the multi-threading paradigm of **RcppParallel** (Allaire et al., 2023). The number of threads may be set directly with the ‘ncores’ argument or with the separate function `setNumThreads`. Either way, the number of threads is stored in the environment variable `RCPP_PARALLEL_NUM_THREADS`.

9.8.7 details – miscellaneous arguments

Many minor or infrequently used arguments are grouped as ‘details’. We mention only the most important ones here:

- `distribution`
- `fastproximity`
- `maxdistance`
- `fixedbeta`
- `LLonly`

See [?details](#) for a full list and description.

9.8.7.1 Distribution of n

This details argument switches between two possibilities for the distribution of n : ‘poisson’ (the default) or ‘binomial’. Binomial n conditions on fixed $N(A)$ where A is the area of the habitat mask. This corresponds to point process with a fixed number of activity centres inside an arbitrary boundary. Estimates of density conditional on $N(A)$ have lower variance, but this is usually an artifact of the conditioning and therefore misleading.

9.8.7.2 Fast proximity

Binary and count data collected over several occasions may be collapsed to a single occasion, under certain conditions. Collapsed data lead to the same estimates (e.g., Efford, Dawson, et al., 2009) with a considerable saving in execution time. Data from binary proximity detectors are modelled as binomial with size equal to the number of occasions. The requirement is that no information is lost that is relevant to the model. This really depends on the model: collapsed data are inadequate for time-dependent models, including those with behavioural responses (Chapter 10).

By default, data are automatically collapsed to speed up processing when the model allows. This is inconvenient if you wish to use AIC to compare a variety of models. The problem is solved by setting ‘details = list(fastproximity = FALSE)’ for all models. Fitting will be slower.

9.8.7.3 Individual mask subset

Integration by default is performed by summing over all mask cells for each individual. Cells distant from the detection locations of an individual contribute almost nothing to the likelihood, so it is efficient to limit the summation to a certain radius of the centroid of detections. This is achieved by specifying the details argument ‘maxdistance’. A radius similar to the buffer width is appropriate. See Section B.8 for an example.

9.8.7.4 Fixing coefficients

The ‘fixed’ argument of `secr.fit` has the effect of fixing one or more ‘real’ parameters. The ‘details’ component ‘fixedbeta’ provides control at a finer level: it may be used to fix certain coefficients while allowing others to vary. It is a vector of values, one for each coefficient *in the order they appear in the model*. Coefficients that are to be estimated (i.e. not fixed) are given the value NA. Check the order of coefficients by applying `coef` to a fitted model, or by starting to fit a model with `trace = TRUE`.

9.8.7.5 Single likelihood evaluation

Setting `LLonly = TRUE` returns a single evaluation of the likelihood at the parameter values specified in `start`.

9.8.7.6 Saving progress

If there is a risk of a run being terminated arbitrarily (e.g., because it exceeds system time limit) then it can be prudent to save intermediate steps in the likelihood maximisation. This is done by setting the details argument 'saveprogress'. Positive values cause the current data and coefficients to be saved to a permanent RDS file (default 'progress.RDS') at the given frequency. Saving can be costly, so an interval >1 is recommended. The name of the resulting file can be passed to `secl.refit` to resume fitting.

10 Detection model

The detection model in SECR most commonly models the probability that an individual with a particular activity centre will be detected on a particular occasion at a particular place (detector). If the detector type allows for multiple detections (cues or visits) the model describes the number of detections rather than probability.

A bare-bones detection model is a distance-detection function (or simply ‘detection function’) with two parameters, intercept and spatial scale. There are three sources of complexity and opportunities for customisation:

- function shape (e.g., halfnormal vs negative exponential) and whether it describes the probability or hazard of detection,
- parameters may depend on other known variables, and
- parameters may be modelled as random effects to represent variation of unknown origin.

We consider the shape of detection functions in the next section. Modelling parameters as a function of other variables is addressed in the following section on [linear submodels](#). Random effects in **secr** are limited to finite mixture models that we cover in the chapter on [individual heterogeneity](#).

10.1 Distance-detection functions

The probability of detection $g(d)$ at a detector distance d from an activity centre may take one of the simple forms in Table 10.1. Alternatively, the probability of detection may be derived from $g(d) = 1 - \exp[-\lambda(d)]$ where $\lambda(d)$ is the hazard of detection, itself modelled with one of the simple parametric forms (Table 10.2)¹. Several further options are provided by **secr.fit** (see ? **detectfn**), but only ‘HN’, ‘HR’, and ‘EX’ or their ‘hazard’ equivalents are commonly used.

Table 10.1: Probability detection functions.

Code	Name	Parameters	Function
HN	halfnormal	g_0, σ	$g(d) = g_0 \exp\left(\frac{-d^2}{2\sigma^2}\right)$

¹The transformation is non-linear so, for example, a half-normal form for $g(\cdot)$ does not correspond to half-normal form for $\lambda(\cdot)$.

Code	Name	Parameters	Function
HR	hazard rate ²	g_0, σ, z	$g(d) = g_0[1 - \exp\{-(d/\sigma)^{-z}\}]$
EX	exponential	g_0, σ	$g(d) = g_0 \exp\{-(d/\sigma)\}$

Table 10.2: Hazard detection functions.

Code	Name	Parameters	Function
HHN	hazard halfnormal	λ_0, σ	$\lambda(d) = \lambda_0 \exp\left(\frac{-d^2}{2\sigma^2}\right)$
HHR	hazard hazard rate	λ_0, σ, z	$\lambda(d) = \lambda_0(1 - \exp\{-(d/\sigma)^{-z}\})$
HEX	hazard exponential	λ_0, σ	$\lambda(d) = \lambda_0 \exp\{-(d/\sigma)\}$
HVP	hazard variable power	λ_0, σ, z	$\lambda(d) = \lambda_0 \exp\{-(d/\sigma)^z\}$

The merits of focussing on the hazard³ are a little arcane. We list them here:

1. Quantities on the hazard scale are additive and more tractable for some purposes (e.g. adjusting for effort, computing [expected counts](#)).
2. For some detector types (e.g., Poisson counts) the data are integers, for which $\lambda(d)$ has a direct interpretation as the expected count. However, $\lambda(d)$ can always be derived from $g(d)$ ($\lambda(d) = -\log[1 - g(d)]$).
3. Intuitively, there is a close proportionality between $\lambda(d)$ and the height of an individual's utilization pdf.

The ‘hazard variable power’ function is a 3-parameter function modelled on that of Ergon & Gardner (2013). The third parameter allows for smooth variation of shape, including both HHN ($z = 2$) and HEX ($z = 1$) as special cases.

Tip

Using the ‘complementary log-log’ link function cloglog for a hazard detection function such as $\lambda(d) = \lambda_0 \exp[-d^2/(2\sigma^2)]$ is equivalent to modelling λ with a log link, as $p = 1 - \exp(-\lambda)$ and $\lambda = -\log(1 - p)$. For a halfnormal function, the quantity y on the cloglog scale is then a linear function of $\exp(-d^2)$ with intercept $\alpha_0 = \log(\lambda_0)$ and slope $\alpha_1 = -1/(2\sigma^2)$ (e.g., Royle, Chandler, Sun, et al., 2013).

10.1.1 Choice of detection function not critical

The variety of detection functions is daunting. You could try them all and select the “best” by AIC, but we do not recommend this. Fortunately, the choice of function is not critical. We

²This use of ‘hazard’ has historical roots in distance sampling (Hayes & Buckland, 1983) and has no real connection to models for hazard as a function of distance.

³Technically this is the cumulative hazard rather than the instantaneous hazard, but we get tired of using the full term.

illustrate this with the snowshoe hare dataset of Chapter 2. The function `list.secr.fit` is used to fit a series of models. Warnings due to the use of a multi-catch likelihood for single-catch traps are suppressed both here and in other examples.

```
df <- c('HHN','HHR','HEX','HVP')
fits <- list.secr.fit(detectfn = df, constant = list(capthist = hareCH6,
  buffer = 250, trace = FALSE), names = df)
```

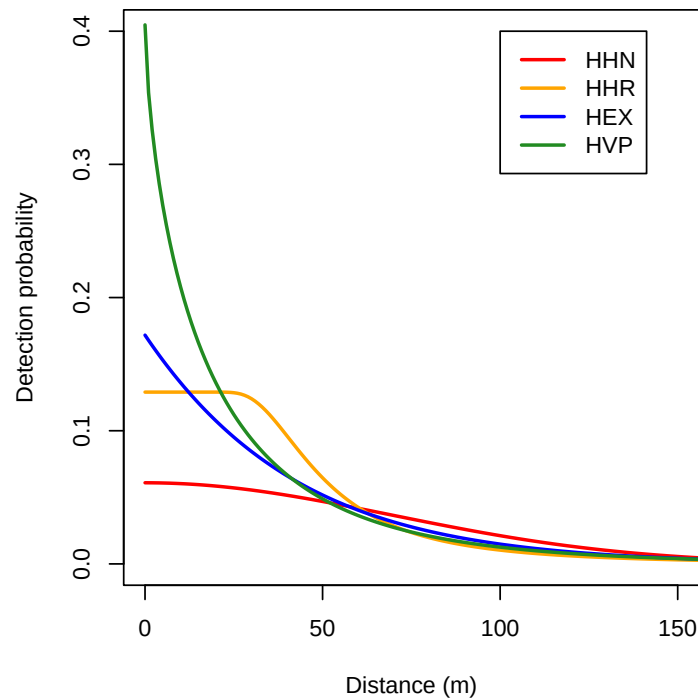


Figure 10.1: Four detection functions fitted to snowshoe hare data.

The relative fit of the HHR, HVP and HEX models is essentially the same, whereas HHN is distinctly worse:

```
AIC(fits)[, c(2,3,4,7,8)]
```

##		<i>detectfn</i>	<i>npar</i>	<i>logLik</i>	<i>dAIC</i>	<i>AICwt</i>
##	HHR	<i>hazard hazard rate</i>	4	-599.68	0.000	0.5474
##	HVP	<i>hazard variable power</i>	4	-600.43	1.487	0.2603
##	HEX	<i>hazard exponential</i>	3	-601.73	2.092	0.1923
##	HHN	<i>hazard halfnormal</i>	3	-608.07	14.783	0.0000

The third parameter z was estimated as 3.08 for HHR, and 0.67 for HVP.

Fitting the HVP function with z fixed to different values is another way to examine the effect of shape (Fig. 10.3). Density estimates ranged only from 1.47 to 1.42.

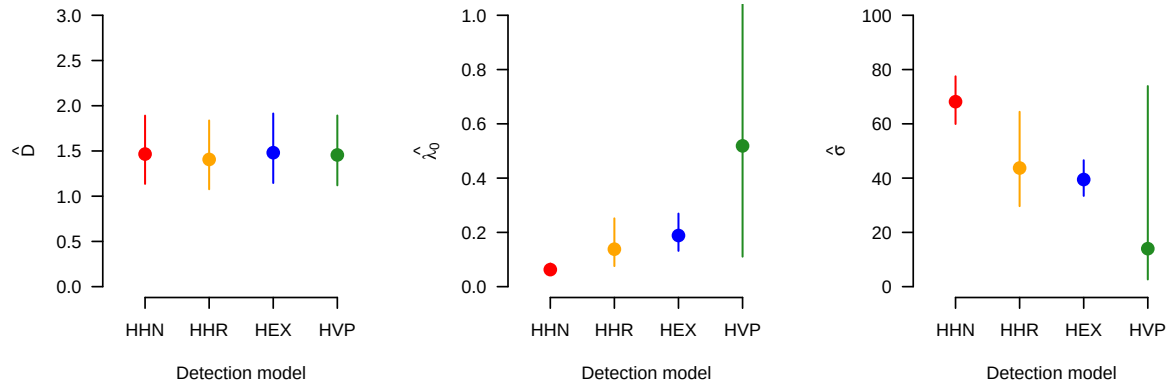


Figure 10.2: Parameter estimates from four detection functions (95% CI).

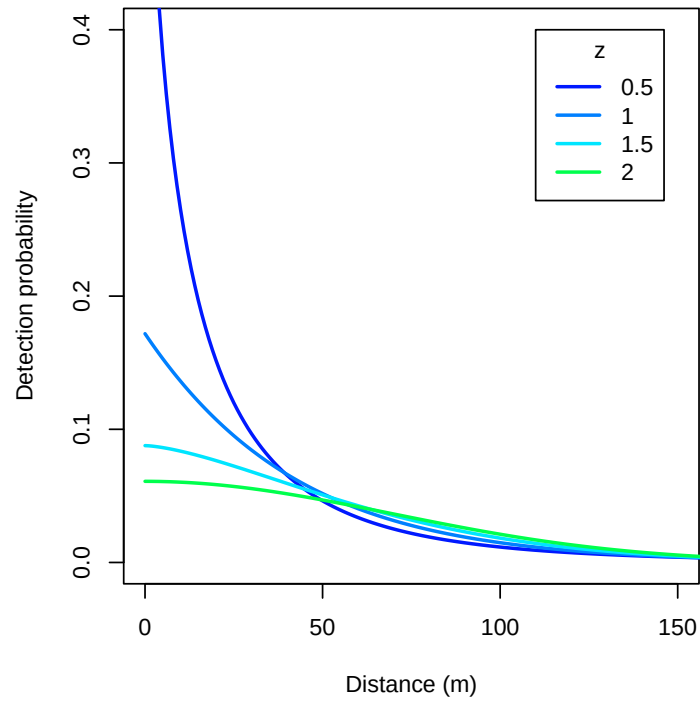


Figure 10.3: HVP function fitted to snowshoe hare data with z parameter fixed at four different values.

How can different functions produce nearly the same estimates? Remember that $\hat{D} = n/a(\hat{\theta})$, and n is the same for all models. Constant $\hat{D}$ therefore implies constant effective sampling area $a(\hat{\theta})$. In other words, variation in $\hat{\lambda}_0$ and $\hat{\sigma}$ ‘washes out’ when they are combined in $a(\hat{\theta})$. Under the four models $a(\hat{\theta})$ is estimated as 46.4, 48.4, 45.9 and 46.7 ha.

10.1.2 Detection parameters are mostly nuisance parameters

The detection model and its parameters (g_0 , λ_0 , σ etc.) provide the link between our observations and the state of the animal population represented by the parameter D (density, distribution in space, trend etc.). Sub-models for D are considered in Chapter 11. But what interpretation should we attach to the detection parameters themselves?

The intercept g_0 is in a sense the probability that an animal will be detected at the centre of its home range. The spatial scale of detection σ relates to the size of the home range. These attributed meanings can aid intuitive understanding. However, we advise against a literal reading. The estimates have meaning only for a specified detection function and cannot meaningfully be compared across functions. Observe the different intercepts of the functions in Fig. 10.2.

The halfnormal function is closest to a standard reference, but estimates of halfnormal σ are sensitive to infrequent large movements. Care is also needed because some early writers omitted the factor 2 from the denominator, increasing estimates of σ by $\sqrt{2}$ ⁴.

Continuing the snowshoe hare example: the estimates of λ_0 and σ from HVP are surprisingly uncertain when considered on their own, yet the HVP estimate of density has about the same precision as other detection functions (Fig. 10.2). How can this be? There is strong covariation in the sampling distributions of the two parameters that we plot using `ellipse.secr` in Fig. 10.4.

10.1.3 SECR is not distance sampling

The idea of a distance-detection function originated in distance sampling (Buckland et al., 2001) and Borchers et al. (2015) provided a unified framework for spatially explicit capture–recapture and distance sampling. Nevertheless, the role of the detection function differs substantially.

In distance sampling, shape matters a lot. In particular, the estimate of density depends on the slope of the detection function near the origin, given the assumption that all animals at the origin are detected (e.g., Buckland et al., 2015).

In SECR, no special significance is attached to the intercept or the shape of the function. The detection function serves as a spatial filter for a modelled 2-dimensional point pattern of activity centres; the filter must ‘explain’ the frequency of recaptures and their spatial spread. These are the components of the effective sampling area.

⁴Examples are Gardner et al. (2009), Royle & Gardner (2011) and Royle et al. (2011), but not Royle et al. (2014) and Royle et al. (2015)).

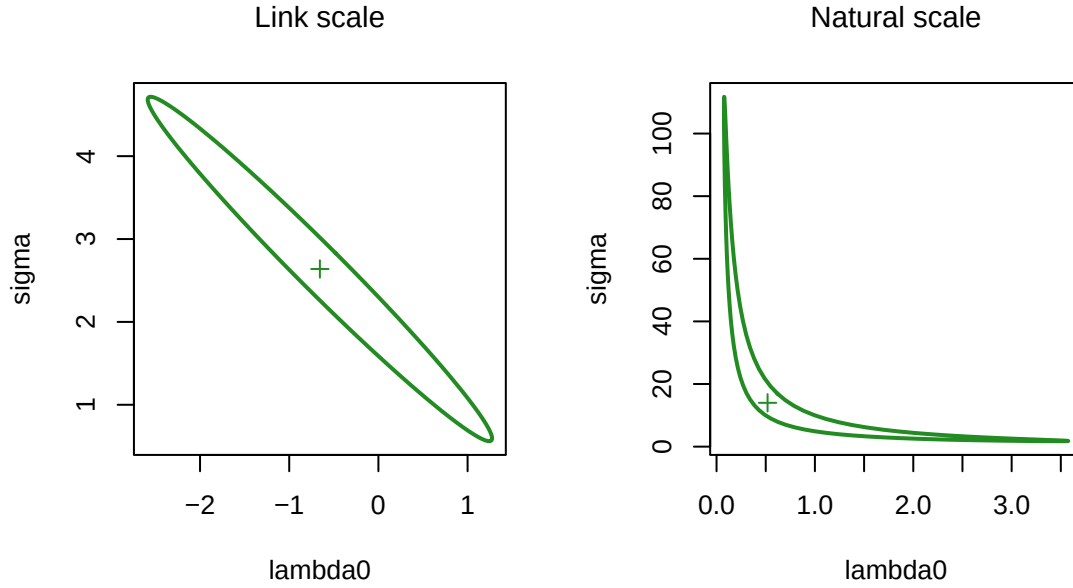


Figure 10.4: Confidence ellipse for HVP detection parameters plotted on the link scale and back-transformed to natural scale. ‘+’ indicates MLE.

The hazard-rate function HR is recommended for distance sampling because it has a distinct ‘shoulder’ near zero distance, and distance sampling is not concerned with the tail (distant observations are often censored). SECR relies on the tail flattening to zero within the region of integration defined by the [habitat mask](#). Otherwise, the population at risk of detection is determined by the choice of mask, which is usually arbitrary and *ad hoc*. The hazard-rate function has an extremely long tail (it is not convergent), so there is always a risk of mask-dependence. As an aside - in the snowshoe hare example with detection function HHR we suppressed the warning “predicted relative bias exceeds 0.01 with buffer = 250” that is due to truncation of the long tail.

10.1.4 Why bother?

Given the preceding comments you may wonder why we bother with different detection functions at all. In part this is historical: it was not obvious in the beginning that density estimates were so robust. Sometimes it’s just nice to have the flexibility to match the model to animal behaviour. Functions with longer tails (e.g., HEX) accommodate occasional extreme movements that can prevent a short-tailed function (HHN) from fitting at all.

Also, it is desirable to account for any significant lack of fit due to the detection function before modelling effects that may have a more critical effect on density estimates, such as individual heterogeneity and learned responses.

10.2 Detection submodels

Until now we have assumed that there is a single beta parameter for each real parameter. A much richer set of models is obtained by treating each real parameter as a function of covariates. For convenience, the function is linear on the appropriate [link](#) scale. The single ‘beta’ coefficient is then replaced by two or more coefficients (e.g., intercept β_0 and slope β_1 of the linear relationship $y = \beta_0 + \beta_1 x_1$ where y is a parameter on the link scale and x_1 is a covariate). Suppose, for example, that y depends on sampling occasion s then $y(s) = \beta_0 + \beta_1 x_1(s)$ and the corresponding real parameter is $y(s)$ back transformed from the link scale.

This may be generalised using the notation of linear models,

$$\mathbf{y} = \mathbf{X}\boldsymbol{\beta}, \quad (10.1)$$

where $\mathbf{X}$ is the design matrix, $\boldsymbol{\beta}$ is a vector of coefficients, and $\mathbf{y}$ is the resulting vector of values on the link scale, one for each row of $\mathbf{X}$. The first column of the design matrix is a column of 1’s for the intercept β_0 . Factor (categorical) predictors will usually be represented by several columns of indicator values (0’s and 1’s coding factor levels). See Cooch & White (2023) Chapter 6 for an accessible introduction to linear models and design matrices.

In **secr** each detection parameter $(g_0, \lambda_0, \sigma, z)$ is controlled by a linear sub-model on its link scale, i.e. each has its own design matrix. *Rows* of the design matrix correspond to combinations of session, individual, occasion, and detector, omitting any of these four that is constant (perhaps because there is only one level). Finite-mixture models add further rows to the design matrix that we leave aside for now. *Columns* after the first are either (i) indicators to represent effects that can be constructed automatically (Table 10.3), or (ii) user-supplied covariates associated with sessions, individuals, occasions or detectors.

Table 10.3: Automatically generated predictor variables for detection models

Variable	Description	Notes
g	group	groups are defined by the individual covariate(s) named in the ‘groups’ argument
t	time factor	one level for each occasion
T	time trend	linear trend over occasions on link scale
b	learned response	step change after first detection
B	transient response	depends on detection at preceding occasion (Markovian response)
bk	animal x site response	site-specific step change
Bk	animal x site response	site-specific transient response
k	site learned response	site effectiveness changes once any animal caught

Variable	Description	Notes
K	site transient response	site effectiveness depends on preceding occasion
session	session factor	one level for each session
Session	session trend	linear trend on link scale
h2	2-class mixture	finite mixture model with 2 latent classes
ts	marking vs sighting	two levels (marking and sighting occasions)

Each design matrix is constructed automatically when `secr.fit` is called, using the data and a model formula. Computation of the linear predictor (Eq. 10.1) and back-transformation to the real scale are also automatic: the user need never see the design matrix.

i Spatially varying detection

The hard-wired structure of the design matrices precludes some possible sub-models: there is no direct way to model *spatial* variation in a detection parameter. This was a choice made in the design of the software. It aimed to tame the complexity and resource demands that would result if λ_0 , g_0 and σ were allowed to vary continuously in space. However, spatial effects may be modelled efficiently using detector-level covariates, i.e. as a function of detector location rather than AC location, and a further workaround for parameter σ is shown in Appendix F.

The formula may be constant (~ 1 , the default) or some combination of terms in standard R formula notation (see `?formula`). For example, $g_0 \sim b + T$ specifies a model with a learned response and a linear time trend in g_0 ; the effects are additive on the link scale. Table 10.4 has some examples.

Table 10.4: Some examples of the ‘model’ argument in `secr.fit`

Formula	Effect
$g_0 \sim 1$	g_0 constant across animals, occasions and detectors
$g_0 \sim b$	learned response affects g_0
$\text{list}(g_0 \sim b, \sigma \sim b)$	learned response affects both g_0 and σ
$g_0 \sim h2$	2-class finite mixture for heterogeneity in g_0
$g_0 \sim b + T$	learned response in g_0 combined with trend over occasions
$\sigma \sim g$	detection scale σ differs between groups
$\sigma \sim g * T$	group-specific trend in σ

The common question of how to model sex differences can be answered in several ways. we devote Chapter 16 to the possibilities (groups, individual covariate, hybrid mixtures etc.).

i Note

Linear sub-models for parameters are considered by Cooch & White (2023) as a *constraint* on a more general model. Their default is for each parameter to be fully-time-specific e.g., a Cormack-Jolly-Seber open population survival model would fit a unique detection probability p and survival rate ϕ at each time. Our default is for each parameter to be constant (i.e. maximally constrained), and for linear sub-models to introduce variation.

10.2.1 Covariates

Any name in a formula that is not listed as a variable in Table 10.3 is assumed to refer to a user-supplied covariate. `secr.fit` looks for user-supplied covariates in data frames embedded in the ‘capthist’ argument, or supplied in the ‘timecov’ and ‘sessioncov’ arguments, or named with the ‘timevaryingcov’ attribute of a traps object, using the first match (Table 10.5).

Table 10.5: Types of user-provided covariate for parameters of detection models. The names of columns in the respective dataframes may be used in model formulae. Time-varying detector covariates are a special case considered below.

Covariate type	Data source
Individual	covariates(capthist)
Time	timecov argument
Detector	covariates(traps(capthist))
Detector x Time	covariates(traps(capthist))
Session	sessioncov argument

A continuous covariate that takes many unique values poses problems for the implementation in `secr`. A multiplicity of values inflates the size of internal lookup tables, both slowing down each likelihood evaluation and potentially exceeding the available memory⁵. A binned covariate should do the job equally well, while saving time and space (see function `binCovariate`).

⁵In the C++ code, two real-valued 3-dimensional arrays are populated with pre-computed values of $p_{sk}(\mathbf{x})$ (gk) and $h_{sk}(\mathbf{x})$ (hk). The dimensions are the number of unique parameter combinations C , the number of detectors K and the number of mask points M . The memory requirement for these arrays alone is $2.8.C.K.M$ bytes, which for 200 detectors, 10000 mask points, and 100 parameter levels is 3.2 Gb. This is on top of the two parameter index arrays requiring $2.4.R.n.S.K.U$ bytes for R sessions and U mixture classes (e.g. 10 sessions, 200 animals, 6 occasions, 200 detectors and 2 mixture classes, 0.0384 Gb), and a number of smaller objects.

10.2.2 Time-varying trap covariates

A special mechanism is provided for detector-level covariates that take different values on each occasion. Then we expect the dataframe of detector covariates to include a column for each occasion.

A ‘traps’ object may have an attribute ‘timevaryingcov’ that is a list in which each named component is a vector of indices identifying which covariate column to use on each occasion. The name may be used in model formulae. Use `timevaryingcov()` to extract or replace the attribute.

10.2.3 Regression splines

Modelling a link-linear⁶ relationship between a covariate and a parameter may be too restrictive.

Regression splines are a very flexible way to represent non-linear responses in generalized additive models, implemented in the R package **mgcv** (Wood, 2006). Borchers & Kidney (2014) showed how they may be used to model 2-dimensional trend in density. They used **mgcv** to construct regression spline basis functions from mask x- and y-coordinates, and possibly additional mask covariates, and then passed these as covariates to **secr.fit**. Smooth, semi-parametric responses are also useful for modelling variation in detection parameters such as g_0 and σ over time, or in response to numeric individual- or detector-level covariates, when (1) a linear or other parametric response is arbitrary or implausible, and (2) sampling spans a range of times or levels of the covariate(s).

Smooth terms may be used in **secr** model formulae for both density and detection parameters. The covariate is merely wrapped in a call to the smoother function `s()`. Smoothness is controlled by the argument ‘k’.

For a concrete example, consider a population sampled monthly for a year (i.e. 12 ‘sessions’). If home range size varies seasonally then the parameter sigma may vary in a more-or-less sinusoidal fashion. A linear trend is obviously inadequate, and a quadratic is not much better. However, a sine curve is hard to fit (we would need to estimate its phase, amplitude, mean and spatial scale) and assumes the increase and decrease phases are equally steep. An extreme solution is to treat month as a factor and estimate a separate parameter for each level (month). A smooth (semi-parametric) curve may capture the main features of seasonal variation with fewer parameters.

There are some drawbacks to using smooth terms. The resulting fitted objects are large, on account of the need to store setup information from **mgcv**. The implementation may change in later versions of **mgcv** and **secr**, and smooth models fitted now will not necessarily be compatible with later versions. Setting the intercept of a smooth to zero is not a canned option in **mgcv**, and is not offered in **secr**. It may be achieved by placing a knot at zero and hacking the matrix of basis functions to drop the corresponding column, plus some more jiggling.

⁶We use ‘link-linear’ to describe a linear model on the link scale, where this may be log-linear, logit-linear etc.

10.2.4 Why bother?

Detailed modelling of detection parameters may be a waste of energy for the same reasons that the [choice of detection function](#) itself has limited interest. See, for example, the simulation results of Sollmann (2024) on occasion-specific models ($\sim t$). However, behavioural responses and individual heterogeneity can have a major effect on density estimates, and these deserve attention.

10.2.4.1 Behavioural responses

An individual behavioral response is a change in the probability or hazard of detection on the occasions that follow a detection. Trapping of small mammals provides evidence of species that routinely become trap happy (presumably because they enjoy the bait) or trap shy (presumably because the experience of capture and handling is unpleasant). Positive or negative responses are modelled as a step change in a detection parameter, usually the intercept of the detection function (g_0, λ_0).

The response may be permanent (b) or transient (B) (i.e. applying only on the next occasion). In spatial models we also distinguish between a global response, across all detectors, and a local response, specific to the initial detector (suffix 'k'). This leads to four response models: b, bk, B, and Bk.

We explore these options with Reid's Wet Switzer Gulch deer mouse (*Peromyscus maniculatus*) dataset from Otis et al. (1978). Mice were trapped on a grid of 99 traps over 6 days. The Sherman traps were treated as multi-catch traps for this analysis. We fit the four behavioural response models and the null model to the morning data.

```
cm0d <- paste0('g0~', c('1','b','B','bk','Bk'))
# convert each character string to a formula and fit the models
fits <- list.secr.fit(model = sapply(cm0d, formula), constant =
  list(capthist = 'deermouse.WSG', trace = FALSE, buffer = 80),
  names = cm0d)
AIC(fits, sort = FALSE)[c(3:5,7,8)]
```

	npar	logLik	AIC	dAIC	AICwt
g0~1	3	-663.54	1333.1	129.938	0
g0~b	4	-643.72	1295.4	92.298	0
g0~B	4	-651.62	1311.2	108.099	0
g0~bk	4	-597.57	1203.1	0.000	1
g0~Bk	4	-621.02	1250.0	46.910	0

All response models are preferred to the null model, but the differences among them are marked: the evidence supports a persistent local response (bk). The density estimates for bk and Bk are close to the null model, whereas the b and B estimates are greater. In our experience this is a common result: a local response is preferred by AIC and has less impact

on density estimates than a global response, and there is little penalty for omitting the response from the model.

```
collate(fits, realnames = 'D')[1,,]
```

	estimate	SE.estimate	lcl	ucl
g0~1	14.089	2.0364	10.629	18.676
g0~b	18.531	3.3460	13.044	26.324
g0~B	15.726	2.3351	11.774	21.005
g0~bk	13.884	2.1108	10.324	18.672
g0~Bk	13.883	2.0472	10.414	18.506

The estimated magnitude of the responses may be examined with `predict(fits[2:5], all.levels = TRUE)` but the output is long and we show only the global (b) and local (bk) enduring responses:

```
$`g0~b`
              estimate    lcl    ucl
session = WSG, b = 0    0.055 0.031 0.096
session = WSG, b = 1    0.210 0.163 0.267

$`g0~bk`
              estimate    lcl    ucl
session = WSG, bk = 0   0.061 0.044 0.085
session = WSG, bk = 1   0.596 0.454 0.723
```

Here ‘b = 0’ and ‘bk = 0’ refer to g_0 for a naive animal and ‘b = 1’ and ‘bk = 1’ refer to the post-detection values (estimates are shown with 95% limits). It appears that deermice are highly likely to return to traps where they have been caught.

Detector-level ‘behavioural’ response is also possible (predictors k, K in `secre.fit`). Detection of any individuals at a detector may in principle be facilitated or inhibited by a previous detection there of any other individual. We are not aware of published examples.

The trap-facilitation model fitted to the deer mouse data results in a larger and less precise estimate of density (16.78/ha, SE 2.59/ha), with AIC intermediate between the null model and bk (facilitation model $\Delta\text{AIC} = 73.7$ relative to bk). There is a risk of confusing such an effect with simple heterogeneity in the performance of detectors or clumping of activity centres or an individual local response (bk). More investigation is needed.

10.3 Varying effort

Researchers are often painfully aware of glitches in their data gathering - traps that were not set, sampling occasions missed or delayed due to weather etc. Even when the actual estimates are robust, as in an example below, it is desirable (therapeutic and scientific) to allow for known irregularities in the data. This is the role of the ‘usage’ matrix as [described](#) in Chapter 3.

The ‘usage’ attribute of a ‘traps’ object in **secr** is a $K \times S$ matrix recording the effort (T_{sk}) at each detector $k = 1 \dots K$ and occasion $s = 1 \dots S$. Effort may be binary (0/1) or continuous. If the attribute is missing (NULL) it will be treated as all ones. Extraction and replacement functions are provided (`usage` and `usage<-`, as demonstrated below). All detector types accept usage data in the same format. The usage matrix for polygon and transect detectors has one row for each polygon or transect, rather than one row per vertex.

Binomial count detectors are a special case. When the `secr.fit` argument `binomN = 1`, or equivalently `binomN = ‘usage’`, usage is interpreted as the size of the binomial distribution (the maximum possible number of detections of an animal at a detector on one occasion).

10.3.1 Input of usage data

Usage data may be input as extra columns in a file of detector coordinates (see `?read.traps` and [secr-datainput.pdf](#)).

Usage data also may be added to an existing traps object, even after it has been included in a capthist object. For example, the traps object in the demonstration dataset ‘captdata’ starts with no usage attribute, but we can add one. Suppose that traps 14 and 15 were not set on occasions 1–3. We construct a binary usage matrix and assign it to the traps object like this:

```
K <- nrow(traps(captdata))
S <- ncol(captdata)
mat <- matrix(1, nrow = K, ncol = S)
mat[14:15,1:3] <- 0 # traps 14:15 not set on occasions 1:3
usage(traps(captdata)) <- mat
```

10.3.2 Models

The usage attribute of a traps object is applied automatically by `secr.fit`. Following on from the preceding example, we can confirm our assignment and fit a new model.

```
summary(traps(captdata)) # confirm usage attribute
```

```

Object class      traps
Detector type     single
Detector number   100
Average spacing   30 m
x-range           365 635 m
y-range           365 635 m

```

```

Usage range by occasion
  1 2 3 4 5
min 0 0 0 1 1
max 1 1 1 1 1

```

```

fit <- secr.fit(captdata, buffer = 100, trace = FALSE, biasLimit = NA)
predict(fit)

```

	link	estimate	SE.estimate	lcl	ucl
D	log	5.47347	0.645992	4.34660	6.89249
g0	logit	0.27472	0.027164	0.22479	0.33102
sigma	log	29.39669	1.308421	26.94206	32.07494

The result in this case is only subtly different from the model with uniform usage (compare `predict(secrdemo.0)`). Setting `biasLimit = NA` avoids a warning message from `secr.fit` regarding `bias.D`: this function is usually run by `secr.fit` after any model fit using the ‘buffer’ argument, but it does not handle varying effort.

Usage is hardwired and will be applied whenever a model is fitted. There are two ways to suppress this. The first is to remove the usage attribute (`usage(traps(captdata)) <- NULL`). The second is to bypass the attribute for a single fit by calling `secr.fit` with ‘details = list(ignoreusage = TRUE)’.

For a more informative example, we simulate data from an array of binary proximity detectors (such as automatic cameras) operated over 5 occasions, using the default density (5/ha) and detection parameters ($g_0 = 0.1$, $\sigma = 25$ m) in `sim.caphist`. We choose to expose all detectors twice as long on occasions 2 and 3 as on occasion 1, and three times as long on occasions 4 and 5:

```

simgrid <- make.grid(nx = 10, ny = 10, detector = 'proximity')
usage(simgrid) <- matrix(c(1,2,2,3,3), byrow = TRUE, nrow = 100,
  ncol = 5)
simCH <- sim.caphist(simgrid, popn = list(D = 5, buffer = 100),
  detectpar = list(g0 = 0.1, sigma = 25), noccasions = 5,
  seed = 123)
summary(simCH)

```



```
Object class      capthist
Detector type     proximity (5)
Detector number   100
Average spacing   20 m
x-range           0 180 m
y-range           0 180 m
```

```
Usage range by occasion
  1 2 3 4 5
min 1 2 2 3 3
max 1 2 2 3 3
```

```
Counts by occasion
      1    2    3    4    5 Total
n      15   18   23   29   23   108
u      15    7    6    7    1    36
f       8    6    9    4    9    36
M(t+1)  15   22   28   35   36    36
losses    0    0    0    0    0     0
detections 26   32   39   54   55   206
detectors visited 24  28  33  41  44   170
detectors used   100 100 100 100 100   500
```

Now we fit four models with a half-normal detection function. The first model (`fit.null`) has no adjustment because we ignore the usage information. The second (`fit.usage`) automatically adjusts for effort. The third (`fit.tcov1`) again ignores effort, but fits a distinct g_0 for each level of effort. The fourth (`fit.tcov2`) uses a numerical covariate equal to the known effort. The setting `fastproximity = FALSE` allows all models can be compared by AIC.

```
# shared arguments for model fits 1-4
timedf <- data.frame(tfactor = factor(c(1,2,2,3,3)), tnumeric =
  c(1,2,2,3,3))
args <- list(capthist = simCH, buffer = 100, biasLimit = NA,
  timecov = timedf, trace = FALSE)
models <- c(g0 ~ 1, g0 ~ 1, g0 ~ tfactor, g0 ~ tnumeric)
details <- rep(list(list(ignoreusage = TRUE, fastproximity =
  FALSE)), 4)
details[[2]]$ignoreusage <- FALSE

# review arguments
data.frame(model = format(models), ignoreusage = sapply(details,
  '[[', 'ignoreusage'))
```

```
      model ignoreusage
1      g0 ~ 1         TRUE
```

```

2      g0 ~ 1      FALSE
3  g0 ~ tfactor    TRUE
4  g0 ~ tnumeric   TRUE

```

```

# fit
fits <- list.secr.fit(model = models, details = details, constant =
  args, names = c('null','usage','tfactor','tnumeric'))
AIC(fits)[,-c(2,5,6)]

```

	model	npar	logLik	dAIC	AICwt
usage	D~1 g0~1 sigma~1	3	-737.20	0.000	0.4212
tnumeric	D~1 g0~tnumeric sigma~1	4	-736.24	0.072	0.4063
tfactor	D~1 g0~tfactor sigma~1	5	-736.09	1.785	0.1725
null	D~1 g0~1 sigma~1	3	-744.90	15.390	0.0000

From the likelihoods we can see that failure to allow for effort (model ‘null’) dramatically reduces model fit. The model with a factor covariate (‘tfactor’) captures the variation in detection probability, but at the cost of fitting two additional parameters. The model with built-in adjustment for effort (‘usage’) has AIC similar to one with effort as a numeric covariate (‘tnumeric’). How do the estimates compare? This is a task for the `collate` function.

```
collate(fits, newdata = timedf)[,,'estimate','g0']
```

	null	usage	tfactor	tnumeric
tfactor=1,tnumeric=1	0.21001	0.10112	0.13349	0.12551
tfactor=2,tnumeric=2	0.21001	0.10112	0.18047	0.18851
tfactor=2,tnumeric=2	0.21001	0.10112	0.18047	0.18851
tfactor=3,tnumeric=3	0.21001	0.10112	0.27772	0.27324
tfactor=3,tnumeric=3	0.21001	0.10112	0.27772	0.27324

The ‘null’ model fits a single g_0 across all occasions that is approximately twice the true rate on occasion 1 (0.1). The estimates of g_0 from ‘tfactor’ and ‘tnumeric’ mirror the variation in effort. The effort-adjusted ‘usage’ model estimates the fundamental rate for one unit of effort (0.1).

```
collate(fits)[,,'D']
```

	estimate	SE.estimate	lcl	ucl
null	4.9709	0.84830	3.5663	6.9287
usage	4.9755	0.84881	3.5699	6.9344
tfactor	4.9703	0.84811	3.5660	6.9277
tnumeric	4.9698	0.84805	3.5656	6.9271

The density estimates themselves are almost entirely unaffected by the choice of model for g_0 . This is not unusual (Sollmann, 2024). Nevertheless, the example shows how ‘usage’ allows unbalanced data to be analysed with a minimum of fuss.

10.3.3 Further notes on varying effort

1. Adjustment for varying effort will be more critical in analyses where (i) the variation is confounded with temporal (between-session) or spatial variation in density, and (ii) it is important to estimate the temporal or spatial pattern. For example, if detector usage was consistently high in one part of a landscape, while true density was constant, failure to allow for varying usage might produce a spurious density pattern.
2. The units of usage determine the units of g_0 or λ_0 in the fitted model. This must be considered when choosing starting values for likelihood maximization. Ordinarily one relies on `secr.fit` to determine starting values automatically (via `autoini`), and a simple linear adjustment for usage, averaged across non-zero detectors and occasions, is applied to the value of g_0 from `autoini`.
3. When occasions are collapsed or detectors are lumped with the `reduce` method for capthist objects, usage is summed for each aggregated unit.
4. The function `usagePlot` displays a bubble plot of spatially varying detector usage on one occasion. The arguments ‘markused’ and ‘markvarying’ of `plot.traps` may also be useful.
5. Absolute duration does not always equate with effort. Animal activity may be concentrated in part of the day, or older DNA samples from hair snares may fail to amplify (Efford et al., 2013).
6. Binary or count data from searches of polygons or transects (Efford, 2011) do not raise any new issues for including effort, at least when effort is homogeneous across each polygon or transect. Effects of varying polygon or transect size are automatically accommodated in the models of Chapter 4. Models for varying effort within polygons or transects have not been needed for problems encountered to date. Such variation might in any case be accommodated by splitting the searched areas or transects into smaller units that were more nearly homogeneous (see the `snip` function for splitting transects).

11 Density model

Spatially explicit capture–recapture models allow for population density to vary over space (Borchers & Efford, 2008). Density is the intensity of a spatial Poisson process for activity centres. Models for density may include spatial covariates (e.g., vegetation type, elevation) and spatial trend.

In Chapter 10 we introduced [linear sub-models](#) for detection parameters. Here we consider SECR models in which the population density at any point, considered on the link scale, is also a linear function of V covariates¹. For density this means

$$D(\mathbf{x}; \phi) = f^{-1}[\phi_0 + \sum_{v=1}^V c_v(\mathbf{x}) \phi_v], \quad (11.1)$$

where $c_v(\mathbf{x})$ is the value of the v -th covariate at point $\mathbf{x}$, ϕ_0 is the intercept, ϕ_v is the coefficient for the v -th covariate, and f^{-1} is the inverse of the link function. Commonly we model the logarithm of density and f^{-1} is the exponential function.

Although $D(\mathbf{x}; \phi)$ is often a smooth function, in **secr** we evaluate it only at the fixed points of the habitat mask (Chapter 12). A mask defines the region of habitat relevant to a particular study: in the simplest case it is a buffered zone inclusive of the detector locations. More complex masks may exclude interior areas of non-habitat or have an irregular outline.

A density model $D(\mathbf{x}; \phi)$ is specified in the ‘model’ argument of **secr.fit**². Spatial covariates, if any, are needed for each mask point; they are stored in the ‘covariates’ attribute of the mask. Results from fitting the model (including the estimated coefficients ϕ) are saved in an object of class ‘secr’. To visualise a fitted density model we first evaluate it at each point on a mask with the function **predictDsurface**. This creates an object of class ‘Dsurface’. A Dsurface is a mask with added density data, and plotting a Dsurface is like plotting a mask covariate.

Table 11.1: Some examples of models for density in **secr.fit**.

Formula	Effect
$D \sim \text{cover}$	density varies with ‘cover’, a variable in covariates(mask)
$\text{list}(D \sim g, g0 \sim g)$	both density and $g0$ differ between groups

¹We can also express the model as before $\mathbf{y} = \mathbf{X}\boldsymbol{\beta}$, where $\mathbf{X}$ is the design matrix, $\boldsymbol{\beta}$ is a vector of coefficients, and $\mathbf{y}$ is the resulting vector of densities on the link scale. Rows of $\mathbf{X}$ and elements of $\mathbf{y}$ correspond to points on the habitat mask, possibly replicated in the case of group and session effects.

²It may also be specified in a user-written function supplied to **secr.fit** (see Appendix I), but you are unlikely to need this.

Formula	Effect
D ~ session	session-specific density

11.1 Absolute vs relative density

The conventional approach to density surfaces is to fit a model of absolute density by maximizing the ‘full’ likelihood. Until recently, maximizing the likelihood [conditional on \$n\$](#) , the number detected, was thought to work only when density was assumed to be uniform (homogeneous), placing it outside the scope of this chapter. However, recent extensions to **secr** allow models of *relative* density to be fitted by maximizing the conditional likelihood (CL = TRUE in **secr.fit**), and this has some advantages (Efford, 2025c). Relative density differs from absolute density by a constant factor. This chapter stresses modelling absolute density and keeps [relative density](#) for a section at the end.

11.2 Brushtail possum example

For illustration we use a brushtail possum (*Trichosurus vulpecula*) dataset from the Orongorongo Valley, New Zealand. Possums were live-trapped in mixed evergreen forest near Wellington for nearly 40 years (Efford & Cowan, 2004). Single-catch traps were set for 5 consecutive nights, three times a year. The dataset ‘OVpossumCH’ has data from the years 1996 and 1997. The study grid was bounded by a shingle riverbed to the north and west. See ?OVpossum in **secr** for more details.

First we import data for the habitat mask from a polygon shapefile included with the package:

```
datadir <- system.file("extdata", package = "secr")
OVforest <- sf::st_read(paste0(datadir, "/OVforest.shp"),
  quiet = TRUE)
# drop points we don't need
leftbank <- read.table(paste0(datadir, "/leftbank.txt"))[21:195,]
options(digits = 6, width = 95)
```

OVforest is now a simple features (sf) object defined in package **sf**. We build a habitat mask object, selecting the first two polygons in OVforest and discarding the third that lies across the river. The attribute table of the shapefile (and hence OVforest) includes a categorical variable ‘forest’ that is either ‘beech’ (*Nothofagus* spp.) or ‘nonbeech’ (mixed podocarp-hardwood); **addCovariates** attaches these data to each cell in the mask.

```
ovtrap <- traps(OVpossumCH[[1]])
ovmask <- make.mask(ovtrap, buffer = 120, type = "trapbuffer",
  poly = OVforest[1:2,], spacing = 7.5, keep.poly = FALSE)
ovmask <- addCovariates(ovmask, OVforest[1:2,])
```

Plotting is easy:

```
par(mar = c(1,6,2,8))
forestcol <- terrain.colors(6)[c(4,2)]
plot(ovmask, cov="forest", dots = FALSE, col = forestcol)
plot(ovtrap, add = TRUE)
par(cex = 0.8)
terra::sbar(d = 200, xy = c(2674670, 5982930), type = 'line',
  divs = 2, below = "metres", labels = c("0","100","200"),
  ticks = 10)
terra::north(xy = c(2674670, 5982830), d = 40, label = "N")
```

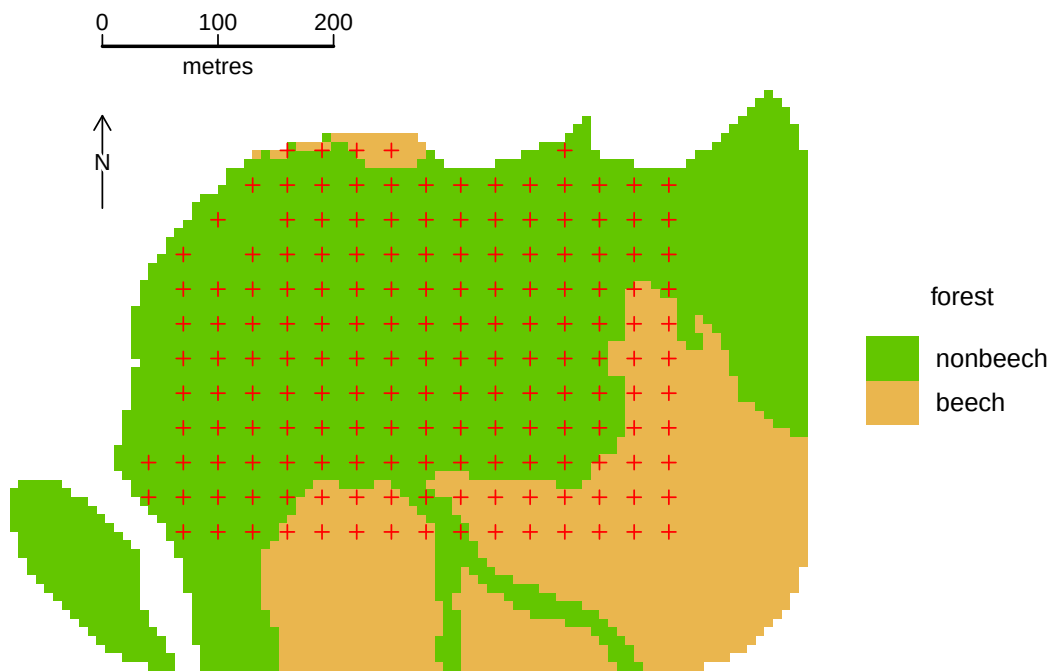


Figure 11.1: Orongorongo Valley possum study area

We next fit some simple models to data from February 1996 (session 49). Some warnings are suppressed for clarity.

```
args <- list(capthist = OVpossumCH[['49']], mask = ovmask, trace = FALSE)
models <- list(D ~ 1, D ~ x + y, D ~ x + y + x2 + y2 + xy, D ~ forest)
```

```
names <- c('null','Dxy','Dxy2', 'Dforest')
fits <- list.secr.fit(model = models, constant = args, names = names)
```

```
AIC(fits)[,-c(1,2,5,6)]
```

	npar	logLik	dAIC	AICwt
Dxy2	8	-1549.32	0.000	0.4429
Dxy	5	-1552.86	1.086	0.2573
Dforest	4	-1554.15	1.653	0.1938
null	3	-1555.75	2.860	0.1060

Each of the inhomogeneous models seems marginally better than the null model, but there is little to choose among them.

To visualise the entire surface we compute predicted density at each mask point. For example, we can plot the quadratic surface like this:

```
par(mar = c(1,6,2,8))
surfaceDxy2 <- predictDsurface(fits$Dxy2)
plot(surfaceDxy2, plottype = "shaded", poly = FALSE, breaks =
      seq(0,22,2), title = "Density / ha", text.cex = 1)
# graphical elements to be added, including contours of Dsurface
plot(ovtrap, add = TRUE)
plot(surfaceDxy2, plottype = "contour", poly = FALSE, breaks =
      seq(0,22,2), add = TRUE)
lines(leftbank)
```

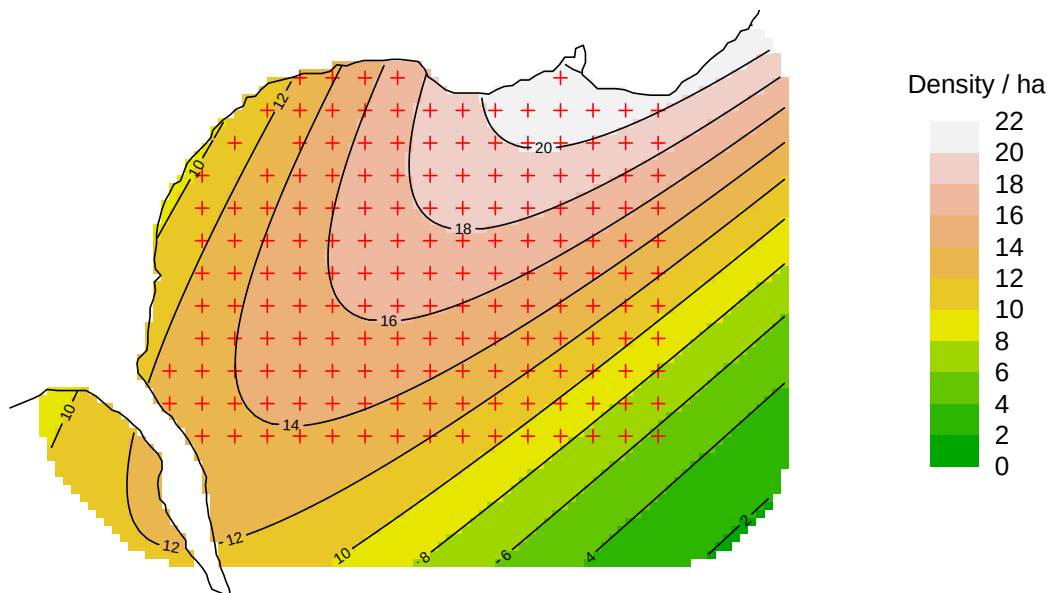


Figure 11.2: Quadratic possum density surface

Following sections expand on the options for specifying and displaying density models.

11.3 Using the ‘model’ argument

A model formula defines variation in each parameter as a function of covariates (including geographic coordinates and their polynomial terms) that is linear on the ‘link’ scale, as in a generalized linear model.

The options differ between the state and observation models. D may vary with respect to group, session or point in space

The predictors ‘group’ and ‘session’ behave for D as they do for other real parameters. They determine variation in the expected density for each group or session that is (by default) uniform across space, leading to a homogeneous Poisson model and a flat surface. No further explanation is therefore needed.

11.3.1 Link function

The default link for D is ‘log’. It is equally feasible in most cases to choose ‘identity’ as the link (see the `secr.fit` argument ‘link’), and for the null model $D \sim 1$ the estimate will be the same to numerical accuracy, as will estimates involving only categorical variables (e.g., session). However, with an ‘identity’ link the usual (asymptotic) confidence limits will be symmetrical (unless truncated at zero) rather than asymmetrical. In models with continuous predictors, including spatial trend surfaces, the link function will affect the result, although the difference may be small when the amplitude of variation on the surface is small. Otherwise, serious thought is needed regarding which model is biologically more appropriate: logarithmic or linear.

The ‘identity’ link may cause problems when density is very small or very large because, by default, the maximization method assumes all parameters have similar scale (e.g., `typsize = c(1,1,1)` for default constant models). Setting `typsize` manually in a call to `secr.fit` can fix the problem and speed up fitting. For example, if density is around 0.001/ha (10 per 100 km²) then call `secr.fit(..., typsize = c(0.001,1,1))` (`typsize` has one element for each beta parameter). Problems with the identity link mostly disappear when modelling relative density (`CL = TRUE`) because coefficients are automatically scaled by the intercept. See Appendix I for more on link functions.

You may wonder why `secr.fit` is ambivalent regarding the link function: link functions have seemed a necessary part of the machinery for capture–recapture modelling since Lebreton et al. (1992). Their key role is to keep the ‘real’ parameter within feasible bounds (e.g., 0-1 for probabilities). In `secr.fit` any modelled value of D that falls below zero is truncated at zero (of course this condition will not arise with a log link).

11.3.2 Built-in variables

`secl.fit` automatically recognises the spatial variables `x`, `y`, `x2`, `y2` and `xy` if they appear in the formula for `D`. These refer to the x-coordinate, y-coordinate, x-coordinate² etc. for each mask point, and will be constructed automatically as needed. The built-in spatial variables offer limited model possibilities (Table 11.2).

The formula for `D` may also include the non-spatial variables `g` (group), `session` (categorical), and `Session` (continuous), defined as for modelling `g0` and `sigma` in Chapter 10.

Table 11.2: Examples of density models using built-in variables.

Formula	Interpretation
$D \sim 1$	flat surface (default)
$D \sim x + y$	linear trend surface (planar)
$D \sim x + x^2$	quadratic trend in east-west direction only
$D \sim x + y + x^2 + y^2 + xy$	quadratic trend surface

11.3.3 User-provided variables

More interesting models can be made with variables provided by the user. These are stored in a data frame as the ‘covariates’ attribute of a mask object. Covariates must be defined for every point on a mask.

Variables may be categorical (a factor or character value that can be coerced to a factor) or continuous (a numeric vector). The habitat variable ‘habclass’ constructed in the Examples section of the `skink` help is an example of a two-class categorical covariate. Remember that categorical variables entail one additional parameter for each extra level.

There are several ways to create or input mask covariates.

1. Read columns of covariates along with the x- and y-coordinates when creating a mask from a dataframe or external file (`read.mask`)
2. Read the covariates dataframe separately from an external file (`read.table`)
3. Infer covariate values by computation on in existing mask (see below).
4. Infer values for points on an existing mask from a GIS data source, such as a polygon shapefile or other spatial data source (see Appendix C).

Use the function `addCovariates` for the third and fourth options.

11.3.4 Covariates computed from coordinates

Higher-order polynomial terms may be added as covariates if required. For example,

```
covariates(ovmask)[,"x3"] <- covariates(ovmask)$x^3
```

allows a model like $D \sim x + x^2 + x^3$.

If you have a strong prior reason to suspect a particular ‘grain’ to the landscape then this may be also be computed as a new, artificial covariate. This code gives a covariate representing a northwest – southeast trend:

```
covariates(ovmask)[,"NWSE"] <- ovmask$y - ovmask$x -  
  mean(ovmask$y - ovmask$x)
```

Another trick is to compute distances to a mapped landscape feature. For example, possum density in our Orongorongo example may relate to distance from the river; this corresponds roughly to elevation, which we do not have to hand. The `distancetotrap` function of **secr** computes distances from mask cells to the nearest vertex on the riverbank, which are precise enough for our purpose.

```
covariates(ovmask)[,"DTR"] <- distancetotrap(ovmask, leftbank)
```

```
par(mar = c(1,6,2,8))  
plot(ovmask, covariate = "DTR", breaks = seq(0,500,50),  
     title = "Distance to river m", dots = FALSE, inset= 0.07)
```



Figure 11.3: Orongorongo Valley possum study: distance to river

11.3.5 Pre-computed resource selection functions

A resource selection function (RSF) was defined by Boyce et al. (2002) as “any model that yields values proportional to the probability of use of a resource unit”. An RSF combines habitat information from multiple sources in a single variable. Typically the function is estimated from telemetry data on marked individuals, and primarily describes individual-level behaviour (3rd-order habitat selection of D. H. Johnson, 1980).

However, the individual-level RSF is also a plausible hypothesis for 2nd-order habitat selection i.e. for modelling the relationship between habitat and population density. Then we interpret the RSF as a single variable that is believed to be proportional to the expected population density in each cell of a habitat mask. For example, Proffitt et al. (2015) used an RSF developed independently from telemetry data on utilisation in relation to terrain and land cover to model spatial variation in the density of mountain lions *Puma concolor* in Montana.

Suppose, for example, in folder `datadir` we have a polygon shapefile (`RSF.shp`, `RSF.dbf` etc.) with the attribute “`rsf`” defined for each polygon. Given `mask` and `capthist` objects “`habmask`” and “`myCH`”, this code fits a SECR model that calibrates the RSF in terms of population density:

```
rsfshape <- sf::st_read(paste0(datadir, "/RSF.shp"))
habmask <- addCovariates(habmask, rsfshape, columns = "rsf")
secr.fit(myCH, mask = habmask, model = D ~ rsf - 1)
# alternatively, fit a relative density model
secr.fit(myCH, mask = habmask, CL = TRUE, model = D ~ rsf)
```

- “`rsf`” must be known for every pixel in the habitat mask
- Usually it make sense to fit the density model through the origin (`rsf = 0` implies $D = 0$). This is not true of habitat suitability indices in general.

This is a quite different approach to fitting multiple habitat covariates within `secr`, and one that should be considered. There are usually too few individuals in a SECR study to usefully fit models with multiple covariates of density, even given a large dataset such as our possum example. However, 3rd-order and 2nd-order habitat selection are conceptually distinct, and their relationship is an interesting research topic (e.g., Linden et al., 2018).

11.3.6 Regression splines

Regression splines are a flexible alternative to polynomials for spatial trend analysis. Regression splines are familiar as the smooth terms in ‘generalized additive models’ (gams) implemented (differently) in the base R package `gam` and in R package `mgcv` (Wood, 2006).

Some of the possible smooth terms from `mgcv` can be used in model formulae for `secr.fit` – see the help page for ‘smooths’ in `secr`. Smooths are specified with terms that look like calls to the functions `s` and `te`. Smoothness is determined by the number of knots which is set

by the user via the argument 'k'. The number of knots cannot be determined automatically by the penalty algorithms of **mgcv**.

Here we fit a regression spline with the same number of parameters as a quadratic polynomial, a linear effect of the 'distance to river' covariate on $\log(D)$, and a nonlinear smooth.

```
args <- list(capthist = OVpossumCH[[1]], mask = ovmask, trace =
  FALSE)
models <- list(D ~ s(x,y, k = 6), D ~ DTR, D ~ s(DTR, k = 3))
RSfits <- list.secr.fit(model = models, constant = args,
  prefix = "RS")
```

Now add these to the AIC table and plot the 'AIC-best' model:

```
AIC(c(fits, RSfits))[, -c(1,2,5,6)]
```

	npar	logLik	dAIC	AICwt
RS3	5	-1552.00	0.000	0.2667
Dxy2	8	-1549.32	0.628	0.1948
RS2	4	-1553.36	0.705	0.1875
Dxy	5	-1552.86	1.714	0.1132
RS1	8	-1549.93	1.847	0.1059
Dforest	4	-1554.15	2.281	0.0853
null	3	-1555.75	3.488	0.0466

```
newdat <- data.frame(DTR = seq(0,400,5))
tmp <- predict(RSfits$RS3, newdata = newdat)
par(mar=c(5,8,2,4), pty = "s")
plot(seq(0,400,5), sapply(tmp, "[", "D","estimate"),
  ylim = c(0,20), xlab = "Distance from river (m)",
  ylab = "Density / ha", type = "l")
```

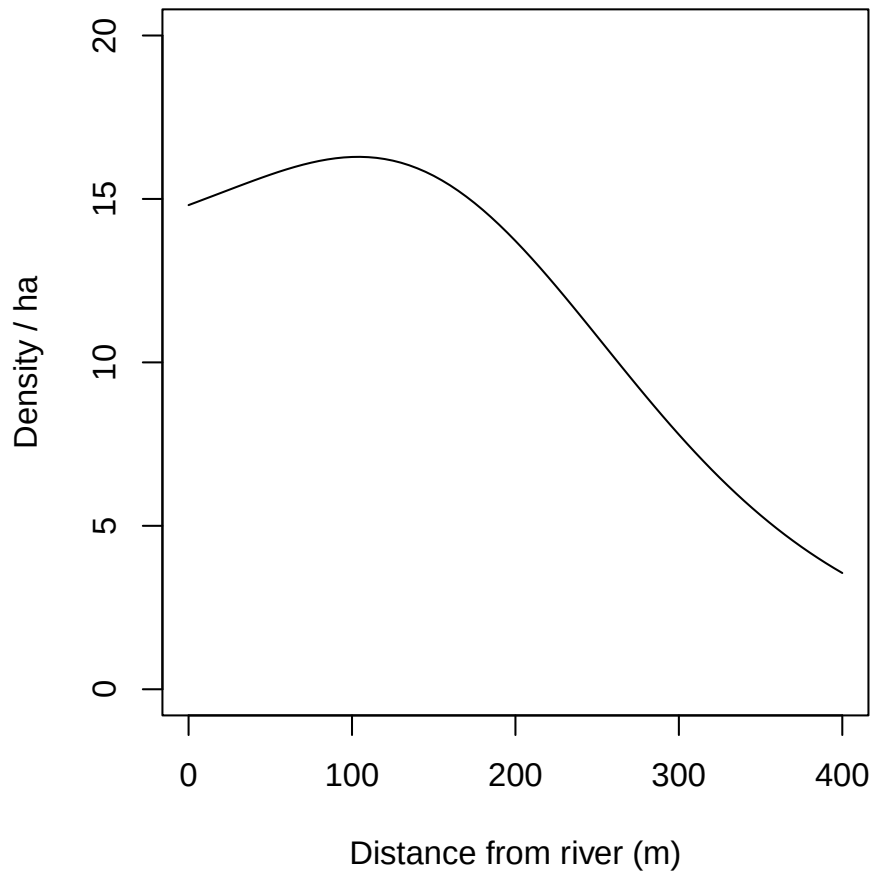


Figure 11.4: Possum density vs distance to river: regression spline $k = 3$

Confidence intervals are computed in `predictDsurface` by back-transforming $\pm 2\text{SE}$ from the link (log) scale:

```
par(mar = c(1,1,1,1), mfrow = c(1,2), xpd = FALSE)
surfaceDDTR3 <- predictDsurface(RSfits$RS3, cl.D = TRUE)
plot(surfaceDDTR3, covariate= "lcl", breaks = seq(0,22,2),
      legend = FALSE)
mtext(side = 3, line = -1.5, cex = 0.8,
      "Lower 95% confidence limit of D (possums / ha)")
plot(surfaceDDTR3, plottype = "contour", breaks = seq(0,22,2),
      add = TRUE)
lines(leftbank)
plot(surfaceDDTR3, covariate= "ucl", breaks = seq(0,22,2),
      legend = FALSE)
mtext(side = 3, line = -1.5, cex = 0.8,
      "Upper 95% confidence limit of D (possums / ha)")
plot(surfaceDDTR3, covariate = "ucl", plottype = "contour",
      breaks = seq(0,22,2), add = TRUE)
```

```
lines(leftbank)
mtext(side=3, line=-1, outer=TRUE, "s(DTR, k = 3) model", cex = 0.9)
```

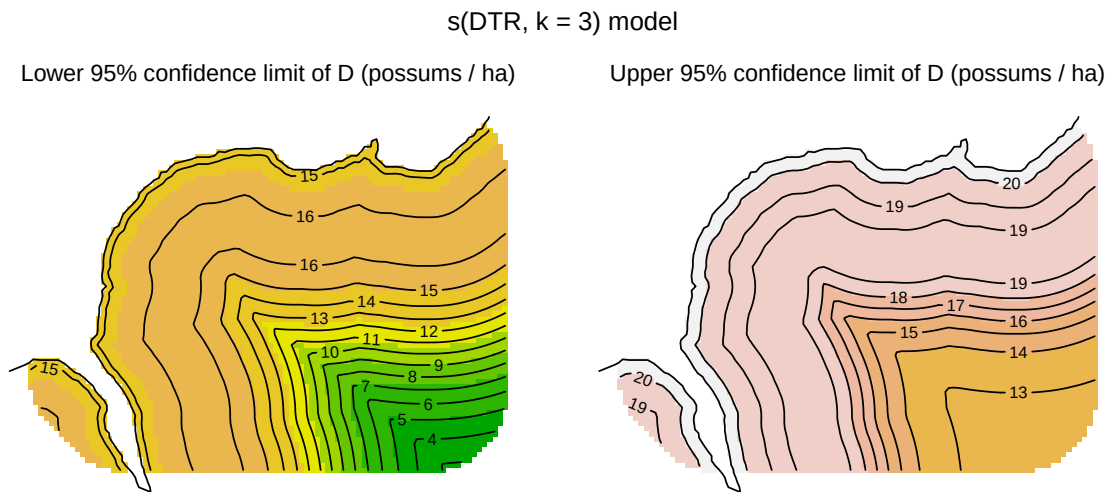


Figure 11.5: Confidence surfaces

Multiple predictors may be included in one ‘s’ smooth term, implying interaction. This assumes isotropy – equality of scales on the different predictors – which is appropriate for geographic coordinates such as x and y in this example. In other cases, predictors may be measured on different scales (e.g., structural complexity of vegetation and elevation) and isotropy cannot be assumed. In these cases a tensor-product smooth (**te**) is appropriate because it is scale-invariant. For **te**, ‘ k ’ represents the order of the smooth on each axis, and we must fix the number of knots with ‘ $fx = TRUE$ ’ to override automatic selection.

For more on the use of regression splines see the documentation for **mgcv**, the **sekr** help page ‘?smooths’, Wood (2006), and Borchers & Kidney (2014).

11.3.7 Scale of effect

Modelling density as a function of covariate(s) at a point (the centroid of a mask cell) lacks biological realism. Individuals exploit resources across their home range, and density may be affected by regional dynamics over a much wider area (e.g., Jackson & Fahrig, 2014).

The scale at which environmental predictors influence density is usually unknown, and almost certainly does not correspond to either a point or the arbitrary size of a mask cell. Some progress has been made in estimating the scale from data (e.g., R. Chandler & Hepinstall-Cymerman, 2016), but methods have not yet been integrated into SECR.

The best we can do at present is to construct spatial layers in GIS software that represent various levels of smoothing or spatial aggregation, corresponding to different scales of effect. Each layer may then be imported as a mask covariate that is evaluated at the cell centroids to represent the surrounding area.

This example uses the `focal` function in the **terra** package (Hijmans, 2023b) to compute both the proportion of forest cells in a square window and a Gaussian-smoothed proportion.

```
# binary SpatRaster 0 = nonbeech, 1 = beech
covariates(ovmask)$forest_num <- ifelse (covariates(ovmask)$forest == 'beech', 1,0)
tmp <- terra::rast(ovmask, covariate = 'forest_num')

# square smoothing window: 5 cells
tmpw5 <- terra::focal(tmp, w = 5, "mean", na.policy = "omit", na.rm = TRUE)
names(tmpw5) <- 'w5'

# Gaussian window, sigma 50 m
# window radius is 3 sigma
g50 <- terra::focalMat(tmp, d = 50, type = "Gauss")
tmpg50 <- terra::focal(tmp, w = g50, "sum", na.policy = "omit", na.rm = TRUE)
names(tmpg50) <- 'g50'

# add to mask
ovmask <- addCovariates(ovmask, tmpw5)
ovmask <- addCovariates(ovmask, tmpg50)

# plot
par(mar = c(1,1,1,1), mfrow = c(1,2), xpd = FALSE)

plot(ovmask, cov = 'w5', dots = FALSE, legend = FALSE)
plot(ovtrap, add = TRUE)
text (2674600, 5982892, 'a.', cex = 1.2)

plot(ovmask, cov = 'g50', dots = FALSE, legend = FALSE)
plot(ovtrap, add = TRUE)
text (2674600, 5982892, 'b.', cex = 1.2)
```

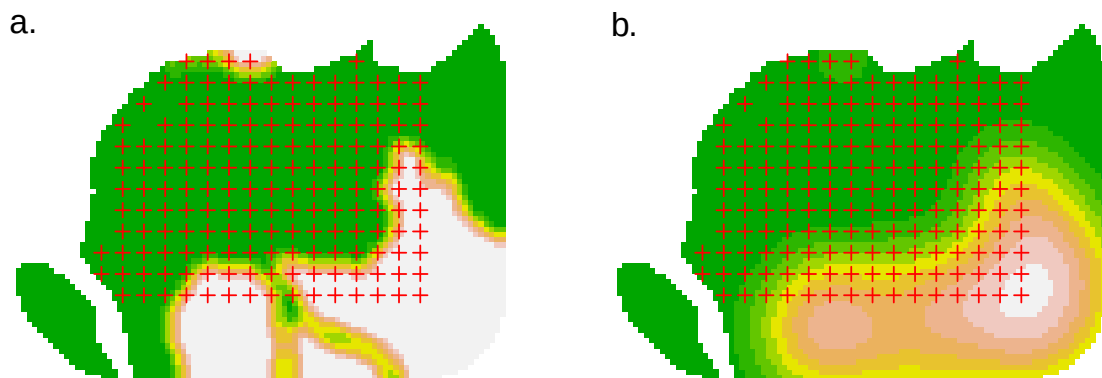


Figure 11.6: Smoothed forest covariate (proportion ‘beech’: 0% dark green, 100% white).
(a) square window (5×5 cells), (b) Gaussian smooth ($\sigma = 50$ m).

11.4 Prediction and plotting

Fitting a model provides estimates of its coefficients or ‘beta parameters’; use the `coef` method to extract these from an `secr` object. The coefficients are usually of little use in themselves, but we can use them to make predictions. In order to plot a fitted model we first predict the height of the density surface at each point on a mask. As we have seen, this is done with `predictDsurface`, which has arguments (`object`, `mask = NULL`, `se.D = FALSE`, `cl.D = FALSE`, `alpha = 0.05`). By default, prediction is at the mask points used when fitting the model (i.e. `object$mask`); specify the `mask` argument to extrapolate the model to a different area.

The output from `predictDsurface` is a specialised mask object called a `Dsurface` (class `c('Dsurface', 'mask', 'data.frame')`). The covariate dataframe of a `Dsurface` has columns for the predicted density of each group (`D.0` if there is only one). Usually when you print a mask you see only the x- and y-coordinates. The `print` method for `Dsurface` objects displays both the coordinates and the density values as one dataframe, as also do the `head` and `tail` methods.

Use the arguments ‘`se.D`’ and ‘`cl.D`’ to request computation of the estimated standard error and/or upper and lower confidence limits for each mask point³. If requested, values are saved as additional covariates of the output `Dsurface` (`SE.0`, `lcl.0`, and `ucl.0` if there is only one group).

The plot method for a `Dsurface` object has arguments (`x`, `covariate = "D"`, `group = NULL`, `plottype = "shaded"`, `scale = 1`, ...). `covariate` may either be a prefix (one of “`D`”, “`SE`”, “`lcl`”, “`ucl`”) or any full covariate name. ‘`plottype`’ may be one of “`shaded`”, “`dots`”, “`persp`”, or “`contour`”. A coloured legend is displayed centre-right (see `?plot.mask` and `?strip.legend` for options).

For details on how to specify colours, levels etc. read the help pages for `plot.mask`, `contour` and `persp` (these functions may be controlled by extra arguments to `plot.Dsurface`, using the ‘`dots`’ convention).

A plot may be enhanced by the addition of contours. This is a challenge, because the `contour` function in R requires a rectangular matrix of values, and our mask is not rectangular. We could make it so with the `secr` function `rectangularMask`, which makes a rectangular `Dsurface` with missing (NA) values of density at all the external points. `plot.Dsurface` recognises an irregular mask and attempts to fix this with an internal call to `rectangularMask`.

11.5 Scaling

So far we have ignored the scaling of covariates, including geographic coordinates.

³Option available only for models specified in generalized linear model form with the ‘`model`’ argument of `secr.fit`, not for user-defined functions.

`secr.fit` scales the x- and y-coordinates of mask points to mean = 0, SD = 1 before using the coordinates in a model. Remember this when you come to use the coefficients of a density model. Functions such as `predictDsurface` take care of scaling automatically. `predict.secr` uses the scaled values ('newdata' x = 0, y = 0), which provides the predicted density at the mask centroid. The mean and SD used in scaling are those saved as the 'meanSD' attribute of a mask (dataframe with columns 'x' and 'y' and rows 'mean' and 'SD').

Scaling of covariates other than x and y is up to the user. It is not usually needed.

The numerical algorithms for maximizing the likelihood work best when the absolute expected values are roughly similar for all parameters on their respective 'link' scales (i.e. all beta parameters) rather than varying by orders of magnitude. The default link function for D and sigma (log) places the values of these parameters on a scale that is not wildly different to the variation in g0 or lambda0, so this is seldom an issue. In extreme cases you may want to make allowance by setting the `typsize` argument of `nlm` or the `parscale` control argument of `optim` (via the ... argument of `secr.fit`).

Scaling is not performed routinely by `secr.fit` for distance calculations. Sometimes, large numeric values in coordinates can cause loss of precision in distance calculations (there are a lot of them at each likelihood evaluation). The problem is serious in datasets that combine large coordinates with small detector spacing, such as the Lake Station `skink` dataset. Set `details = list(centred = TRUE)` to force scaling; this may become the default setting in a future version of `secr`.

11.6 This is not a density surface

The surfaces we have fitted involve inhomogeneous Poisson models for the distribution of animal home range centres. The models have parameters that determine the relationship of expected density to location or to habitat covariates.

Another type of plot is sometimes presented and described as a 'density surface' – the summed posterior distribution of estimated range centres from a Bayesian fit of a homogeneous Poisson model. A directly analogous plot may be obtained from the `secr` function `fxTotal` (see also Borchers & Efford (2008) Section 4.3). The contours associated with the home range centre of each detected individual essentially represent 2-D confidence intervals for its home range centre, given the fitted observation model. Summing these gives a summed probability density surface for the centres of the observed individuals ('D.fx'), and to this we can add an equivalent scaled probability density surface for the individuals that escaped detection ('D.nc'). Both components are reported by `fx.total`, along with their sum ('D.sum') which we plot here for the flat possum model:

```
fxsurface <- fxTotal(fits$null)
```

```

par(mar = c(1,6,2,8))
plot(fxsurface, covariate = "D.sum", breaks = seq(0,30,2),
     poly = FALSE)
plot(ovtrap, add = TRUE)

```

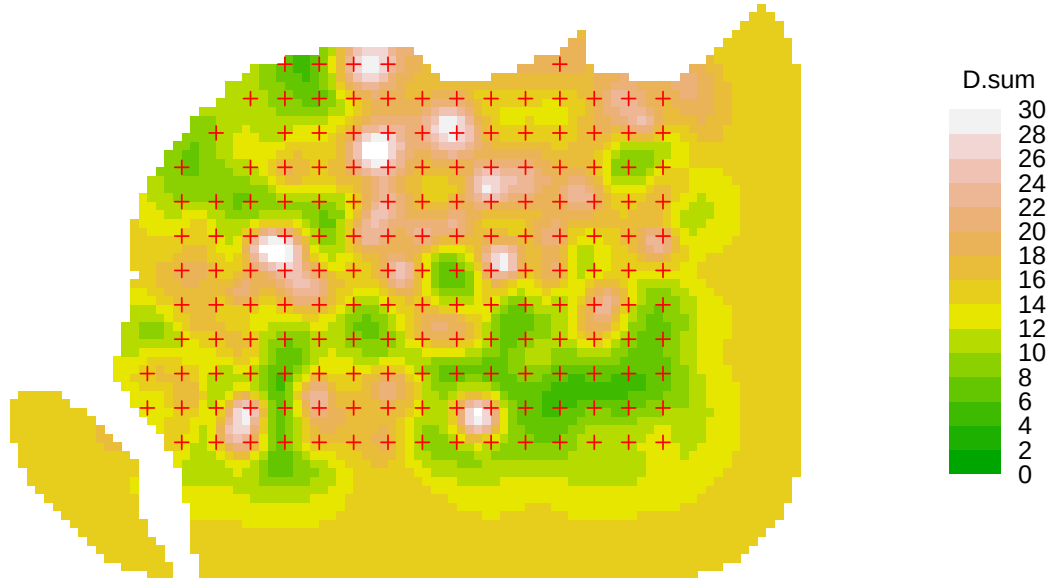


Figure 11.7: Total fx surface

The plot concerns only one realisation from the underlying Poisson model. It visually invites us to interpret patterns in that realisation that we have not modelled. There are serious problems with the interpretation of such plots as ‘density surfaces’:

- attention is focussed on the individuals that were detected; others that were present but not detected are represented by a smoothly varying base level that dominates in the outer region of the plot (contrast this figure with the previous quadratic and DTR3 models).
- the surface depends on sampling intensity, and as more data are added it will change shape systematically. Ultimately, the surface near the centre of a detector array becomes a set of emergent peaks rising from an underwater plain of zero density, below the plateau of average density outside the array.
- the ‘summed confidence interval’ plot is easily confused with the 2-D surface obtained by summing utilisation distributions across animals
- confidence intervals are not available for the height of the probability density surface.

The plots are also prone to artefacts. In some examples we see concentric clustering of estimated centres around the trapping grids, apparently ‘repelled’ from the traps themselves (e.g., plot below for a null model of the Waitarere ‘possumCH’ dataset in **secr**). This phenomenon appears to relate to lack of model fit (unpubl. results).

```
fxsurfaceW <- fxTotal(possum.model.0)
```

```
par(mar = c(1,5,1,8))
plot(fxsurfaceW, covariate = "D.sum", breaks = seq(0,5,0.5),
     poly = FALSE)
plot(traps(possumCH), add = TRUE)
```



Figure 11.8: Waitarere possum fx surface

See Durbach et al. (2024) for further critique.

11.7 Relative density

Theory for relative density models was given [earlier](#) (see also (Efford, 2025c)). A spatial model for relative density is fitted in **secr** by setting `CL = TRUE` and providing a model for `D` in the call to `secr.fit` (`secr` $\geq$ 5.2.0). For example

```
fitrd1 <- secr.fit(capthist = OVpossumCH[[1]], mask = ovmask,
                  trace = FALSE, model = D ~ x + y + x2 + y2 + xy, CL = TRUE)
options(digits = 4)
coef(fitrd1)[,1:2]
```

	beta	SE.beta
D.x	-0.19872	0.11851
D.y	0.30999	0.13469

```
D.x2 -0.22472 0.13345
D.y2 -0.09673 0.12818
D.xy  0.29672 0.13208
g0    -2.22032 0.09422
sigma  3.31767 0.03544
```

```
# compare coefficients from full fit:
coef(fits$Dxy2)[1:2]
```

```
      beta SE.beta
D      2.72539 0.12781
D.x    -0.19871 0.11851
D.y     0.30999 0.13468
D.x2   -0.22472 0.13344
D.y2   -0.09673 0.12817
D.xy    0.29672 0.13208
g0     -2.22032 0.09422
sigma   3.31767 0.03544
```

The relative density model is fitted by maximizing the likelihood conditional on n and has one fewer coefficients than the absolute density model. Estimates of other coefficients are the same within numerical error (log link) or are scaled by the intercept (identity link).

The density intercept and other coefficients may be retrieved with the function `derivedDcoef`:

```
derivedDcoef(fitrd1) # delta-method variance suppressed as unreliable
```

```
      beta  SE.beta      lcl      ucl
D      2.7253879      NA      NA      NA
D.x    -0.1987152 0.1185097 -0.4309899  0.0335595
D.y     0.3099851 0.1346875  0.0460025  0.5739678
D.x2   -0.2247188 0.1334476 -0.4862713  0.0368337
D.y2   -0.0967316 0.1281839 -0.3479674  0.1545042
D.xy    0.2967234 0.1320827  0.0378460  0.5556008
g0     -2.2203192 0.0942190 -2.4049850 -2.0356534
sigma   3.3176746 0.0354357  3.2482218  3.3871274
```

To plot the full density surface it is first necessary to infer the missing intercept. This is done automatically by the function `derivedDsurface`:

```

par(mar = c(1,6,2,8))
derivedD <- derivedDsurface(fitrd1)
plot(derivedD, plottype = "shaded", poly = FALSE, breaks =
      seq(0,22,2), title = "Density / ha", text.cex = 1)
plot(surfaceDxy2, plottype = "contour", poly = FALSE, breaks =
      seq(0,22,2), add = TRUE)
lines(leftbank)

```

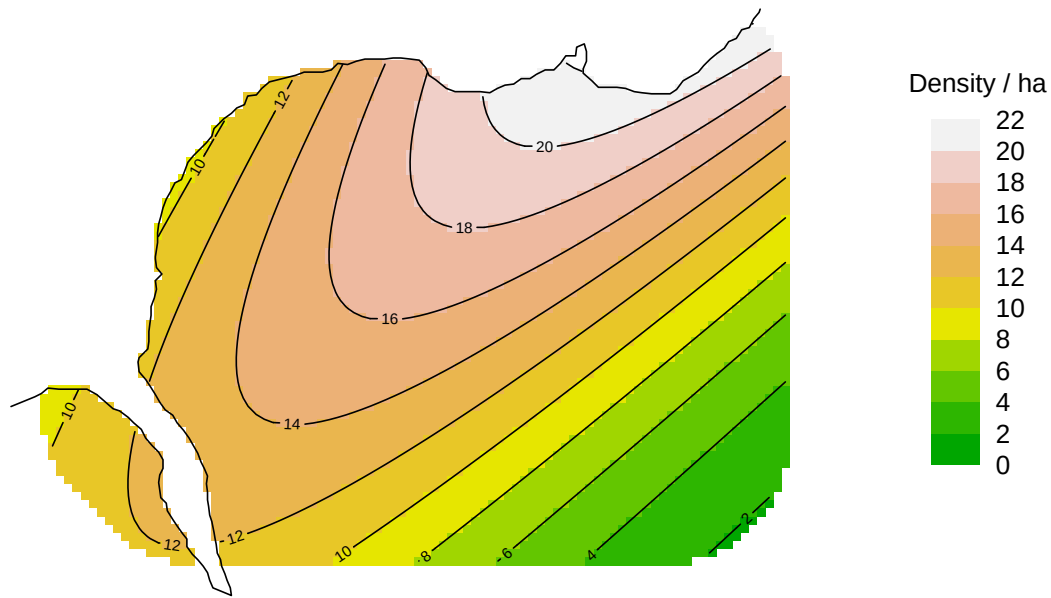


Figure 11.9: Possum density derived from quadratic relative density surface. Contour lines are from the previous full fit.

The density in Fig. 11.9 nearly matches the absolute density in Fig. 11.2. This will not be the case if the relative density model is fitted to data from animals tagged elsewhere or on a subset of the area. Tagging then imposes differential spatial weighting that must be made explicit in the model to recover the correct pattern of density in relation to covariates. One scenario involves acoustic telemetry or other automated detection for which the only animals at risk of detection are those previously marked (cf resighting data, in which unmarked animals are detected and counted, but not identified).

12 Habitat mask

A mask represents habitat in the vicinity of detectors that is potentially occupied by the species of interest. Understanding habitat masks and how to define and manipulate them is central to spatially explicit capture–recapture. This chapter summarises what users need to know about masks in **secr**. Early sections apply regardless of software.

12.1 Background

We start with an intuitive explanation of the need for habitat masks. Devices such as traps or cameras record animals moving in a general region. If the devices span a patch of habitat with a known boundary then we use a mask to define that geographical unit. More commonly, detectors are placed in continuous habitat and the boundary of the region sampled is ill-defined. This is because the probability of detecting an animal tapers off gradually with distance.

Vagueness regarding the region sampled is addressed in spatially explicit capture–recapture by considering a larger and more inclusive region, the habitat mask. Its extent is not critical, except that it should be at least large enough to account for all detected animals.

Next we refine this intuitive explanation for each of the dominant methods for fitting SECR models: maximum likelihood and Markov-chain Monte Carlo (MCMC) sampling. Each grapples in a slightly different way with the awkward fact that, although we wish to model detection as a function of distance from the activity centre, the activity centre of each animal is at an unknown location.

12.1.1 Maximum likelihood and the area of integration

The likelihood developed for SECR by Borchers & Efford (2008) allows for the unknown centres by numerically integrating them out of the likelihood (crudely, by summing over all possible locations of detected animals, weighting each by a detection probability). Although the integration might, in principle, have infinite spatial bounds, it is practical to restrict attention to a smaller region, the ‘area of integration’. As long as the probability weights get close to zero before we reach the boundary, we don’t need to worry too much about the size of the region.

In **secr** the habitat mask equates to the area of integration: the likelihood is evaluated by summing values across a fine mesh of points. This is the primary function of the habitat mask; we consider other functions in Section [12.2](#).

12.1.2 MCMC and the Bayesian ‘state space’

MCMC methods for spatial capture–recapture developed by Royle and coworkers (Royle et al., 2014) take a slightly different tack. The activity centres are treated as a large number of unobserved (latent) variables. The MCMC algorithm ‘samples’ from the posterior distribution of location for each animal, whether detected or not. The term ‘state space’ is used for the set of permitted locations; usually this is a continuous (not discretized) rectangular region.

12.2 What is a mask for?

Masks serve multiple purposes in addition to the basic one we have just introduced. We distinguish five functions of a habitat mask and there may be more:

1. To define the outer limit of the area of integration. Habitat beyond the mask may be occupied, but animals centred there have negligible chance of being detected.
2. To facilitate computation. By defining the area of integration as a list of discrete points (the centres of grid cells, each with notionally uniform density) we transform the relatively messy task of numerical integration into the much simpler one of summation.
3. To distinguish habitat sites from non-habitat sites within the outer limit. Habitat cells have the potential to be occupied. Treating non-habitat as if it is habitat can cause habitat-specific density to be underestimated.
4. To store habitat covariates for spatial models of density. Covariates for modelling a density surface are provided for each point on the mask.
5. To define a region for which a post-hoc estimate of population size is required. This may differ from the mask used to fit the model.

The first point raises the question of where the outer limit should lie (i.e., the buffer width), and the second raises the question of how coarse the discretization (i.e., the cell size) can be without damaging the estimates. Later sections address each of the five points in turn, after an introductory section describing the particular implementation of habitat masks in the R package **secr**.

12.3 Masks in **secr**

A habitat mask is represented in **secr** by a set of *square* grid cells. Their combined area may be almost any shape and may include holes. An object of class ‘mask’ is a 2-column dataframe with additional attributes (cell area etc.); each row gives the x- and y-coordinates of the centre of one cell.

12.3.1 Masks generated automatically by `secr.fit`

A mask is used whenever a model is fitted with the function `secr.fit`, even if none is specified in the ‘mask’ argument. When no mask is provided, one is constructed automatically using the value of the ‘buffer’ argument. For example

```
library(secr, quietly = TRUE)
fit <- secr.fit(captdata, buffer = 80, trace = FALSE)
```

The mask is saved as a component of the fitted model (‘secr’ object); we can plot it and overlay the traps:

```
par(mar = c(1,1,1,1))
plot(fit$mask, dots = FALSE, mesh = "grey", col = "white")
plot(traps(captdata), detpar = list(pch = 16, cex = 1), add = TRUE)
```

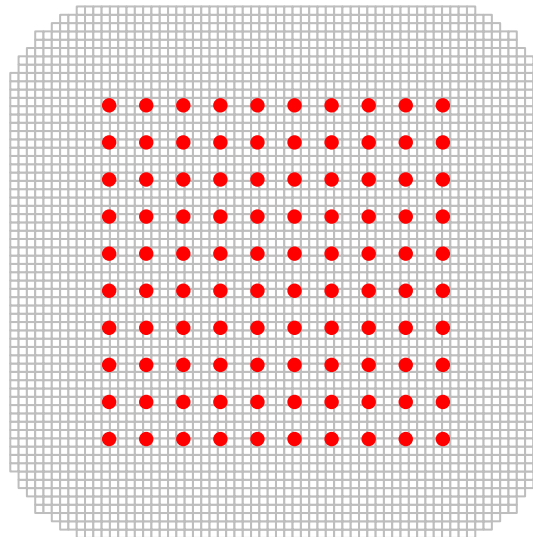


Figure 12.1: Mask (grey grid) generated automatically in `secr.fit` by buffering around the detectors (red dots) (80-m buffer, 30-m detector spacing)

The mask is generated by forming a grid that extends ‘buffer’ metres north, south, east and west of the detectors and dropping centroids that are more than ‘buffer’ metres from the nearest detector (hence the rounded corners). The obvious question “How wide should the buffer be?” is addressed in a later section. The spacing of mask points (i.e. width of grid cells) is set arbitrarily to 1/64th of the east-west dimension - in this example the spacing is 6.7 metres.

12.3.2 Masks constructed with `make.mask`

A mask may also be prepared in advance and provided to `secr.fit` in the ‘mask’ argument. This overrides the automatic process of the preceding section, and the value of ‘buffer’ is discarded. The function `make.mask` provides precise control over the the size of the cells, the extent of the mask, and much more. We introduce `make.mask` here with a simple example based on a ‘hollow grid’:

```
hollowgrid <- make.grid(nx = 10, ny = 10, spacing = 30, hollow = TRUE)
hollowmask <- make.mask(hollowgrid, buffer = 80, spacing = 15, type = "trapbuffer")
```

```
par(mar = c(1,1,1,1))
plot(hollowmask, dots = FALSE, mesh = "grey", col = "white")
plot(hollowgrid, detpar = list(pch = 16, cex = 1), add = TRUE)
```

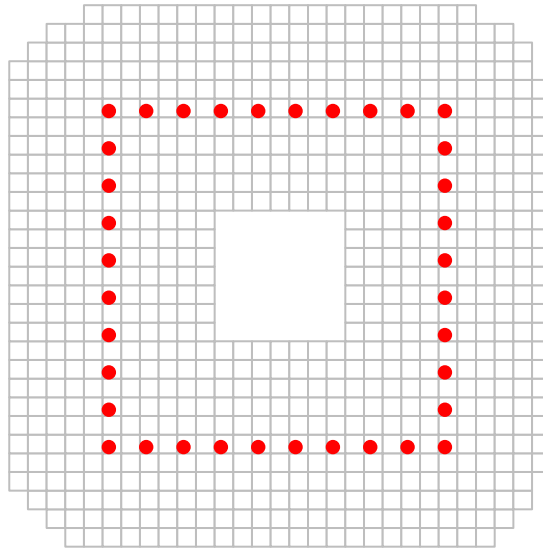


Figure 12.2: Mask (grey grid) generated with `make.mask` (80-m buffer, 30-m trap spacing, 15-m mask spacing). Grid cells in the centre were dropped because they were more than 80 m from any trap.

We chose a coarser grid (spacing 15 metres) relative to the trap spacing. This, combined with the hole in the centre, results in a mask with many fewer rows (764 rows compared to 3980). Setting the type to “trapbuffer” trims grid cells from the corners and the centre.

If we collected data `hollowCH` with the hollow grid we could fit a SECR model using `hollowmask`. For illustration we simulate some data using default settings in `sim.caphist` (5 occasions, $D = 5/\text{ha}$, $g_0 = 0.2$, $\sigma = 25$ m).

```
hollowCH <- sim.caphist(hollowgrid, seed = 123)
fit2 <- secr.fit(hollowCH, mask = hollowmask, trace = FALSE)
predict(fit2)
```

	link	estimate	SE.estimate	lcl	ucl
D	log	5.0807	1.078535	3.36684	7.66711
g0	logit	0.2047	0.044309	0.13117	0.30497
sigma	log	26.1801	3.241670	20.55772	33.34021

Fitting is fast because there are few traps and few mask points. As before, the mask is retained in the output, so –

```
cat("Number of rows in hollow mask =", nrow(fit2$mask), "\n")
```

Number of rows in hollow mask = 764

12.4 How wide should the buffer be?

The general answer is ‘Wide enough that any bias in estimated densities is negligible’. Excessive truncation of the mask results in positive bias that depends on the sampling regime (detector layout and sampling duration) and the detection function, particularly its spatial scale and shape.

The penalty for using an over-wide buffer is that fitting will be slower for a given mask spacing. *It is usually smart to accept this penalty rather than search for the narrowest acceptable buffer.*

Two factors are critical when selecting a buffer width –

1. The spatial scale of detection, which is usually a function of home-range movements.
2. The shape of the detection function, particularly the length of its tail.

These must be considered together. The following comments assume the default half-normal detection function, which has a short tail and spatial scale parameter σ_{HN} , unless stated otherwise.

12.4.1 A rule of thumb for buffer width

As a rule of thumb, a buffer of $4\sigma_{HN}$ is likely to be adequate (result in truncation bias of less than 0.1%). A pilot estimate of σ_{HN} may be found for a particular dataset (caphist object) with the function RPSV with the argument ‘CC’ set to TRUE:

```
RPSV(captdata, CC = TRUE)
```

```
[1] 25.629
```

This is an approximation based on a circular bivariate normal distribution that ignores the truncation of recaptures due to the finite extent of the detector array (Calhoun & Casby, 1958).

12.4.2 Buffer width for heavy-tailed detection functions

Heavy-tailed detection functions such as the hazard-rate (HR, HHR) can be problematic because they require an unreasonably large buffer for stable density estimates. They are better avoided unless there is a natural boundary.

12.4.3 Hands-free buffer selection: `suggest.buffer`

The `suggest.buffer` function is an alternative to the $4\sigma_{HN}$ rule of thumb for data from point detectors (not polygon or transect detectors). It has the advantage of allowing for the geometry of the detector array (specifically, the length of edge) and the duration of sampling. The algorithm is obscure and undocumented (this is only a suggestion!) and uses an approximation to the bias, computed by function `bias.D`. The first argument of `suggest.buffer` may be a capthist object or a fitted model. With a capthist object as input:

```
suggest.buffer(captdata, detectfn = 'HN', RBtarget = 0.001)
```

```
Warning: using automatic 'detectpar' g0 = 0.2339, sigma = 30.75
```

```
[1] 104
```

When the input is only a capthist object, the suggested buffer width relies on an estimate of σ_{HN} that is itself biased (`RPSV(captdata, CC=TRUE)`). Section 12.4.4 shows how the suggested buffer width changes when we use a better estimate of σ_{HN} . Actual bias due to mask truncation will also exceed the target ($RB = 0.1\%$) due to limitations of the *ad hoc* algorithm in `bias.D`.

12.4.4 Retrospective buffer checks

Once a model has been fitted with a particular buffer width or mask, the estimated detection parameters may be used to check whether the buffer width is likely to have resulted in mask truncation bias. We highlight two of these:

1. `secr.fit` automatically checks a mask generated from its ‘buffer’ argument (i.e. when the ‘mask’ argument is missing), using `bias.D` as in `suggest.buffer`. A warning is given when the predicted truncation bias exceeds a threshold. The threshold is controlled by the ‘biasLimit’ argument (default 0.01). To suppress the check set ‘biasLimit = NA’, or provide a pre-defined mask.

The check is performed by a cunning custom algorithm in function `bias.D`. This uses one-dimensional numerical integration of a polar approximation to site-specific detection probability, coupled to a 3-part linear approximation for the length of contours of distance-to-nearest-detector. The check cannot be performed for some detector types, and the embedded integration can give rise to cryptic error messages.

2. `esaPlot` provides a quick visualisation of the change in estimated density as buffer width changes. It is a handy check on any fitted model, and may also be used with pilot parameter values. The name of the function derives from its reliance on calculation of the [effective sampling area](#) (esa or $a(\hat{\theta})$).

```
fit <- secr.fit(captdata, buffer = 100, trace = FALSE)
```

```
par(pty = "s", mar = c(4,4,2,2), mgp = c(2.5,0.8,0), las = 1)
esaPlot(fit, ylim = c(0,10))
abline(v = 4 * 25.6, col = "red", lty = 2)
```

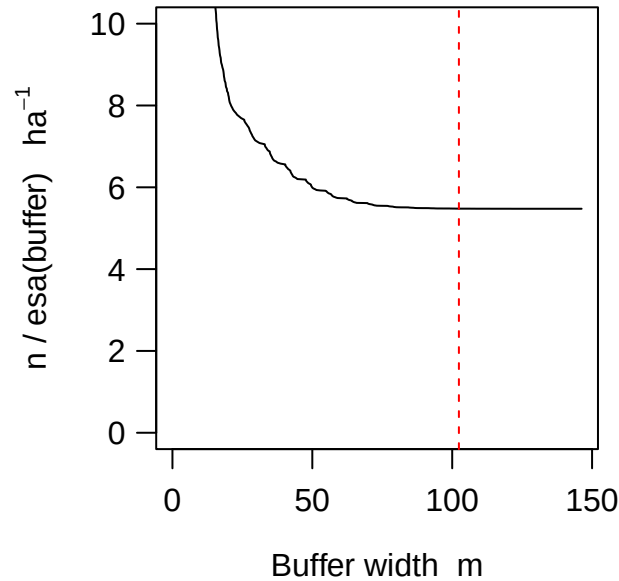


Figure 12.3: Effect of varying buffer width on estimated density (y-axis). Vertical line indicates rule-of-thumb buffer width.

The `esa` plot supports the prediction that increasing buffer width beyond the rule-of-thumb value has no discernable effect on the estimated density (Fig. 12.3).

The function `mask.check` examines the effect of buffer width and mask spacing (cell size) by computing the likelihood or re-fitting an entire model. The function generates either the log likelihood or the estimated density for each cell in a matrix where rows correspond to different buffer widths and columns correspond to different mask spacings. The function is limited to single-session models and is slow compared to `esaPlot`. See `?mask.check` for more.

Note also that `suggest.buffer` may be used retrospectively (with a fitted model as input), and

```
suggest.buffer(fit)
```

```
[1] 100
```

which is coincidental, but encouraging!

12.4.5 Buffer using non-Euclidean distances

If you intend to use a non-Euclidean distance metric then it makes sense to use this also when defining the mask, specifically to drop mask points that are distant from any detector according to the metric. See Appendix F for an example. Modelling with non-Euclidean distances also requires the user to provide `secl.fit` with a matrix of user-computed distances between detectors and mask points.

12.5 Grid cell size

Using a set of discrete locations (mask points) to represent the locations of animals is numerically convenient, and by making grid cells small enough we can certainly eliminate any effect of discretization. However, reducing cell size increases the number of cells and slows down model fitting. Trials with varying cell size (mask spacing) provide reassurance that discretization has not distorted the analysis.

In this section we report results from trials with four very different datasets for which details are given in the **secre** documentation: Maryland ovenbirds (ovenCH), Waitarere possums (possumCH), Arizona horned lizards (hornedlizardCH) and Tennessee black bears (blackbearCH). These studies used respectively mist nets, cage trapping, area search and DNA identification of hairs from barbed wire snares.

The reference scale was σ estimated earlier by fitting a half-normal detection model. In each case masks were constructed with constant buffer width 4σ and different spacings in the range 0.2σ to 3σ . This resulted in widely varying numbers of mask points (Fig. 12.4).

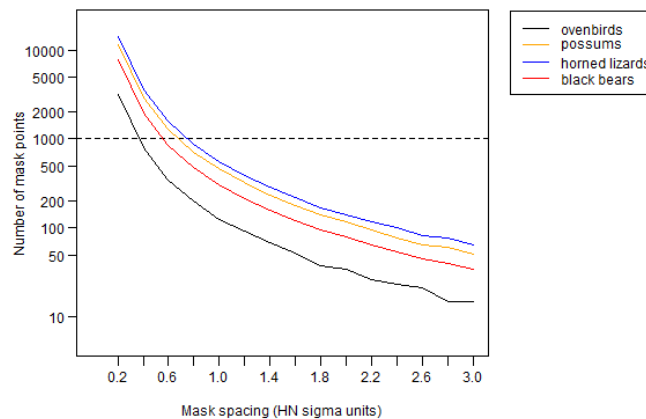


Figure 12.4: Effect of mask spacing on number of mask points for four test datasets. Detector configurations varied: a single searched square (horned lizards), a single elongated hollow grid of mistnets (ovenbirds), multiple hollow grids of cage traps (Waitarere possums), hair snares along a dense irregular network of trails (black bears).

The results in Fig. 12.5 suggest that, for a uniform density model, any mask spacing less than the half-normal σ is adequate; 0.6σ provides a considerable safety margin. The effect of detector spacing on the relationship has not been examined. Referring back to Fig. 12.4, a mask of about 1000 points will usually be adequate with a 4σ buffer.

The default spacing in **secre.fit** and **make.mask** is determined by dividing the x-dimension of the buffered area by 64. The resulting mask typically has about 4000 points, which is overkill. Substantial improvements in speed can be obtained with coarser masks, obtained by reducing ‘nx’ or ‘spacing’ arguments of **make.mask**.

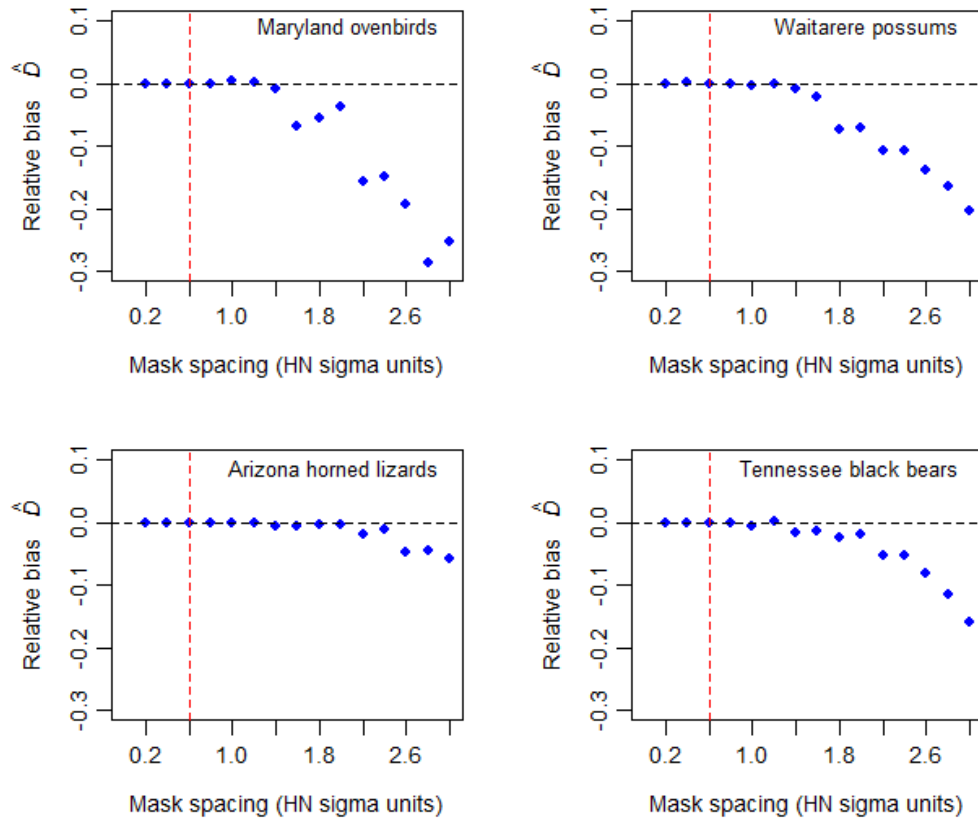


Figure 12.5: Effect of mask spacing on estimates of density from null model. Bias is relative to the estimate using the narrowest spacing. The Arizona horned lizard data appeared especially robust to mask spacing, which may be due to the method (search of a large contiguous area) or duration (14 sampling occasions) (Royle & Young, 2008).

For completeness, we revisit the question of buffer width using the `esaPlot` function with each of the four test datasets (Fig. 12.6).

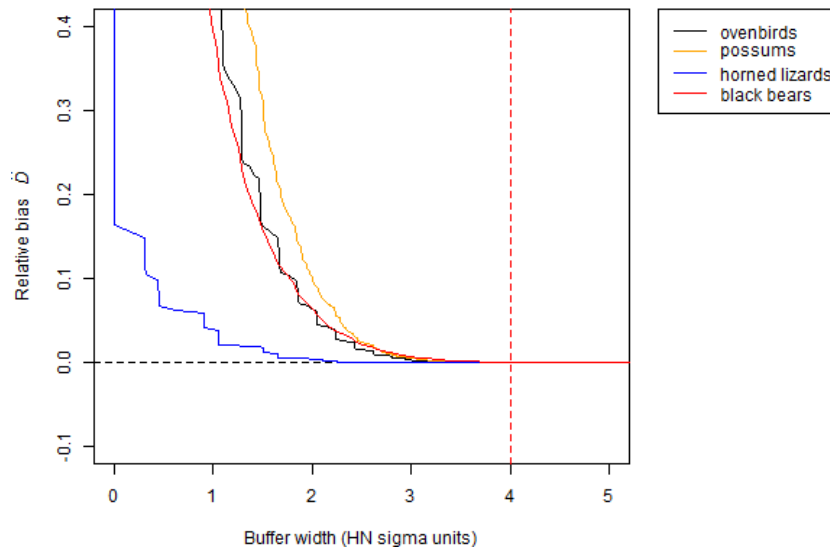


Figure 12.6: Approximate relative bias due to mask truncation for four datasets. Bias is relative to the estimate using the widest buffer.

12.6 Excluding non-habitat

Our focus so far has been on choosing a buffer width to set the outer boundary of a habitat mask, assuming that the actual boundary is arbitrary. We can call these ‘masks of convenience’ (Fig. 12.7 a); numerical accuracy and computation speed are the only constraints. At the other extreme, a mask may represent a natural island of habitat surrounded by non-habitat (Fig. 12.7 c). A geographical map, possibly in the form of an ESRI shapefile, is then sufficient to define the mask. Between these extremes there may be a habitat mosaic including both some non-habitat near the detectors and some habitat further away, so neither the buffered mask of convenience nor the habitat island is a good match (Fig. 12.7 b).

The virtue of clipping non-habitat is that the estimate of density then relates to the area of habitat rather than the sum of habitat and non-habitat. For most uses habitat-based density would seem the more meaningful parameter.

Exclusion of non-habitat (Fig. 12.7 b,c) is achieved by providing `make.mask` with a digital map of the habitat in the ‘poly’ argument. The digital map may be an R object defining spatial polygons as described in Appendix C, or simply a 2-column matrix or dataframe of coordinates. This simple example uses the coordinates in `possumarea`:

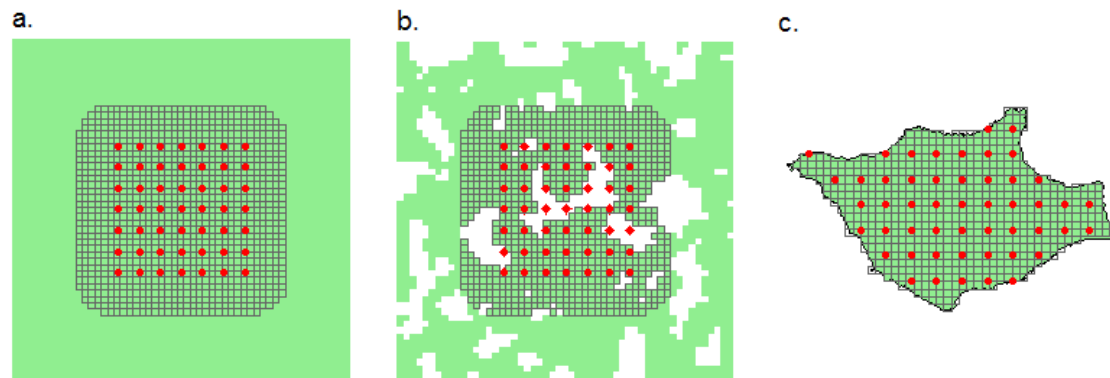


Figure 12.7: Types of habitat mask (grey mesh) defined in relation to habitat (green) and detectors (red dots). (a) mask of convenience defined by a buffer around detectors in continuous habitat, (b) mask of convenience excluding non-habitat (c) fully sampled habitat island.

```
clippedmask <- make.mask(traps(possumCH), type = 'trapbuffer', buffer = 400,
                        poly = possumarea)
```

```
par(mfrow = c(1,1), mar = c(1,1,1,1))
plot(clippedmask, border = 100, ppoly = FALSE)
polygon(possumarea, col = 'lightgreen', border = NA)
plot(clippedmask, dots = FALSE, mesh = grey(0.4), col = NA,
     add = TRUE, polycol = 'blue')
plot(traps(possumCH), detpar = list(pch = 16, cex = 0.8),
     add = TRUE)
```

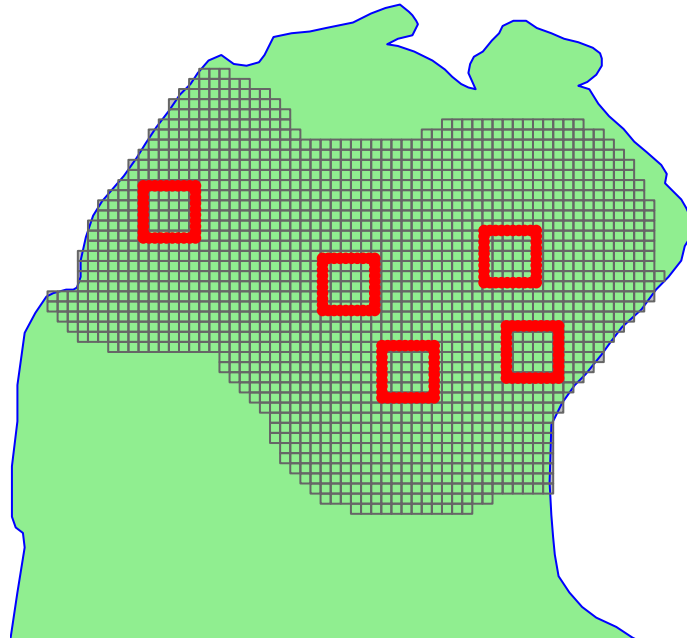


Figure 12.8: Mask computed by clipping to a polygon – the shoreline of the ‘peninsula’ at Waitarere separating the Tasman Sea (left) from the estuary of the Manawatu River (right).

 Tip

By default, data for the ‘poly’ argument are retained as an attribute of the mask. With some data sources this grossly inflates the size of the mask, and it is better to discard the attribute with `keep.poly = FALSE`.

12.7 Mask covariates

Masks may have a ‘covariates’ attribute that is a dataframe just like the ‘covariates’ attributes of traps and capthist objects. The data frame has one row for each row (point) on the mask, and one column for each covariate. The dataframe may include unused columns. Mask covariates are used for modelling [density surfaces] (D), not for modelling detection parameters (g0, lambda0, sigma).

Covariates may be categorical (factor-valued) or continuous. Character-valued covariates will be coerced to factors. Covariates in detection models ideally take a small number of discrete values, but there is no such constraint on mask covariates, which may be continuous.

12.7.1 Adding covariates

Mask covariates are always added after a mask is first constructed. Extending the earlier example, we can add a covariate for the computed distance to shore:

```
covariates(clippedmask) <- data.frame(d.to.shore =  
  distancetotrap(clippedmask, possumarea))
```

The function `addCovariates` makes it easy to extract data for each mask cell from a spatial data source. Its usage is

```
addCovariates (object, spatialdata, columns = NULL,  
  strict = FALSE, replace = FALSE)
```

Values are extracted for the point in the data source corresponding to the centre point of each grid cell. The spatial data source (`spatialdata`) should be one of

- ESRI polygon shapefile name (excluding .shp)
- sf spatial object, package `sf`
- RasterLayer, package `raster`
- SpatRaster, package `terra`
- SpatialPolygonsDataFrame, package `sp`
- SpatialGridDataFrame, package `sp`
- another mask with covariates
- a traps object with covariates

One or more input columns may be selected by name. The argument ‘strict’ generates a warning if points lie outside a mask used as a spatial data source. Appendix C has more about spatial data in `secr`.

See Chapter 11 for further applications, including smooths to represent the [scale of effect](#).

12.7.2 Repairing missing values

Covariate values become NA for points not in the data source for `addCovariates`. Modelling will fail until a valid value is provided for every mask point (ignoring covariates not used in models). If only a few values are missing at only a few points it is usually acceptable to interpolate them from surrounding non-missing values. For continuous covariates we suggest linear interpolation with the function `interp` in the **akima** package (Akima & Gebhardt, 2022). The following short function provides an interface:

```

repair <- function (mask, covariate, ...) {
  NAcov <- is.na(covariates(mask)[,covariate])
  OK <- subset(mask, !NAcov)
  require(akima)
  irect <- akima::interp (x = OK$x, y = OK$y, z =
    covariates(OK)[,covariate],...)
  irectxy <- expand.grid(x = irect$x, y = irect$y)
  i <- nearesttrap(mask[NAcov,], irectxy)
  covariates(mask)[,covariate][NAcov] <- irect[[3]][i]
  mask
}

```

To demonstrate `repair` we deliberately remove a swathe of covariate values from a copy of our clippedmask and then attempt to interpolate them (Fig. 12.9):

```

damagedmask <- clippedmask
covariates(damagedmask)$d.to.shore[500:1000] <- NA
repaired <- repair(damagedmask, 'd.to.shore', nx=60, ny=50)

```

The interpolation may potentially be improved by varying the `interp` arguments `nx` and `ny` (passed via the `...` argument of `repair`). Although extrapolation is available (with `linear = FALSE`, `extrap = TRUE`) it did not work in this case, and there remain some unfilled cells (Fig. 12.9 c).

Categorical covariates pose a larger problem. Simply copying the closest valid value may suffice to allow modelling to proceed, and this is a good solution for the few NA cells in Fig. 12.9 c. The result should always be checked visually by plotting the covariate: strange patterns may result.

```

copynearest <- function (mask, cov) {
  NAcov <- is.na(covariates(mask)[,cov])
  OK <- subset(mask, !NAcov)
  i <- nearesttrap(mask, OK)
  covariates(mask)[,cov][NAcov] <- covariates(OK)[i[NAcov],cov]
  mask
}
completed <- copynearest(repaired, 'd.to.shore')

```

See Chapter 11 for more on the use of mask covariates.

12.8 Regional population size

Population density D is the primary parameter in this implementation of spatially explicit capture–recapture (SECR). The number of individuals (population size $N(A)$) is treated as

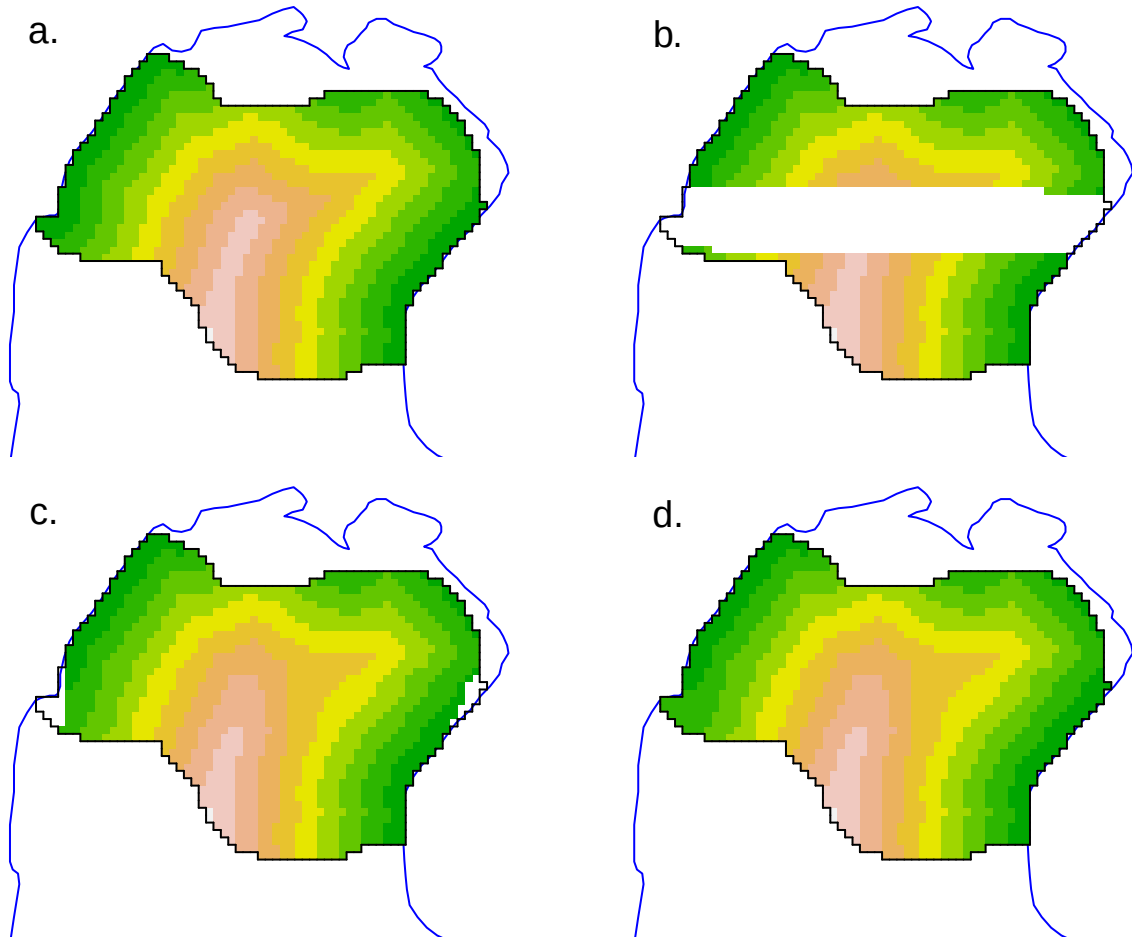


Figure 12.9: Interpolation of missing values in mask covariate (artificial example). (a) True coverage, (b) Swathe of missing values, (c) Repaired by linear interpolation. Cells in the west and east that lie outside the convex hull of non-missing points are not interpolated and remain missing, (d) repair completed by filling remaining NA cells with value from nearest non-missing cell.

a derived parameter. The rationale for this is that population size is ill-defined in many classical sampling scenarios in continuous habitat (Figs. 7a,b). Population size is well-defined for a habitat island (Fig. 12.7 c). Population size may also be well-defined for a persistent swarm, colony, herd, pack or flock, although group living is incompatible with the usual SECR assumption of independence e.g. Bischof et al. (2020).

Population size on a habitat island A may be derived from an SECR model by the simple calculation $\hat{N} = \hat{D}|A|$ if density is uniform. The same calculation yields the expected population in any area A' . Calculations get more tricky if density is not uniform as then $\hat{N} = \int_{A'} D(\mathbf{x}) d\mathbf{x}$ (computing the volume under the density surface) (Efford & Fewster, 2013). `secr` provides the function `region.N` for this purpose (Section 13.5).

12.9 Plotting masks

The default plot of a mask shows each point as a grey dot. We have used `dots = FALSE` throughout this document to emphasise the gridcell structure. That is especially handy when we use the plot method to display mask covariates:

```
par(mar = c(1,1,2,6), xpd = TRUE)
plot(clippedmask, covariate = 'd.to.shore', dots = FALSE,
     border=100, title = 'Distance to shore m', polycol = 'blue')
plotMaskEdge(clippedmask, add = TRUE)
```

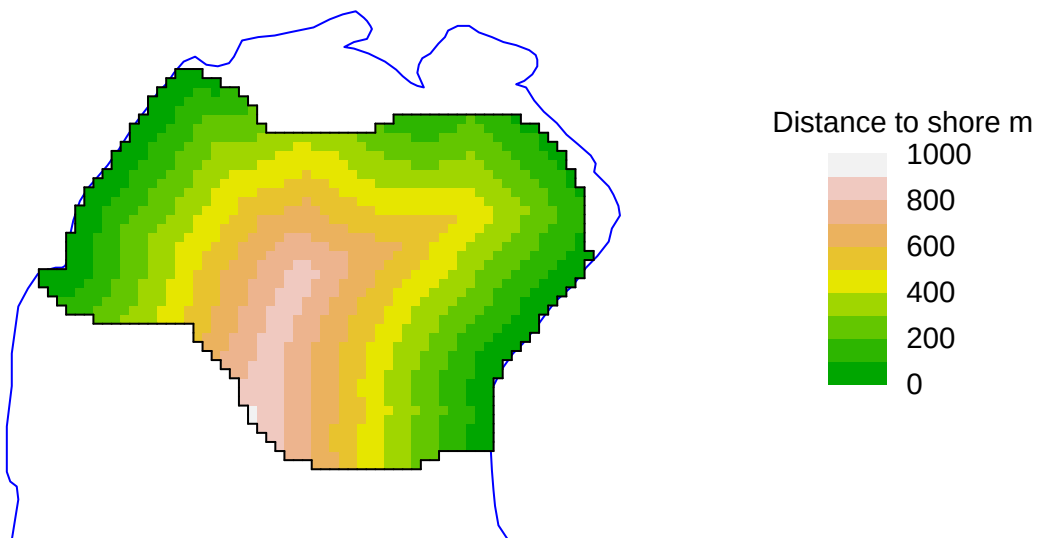


Figure 12.10: Plot of a computed continuous covariate across a clipped mask, with outer margin.

The legend may be suppressed with `legend = FALSE`. See `?plot.mask` for details. We used `plotMaskEdge` to add a line around the perimeter.

12.10 More on creating and manipulating masks

12.10.1 Arguments of `make.mask`

The usage statement for `make.mask` is included as a reminder of various options and defaults that have not been covered in this vignette. See `?make.mask` for details.

```
make.mask (traps, buffer = 100, spacing = NULL, nx = 64, ny = 64,
  type = c("traprect", "trapbuffer", "pdot", "polygon",
    "clusterrect", "clusterbuffer", "rectangular", "polybuffer"),
  poly = NULL, poly.habitat = TRUE, cell.overlap = c("centre",
    "any", "all"), keep.poly = TRUE, check.poly = TRUE,
  pdotmin = 0.001, random.origin = FALSE, ...)
```

12.10.2 Mask attributes saved by `make.mask`

Mask objects generated by `make.mask` include several attributes not usually on view. Use `str` to reveal them. Three are simply saved copies of the arguments ‘polygon’, ‘poly.habitat’ and ‘type’, and ‘covariates’ has been discussed already.

The attribute ‘spacing’ is the distance in metres between adjacent grid cell centres in either x- or y- directions.

The attribute ‘area’ is the area of a single grid cell in hectares (1 ha = 10000 m², so area = spacing² / 10000; retrieve with `attr(mask, 'area')`). Use the function `maskarea` to find the total area of all mask cells; for example, here is the area in hectares of the clipped Waitarere possum mask.

```
maskarea(clippedmask)
```

```
[1] 205.06
```

The attribute ‘boundingbox’ is a 2-column dataframe with the x- and y-coordinates of the corners of the smallest rectangle containing all grid cells.

The attribute ‘meanSD’ is a 2-column dataframe with the means and standard deviations of the x- and y-coordinates. These are used to standardize the coordinates if they appear directly in a model formula (e.g., $D \sim x + y$).

12.10.3 Linear habitat

Models for data from linear habitats analysed in the package **seclinear** (Efford, 2024) use the class ‘linearmask’ that inherits from ‘mask’. Linear masks have additional attributes ‘SLDF’ and ‘graph’ to describe linear habitat networks. See [seclinear-vignette.pdf](#) for details.

12.10.4 Dropping points from a mask

If the mask you want cannot be obtained directly with `make.mask` then use either `subset` (batch) or `deleteMaskPoints` (interactive; unreliable in RStudio). This ensures that the attributes are updated properly. Do not simply extract the required points from the mask dataframe by subscripting (`[]`).

12.10.5 Multi-session masks

Fitting a SECR model to a multi-session capthist requires a mask for each session. If a single mask is passed to `secr.fit` then it will be replicated and must be appropriate for all sessions. The alternative is to provide a list of masks, one per session, in the correct order; `make.mask` generates such a list from a list of traps objects. See Chapter 14 for details.

12.10.6 Artificial habitat maps

Function `randomHabitat` generates somewhat realistic maps of habitat that may be used in simulations. It assigns mask pixels to ‘habitat’ and ‘non-habitat’ according to an algorithm that clusters habitat cells together. The classification is saved as a covariate in the output mask, from which non-habitat cells may be dropped entirely (this was used to generate the green habitat background in Fig. 12.7 b).

12.10.7 Mask vs raster

Mask objects have a lot in common with objects of the `RasterLayer` S4 class defined in the package **raster** (Hijmans, 2023a) and the `SpatRaster` class defined in the package **terra** (Hijmans, 2023b). However, they are much simpler: no projection is specified and grid cells must be square.

A mask object may be exported as a `RasterLayer` using the `raster` method defined in **secr** for mask objects. This allows you to nominate a covariate to provide values for the `RasterLayer`, and to specify a projection.

A mask object may also be exported as a `SpatRaster` using the `rast` method defined in **secr** for mask objects. See [here](#) for an example in which **terra** functions are used to smooth a mask covariate before returning it to the mask.

Warning

Inappropriate mask spacing is a common source of problems. Model fitting can be painfully slow if the mask has too many cells. Choose the spacing (cell size) as described in [Grid cell size](#). A single mask for widely scattered clusters of traps should drop cells from wide inter-cluster spaces (set ‘type = trapbuffer’).

13 Working with fitted models

This chapter covers a miscellany of issues that arise regardless of the particular model to be fitted.

13.1 Bundling fitted models

You may have fitted one model or several. A convenient way to handle several models is to bundle them together as an object of class ‘seclist’:

```
ovenlist <- seclist(ovenbird.model.1, ovenbird.model.D)
```

Most of the following functions accept both ‘secl’ and ‘seclist’ objects. The multi-model fitting function `list.secl.fit` returns an seclist.

13.2 Recognizing failure-to-fit

Sometimes a model may appear to have fitted, but on inspection some values are missing or implausible (near zero or very large). The maximization function `nlm` may return the arcane ‘code 3’, or there may be message that a variance calculation failed.

These conditions must be taken seriously, but they are not always fatal to the analysis. See Appendix A for diagnosis and possible solutions.

If all variances (SE) are missing or extreme then the fit has indeed failed.

13.3 Prediction

A fitted model (‘secl’ object) contains estimates of the coefficients of the model. Use the `coef` method for secl objects to display the coefficients (e.g., `coef(secldemo.0)`). We note

- the coefficients (aka ‘beta parameters’) are on the link scale, and at the very least must be back-transformed to the natural scale, and
- a real parameter for which there is a [linear sub-model](#) is not unique: it takes a value that depends on covariates and other predictors.

These complexities are handled by the `predict` method for `secr` objects, as for other fitted models in R (e.g., `lm`). The desired levels of covariates and other predictors appear as columns of the ‘newdata’ dataframe. `predict` is called automatically when you print an ‘secr’ object by typing its name at the R prompt.

`predict` is commonly called with a fitted model as the only argument; then ‘newdata’ is constructed automatically by the function `makeNewData`. The default behaviour is disappointing if you really wanted estimates for each level of a predictor. This can be overcome, at the cost of more voluminous output, by setting `all.levels = TRUE`, or simply `all = TRUE`. For more customised output (e.g., for a particular value of a continuous predictor), construct your own ‘newdata’.

Density surfaces are a special case. Prediction across space requires the function `predictDsurface` as covered in Chapter 11.

13.4 Tabulation of estimates

The `collate` function is a handy way to assemble tables of coefficients or estimates from several fitted models. The input may be individual models or an `secrlist`. The output is a 4-dimensional array, typically with dimensions corresponding to

- session
- model
- statistic (estimate, SE, lcl, ucl)
- parameter (D, g0, sigma)

The parameters must be common to all models.

```
# [1,,] drops 'session' dimension for printing
collate(secrdemo.0, secrdemo.CL, realnames = c('g0','sigma'))[1,,]
```

```
, , g0
```

	estimate	SE.estimate	lcl	ucl
secrdemo.0	0.27319	0.027051	0.22348	0.32927
secrdemo.CL	0.27319	0.027051	0.22348	0.32927

```
, , sigma
```

	estimate	SE.estimate	lcl	ucl
secrdemo.0	29.366	1.3049	26.918	32.037
secrdemo.CL	29.366	1.3050	26.918	32.037

Here we see that the full- and conditional-likelihood fits produce identical estimates for the detection parameters of a Poisson model.

13.5 Population size

The primary abundance parameter in **secr** is population density D . However, population size, the expected number of AC in a region A , may be predicted from any fitted model that includes D as a parameter.

Function **region.N** calculates $\hat{N}(A)$ along with standard errors and confidence intervals (Efford & Fewster, 2013). The default region is the mask used to fit the model, but this is generally arbitrary, as we have seen, and users would be wise to specify the ‘region’ argument explicitly.

For an example of **region.N** in action, consider a model fitted to [data on black bears](#) from DNA hair snags in Great Smoky Mountains National Park, Tennessee.

```
msk <- make.mask(traps(blackbearCH), buffer = 6000, type = 'trapbuffer',  
                poly = GSM, keep.poly = FALSE) # clipped to park boundary  
bbfit <- secr.fit(blackbearCH, model = g0~bk, mask = msk, trace = FALSE)
```

The region defaults to the extent of the original habitat mask:

```
region.N(bbfit)
```

	estimate	SE.estimate	lcl	ucl	n
E.N	444.6	55.003	349.19	566.08	139
R.N	444.6	50.801	360.11	561.36	139

The two output rows relate to the ‘expected’ and ‘realised’ numbers of AC in the region. The distinction between expected (random) N and realised (fixed) N is explained by Efford & Fewster (2013). Typically the expected and realised $N(A)$ from **region.N** are the same or nearly so, but deviations occur when the density model is not uniform. The estimated sampling error is less for ‘realised’ $N(A)$ as part is fixed at the observed n .

The nominated region is arbitrary. It may be a new mask with different extent and cell size. If the model used spatial covariates the covariates must also be present in the new mask. If the density model did not use spatial covariates then the region may be a simple polygon.

It is interesting to extrapolate the number of black bears in the entire park (Fig. 13.1):

```
region.N(bbfit, region = GSM)
```

	estimate	SE.estimate	lcl	ucl	n
E.N	2091.9	258.80	1643.0	2663.5	139
R.N	2091.4	254.72	1652.5	2657.6	139

Estimates of realised population size (but not expected population size) are misleading if the new region does not cover all n detected animals.

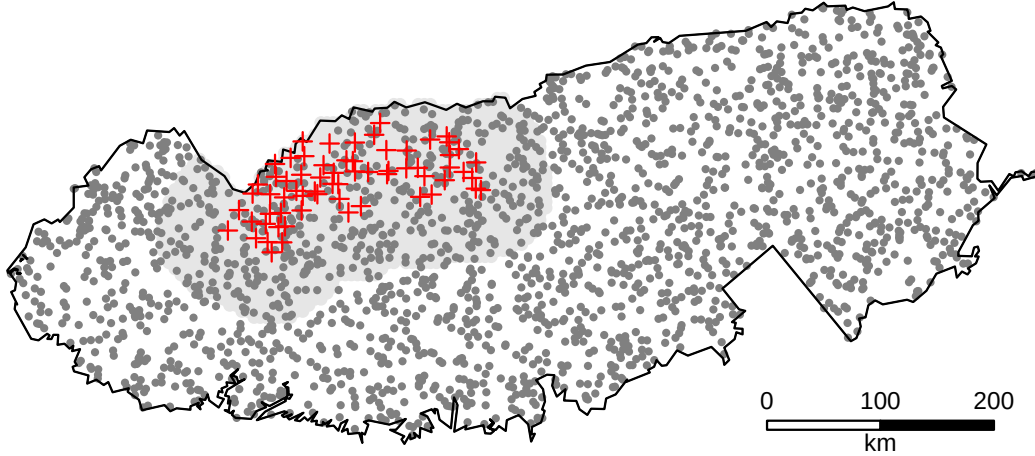


Figure 13.1: Simulated AC of black bears in Great Smoky Mountains National Park, USA. The number of AC (2092) was predicted from a model fitted to hair snags (red crosses) in one sector of the park. The habitat mask used to fit the model is shaded.

13.6 Model selection

The best model is not necessarily the one that most closely fits the data. We need a criterion that penalises unnecessary complexity. For models fitted by maximum likelihood, the obvious and widely used candidate is Akaike's Information Criterion AIC

$$\text{AIC} = -2\log[L(\hat{\theta})] + 2K$$

where K is the number of coefficients estimated and $\log[L(\hat{\theta})]$ is the maximized log likelihood.

Burnham & Anderson (2002), who popularised information-theoretic model selection for biologists, advocated the use of a small-sample adjustment due to Hurvich & Tsai (1989), designated AIC_c :

$$\text{AIC}_c = -2\log(L(\hat{\theta})) + 2K + \frac{2K(K+1)}{n-K-1}.$$

Here we assume the sample size n is the number of individuals observed at least once (i.e. the number of rows in the capthist). This is somewhat arbitrary, and a reason to question the routine use of AIC_c . The additional penalty has the effect of decreasing the attractiveness of models with many parameters. The effect is especially large when $n < 2K$.

Model selection by information-theoretic methods is open to misinterpretation. We do not attempt here to deal with the many subtleties, but raise some caveats:

- Models to be compared by AIC must be based on identical data. Some options in `secl.fit` silently change the structure of the data and make models incompatible ('CL', 'fastproximity', 'groups', 'hcov', 'binomN'). Comparisons should be made only within the family of models defined by constant settings of these options. A check of compatibility (standalone function `AICcompatible`) is applied automatically in `secl`, but this is not guaranteed to catch all misuse.
- AIC_c is widely used. However, there are doubts about the correct sample size for AIC_c , and AIC may be a better basis for model averaging, as demonstrated by simulation for generalised linear models (Fletcher, 2019, p. 60). We use AIC in this book.
- AIC values indicate the relative performance of models. Reporting AIC values *per se* is not helpful; present the differences ΔAIC between each model and the 'best' model.

13.6.1 Model averaging

Model weights may be used to form model-averaged estimates of real or beta parameters with `modelAverage` (see also Buckland et al. (1997) and Burnham & Anderson (2002)). Model weights are calculated as

$$w_i = \frac{\exp(-\Delta_i/2)}{\sum \exp(-\Delta_i/2)},$$

where Δ refers to differences in AIC or AIC_c .

Models for which Δ_i exceeds an arbitrary limit (e.g., 10) are given a weight of zero and excluded from the summation.

13.6.2 Likelihood ratio

If you have only two models, one a more general version of the other, then a [likelihood-ratio test](#) is appropriate. Here we compare an ovenbird model with density constant over time to one with a log-linear trend over years.

```
LR.test(ovenbird.model.1, ovenbird.model.D)
```

```
Likelihood ratio test for two models
```

```
data: ovenbird.model.1 vs ovenbird.model.D
X-square = 0.83, df = 1, p-value = 0.36
```

There is no evidence of a trend.

13.6.3 Model selection strategy

It is almost impossible to fit and compare all plausible SECR models. A strategy is needed to find a path through the possibilities. However, the available strategies are *ad hoc* and we cannot offer strong, evidence-based advice.

One strategy is to determine the best model for the detection process before proceeding to consider models for density. This is intuitively attractive.

Doherty et al. (2010) used simulation to assess model selection strategies for Cormack-Jolly-Seber survival analysis. They reported that the choice of strategy had little effect on bias or precision, but could affect model weights and hence affect model-averaged estimates.

We are not aware of any equivalent study for SECR.

13.6.4 Score tests

Models must usually be fitted to compare them by AIC. Score tests allow models at the next level in a tree of nested models to be assessed without fitting (McCrea & Morgan, 2011). Only the ‘best’ model at each level is fitted, perhaps spawning a further set of comparisons. This method is provided in `secr` (see [?score.test](#)), but it has received little attention and its limits are unknown.

13.7 Assessing model fit

Rigor would seem to require that a model used for inference has been shown to fit the data. This premise has many fishhooks. Tests of absolute ‘goodness-of-fit’ for SECR have uniformly low power. Assumptions are inevitably breached to some degree, and we rely in practice on soft criteria (experience, judgement and the property of robustness) as covered in Chapter 6, and the relative fit of plausible models (indicated by AIC or similar criteria).

Nevertheless, tests of goodness-of-fit are potentially informative regarding ways a model might be improved. Early maximum likelihood and Bayesian approaches to SECR spawned different approaches to goodness-of-fit testing - the parametric bootstrap and Bayesian p-values. We present these in an updated context as this is an area of active research. Each test is introduced, along with a recent approach that attempts to bridge the divide by emulating Bayesian p-values for models fitted by maximum likelihood.

13.7.1 Parametric bootstrap

If an SECR model fits a dataset then a goodness-of-fit statistic computed from the data will lie close to the median of that statistic from replicated Monte Carlo simulations. Borchers & Efford (2008), following an earlier edition of Cooch & White (2023), suggested using a test based on the scaled deviance i.e. $[-2\log\hat{L} + 2\log L_{sat}]/\Delta df$ where $\hat{L}$ is the likelihood evaluated at its maximum, L_{sat} is the likelihood of the saturated model (Table 13.1), and

Δdf is the difference in degrees of freedom between the two. The distribution of the test statistic was estimated by a parametric bootstrap i.e. by simulation from the fitted model with parameters fixed at the MLE.

Table 13.1: Expressions for the saturated likelihood of conditional, full Poisson and full binomial SECR likelihoods. n_ω is the number of individuals with detection history ω and summation is over the unique histories; N is the population in the area A and for evaluation we use an estimate $\hat{N} = \hat{D} \cdot |A|$.

Model	Likelihood of saturated model
conditional	$\log(n!) - \sum_{\omega} \log(n_{\omega}!) + \sum_{\omega} n_{\omega} \log(\frac{n_{\omega}}{n})$
Poisson	$n \log(n) - n - \sum_{\omega} \log(n_{\omega}!) + \sum_{\omega} n_{\omega} \log(\frac{n_{\omega}}{n})$
Binomial	$n \log(\frac{n}{N}) - (N - n) \log(\frac{N-n}{N}) + \log(\frac{N!}{(N-n)!}) - \sum_{\omega} \log(n_{\omega}!) + \sum_{\omega} n_{\omega} \log(\frac{n_{\omega}}{n})$

Warning

Saturated likelihoods are shown in Table 13.1 for the simplest models. There may be complications for models with groups, individual covariates etc. (cf Cooch & White, 2023, Section 5.1).

13.7.2 Bayesian p-values

Royle et al. (2014) (pp. 232–243) treated the problem of assessing model fit in the context of Bayesian SECR using Bayesian p-values (Gelman & Stern, 1996).

In essence, Bayesian p-values compare two discrepancies: the discrepancy between the observed and expected values, and the discrepancy between simulated and expected values. A discrepancy is a scalar summary statistic for the match between two sets of counts; Royle et al. (2014) used the Freeman-Tukey statistic (e.g., C. Brooks S. P. & Morgan, 2000) in preference to the conventional Pearson statistic. The statistic has this general form for M counts y_m with expected value $E(y_m)$: $T = \sum_m [\sqrt{y_m} - \sqrt{E(y_m)}]^2$.

Each pair of discrepancy statistics provides a binary outcome (Was the observed discrepancy greater than the simulated discrepancy?) and these are summarised as a ‘Bayesian p value’ i.e. the proportion of replicates in which the observed discrepancy exceeds the simulated discrepancy. The p value is expected to be around 0.5 for a model that fits.

13.7.3 Emulation of Bayesian p values

Choo et al. (2024) proposed novel simulation-based tests for SECR models fitted by maximum likelihood. Their idea was to emulate Bayesian p values by repeatedly –

1. resampling density and detection parameters from a multivariate normal distribution based on the estimated variance-covariance matrix

2. conditioning on the known (detected individuals) and locating each according to its post-hoc probability distribution (fxi in **secr**), using the resampled detection parameters
3. simulating additional individuals from the post-hoc distribution of unobserved individuals, as required to make up the resampled density, also using the resampled detection parameters
4. computing the expected values of chosen summary statistics (e.g. margins of the capthist array)
5. computing two discrepancy statistics (e.g. Freeman-Tukey) for each realisation: observed vs expected and simulated vs expected.

The explicit ‘point of difference’ of the Choo et al. (2024) approach is the propagation of uncertainty in the parameter values instead of relying on the central estimates for a parametric bootstrap. This alone would not be expected to increase the power of the test over a conventional test, and might even reduce power. However, the comparison of observed vs expected and simulated vs expected is performed separately for each realisation of the parameters, *including the distribution of AC*, and this may add power (think of a paired t-test).

13.7.4 Implementation in **secr**

The parametric bootstrap is implemented in **secr.test** and the Choo et al. (2024) emulation of Bayesian p-values is implemented in **MCgof**. We have been unable to find a convincing application of goodness-of-fit to SECR data. Rather than promote either method we prefer to await developments.

13.8 Model re-fit

The fitting of a part-fitted model saved to a progress file may be resumed with **secr.refit** (Section 9.8.7.6). The function may also be used to re-fit a completed model, or to repeat just the variance estimation (Hessian) step.

14 Multiple sessions

A ‘session’ in **secr** is a block of sampling that may be treated as independent from all other sessions. For example, sessions may correspond to trapping grids that are far enough apart that they sample non-overlapping sets of animals. Multi-session data and models combine data from several sessions. Sometimes this is merely a convenience, but it also enables the fitting of models with parameter values that apply across sessions – data are then effectively pooled with respect to those parameters.

Multi-session data are referred to as ‘stacked’ data in other contexts (Kéry & Royle, 2020, pp. 67, 76).

Dealing with multiple sessions adds another layer of complexity, and raises some entirely new issues. This chapter tries for a coherent view of multi-session analyses, covering material that is otherwise scattered.

14.1 Input

A multi-session **capthist** object is essentially an R list of single-session **capthist** objects. We assume the functions **read.capthist** or **make.capthist** will be used for data input (simulated data are considered separately later on).

14.1.1 Detections

Entering session-specific detections is simple because all detection data are placed in one file or dataframe. Each session uses a character-valued code (the session identifier) in the first column. For demonstration let’s assume you have a file ‘msCHcapt.txt’ with data for 3 sessions, each sampled on 4 occasions.

```
# Session ID Occasion Detector sex
1 1 3 H4 f
1 10 2 A1 f
1 11 4 D2 m
1 12 1 B3 f
.
.
2 1 2 A8 m
2 1 4 A8 m
```

```

2 10 4 A5 f
2 11 2 A6 f
.
.
3 1 4 D6 m
3 10 3 A1 m
3 11 2 G3 m
3 11 4 H6 m
.
.

```

(clipped lines are indicated by ‘. ’).

Given a trap layout file ‘msCHtrap.txt’ with the coordinates of the detector sites (A1, A2 etc.), the following call of `read.caphist` will construct a single-session caphist object for each unique code value and combine these in a multi-session caphist:

```
msCH <- read.caphist('data/msCHcapt.txt', 'data/msCHtrap.txt', covnames = 'sex')
```

No errors found :-)

Use the `summary` method or `str(msCH)` to examine `msCH`. Session-by-session output from `summary` can be excessive; the ‘terse’ option gives a more compact summary across sessions (columns).

```
summary(msCH, terse = TRUE)
```

```

           1  2  3
Occasions  4  4  4
Detections 52 55 42
Animals    31 31 27
Detectors  64 64 64

```

Sessions are ordered in `msCH` according to their identifiers (‘1’ before ‘2’, ‘Albert’ before ‘Beatrice’ etc.). The order becomes important for matching with session-specific trap layouts and masks, as we see later. The vector of session names (identifiers) may be retrieved with `session(msCH)` or `names(msCH)`.

14.1.2 Empty sessions

It is possible for there to be no detections in some sessions (but not all!). To create a session with no detections, include a dummy row with the value of the `noncapt` argument as the animal identifier; the default `noncapt` value is 'NONE'. The dummy row should have occasion number equal to the number of occasions and some nonsense value (e.g. 0) in each of the other fields (`trapID` etc.).

Including individual covariates as additional columns seems to cause trouble in the present version of **secr** if some sessions are empty, and should be avoided. We drop them from the example file 'msCHcapt2.txt':

```
# Session ID Occasion Detector
1 19 2 A1
1 28 2 A5
1 37 2 A6
.
.
3 25 1 A5
3 16 4 A5
3 21 3 A6
3 6 1 A7
.
.
4 NONE 4 0
```

Then,

```
msCH2 <- read.caphist('data/msCHcapt2.txt', 'data/msCHtrap.txt')
```

```
Session 4
No live releases
```

```
summary(msCH2, terse = TRUE)
```

```
      1  2  3  4
Occasions  4  4  4  4
Detections 63 52 50  0
Animals    39 29 31  0
Detectors  64 64 64 64
```

Empty sessions trigger an error in `verify.caphist`; to fit a model suppress verification (e.g., `secr.fit(msCH2, verify = FALSE)`).

If the first session is empty then either direct the `autoini` option to a later session with e.g., `details = list(autoini = 2)` or provide initial values manually in the `start` argument.

14.1.3 Detector layouts

All sessions may share the same detector layout. Then the ‘trapfile’ argument of `read.caphist` is a single name, as in the example above. The trap layout is repeated as an attribute of each component (single-session) `caphist`.

Alternatively, each session may have its own detector layout. Unlike the detection data, each session-specific layout is specified in a separate input file or traps object. For `read.caphist` the ‘trapfile’ argument is then a vector of file names, one for each session. For `make.caphist`, the ‘traps’ argument may be a list of traps objects, one per session. The first trap layout is used for the first session, the second for the second session, etc.

14.2 Manipulation

The standard extraction and manipulation functions of `secr` (`summary`, `verify`, `covariates`, `subset`, `reduce` etc.) mostly allow for multi-session input, applying the manipulation to each component session in turn. The function `ms` returns `TRUE` if its argument is a multi-session object and `FALSE` otherwise.

Plotting a multi-session `caphist` object (e.g., `plot(msCH)`) will create one new plot for each session unless you specify `add = TRUE`.

Methods that extract attributes from multi-session `caphist` object will generally return a list in which each component is the result from one session. Thus for the ovenbird mistnetting data `traps(ovenCH)` extracts a list of 5 traps objects, one for each annual session 2005–2009.

The `subset` method for `caphist` objects has a ‘sessions’ argument for selecting particular session(s) of a multi-session object.

Table 14.1: Manipulation of multi-session `caphist` objects (CH).

Function	Purpose	Input	Output
<code>join</code>	collapse sessions	multi-session CH	single-session CH
<code>MS.caphist</code>	build multi-session CH	several single-session CH	multi-session CH
<code>split</code>	subdivide CH	single-session CH multi-session CH	multi-session CH several multi-session CH

The `split` method for `caphist` objects (`?split.caphist`) may be used to break a single-session `caphist` object into a multi-session object, segregating detections by some attribute of the individuals, or by occasion or detector groupings. Also, from `secr` 5.0.1, the `split` method may be used to group sessions in a multi-session `caphist`.

14.3 Fitting

Given multi-session capthist input, `secre.fit` automatically fits a multi-session model by maximizing the product of session-specific likelihoods (Efford, Borchers, et al., 2009). For fitting a model separately to each session see the later section on [Faster fitting...](#)

14.3.1 Habitat masks

The default mechanism for constructing a habitat mask in `secre.fit` is to buffer around the trap layout. This extends to multi-session data; buffering is applied to each trap layout in turn.

Override the default buffering mechanism by specifying the ‘mask’ argument of `secre.fit`. This is necessary if you want to –

1. reduce or increase mask spacing (pixel size; default 1/64 x-range)
2. clip the mask to exclude non-habitat
3. include mask covariates (predictors of local density)
4. define non-Euclidean distances (Appendix F)
5. specify a rectangular mask (type = “traprect” vs type = “trapbuffer”)

For any of these you are likely to use the `make.mask` function (the manual alternative is usually too painful to contemplate). If `make.mask` is provided with a list of traps objects as its ‘traps’ argument then the result is a list of mask objects - effectively, a multi-session mask.

If `addCovariates` receives a list of masks and a single spatial data source then it will add the requested covariate(s) to each mask and return a new list of masks. The single spatial data source is expected to span all the regions; mask points that are not covered receive NA covariate values. As an alternative to a single spatial data source, the `spatialdata` argument may be a list of spatial data sources, one per mask, in the order of the sessions in the corresponding capthist object.

To eliminate any doubt about the matching of session-specific masks to session-specific detector arrays it is always worth plotting one over the other. We don’t have an interesting example, but

```
mask <- make.mask(traps(msCH), buffer = 80, nx = 32, type = 'trapbuffer')
par (mfrow = c(1,3), mar = c(1,1,3,1))
for (sess in 1:length(msCH)) {
  plot(mask[[sess]])
  plot(traps(msCH)[[sess]], add = TRUE)
  mtext(side=3, paste('session', sess))
}
```

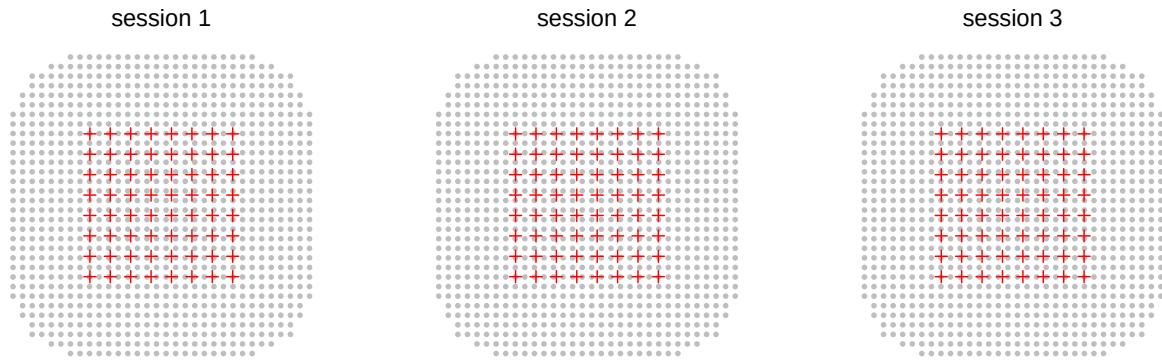


Figure 14.1

14.3.2 Session models

The default in `secr.fit` is to treat all parameters as constant across sessions. For detection functions parameterized in terms of cumulative hazard (e.g., ‘HHN’ or ‘HEX’) this is equivalent to `model = list(D ~ 1, lambda0 ~ 1, sigma ~ 1)`. Two automatic predictors are provided specifically for multi-session models: ‘session’ and ‘Session’.

14.3.2.1 Session-stratified estimates

A model with lowercase ‘session’ fits a distinct value of the parameter (D , g_0 , λ_{00} , σ) for each level of `factor(session(msCH))`.

14.3.2.2 Session covariates

Other variation among sessions may be modelled with session-specific covariates. These are provided to `secr.fit` on-the-fly in the argument ‘sessioncov’ (they cannot be embedded in the `capthist` object like detector or individual covariates). The value for ‘sessioncov’ should be a dataframe with one row per session. Each column is a potential predictor in a model formula; other columns are ignored.

Session covariates are extremely flexible. The linear trend of the ‘Session’ predictor may be emulated by defining a covariate `sessnum = 0:(R-1)` where R is the number of sessions. Sessions of different types may be distinguished by a factor-valued covariate. Supposing for the ovenbird dataset we wished to distinguish years 2005 and 2006 from 2007, 2008 and 2009, we could use `earlylate = factor(c('early','early','late','late','late'))`. Quantitative habitat attributes might also be coded as session covariates.

14.3.2.3 Trend across sessions

secr is primarily for estimating closed population density (density at one point in time), but multi-session data may also be modelled to describe population trend over time. A trend model for density may be interesting if the sessions fall in some natural sequence, such as a series of annual samples (as in the ovenbird dataset ovenCH). A model with initial uppercase ‘Session’ fits a *trend* across sessions using the session number as the predictor. The fitted trend is linear on the link scale; using the default link function for density (‘log’) this corresponds to exponential growth or decline if samples are equally spaced in time.

The pre-fitted model `ovenbird.model.D` provides an example. The coefficient ‘D.Session’ is the rate of change in $\log(D)$:

```
coef(ovenbird.model.D)
```

	beta	SE.beta	lcl	ucl
D	0.031711	0.191460	-0.34354	0.406966
D.Session	-0.063859	0.070151	-0.20135	0.073635
g0	-3.561933	0.150623	-3.85715	-3.266718
sigma	4.364111	0.081149	4.20506	4.523159

The overall finite rate of increase (equivalent to Pradel’s λ) is given by

```
beta <- coef(ovenbird.model.D)['D.Session','beta']
sebeta <- coef(ovenbird.model.D)['D.Session','SE.beta']
exp(beta)
```

```
[1] 0.93814
```

Confidence intervals may also be back-transformed with `exp`. To back-transform the SE use the delta-method approximation $\exp(\beta) * \sqrt{\exp(\text{se}\beta^2)-1} = 0.06589$.

This is fine for a single overall λ . However, if you are interested in successive estimates (session 1 to session 2, session 2 to session 3 etc.) the solution is slightly more complicated. Here we describe a simple option using ‘backward difference’ coding of the levels of the factor session, specified with the details argument ‘contrasts’. This coding is provided by the function `contr.sdif` in the **MASS** package (e.g., Venables and Ripley 1999 Section 6.2).

```
fit <- secr.fit(ovenCH, model = D~session, buffer = 300, trace = FALSE,
  details = list(contrasts = list(session = MASS::contr.sdif)))
coef(fit)
```

A more sophisticated version is provided in Appendix J.

14.4 Simulation

Back at the start of this document we used `sim.caphist` to generate `msCH`, a simple multi-session caphist. Here we look at various extensions. Generating SECR data is a 2-stage process. The first stage simulates the locations of animals to create an object of class ‘popn’; the second stage generates samples from that population according to a particular sampling regime (detector array, number of occasions etc.).

14.4.1 Simulating multi-session populations

By default `sim.caphist` uses `sim.popn` to generate a new population independently for each session. Centres are placed within a rectangular region obtained by buffering around a ‘core’ (the traps object passed to `sim.caphist`).

The session-specific populations may also be prepared in advance as a list of ‘popn’ objects (use `nsessions > 1` in `sim.popn`). This allows greater control. In particular, the population density may be varied among sessions by making argument `D` a vector of session-specific densities. Other arguments of `sim.popn` do not yet accept multi-session input – it might be useful for ‘core’ to accept a list of traps objects (or a list of mask objects if `model2D = “IHP”`).

We can also put aside the basic assumption of independence among sessions and simulate a single population open to births, deaths and movement between sessions. This does not correspond to any model that can be fitted in `secr`, but it allows the effects of non-independence to be examined. See `?turnover` for further explanation.

14.4.2 Multi-session sampling

A multi-session population prepared in advance is passed as the `popn` argument of `sim.caphist`, replacing the usual list (`D`, `buffer` etc.).

The argument ‘traps’ may be a list of length equal to `nsessions`. Each component potentially differs not just in detector locations, but also with respect to detector type (‘detector’) and resighting regime (‘markocc’). The argument ‘noccasions’ may also be a vector with a different number of occasions in each session.

14.5 Problems

There are problems specific to multi-session data.

14.5.1 Failure of `autoini`

Numerical maximization of the likelihood requires a starting set of parameter values. This is either computed internally with the function `autoini` or provided by the user. Given multi-session data, the default procedure is for `secr.fit` to apply `autoini` to the first session only. If the data for that session are inadequate or result in parameter estimates that are extreme with respect to the remaining sessions then model fitting may abort. One solution is to provide start values manually, but that can be laborious. A quick fix is often to switch the session used to compute starting values by changing the details option ‘`autoini`’. For example

```
fit0 <- secr.fit(ovenCH, mask = msk, details = list(autoini = 2), trace = FALSE)
```

A further option is to combine the session data into a single-session capthist object with `details = list(autoini = "all")`; the combined capthist is used only by `autoini`.

14.5.2 Covariates with incompatible factor levels

Individual or detector covariates used in a multi-session model obviously must appear in each of the component sessions. It is less obvious, and sometimes annoying, that a factor (categorical) covariate should have exactly the same levels in the same order in each component session. The `verify` methods for capthist objects checks that this is in fact the case (remember that `verify` is called by `secr.fit` unless you suppress it).

A common example might be an individual covariate ‘sex’ with the levels “f” and “m”. If by chance only males are detected in one of the sessions, and as a result the factor has a single level “m” in that session, then `verify` will give a warning.

The solution is to force all sessions to use the same factor levels. The function `shareFactorLevels` is provided for this purpose. For example

```
msCH <- shareFactorLevels(msCH)
```

14.6 Speed

Fitting a multi-session model with each parameter stratified by session is unnecessarily slow. In this case no data are pooled across sessions and it is better to fit each session separately. If your data are already in a multi-session capthist object then the speedy solution is

```
msk <- make.mask(traps(ovenCH[[1]]), buffer = 300, nx = 25, type = 'trapbuffer')
fits <- lapply(ovenCH, secr.fit, mask = msk, trace = FALSE)
class(fits) <- 'secrlist'
predict(fits)
```

```

$`2005`
      link estimate SE.estimate      lcl      ucl
D      log  0.846481  0.3055456  0.426310  1.680771
g0     logit  0.023247  0.0082002  0.011591  0.046079
sigma   log 88.124789 19.4766018 57.439561 135.202609

$`2006`
      link estimate SE.estimate      lcl      ucl
D      log  1.026929  0.2936740  0.592772  1.779071
g0     logit  0.030126  0.0092789  0.016396  0.054715
sigma   log 73.020562 11.7120712 53.429791 99.794558

$`2007`
      link estimate SE.estimate      lcl      ucl
D      log  1.078170  0.2879557  0.644547  1.803517
g0     logit  0.034768  0.0093266  0.020465  0.058471
sigma   log 76.079446 11.5022377 56.663447 102.148428

$`2008`
      link estimate SE.estimate      lcl      ucl
D      log  1.468097  0.488689  0.77772  2.771316
g0     logit  0.028298  0.011253  0.01289  0.060988
sigma   log 52.223520 10.478476 35.38000 77.085810

$`2009`
      link estimate SE.estimate      lcl      ucl
D      log  0.531269  0.2009517  0.259447  1.087874
g0     logit  0.024442  0.0088054  0.012003  0.049129
sigma   log 98.909108 22.8569815 63.253459 154.663661

```

The first line (`lapply`) creates a list of ‘`secr`’ objects. The `predict` method works once we set the class attribute to ‘`secrlist`’ (or you could `lapply(fits, predict)`).

```
fits2 <- secr.fit(ovenCH, model=list(D~session, g0~session,
  sigma~session), mask = msk, trace = FALSE)
```

What if we wish to compare this model with a less general one (i.e. with some parameter values shared across sessions)? For that we need the number of parameters, log likelihood and AIC summed across sessions:

```
apply(AIC(fits)[,3:5], 2, sum)
```

Warning: models not compatible for AIC

```

      npar  logLik      AIC
15.00 -925.89 1881.78

```

```
AIC(fits2)[,3:5]
```

```

      npar  logLik      AIC
fits2   15 -925.89 1881.8

```

AICc is not a simple sum of session-specific AICc and should be calculated manually (hint: use `sapply(ovenCH, nrow)` for session-specific sample sizes).

The unified model fit and separate model fits with `lapply` give essentially the same answers, and the latter approach is faster by a factor of 182.

Using `lapply` does not work if some arguments of `secr.fit` other than ‘capthist’ themselves differ among sessions (as when ‘mask’ is a list of session-specific masks). Then we can use either a ‘for’ loop or the slightly more demanding function `mapply`, with the same gain in speed.

```

# one mask per session
masks <- make.mask(traps(ovenCH), buffer = 300, nx = 32,
  type = 'trapbuffer')
fits3 <- list.secr.fit(ovenCH, mask = masks, constant =
  list(trace = FALSE))

```

14.7 Caveats

14.7.1 Independence is a strong assumption

If sessions are not truly independent then expect confidence intervals to be too short. This is especially likely when a trend model is fitted to temporal samples with incomplete population turnover between sessions. The product likelihood assumes a new realisation of the underlying population process for each session. If in actuality much of the sampled population remains the same (the same individuals in the same home ranges) then the precision of the trend coefficient will be overstated. Either an open population model is needed (e.g., [openCR](#) (Efford & Schofield, 2020)) or extra work will be needed to obtain credible confidence limits for the trend (probably some form of bootstrapping).

14.7.2 Parameters are assumed constant by default

Output from `predict.secr` for a multi-session model is automatically stratified by session even when the model does not include ‘session’, ‘Session’ or any session covariate as a predictor (the output simply repeats the constant estimates for each session).

15 Individual heterogeneity

In addition to the variation that can be attributed to covariates (e.g., sex) or specific effects (occasion, learned responses etc.) there may be variation in detection parameters among individuals that is unrelated to known predictors. This goes by the name ‘individual heterogeneity’. Unmodelled individual heterogeneity can be a major source of bias in non-spatial capture–recapture estimates of population size (Otis et al., 1978).

It has long been recognised that SECR removes one major source of individual heterogeneity by modelling differential access to detectors. However, each detection parameter (g_0, λ_0, σ) is potentially heterogeneous and a source of bias in SECR estimates of density, as we saw in Chapter 6.

Individual heterogeneity may be addressed by treating the parameter as a random effect. This entails integrating the likelihood over the hypothesized distribution of the parameter. Results are unavoidably dependent on the choice of distribution. The distribution may be continuous or discrete. Finite mixture models with a small number of latent classes (2 or 3) are a form of random effect that is particularly easy to implement (Pledger, 2000).

Continuous random effects are usually assumed to follow a normal distribution on the link scale. There are numerical methods for efficient integration of normal random effects, but these have not been implemented in **secr**. Despite its attractive smoothness, a normal distribution lacks some of the statistical flexibility of finite mixtures. For example, the normal distribution has fixed skewness, whereas a 2-class finite mixture allows varying skewness. The [likelihood](#) for finite mixtures is described separately.

i Interpreting latent classes

It’s worth mentioning a perennial issue of interpretation: Do the latent classes in a finite mixture model have biological reality? The answer is ‘Probably not’ (although the hybrid model blurs this issue). Fitting a finite mixture model does not require or imply that there is a matching structure in the population (discrete types of animal). A mixture model is merely a convenient way to capture heterogeneity.

Mixture models are prone to fitting problems caused by multimodality of the likelihood. Some comments are offered below, but a fuller investigation is needed.

In a finite mixture model no individual is known with certainty to belong to a particular class, although membership probability may be assigned retrospectively from the fitted model. A hybrid model may incorporate the known class membership of some or all individuals, as we described in Chapter 5.

15.1 Finite mixture models in secr

secr allows 2- or 3-class finite mixture models for any ‘real’ detection parameter (e.g., g_0 , λ_0 or σ of a halfnormal detection function). Consider a simple example in which we specify a 2-class mixture by adding the predictor ‘h2’ to the model formula:

Continuing with the snowshoe hares:

```
fit.h2 <- secr.fit(hareCH6, model = lambda0~h2, detectfn = 'HEX',
                  buffer = 250, trace = FALSE)
coef(fit.h2)
```

	beta	SE.beta	lcl	ucl
D	0.4244	0.133371	0.16300	0.68581
lambda0	-1.7604	0.191686	-2.13612	-1.38473
lambda0.h22	2.0999	0.912164	0.31214	3.88776
sigma	3.6616	0.083454	3.49801	3.82514
pmix.h22	-3.6168	0.964986	-5.50814	-1.72546

```
predict(fit.h2)
```

```
$`session = wickershamunburne, h2 = 1`
      link estimate SE.estimate      lcl      ucl
D      log  1.52868    0.204791  1.17704  1.98537
lambda0 log  0.17197    0.033270  0.11811  0.25039
sigma   log 38.92274    3.253912 33.04969 45.83944
pmix    logit 0.97383    0.024589  0.84883  0.99596
```

```
$`session = wickershamunburne, h2 = 2`
      link estimate SE.estimate      lcl      ucl
D      log  1.528679    0.204791  1.1770384  1.98537
lambda0 log  1.404280    1.694529  0.2190648  9.00191
sigma   log 38.922740    3.253912 33.0496948 45.83944
pmix    logit 0.026165    0.024589  0.0040373  0.15117
```

secr.fit has expanded the model to include an extra ‘real’ parameter ‘pmix’, for the proportions in the respective latent classes. You could specify this yourself as part of the ‘model’ argument, but **secr.fit** knows to add it. The link function for ‘pmix’ defaults to ‘mlogit’ (after the mlogit link in MARK), and any attempt to change the link is ignored.

There are two extra ‘beta’ parameters: $\lambda_0.h22$, which is the difference in λ_0 between the classes on the link (logit) scale, and $pmix.h22$, which is the proportion in the second class, also on the logit scale. Fitted (real) parameter values are reported separately for each mixture class ($h2 = 1$ and $h2 = 2$). An important point is that exactly the same

estimate of total density is reported for both mixture classes; the actual abundance of each class is $D \times \text{pmix}$.

Warning

When more than one real parameter is modelled as a mixture, there is an ambiguity: is the population split once into latent classes common to all real parameters, or is the population split separately for each real parameter? The second option would require a distinct level of the mixing parameter for each real parameter. **secr** implements only the ‘common classes’ option, which saves one parameter.

We now shift to a more interesting example based on the Coulombe’s house mouse *Mus musculus* dataset (Otis et al., 1978).

```
morning <- subset(housemouse, occ = c(1,3,5,7,9))
models <- list(lambda0~1, lambda0~h2, sigma~h2, list(lambda0~h2, sigma~h2))
args <- list(capthist = morning, buffer = 25, detectfn = 'HEX', trace = FALSE)
fits <- list.secr.fit(model = models, constant = args, names =
  c('null', 'h2.lambda0', 'h2.sigma', 'h2.lambda0.sigma'))
```

```
AIC(fits, sort = FALSE)[, c(3,4,7,8)]
```

	npar	logLik	dAIC	AICwt
null	3	-1270.4	33.345	0.0000
h2.lambda0	5	-1268.2	32.881	0.0000
h2.sigma	5	-1255.0	6.580	0.0359
h2.lambda0.sigma	6	-1250.7	0.000	0.9641

```
collate(fits, realnames = "D")[1,,]
```

	estimate	SE.estimate	lcl	ucl
null	1291.6	110.91	1091.8	1527.8
h2.lambda0	1312.2	115.63	1104.4	1559.0
h2.sigma	1319.3	119.30	1105.4	1574.6
h2.lambda0.sigma	1285.3	121.70	1068.0	1546.8

Although the best mixture model fits substantially better than the null model ($\Delta\text{AIC} = 33.3$), there is only a 2.6% difference in $\hat{D}$. More complex models are allowed. For example, one might, somewhat outlandishly, fit a learned response to capture that differs between two latent classes, while also allowing sigma to differ between classes:

```
model.h2xbh2s <- secr.fit(morning, model = list(lambda0~h2*bk, sigma~h2),
  buffer = 25, detectfn = 'HEX')
```

15.1.1 Number of classes

The theory of finite mixture models in capture–recapture (Pledger, 2000) allows an indefinite number of classes – 2, 3 or perhaps more. Programmatically, the extension to more classes is obvious (e.g., h3 for a 3-class mixture). The appropriate number of latent classes may be determined by comparing AIC for the fitted models.

Looking on the bright side, it is unlikely that you will ever have enough data to support more than 2 classes. For the data in the example above, the 2-class and 3-class models have identical log likelihood to 4 decimal places, while the latter requires 2 extra parameters to be estimated (this is to be expected as the data were simulated from a null model with no heterogeneity).

15.1.2 Label switching

It is a quirk of mixture models that the labeling of the latent classes is arbitrary: the first class in one fit may become the second class in another. This is the phenomenon of ‘label switching’ (Stephens, 2000).

For example, in the house mouse model ‘h2.lambda0’ the first class is initially dominant, but we can switch that by choosing different starting values for the maximization:

```
args <- list(capthist = morning, model = lambda0~h2, buffer = 25,
             detectfn = 'HEX', trace = FALSE)
starts <- list(NULL, c(7,2,1.3,2,0))
fitsl <- list.secr.fit(start = starts, constant = args,
                      names = c('start1', 'start2'))
AIC(fitsl)[,c(2,3,7,8)]
```

			detectfn	npar	dAIC	AICwt
start1	hazard	exponential		5	0	0.5
start2	hazard	exponential		5	0	0.5

```
round(collate(fitsl, realnames='pmix')[1,,,],3)
```

	estimate	SE.estimate	lcl	ucl
start1	0.922	0.087	0.528	0.992
start2	0.078	0.087	0.008	0.473

Class-specific estimates of the detection parameter (here lambda0) are reversed, but estimates of other parameters are unaffected.

15.1.3 Multimodality

The likelihood of a finite mixture model may have multiple modes (S. P. Brooks et al., 1997; Pledger, 2000). The risk is ever-present that the numerical maximization algorithm will get stuck on a local peak, and in this case the estimates are simply wrong. Slight differences in starting values or numerical method may result in wildly different answers.

The problem has not been explored fully for SECR models, and care is needed. Pledger (2000) recommended fitting a model with more classes as a check in the non-spatial case, but this is not proven to work with SECR models. It is desirable to try different starting values. This can be done simply using another model fit. For example:

```
fit.h2.2 <- secr.fit(hareCH6, model = lambda0~h2, buffer = 250,  
  detectfn = 'HEX', trace = FALSE, start = fit.h2)
```

A more time consuming, but illuminating, check on a 2-class model is to plot the profile log likelihood for a range of mixture proportions (S. P. Brooks et al., 1997). We can use the function `pmixprofileLL` in `secr` to calculate these profile likelihoods. This requires a maximization step for each value of 'pmix'; multiple cores may be used in parallel to speed up the computation. `pmixprofileLL` expects the user to identify the coefficient or 'beta parameter' corresponding to 'pmix' (argument 'pmi'):

```
pmvals <- seq(0.01,0.99,0.01)  
# use a coarse mask to make it faster  
mask <- make.mask(traps(ovenCH[[1]]), nx = 32, buffer = 200,  
  type = "trapbuffer")  
profileLL <- pmixProfileLL(ovenCH[[1]], model = list(lambda0~h2,  
  sigma~h2), pmi = 5, detectfn = 'HEX', CL = TRUE, pmvals =  
  pmvals, mask = mask, trace = FALSE)  
par(mar = c(4,4,2,2))  
plot(pmvals, profileLL, xlim = c(0,1), xlab = 'Fixed pmix',  
  ylab = 'Profile log-likelihood')
```

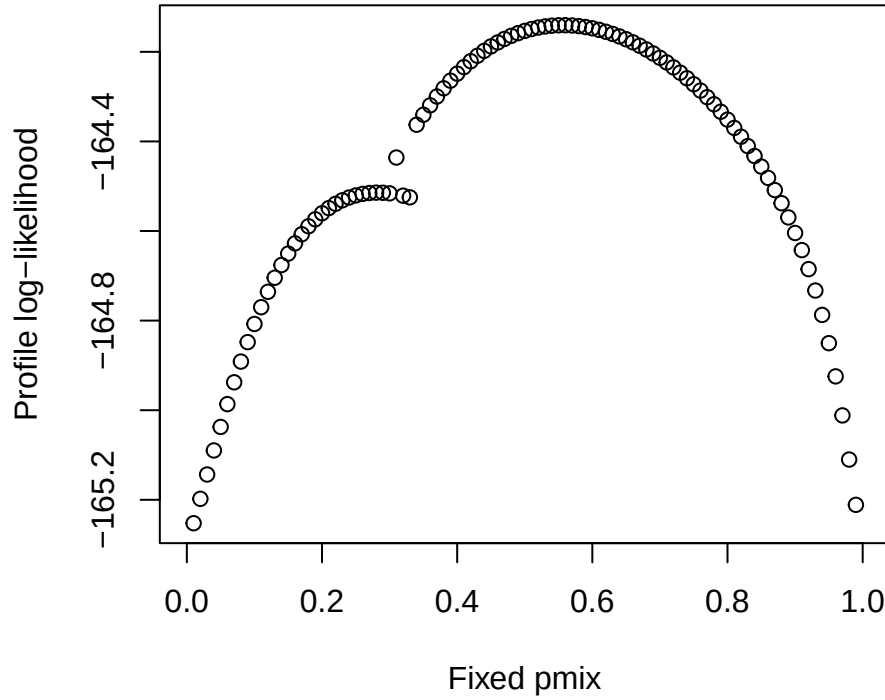



Figure 15.1: Profile log-likelihood for mixing proportion between 0.01 and 0.99 in a 2-class finite mixture model for ovenbird data from 2005

Multimodality is likely to show up as multiple rounded peaks in the profile likelihood. Label switching may cause some ghost reflections about $\text{pmix} = 0.5$ that can be ignored. If multimodality is found one should accept only estimates for which the maximized likelihood matches that from the highest peak. In the ovenbird example, the maximized log likelihood of the fitted h2 model was -163.8 and the estimated mixing proportion was 0.51, so the correct maximum was found.

Maximization algorithms (argument ‘method’ of `secl.fit`) differ in their tendency to settle on local maxima; ‘Nelder-Mead’ is probably better than the default ‘Newton-Raphson’. Simulated annealing is sometimes advocated, but it is slow and has not been tried with SECR models.

15.2 Mitigating factors

Heterogeneity may be demonstrably present yet have little effect on density estimates. Bias in density is a non-linear function of the coefficient of variation of $a_i(\theta)$. For $\text{CV} < 20\%$ the bias is likely to be negligible (Efford & Mowat, 2014).

Individual variation in λ_0 and σ may be inversely correlated and therefore compensatory, reducing bias in $\hat{D}$ (Efford & Mowat, 2014). Bias is a function of heterogeneity in the [effective sampling area](#) $a(\theta)$ which may vary less than each of the components λ_0 and σ .

It can be illuminating to [re-parameterize](#) the detection model.

15.3 Hybrid ‘hcov’ model

The hybrid mixture model accepts a 2-level categorical (factor) individual covariate for class membership that may be missing (NA) for any fraction of animals. The name of the covariate to use is specified as argument ‘hcov’ in `secr.fit`. If the covariate is missing for all individuals then a full 2-class finite mixture model will be fitted (i.e. mixture as a random effect). Otherwise, the random effect applies only to the animals of unknown class; others are modelled with detection parameter values appropriate to their known class. If class is known for all individuals the model is equivalent to a covariate (CL = TRUE) or grouped (CL = FALSE) model. When many or all animals are of known class the mixing parameter may be treated as an estimate of population proportions (probability a randomly selected individual belongs to class u). This is obviously useful for estimating sex ratio free of detection bias. See the hcov help page (`?hcov`) for implementation details, and [here](#) for the theory.

The house mouse dataset includes an individual covariate ‘sex’ with 81 females, 78 males and one unknown.

```
fit.h <- secr.fit(morning, model = list(lambda0~h2, sigma~h2),
  hcov = 'sex', buffer = 25, detectfn = 'HEX', trace = FALSE)
predict(fit.h)
```

```
$`session = coulombe, h2 = f`
      link estimate SE.estimate      lcl      ucl
D      log 1310.94402  113.457140 1106.75969 1552.79799
lambda0 log    0.28640    0.041401   0.21605   0.37965
sigma    log    2.17857    0.153649   1.89763   2.50109
pmix     logit    0.46346    0.043121   0.38077   0.54822
```

```
$`session = coulombe, h2 = m`
      link estimate SE.estimate      lcl      ucl
D      log 1310.94402  113.457140 1106.75969 1552.79799
lambda0 log    0.19836    0.035648   0.13986   0.28133
sigma    log    2.04753    0.173459   1.73479   2.41664
pmix     logit    0.53654    0.043121   0.45178   0.61923
```

16 Sex differences

It is common for males and females to differ in their behaviour in relation to detectors. Sex is commonly recorded for trapped animals or those detected with automatic cameras, and hair samples used to identify individuals by their DNA can also reveal sex given suitable sex-specific markers. SECR models that allow for sex differences are therefore of particular interest.

There are many ways to model sex differences in **secr**, and each of these has already been mentioned in a general context. Here we enumerate the possibilities and comment on their usefulness.

Unlike some effects, the relevance of sex may be obvious from the beginning. You may therefore be happy to structure other aspects of the model around your chosen way to include sex. Alternately, other considerations may constrain the treatment of sex. The example below shows that several different ways of including sex lead to the same estimates.

16.1 Models

We list the possible models in order of usefulness (your mileage may vary; see also the [decision chart](#) below):

1. Hybrid mixture model
2. Conditional likelihood with individual covariate
3. Separate sessions
4. Full likelihood with groups

16.1.1 Hybrid mixtures

This method accommodates occasional missing values and estimates the sex ratio as a parameter (pmix). The method works for both conditional and full likelihood. If estimated (CL = FALSE) the absolute density D includes both classes. The sex ratio is estimated as a parameter (pmix).

16.1.2 Individual covariate

Including sex as an individual covariate in the model requires the model to be fitted by maximizing the conditional likelihood (`CL = TRUE`). Include a categorical (factor) covariate in model formulae (e.g., $g0 \sim \text{sex}$).

Tip

Derived densities are computed from a CL model with function `derived`. To get sex-specific densities specify `groups = "sex"` in `derived`.

16.1.3 Sex as session

It is possible to model data for the two sexes as different sessions (most easily, by coding ‘female’ or ‘male’ in the first column of the capture file read with `read.caphist`). Sex differences are then modelled by including a ‘session’ term in relevant model formulae (e.g., $g0 \sim \text{session}$).

Warning

Parameters not explicitly modelled as session-specific are assumed to be shared. In a full-likelihood model you also need $D \sim \text{session}$ to allow for an uneven sex ratio.

16.1.4 Groups

[Groups](#) were described earlier. Use full likelihood (`CL = FALSE`), define `groups = "sex"` or similar, and include a group term ‘g’ in relevant formulae (e.g., $g0 \sim g$). ‘CL’ and ‘groups’ are arguments of `secl.fit`. The preceding Warning applies also to groups.

16.2 Demonstration

We re-analyse the morning house mouse data already analysed in Chapter [15](#). For all methods except the hybrid mixture we have to discard one individual of unknown sex (hence `capthist` marked with ‘x’). Close inspection of the results shows that the methods are equivalent.

```
morning <- subset(housemouse, occ = c(1,3,5,7,9)) # includes one mouse with sex unknown
morningx <- subset(housemouse, !is.na(covariates(housemouse)$sex), occ = c(1,3,5,7,9))
morningxs <- split(morningx, covariates(morningx)$sex) # split into two sessions
```

```
# conditional likelihood
hybridxCL <- secr.fit(morningx, model = list(lambda0~h2, sigma~h2),
  CL = TRUE, buffer = 25, detectfn = 'HEX',
  trace = FALSE, hcov = 'sex')
xCL <- secr.fit(morningx, model = list(lambda0~sex,
  sigma~sex), CL = TRUE, buffer = 25, detectfn = 'HEX',
  trace = FALSE)
sessionxCL <- secr.fit(morningxs, model = list(lambda0~session,
  sigma~session), CL = TRUE, buffer = 25,
  detectfn = 'HEX', trace = FALSE)
```

```
fitsCL <- secrlist(hybridxCL,xCL,sessionxCL)
predict(fitsCL, all.levels = TRUE)
```

```
$hybridxCL
```

```
$hybridxCL$`session = coulombe, h2 = f`
      link estimate SE.estimate      lcl      ucl
lambda0  log  0.28819    0.041538 0.21758 0.38171
sigma    log  2.17887    0.153223 1.89866 2.50043
pmix     logit 0.46619    0.043007 0.38363 0.55064
```

```
$hybridxCL$`session = coulombe, h2 = m`
      link estimate SE.estimate      lcl      ucl
lambda0  log  0.20098    0.036023 0.14183 0.28478
sigma    log  2.05124    0.173554 1.73830 2.42051
pmix     logit 0.53381    0.043007 0.44936 0.61637
```

```
$xCL
```

```
$xCL$`session = coulombe, sex = f`
      link estimate SE.estimate      lcl      ucl
lambda0  log  0.28819    0.041538 0.21758 0.38171
sigma    log  2.17887    0.153222 1.89866 2.50043
```

```
$xCL$`session = coulombe, sex = m`
      link estimate SE.estimate      lcl      ucl
lambda0  log  0.20098    0.036023 0.14183 0.28478
sigma    log  2.05123    0.173555 1.73829 2.42051
```

```
$sessionxCL
```

```
$sessionxCL$`session = f`
      link estimate SE.estimate      lcl      ucl
lambda0  log  0.28818    0.041538 0.21758 0.38171
sigma    log  2.17887    0.153222 1.89866 2.50044
```

```

$sessionxCL$`session = m`
      link estimate SE.estimate      lcl      ucl
lambda0 log  0.20098    0.036023 0.14183 0.28478
sigma    log  2.05123    0.173555 1.73829 2.42051

# full likelihood
hybridx <- secr.fit(morningx, model = list(lambda0~h2, sigma~h2),
                    hcov = 'sex', CL = FALSE, buffer = 25, detectfn = 'HEX',
                    trace = FALSE)
sessionx <- secr.fit(morningxs, model = list(D ~ session,
      lambda0~session, sigma~session), CL = FALSE, buffer = 25,
      detectfn = 'HEX', trace = FALSE)
# suppress bias check that gives ignorable error in secr 5.2
groupx <- secr.fit(morningx, model = list(D ~ g, lambda0~g, sigma~g),
                  groups = 'sex', CL = FALSE, buffer = 25, detectfn = 'HEX',
                  biasLimit = NA, trace = FALSE)

```

```

fits <- secrlist(hybridx,sessionx, groupx)
predict(fits, all.levels = TRUE)

```

```

$hybridx
$hybridx$`session = coulombe, h2 = f`
      link estimate SE.estimate      lcl      ucl
D        log 1296.93953 112.480024 1094.54998 1536.75226
lambda0  log   0.28819   0.041538   0.21758   0.38171
sigma    log   2.17887   0.153221   1.89866   2.50043
pmix     logit   0.46619   0.043007   0.38363   0.55064

```

```

$hybridx$`session = coulombe, h2 = m`
      link estimate SE.estimate      lcl      ucl
D        log 1296.93953 112.480024 1094.54998 1536.75226
lambda0  log   0.20098   0.036023   0.14183   0.28478
sigma    log   2.05123   0.173553   1.73830   2.42051
pmix     logit   0.53381   0.043007   0.44936   0.61637

```

```

$sessionx
$sessionx$`session = f`
      link estimate SE.estimate      lcl      ucl
D        log 604.61551  72.458474 478.44590 764.05695
lambda0  log   0.28819   0.041538   0.21758   0.38171
sigma    log   2.17887   0.153220   1.89866   2.50043

```

```

$sessionx$`session = m`

```

	link	estimate	SE.estimate	lcl	ucl
D	log	692.32637	86.800463	542.00997	884.33025
lambda0	log	0.20098	0.036023	0.14184	0.28478
sigma	log	2.05123	0.173552	1.73830	2.42050

\$groupx

\$groupx\$`session = coulombe, g = f`

	link	estimate	SE.estimate	lcl	ucl
D	log	604.61508	72.458478	478.44548	764.05655
lambda0	log	0.28819	0.041538	0.21758	0.38171
sigma	log	2.17887	0.153220	1.89866	2.50043

\$groupx\$`session = coulombe, g = m`

	link	estimate	SE.estimate	lcl	ucl
D	log	692.32574	86.800472	542.00934	884.32965
lambda0	log	0.20098	0.036023	0.14183	0.28478
sigma	log	2.05123	0.173552	1.73830	2.42050

Execution time varies considerably:

Timing, conditional likelihood

hybridxCL.elapsed	xCL.elapsed	sessionxCL.elapsed
7.02	3.66	20.20

Timing, full likelihood

hybridx.elapsed	sessionx.elapsed	groupx.elapsed
10.05	44.23	8.77

16.3 Choosing a model

16.3.1 Sex-specific detection

We suggest the decision chart in Fig. 16.1. This omits ‘session’ approaches that we previously included for [comparison](#), because they are rather slow and clunky and group models are equivalent.

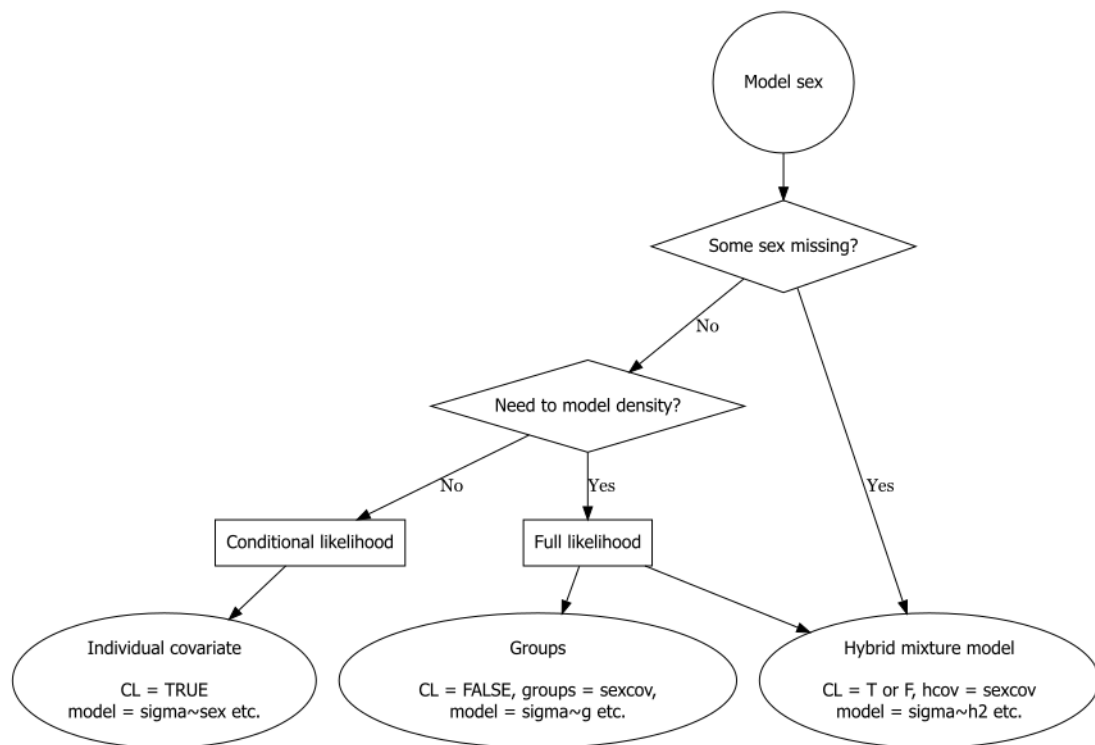


Figure 16.1: Decision chart for including sex in detection model. ‘sexcov’ is a character value naming a 2-level character or factor individual covariate after some trial and error with out.width units etc.

16.3.2 Sex-specific density

Fig. 16.1 is concerned solely with the detection model. Code for estimating sex-specific densities is available for each option as shown in Table 16.1. Sex ratio (pmix) is estimated directly only from hybrid mixture models. Post-hoc specification of ‘groups’ in `derived` works for both conditional and full likelihood models when density is constant. It is a common mistake to omit `D~g` or `D~session` from full-likelihood sex models – this forces a 1:1 sex ratio.

Table 16.1: Sex-specific estimates of density from various models. Any detection parameter may precede ‘`~`’

Fitting method	Model	Sex-specific density
Conditional likelihood	<code>~sex</code> ¹	<code>derived(fit, groups = sexcov)</code>
	<code>~h2, hcov = sexcov</code> <code>~session</code>	<code>derived(fit, groups = sexcov)</code> <code>derived(fit)</code>
Full likelihood	<code>D~g, ~g, groups = sexcov</code>	<code>predict(fit)</code>
	<code>~h2, hcov = sexcov</code> <code>D~session, ~session</code>	<code>derived(fit, groups = sexcov)</code> ² <code>predict(fit)</code>

16.4 Final comments

We note

- Only hybrid mixtures can cope with missing values of the sex covariate.
- The implementation of groups is less comprehensive and may not be available for extensions in the Appendices.
- Options should not be mixed for comparing model fit by AIC.

Sex differences in home-range size (and hence σ) may be mitigated by compensatory variation in g_0 or λ_0 (Efford & Mowat, 2014) (see also Mitigating factors in Chapter 15).

17 Simulation

Simulation has many uses in SECR; we focus particularly on

- determining properties of estimators under breaches of assumptions (Chapter 6),
- assessing candidate study designs (Chapter 8), and
- assessing goodness of fit (Chapter 13).

The first two of these are not related to the analysis of a dataset and the required ‘de novo’ simulations follow essentially the same steps in **secr**, as detailed below. The third entails simulation from a particular model fitted to data, as considered [earlier](#). Other uses of simulation are more niche – parameter estimation (Efford, 2023), testing software implementations, and the computation of parametric bootstrap confidence intervals.

This chapter aims to demystify simulation and make it available to a wider range of SECR users. We start by breaking a simulation down into the component steps, showing how each of these may be performed in **secr**. Later we introduce the package **secrdesign** that manages these steps for greater efficiency.

17.1 Simulation step by step

The essential steps for each replicate of a *de novo* SECR simulation are

1. Generate an instance of the population process, i.e. the distribution of activity centres AC.
2. Generate a set of ‘observed’ spatial detection histories from the AC distribution according to some study design (detector layout, sampling duration), and detection parameters (e.g., λ_0, σ).
3. Fit an SECR model to estimate parameters of the population and detection processes, along with their sampling variances.

It is simple enough to perform steps 1 & 2 directly in R without the use of **secr** functions. We give an example before describing functions that cover many variations with less code.

```
# Step 1. Generate population
D      <- 20                      # AC per hectare
EN     <- D * 9                  # expected number in 9 ha
N      <- rpois(1, EN)           # realised number of AC
popxy  <- matrix(runif(2*N)*300, ncol = 2) # points in 300-m square
```

```

# Step 2a. Sample from population with central 6 x 6 detector array
detxy <- expand.grid(x = seq(100,200,20), y = seq(100,200,20))
K      <- nrow(detxy)                # number of detectors
S      <- 5                          # number of occasions
g0     <- 0.2                        # detection parameter
sigma  <- 20                         # detection parameter
d      <- secr::edist(popxy, detxy)  # distance to detectors
p      <- g0 * exp(-d^2/2/sigma^2)   # probability detected
detected <- as.numeric(runif(N*K*S) < rep(p,S))
ch     <- array(detected, dim = c(N,K,S), dimnames = list(1:N, 1:K, 1:S))
ch     <- ch[apply(ch,1,sum)>0,,]      # reject null histories

# Step 2b. Cast simulated data as an secr 'capthist' object
class(detxy) <- c('traps', 'data.frame')
detector(detxy) <- 'proximity'
ch         <- aperm(ch, c(1,3,2))    # permute dim to N, S, K
class(ch)  <- 'capthist'
traps(ch)  <- detxy

```

Now that the data are in the right shape we can proceed to fit a model:

```

# Step 3. Fit model to simulated capthist
fit <- secr.fit(ch, buffer = 100, trace = FALSE)
predict(fit)

```

		link estimate	SE.estimate	lcl	ucl
D	log	19.91724	2.955932	14.91383	26.59924
g0	logit	0.21721	0.024712	0.17267	0.26951
sigma	log	18.82012	1.000238	16.95959	20.88476

This is rather laborious and it's easy to slip up. We next outline the **secr** functions `sim.popn` and `sim.capthist` that conveniently wrap Steps 1 & 2, respectively, along with many extensions.

A further level of wrapping is provided by package [secrdesign](#) that manages simulation scenarios, their replicated execution (Steps 1–3), and the summarisation of results.

17.1.1 Simulating AC distributions with `sim.popn`

A simulated AC distribution is an object of class 'popn' in **secr**. This is usually a distribution of points within a rectangular region that is the bounding box of a 'core' object (most likely a 'traps' object) inflated by a certain 'buffer' distance in metres. `sim.popn` generates a 'popn' object, for which there is a `plot` method.

```
tr <- make.grid() # a traps object, for demonstration
pop <- sim.popn(D = 10, core = tr, buffer = 100,
               model2D = "poisson")
par(mar = c(1,1,1,1))
plot(pop)
plot(tr, add = TRUE)
```

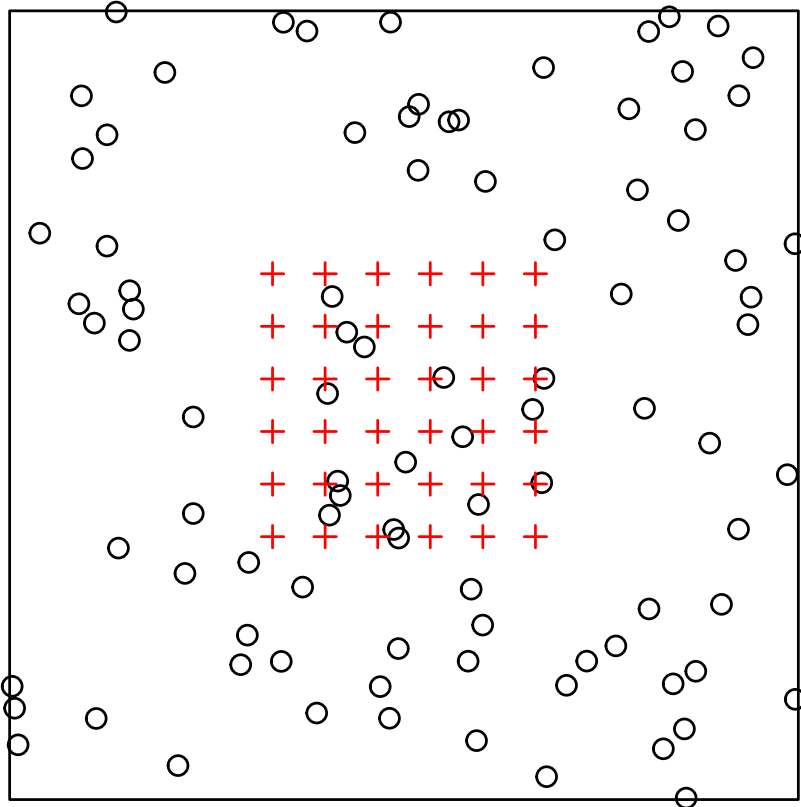


Figure 17.1

The default 2-D distribution is random uniform - a Poisson distribution, as shown here. The expected number of activity centres $E(N_A)$ is determined by the argument D , the density in AC per hectare. By default N_A is a Poisson random variable, but it may be fixed by setting `Ndist = "fixed"`. Published simulations often fix N_A for reasons of convenience such as avoiding realisations that may be hard to model or reducing the variance among replicates. Comparisons must therefore be made with care.

For `Ndist = "poisson"` the choice of buffer is not critical so long as it is large enough to include all potentially detected AC. Simulated populations with excessively large N_A take longer to sample (the next step - generating a capthist object), but the ultimate capthist is no larger and model fitting takes the same time.

Inhomogeneous alternatives to a uniform random distribution may be specified using the arguments ‘model2D’ and ‘details’. We elaborate on these [later](#).

17.1.2 Simulating detection with `sim.capthist`

To simulate sampling of a given population we specify the type and spatial distribution of detectors, the detection function, and the duration of sampling. Detectors are prepared in advance as an object of class ‘traps’, either input from a data file or generated by one of the functions for a specific geometry (`make.grid`, `trap.builder`, `make.systematic`, `make.circle` etc.).

```
ch <- sim.capthist(traps = tr, popn = pop, detectpar = list(
  g0 = 0.1, sigma = 20), nooccasions = 5, renumber = FALSE)
summary(ch)
```

```
Object class      capthist
Detector type     multi (5)
Detector number   36
Average spacing   20 m
x-range           0 100 m
y-range           0 100 m

Counts by occasion
```

	1	2	3	4	5	Total
n	11	7	7	6	7	38
u	11	3	2	0	3	19
f	8	5	5	0	1	19
M(t+1)	11	14	16	16	19	19
losses	0	0	0	0	0	0
detections	11	7	7	6	7	38
detectors visited	9	7	7	6	7	36
detectors used	36	36	36	36	36	180

```
Individual covariates
```

```
sex
F: 9
M:10
```

```
par(mar = c(1,1,1,1))
plot(ch, tracks = TRUE, rad = 3)
plot(pop, add = TRUE)
captured <- subset(pop, rownames(pop) %in% rownames(ch))
plot(captured, pch = 16, add = TRUE)
```

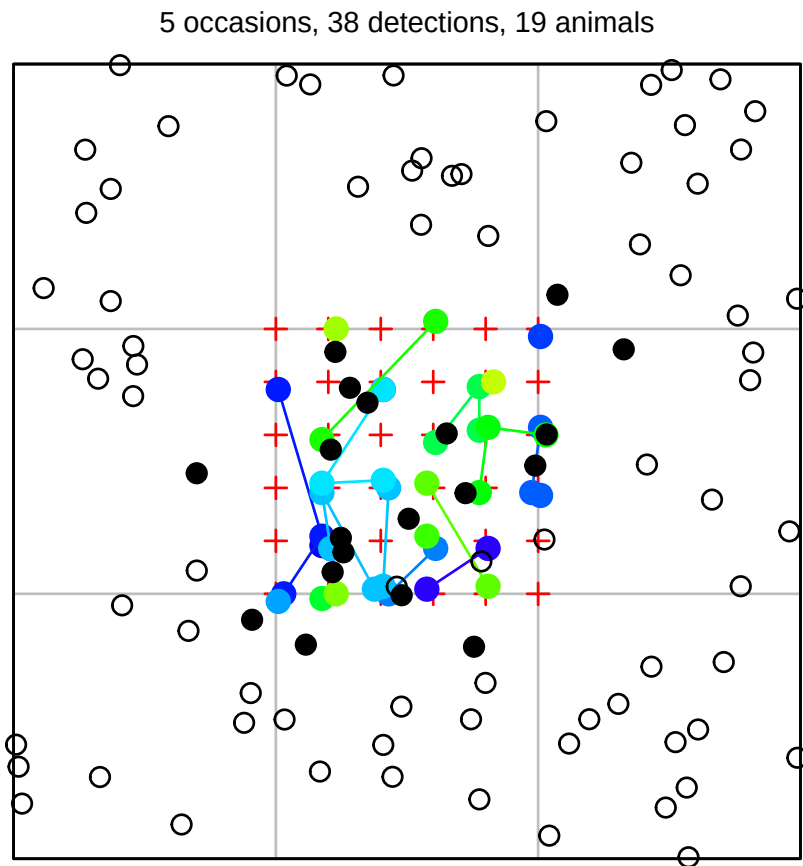


Figure 17.2

17.1.3 Combining data generation and model fitting

We loop over the AC-generation, sampling, and model-fitting steps, recording the results for later.

```
tr <- make.grid() # a traps object
nrepl <- 10
set.seed(123)
out <- list()
for (r in 1:nrepl) {
  pop <- sim.popn(D = 20, core = tr, buffer = 100,
    model2D = "poisson")
  ch <- sim.capthist(traps = tr, popn = pop, detectpar = list(
    g0 = 0.2, sigma = 20), nooccasions = 5)
  fit <- secr.fit(ch, buffer=100, trace = FALSE)
  out[[r]] <- predict(fit)
}
```

Each output returned by the `predict` method is a dataframe from which we extract the density estimate and its standard error to form the summary:

```
trueD <- 20
Dhat  <- sapply(out, '[', 'D', 'estimate')
seDhat <- sapply(out, '[', 'D', 'SE.estimate')
cat ("mean  ", mean(Dhat), '\n')
cat ("RB    ", (mean(Dhat)-trueD) / trueD, '\n')
cat ("RSE   ", mean(seDhat / Dhat), '\n')
cat ("rRMSE ", sqrt(mean((Dhat-trueD)^2)) / mean(Dhat), '\n')
```

```
mean    20.52
RB      0.025987
RSE     0.15481
rRMSE   0.14281
```

17.2 Simulation in secrdesign

The R package **secrdesign** takes care of a lot of the coding needed to specify, execute and summarise SECR simulations. Full details are in its documentation, particularly [secrdesign-vignette.pdf](#).

The heart of **secrdesign** is a dataframe of one or more *scenarios*. Usually there is one row per scenario. The scenario dataframe is constructed with `make.scenarios`. Some properties of scenarios, such as expected sample sizes, may be extracted without simulation using `scenarioSummary`. This is a good first check.

Simulations are executed with `run.scenarios`, with or without model fitting. The resulting object is summarised either directly (with the `summary` method) or after further processing to select fields and statistics. Several arguments of `run.scenarios` comprise lookup lists from which each scenario selects according to a numeric index field. For example, ‘trapset’ is a list of detector layouts, from which each scenario selects via its ‘trapsindex’ column. Likewise ‘pop.args’, ‘det.args’, and ‘fit.args’ are lists corresponding to the ‘popindex’, ‘detindex’ and ‘fitindex’ columns of the scenario data.frame. These optionally provide fine control of `sim.popn`, `sim.caphist` and `secr.fit`, respectively. The saved output may be customized via the ‘extractfn’ argument.

This small demonstration repeats the simulation in the preceding code. ‘xsigma’ specifies the buffer width as a multiple of ‘sigma’.

```
library(secrdesign)
tr  <- make.grid()
scen <- make.scenarios (D = 20, g0 = 0.2, sigma = 20,
  noccasions = 5)
sims <- run.scenarios(10, scen, tr, xsigma = 5, fit = TRUE,
```

```

    seed = 123)
## Completed scenario 1
## Completed in 0.043 minutes

```

We can get a compact summary like this:

```
estimateSummary(sims)
```

	scenario	true.D	nvalid	EST	seEST	RB	seRB	RSE	RMSE	rRMSE	COV
1	1	20	10	20.511	1.2833	0.025526	0.064167	0.15498	3.8837	0.19419	0.9

The default sampling function `sim.caphist` is usually adequate, but an alternative may be specified via the argument ‘CH.function’.

Simulation quite often results in data that are too sparse for analysis, and other malfunctions are possible. As a user you should be on your guard for meaningless, extreme values in the estimates. The `validate` function provides a mechanism for filtering these out.

17.3 Advanced simulations

17.3.1 Inhomogeneous populations

Variations on a uniform distribution of AC are selected with the `sim.popn` argument ‘model2D’. This may be done directly, as when we first introduced `sim.popn`, or indirectly via the ‘pop.args’ argument of `run.scenarios`. While almost any process may be used to generate data, only the parameters of an inhomogeneous Poisson process may be estimated by fitting models in `secl`. Alternative methods are available for some other generating models (Dey et al., 2023; Stevenson et al., 2021).

We next consider three types of point process that may be used to simulate inhomogeneous distributions of activity centres in `sim.popn`.

17.3.1.1 Inhomogeneous Poisson Process

An inhomogeneous Poisson process (IHPP) is specified by setting ‘model2D = “IHP”’. The argument ‘core’ should be a [habitat mask](#). Cell-specific density (expected number of individuals per hectare) may be given either in a covariate of the mask (named in argument ‘D’) or as a vector of values in argument ‘D’. The covariate option allows you to simulate from a previously fitted `Dsurface` (output from `predictDsurface`). Special cases of IHPP are provided in the ‘hills’ and ‘coastal’ options for ‘model2D’.

17.3.1.2 Cox Processes

In a Cox process the expected density surface of the IHPP varies randomly between realisations (i.e. replicates). For example, **secr** has the function **randomDensity** that may be used in **sim.popn** to generate a new binary density mosaic at each call (see Examples on its [help](#) page).

A flexible model for continuous variation in density is the log-Gaussian Cox Process (LGCP) (G. Dupont et al., 2021; Johnson et al., 2010; **Efford2024?**). The IHPP log-intensity surface of a LGCP is Gaussian (normally distributed) at each point, but adjacent points co-vary; autocorrelation declines with distance. The notation and parameterisation can be confusing. The term ‘Gaussian’ refers to the marginal intensity, not the autocorrelation function which is commonly exponential. The Gaussian variance is on the log scale, so even a variance as low as 1.0 indicates substantial heterogeneity. The exponential scale parameter naturally has the same units as the SECR detection function, although G. Dupont et al. (2021) rescaled it as 6% or 100% of the width of the study area. **secr** uses the function **rLGCP** from **spatstat** (Baddeley et al., 2015).

17.3.1.3 Cluster Processes

The points in a 2-D cluster process are no longer independent, as in an IHPP. Each belongs to a cluster. For the Thomas process that has been applied in SECR (e.g., Efford, Borchers, et al., 2009) there is a Poisson distribution of ‘parents’ and a bivariate-normal scatter of offspring points about each parent. Both the number of parents and the number of offspring per parent are Poisson variables, and hence some clusters may be empty. Thomas processes belong to the broader class of Neyman-Scott cluster processes. **secr** implements an interface to the function **rThomas** from **spatstat** (Baddeley et al., 2015) (`model2D = “rThomas”`).

17.3.2 Summary of inhomogeneous options

Both Cox and cluster processes require additional parameters to be specified in the ‘details’ argument of **sim.popn** (Table 17.1). Fig. 17.3 provides examples.

Table 17.1: Distribution of activity centres in **sim.popn** controlled by argument ‘model2D’ with additional parameters specified in the ‘details’ argument

Type	model2D	details	Note
Uniform	“poisson”	—	
IHPP	“IHP”	—	cell densities pre-computed
	“hills”	hills	sine-curve density in 2-D
	“coastal”	Beta	extreme gradients cf Fewster & Buckland (2004)
Cox process	“rLGCP”	var, scale	
Cluster process	“rThomas”	mu, scale	

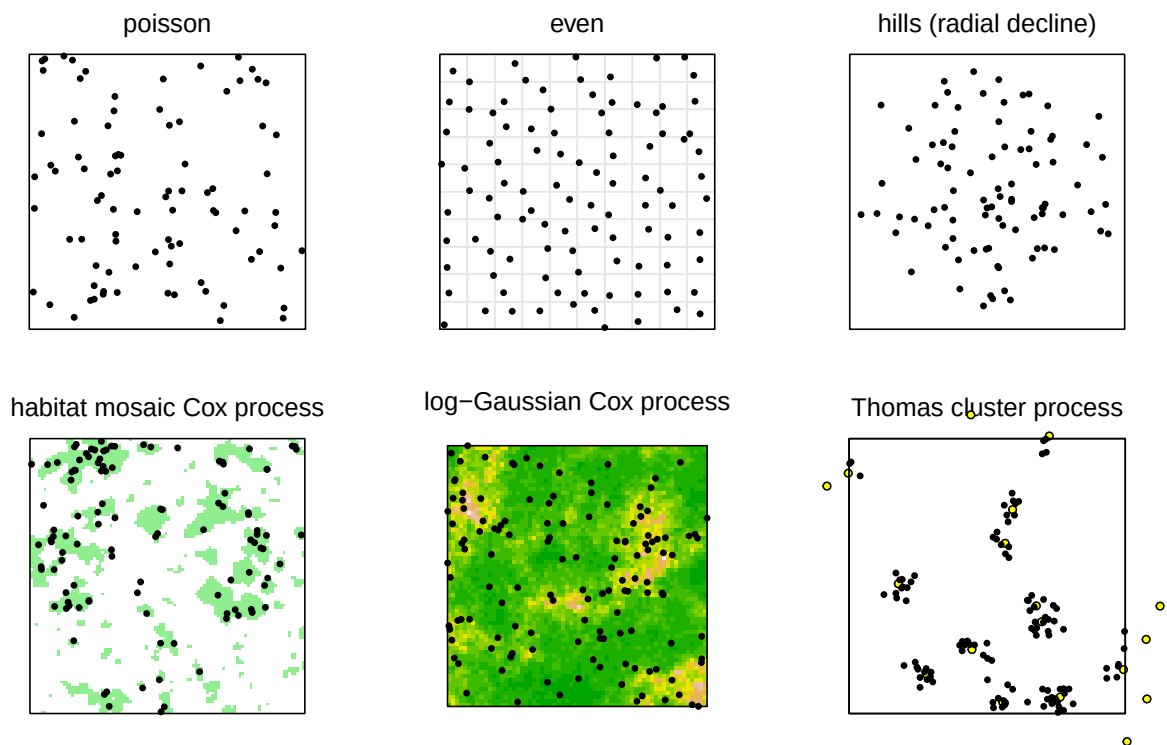


Figure 17.3: Examples of AC distributions simulated with `sim.popn`. Yellow dots indicate location of cluster ‘parents’, some of which lie outside the frame.

17.3.3 Spatial repulsion

We can imagine scenarios in which AC are distributed more evenly than expected from a random uniform distribution, perhaps due to territorial behaviour. A point process model with repulsion between AC was considered by Reich & Gardner (2014). There is no equivalent model in `sim.popn`. However, the option ‘model2D = “even”’ goes some way towards it: space is divided into notional square grid cells of area $1/D$ and one AC is located randomly within each cell (Fig. 17.3).

17.3.4 Inhomogeneous detection

Simulating spatial variation in detection parameters is a minefield: there are many subtly different approaches and *ad hoc* variations (Dey et al., 2023; Efford, 2014; Hooten et al., 2024; Moqanaki et al., 2021; Royle, Chandler, Sun, et al., 2013; Stevenson et al., 2021).

The first question is whether to focus on the location of the detector or the location of the activity centres. We label these approaches ‘detector-centric’ and ‘AC-centric’. Focusing on detectors is simpler as they comprise a few discrete locations whereas AC are at unknown locations in continuous space.

`sim.caphist` allows values of `g0` and `lambda0` to be detector-specific. Values are provided as an occasion x detector matrix.

The authors cited at the start of this section were mainly concerned with [detector-centric] spatially auto-correlated random effects (‘SARE’, following Dey et al. (2023)). Random variation among detectors may be modelled on the link scale by a Gaussian random field. Simulation is simpler than the `LGCP` for AC because values are required only at the K detectors. Deviates are obtained from a random K -dimensional multivariate normal distribution with distance-dependent covariance matrix (e.g., Moqanaki et al., 2021). Simulation in `secr` uses custom functions, for which there are several examples on [GitHub](#) with comments on parameterisation.

A purely detector-centric approach does not allow for a model in which the activity of an individual at one point in its home range depends on resource availability both there and at other points. Normalisation of activity must then account for resource distribution in continuous space (see Efford, 2014 and the preceding [GitHub](#) site). Moqanaki et al. (2021) transformed to uniform (-1.96, 1.96)

There are more arcane options for simulating AC-centric variation in detection:

- covariates ‘lambda0’ and ‘sigma’ may be added to simulated populations on the basis of the x-y coordinates of the AC, and used in `sim.caphist` by setting ‘detectpar = list(individual = TRUE)’ (requires `secr` $\geq 4.6.7$)
- the ‘userdist’ argument may be used to ‘warp’ space and hence the effective sigma (cf Appendix F)
- discrete non-overlapping populations may be simulated and pasted together *post hoc*.

A Troubleshooting secr

What could possibly go wrong when fitting spatially explicit capture–recapture models? Quite a lot. This appendix assembles a list of known difficulties with **secr** 5.2, with examples, and suggests some solutions. The scaling of covariates and coordinates was discussed earlier in relation to [density surface models](#).

A.1 Bias limit exceeded

Suppose we omitted to specify the `buffer` argument for the first snowshoe hare model in Chapter 2:

```
fit <- secr.fit (hareCH6, trace = FALSE)
```

```
Warning: using default buffer width 100 m
```

```
Warning: predicted relative bias exceeds 0.01 with buffer = 100
```

The second warning message is clearly a consequence of the first: relying on the 100-m default buffer for an animal as mobile as the snowshoe hare is likely to cause problems. See the advice on choosing the buffer width in Chapter 12.

Density is overestimated when the buffer width for a detector array in continuous habitat is too small to encompass the centres of all animals potentially detected. A check for this ‘mask truncation bias’ is performed routinely by `secr.fit` after fitting a model that uses on the ‘buffer’ argument. It may be avoided by setting `biasLimit = NA` or providing a pre-computed habitat mask in the ‘mask’ argument.

A.2 Initial log likelihood NA

Maximization will fail completely if the likelihood cannot be evaluated at the starting values. This will be obvious with `trace = TRUE`. Otherwise, the first indication will be a premature end to fitting and a lot of NAs in the estimates.

💡 Tip

It is common to see occasional NA values among the likelihoods evaluated during maximization. This is not a problem.

For an example, this section previously used the default starting values for the dataset `infraCH` (*Oligosoma infrapunctatum* skinks sampled with pitfall traps over two 3-occasion sessions labelled '6' and '7'). Unfortunately, those now seem to work, so we have to contrive an example by specifying a bad starting value for sigma:

```
fit <- secr.fit(infraCH, buffer = 25, start = list(sigma = 2),
               trace = TRUE)
```

Checking data

Preparing detection design matrices

Preparing density design matrix

Finding initial parameter values...

Initial values D = 249.78708, g0 = 0.12696, sigma = 2.87777

Maximizing likelihood...

Eval	Loglik	D	g0	sigma
1	NA	5.5206	-1.9281	0.6931
2	NA	5.5206	-1.9281	0.6931
3	NA	5.5206	-1.9281	0.6931
4	NA	5.5206	-1.9281	0.6931
5	NA	5.5206	-1.9281	0.6931
6	NA	5.5212	-1.9281	0.6931
7	NA	5.5206	-1.9280	0.6931
8	NA	5.5206	-1.9281	0.6932
9	NA	5.5217	-1.9281	0.6931
10	NA	5.5212	-1.9280	0.6931
11	NA	5.5212	-1.9281	0.6932
12	NA	5.5206	-1.9279	0.6931
13	NA	5.5206	-1.9280	0.6932
14	NA	5.5206	-1.9281	0.6933

Completed in 4.3 seconds at 17:29:39 17 Mar 2025

Unsurprisingly, the problem can be addressed by manually providing a better starting value for sigma:

```
fit1 <- secr.fit(infraCH, buffer = 25, start = list(sigma = 5),
                 trace = FALSE)
```

The original problem seems to have been due to a discrepancy between the two sessions (try `RPSV(infraCH, CC = TRUE)`). The default sigma was suitable for the first session and not

the second, whereas a larger sigma suits both. Rather than manually providing the starting value we could have directed `secr.fit` to use the second session for determining starting values:

```
fit2 <- secr.fit(infraCH, buffer = 25, details = list(autoini = 2),
               trace = FALSE)
```

A.3 Variance calculation failed

If warnings had not been suppressed in the preceding example we would have seen

```
Warning in secr.fit(infraCH, buffer = 25, details = list(autoini = 2), trace =
FALSE): at least one variance calculation failed
```

Warning message:

```
In secr.fit(infraCH, buffer = 25, details = list(autoini = 2), trace =
FALSE) :
at least one variance calculation failed
```

Examination of the output would reveal missing values for SE, lcl and ucl in both the coefficients and predicted values for `g0`.

Failure to compute the variance can be a sign that the model is inherently non-identifiable or that the particular dataset is inadequate (e.g., Gimenez et al., 2004). However, here it is due to a weakness in the default algorithm. Call `secr.refit` with `method = "none"` to recompute the variance-covariance matrix using a more sophisticated approach:

```
fit2r <- secr.refit(fit2, method = "none")
```

Checking data

Preparing detection design matrices

Preparing density design matrix

Computing Hessian with `fdHess` in `nlme`

1	-2260.348	5.5122	-2.2341	1.4912
2	-2260.355	5.5177	-2.2341	1.4912
3	-2260.348	5.5122	-2.2318	1.4912
4	-2260.350	5.5122	-2.2341	1.4927
5	-2260.355	5.5067	-2.2341	1.4912
6	-2260.349	5.5122	-2.2363	1.4912
7	-2260.350	5.5122	-2.2341	1.4898
8	-2260.357	5.5177	-2.2318	1.4912
9	-2260.360	5.5177	-2.2341	1.4927
10	-2260.352	5.5122	-2.2318	1.4927

```
11 -2260.348 5.5122 -2.2341 1.4912
Completed in 3.24 seconds at 17:30:54 17 Mar 2025
```

```
predict(fit2r)
```

```
$`session = 6`
      link estimate SE.estimate      lcl      ucl
D      log 247.686669 13.9919012 221.746245 276.66167
g0     logit 0.096732 0.0085376 0.081242 0.11481
sigma   log 4.442619 0.1636278 4.133318 4.77507
```

```
$`session = 7`
      link estimate SE.estimate      lcl      ucl
D      log 247.686669 13.9919012 221.746245 276.66167
g0     logit 0.096732 0.0085376 0.081242 0.11481
sigma   log 4.442619 0.1636278 4.133318 4.77507
```

The trapping sessions were only 4 weeks apart in spring 1995. We can further investigate session differences by fitting a session-specific model. The fastest way to fit fully session-specific models is to fit each session separately; `lapply` here applies `secr.fit` separately to each component of `infraCH`:

```
fits3 <- lapply(infraCH, secr.fit, buffer = 25, trace = FALSE)
class(fits3) <- "seclist" # ensure secr will recognise the fitted models
predict(fits3)
```

```
$`6`
      link estimate SE.estimate      lcl      ucl
D      log 255.75797 28.379462 205.90496 317.68122
g0     logit 0.17127 0.029913 0.12029 0.23802
sigma   log 2.51903 0.181318 2.18798 2.90017
```

```
$`7`
      link estimate SE.estimate      lcl      ucl
D      log 278.022858 19.009440 243.191611 317.84283
g0     logit 0.098943 0.011942 0.077877 0.12494
sigma   log 4.951085 0.226951 4.525879 5.41624
```

Notice that there is no issue with starting values when the sessions are treated separately. The skinks appeared to enlarge their home ranges as the weather warmed; they may also have become more active overall¹. It is plausible that density did not change: the estimate increased slightly, but there is substantial overlap of confidence intervals.

¹Home-range area increased about 4-fold; g0 showed some compensatory decrease, but compensation was incomplete, implying increased total activity (treating g_0 as an approximation to λ_0 ; see Efford and Mowat 2014).

A.4 Log likelihood becomes NA or improbably large after a few evaluations

The default maximization method (Newton-Raphson in function `nlm`) takes a large step away from the initial values at evaluation `np + 3` where `np` is the number of estimated coefficients. This often results in a very negative or NA log likelihood, from which the algorithm quickly returns to a reasonable part of the parameter space. However, for some problems it does not return and estimation fails, possibly with a weird message about infinite density. Two solutions are suggested:

- change to the more robust Nelder-Mead maximization algorithm

```
secr.fit(CH, method = "Nelder-Mead", ...)
```

- vary the scaling of each parameter in `nlm` by passing the `typsize` (typical size) argument. The default is `typsize = rep(1, np)`. Suppose your model has four coefficients and it is the second one that appears to be behaving wildly:

```
secr.fit(CH, typsize = c(1, 0.1, 1, 1), ...)
```

In these examples `CH` is your capthist object and `...` indicates other arguments of `secr.fit`.

A.5 Possible maximization error: `nlm` code 3

The default algorithm for numerical maximization of the likelihood is `nlm` in the base R **stats** package. That uses a Newton-Raphson algorithm and numerical derivatives. It was chosen because it is significantly faster than the alternatives. However, it sometimes returns estimates with the ambiguous result code 3, which means that the optimization process terminated because “[the] last global step failed to locate a point lower than estimate. Either estimate is an approximate local minimum of the function or `steptol` is too small.”

Here is an example:

```
fit3 <- secr.fit(infraCH, buffer = 25, model = list(g0~session,  
  sigma~session), details = list(autoini = 2), trace = FALSE)
```

Warning message:

```
In secr.fit(infraCH, buffer = 25, model = list(g0 session, sigma :  
possible maximization error: nlm returned code 3. See ?nlm
```

The results seem usually to be reliable even when this warning is given. If you are nervous, you can try a different algorithm in `secr.fit` – “Nelder-Mead” is recommended. We can derive starting values from the earlier fit:


```
fit3nm <- secr.fit(infraCH, buffer = 25, model = list(g0~session,
  sigma~session), method = "Nelder-Mead", start = fit3, trace = FALSE)
```

There is no warning. Comparing the density estimates we see a trivial difference in the SE and confidence limits and none at all in the estimates themselves:

```
collate(fit3, fit3nm)[1,, 'D']
```

	estimate	SE.estimate	lcl	ucl
fit3	271.95	15.728	242.83	304.56
fit3nm	271.95	15.810	242.68	304.74

This suggests that only the variance-covariance estimates were in doubt, and it would have been much quicker merely to check them with `method = "none"` as in the previous section.

A.6 Possible maximization error: nlm code 4

The `nlm` Newton-Raphson algorithm may also finish with the result code 4, which means that the optimization process terminated when the maximum number of iterations was reached (“iteration limit exceeded”). The maximum is set by the argument `iterlim` which defaults to 100 (each ‘iteration’ uses several likelihood evaluations to numerically determine the gradient for the Newton-Raphson algorithm).

The number of iterations can be checked retrospectively by examining the `nlm` output saved in the ‘fit’ component of the fitted model. Ordinarily `nlm` uses less than 50 iterations (for example `fit3$fit$iterations = 41`).

A ‘brute force’ solution is to increase `iterlim` (`secr.fit()` passes `iterlim` directly to `nlm()`) or to re-fit the model starting at the previous solution (`start = oldfit`). This is not guaranteed to work. Alternative algorithms such as `method = 'Nelder-Mead'` are worth trying, but they may struggle also.

There does not appear to be a universal solution. Slow or poor fitting seems more common when there are many beta parameters, and when one or more parameters is very imprecise, at a boundary, or simply unidentifiable. It is suggested that you examine the coefficients of the provisional result with `coef(fit)` and seek to eliminate those that are not identifiable.

Tricks include:

- combining levels of poorly supported factor covariates
- fixing the value of non-identifiable beta parameters with details argument `fixedbeta`
- ensuring that all levels of a factor x factor interaction are represented in the data (possibly by defining a single factor with valid levels)
- changing the coding of factor covariates with details argument `contrasts`.

The following code demonstrates fixing a beta parameter, although it is neither needed nor recommended in this case.

```
# review the fitted values
coef(fit3)
```

	beta	SE.beta	lcl	ucl
D	5.60561	0.057786	5.49235	5.71887
g0	-1.62698	0.196979	-2.01305	-1.24091
g0.session7	-0.57547	0.219967	-1.00659	-0.14434
sigma	0.91977	0.071337	0.77995	1.05959
sigma.session7	0.68209	0.082571	0.52026	0.84393

```
# extract the coefficients
betafix <- coef(fit3)$beta
# set first 4 values to NA as we want to estimate these
betafix[1:4] <- NA
betafix
```

```
[1]      NA      NA      NA      NA 0.68209
```

```
# refit, holding last coefficient constant
fit3a <- secr.fit(infraCH, buffer = 25, model = list(g0~session, sigma~session),
  details = list(autoini = 2, fixedbeta = betafix), trace = FALSE)
coef(fit3a)
```

	beta	SE.beta	lcl	ucl
D	5.60561	0.058632	5.49069	5.72052
g0	-1.62693	0.148920	-1.91881	-1.33505
g0.session7	-0.57556	0.133777	-0.83776	-0.31336
sigma	0.91978	0.040899	0.83962	0.99994

Note that the estimated coefficients ('beta') have not changed, but the estimated 'SE.beta' of each detection parameter has dropped - a result of our spurious claim to know the true value of 'sigma.session7'.

There is no direct mechanism for holding the beta parameters for different levels of a factor (e.g., `session`) at a single value. The effect can be achieved by defining a new factor covariate with combined levels.

A.7 `secr.fit` requests more memory than is available

In `secr` 5.2 the memory required by the external C code is at least $32 \times C \times M \times K$ bytes, where C is the number of distinct sets of values for the detection parameters (across all individuals, occasions, detectors and finite mixture classes), M is the number of points in the habitat mask and K is the number of detectors. Each distinct set of values appears as a row in a lookup table² whose columns correspond to real parameters; a separate parameter index array (PIA) has entries that point to rows in the lookup table. Four arrays with dimension $C \times M \times K$ are pre-filled with, for example, the double-precision (8-byte) probability an animal in mask cell m is caught in detector k under parameter values c .

The number of distinct parameter sets C can become large when any real parameter (g_0 , λ_0 , σ) is modelled as a function of continuous covariates, because each unit (individual, detector, occasion) potentially has a unique level of the parameter. A rough calculation may be made of the maximum size of C for a given amount of available RAM. Given say 6GB of available RAM, $K = 200$ traps, and $M = 4000$ mask cells, C should be substantially less than $6e9 / 200 / 4000 / 32 \approx 234$. Allowance must be made for other memory allocations; this is simply the largest.

There is a different lookup table for each session; the limiting C is for the most complex session. The memory constraint concerns detection parameters only.

Most analyses can be configured to bring the memory request down to a reasonable number.

1. C may be reduced by replacing each continuous covariate with one using a small number of discrete levels (e.g. the mid-points of weight classes). For example, `weightclass <- 10 * trunc(weight/10) + 5` for midpoints of 10-g classes.
2. M can be reduced by building a habitat mask with an appropriate spacing (see [secr-habitatmasks.pdf]).
3. K might seem to be fixed by the design, but in extreme cases it may be appropriate to combine data from adjacent detectors (see [Collapsing detectors](#)).

The `mash` function (see [Mashing](#)) may be used to reduce the number of detectors when the design uses many identical and independent clusters. Otherwise, apply your ingenuity to simplify your model, e.g., by casting ‘groups’ as ‘sessions’. Memory is less often an issue on 64-bit systems (see also `?Memory-limits`).

A.8 Covariate factor levels differ between sessions

This is fairly explicit; `secr.fit` will stop if you include in a model any covariate whose factor levels vary among sessions, and `verify` will warn if it finds any covariate like this. This commonly occurs in multi-session datasets with ‘sex’ as an individual covariate when only

²You can see this table by running `secr.fit` with `details = list(debug = 3)` and typing `Xrealparval` at the browser prompt (type Q to exit).

males or only females are detected in one session. Use the function [shareFactorLevels](#) to force covariates to use the same superset of levels in all sessions.

A.9 Estimates depend on starting values

Instability of the estimates can result when the likelihood surface has a local maximum and is said to be ‘multimodal’. Numerical maximization may then fail to find the true maximum from a given starting point. Nelder-Mead is more robust than other methods.

Finite mixture models have known problems due to multimodality of the likelihood, as discussed separately (Chapter 15). See Dawson & Efford (2009) and the vignette [secre-sound.pdf](#) for another example of a multimodal likelihood in SECR.

B Faster is better

There is nothing virtuous about waiting days for a model to fit if there is a faster alternative. Here are some things you can do.

B.1 Mask tuning

Consider carefully the necessary extent of your habitat mask and the acceptable cell size (Chapter 12 has detailed advice). If your detectors are clustered then your mask may have gaps between the clusters (use `type = 'trapbuffer'` in `make.mask`). Masks with more than 2000 points are generally excessive (and the default is about 4000!).

B.2 Conditional likelihood

The default in `secr.fit` is to maximize the full likelihood (i.e., to jointly fit both the state process and the observation process). If you do not need to model spatial, temporal or group-specific variation in density (the sole real parameter of the state model) then you can save time by first fitting only the observation process. This is achieved by maximizing only the likelihood conditional on n , the number of detected individuals (Borchers & Efford, 2008). Conditional likelihood maximization is selected in `secr.fit` by setting `CL = TRUE`.

Selecting an observation model with `CL = TRUE` (and first focussing on detection parameters) is a good strategy even if you intend to model density later. It may occasionally be desirable to re-visit the selection if covariates can affect both density and detection parameters.

```
fit <- secr.fit(hareCH6, buffer = 250, CL = FALSE, trace = FALSE) # default
fitCL <- secr.fit(hareCH6, buffer = 250, CL = TRUE, trace = FALSE)
```

Fitting time is reduced by 42% because maximization is over two parameters (g_0 , σ) instead of three. The relative reduction will be less for more complex detection models, but still worthwhile.

Having selected a suitable observation model with `CL = TRUE` you can then either resort to a full-likelihood fit to estimate density, or compute a Horvitz-Thompson-like (HT) estimate in function `derived`. In models without individual covariates the HT estimate is $n/a(\hat{\theta})$ where n is the number of detected individuals, θ represents the parameters of the observation model, and a is the effective sampling area as a function of the estimated detection parameters.

Compare

```
predict(fit)      # CL = FALSE
```

	link	estimate	SE.estimate	lcl	ucl
D	log	1.465987	0.1913125	1.136361	1.891228
g0	logit	0.061584	0.0093005	0.045685	0.082536
sigma	log	68.345777	4.4693126	60.132424	77.680973

```
derived(fitCL)    # CL = TRUE
```

	estimate	SE.estimate	lcl	ucl	CVn	CVa	CVD
esa	46.385	NA	NA	NA	NA	NA	NA
D	1.466	0.19051	1.1376	1.8892	0.12127	0.046715	0.12995

Estimated density is exactly the same, to 6 significant figures (1.46599). This is expected when n is Poisson; slight differences arise when n is binomial (because the number of animals N in the masked area is considered fixed rather than random). The estimated variance differs slightly - that from `derived` follows an unpublished and slightly *ad hoc* procedure (Borchers & Efford, 2007).

B.3 Mashing

Mashing is a very effective way of speeding up estimation when the design uses many replicate clusters of detectors, each with the same geometry, and clusters are far enough apart that animals are not detected on more than one. The approach for M clusters is to overlay all data as if from a single cluster; the estimated density will be M times the per-cluster estimate, and SE etc. will be inflated by the same factor. This relies on individuals being detected independently of each other, which is a standard assumption in any case. The present implementation assumes density is uniform.

The capthist object is first collapsed with function `mash` into one using the geometry of a single cluster. The object retains a memory of the number of individuals from each original cluster in the attribute 'n.mash'. Functions `derived`, `derivedMash` and the method `predict.secr` use 'n.mash' to adjust their output density, SE, and confidence limits.

We describe in general terms an actual example in which 18 separated clusters of 12 traps were operated on 6 occasions. Each cluster had the same geometry (two parallel rows of traps separated by 200 m along and between rows). Trap numbering was consistently up one row and down the other. The capthist object CH included detections of 150 animals in the 216 traps. To mash these data we first assign attributes for the cluster number (`clusterID`) and the sequence number of each trap within its cluster (`clustertrap`). The function `mash` then collapses the data as if all detections were made on one cluster. A mask based on this single notional cluster has many fewer points than a mask with the same spacing spanning all clusters.

```
clustertrap(traps(CH)) <- rep(1:12,18)
clusterID(traps(CH)) <- rep(1:18, each = 12)
mashedCH <- mash(CH)
mashedmask <- make.mask(traps(mashedCH), buffer = 900, spacing = 100,
                        type = "trapbuffer")
fitmask <- secr.fit(mashedCH, mask = mashedmask)
```

The model for the mashed data fitted in 4% of the time required for the original. The mashed estimate of density shrank by 2% in this case, which is probably due to slight variation among clusters in the actual spacing of traps (one cluster was arbitrarily chosen to represent all clusters). Mashing prevents the inclusion of cluster-specific detail in the model (such as discontinuous habitat near the traps). For further details see `?mash`.

B.4 Parallel fitting

Your computer almost certainly has multiple cores, allowing computations to run in parallel.

Multi-threading in `secr.fit` uses multiple cores. By default only 2 cores are used (a limit set by CRAN), and this should almost certainly be increased. The number of threads is set with `setNumThreads()` or the `'ncores'` argument. The marginal benefit of increasing the number of threads declines as more are added. Modify the following benchmark code for your own example. For models that are very slow to fit, relative values may be got more quickly by performing a single likelihood evaluation with `secr.fit(..., details = list(LLonly = TRUE))`.

```
library(microbenchmark) # install this package first
cores <- c(2,4,6,8)
jobs <- lapply(cores, function(nc) {
  bquote(suppressWarnings(
    secr.fit(captdata, trace = FALSE, ncores = .(nc))
  )))
names(jobs) <- paste("ncores = ", cores)
microbenchmark(list = jobs, times = 10, unit = "seconds")
```

Unit: seconds

	expr	min	lq	mean	median	uq	max	neval
ncores = 2		1.87131	1.89951	2.3122	2.38304	2.43195	2.9935	10
ncores = 4		1.11485	1.20758	1.3382	1.36394	1.41183	1.5393	10
ncores = 6		0.86815	0.96450	1.1706	1.14293	1.35131	1.5070	10
ncores = 8		0.82154	0.83097	0.9491	0.94394	0.96588	1.1944	10

See [?Parallel](#) for more.

B.5 Reducing complexity (session- or group-specific models)

Simultaneous estimation of many parameters can be painfully slow, but it can be completely avoided. If your model is fully session- or group-specific then it is much faster to analyse each group separately. For sessions this is can be done simply with `lapply` (Section 14.6). For groups you may need to construct new `capthist` objects using `subset` to extract groups corresponding to the levels of one or more individual covariates.

B.6 Reducing number of levels of detection covariates

`secr.fit` pre-computes a lookup table of values for detection parameters. The size of the table increases with the number of unique levels of any covariates. A continuous individual or detector covariate can have many similar levels. Binning covariate values can give a large saving in memory and time (see [?binCovariate](#)).

B.7 Collapsing occasions

If there is no temporal aspect to the model you want to fit (such as a behavioural response) and detectors are independent (not true for traps i.e. “multi”) then it is attractive to collapse all sampling occasions. This happens automatically by default for ‘proximity’ and ‘count’ detectors (details argument ‘fastproximity = TRUE’). For example, with the Tennessee black bear data we get:

```
# Great Smoky Mountains black bear hair snag data
# Deliberately slow both fits down by setting ncores = 2
old <- setNumThreads(2)

# mask using park boundary GSM
msk <- make.mask(traps(blackbearCH), buffer = 6000, type = 'trapbuffer',
                 poly = GSM, keep.poly = FALSE)

# 'fastproximity' On (default for 'proximity' detectors)
bbfitfaston <- secr.fit(blackbearCH, mask = msk, trace = FALSE)

# 'fastproximity' Off
bbfitfastoff <- secr.fit(blackbearCH, mask = msk, trace = FALSE,
                        details = list(fastproximity = FALSE))

# summary
fits <- secrlist(bbfitfaston, bbfitfastoff)

cat("Compare density estimates\n")
```



```
collate(fits)[,, 'D']
cat ("\nCompare timing\n")
sapply(fits, '[', 'proctime')
```

Compare density estimates

	estimate	SE.estimate	lcl	ucl
bbfitfaston	0.0084355	0.00081891	0.006977	0.010199
bbfitfastoff	0.0084355	0.00081890	0.006977	0.010199

Compare timing

bbfitfaston.elapsed	bbfitfastoff.elapsed
3.97	8.08

Data may be collapsed in `reduce.caphist` without loss of data by choosing ‘outputdetector’ carefully and setting `by = "ALL"`. The collapsed caphist object receives a `usage` attribute equal to the sum of occasion-specific usages. In this case `usage` is the number of collapsed occasions (10) and the collapsed model fits a Binomial probability with `size = 10` rather than a Bernoulli probability per occasion.

B.8 Individual mask subsets

`secr` allows the user to customise the mask for each detected animal by considering only a subset of points. The subset is defined by a radius in metres around the centroid of detections; set this using the `details` argument ‘`maxdistance`’. Speed gains vary with the layout, but can exceed 2-fold. Bias results when the radius is too small (try 5σ or the buffer distance).

Here we extend the preceding black bear example. The mask used for each bear is restricted to points within 6 km of the centroid of its detections. There is a $> 2\times$ speed gain without significant change in the density estimates.

```
centr <- centroids(blackbearCH)
par(mfrow=c(3,4), mar=c(1,1,1,1))
for (i in sort(sample.int(139, size=12))) {
  plot(msk, border = 10)
  plot(subset(msk, distancetotrap(msk, centr[i,])<6000), add = TRUE,
        col = 'red')
  plot(subset(blackbearCH, i), varycol = FALSE, add = TRUE, title = '',
        subtitle = '')
  mtext(side = 3, i, line = -1, cex = 0.9 )
}
```

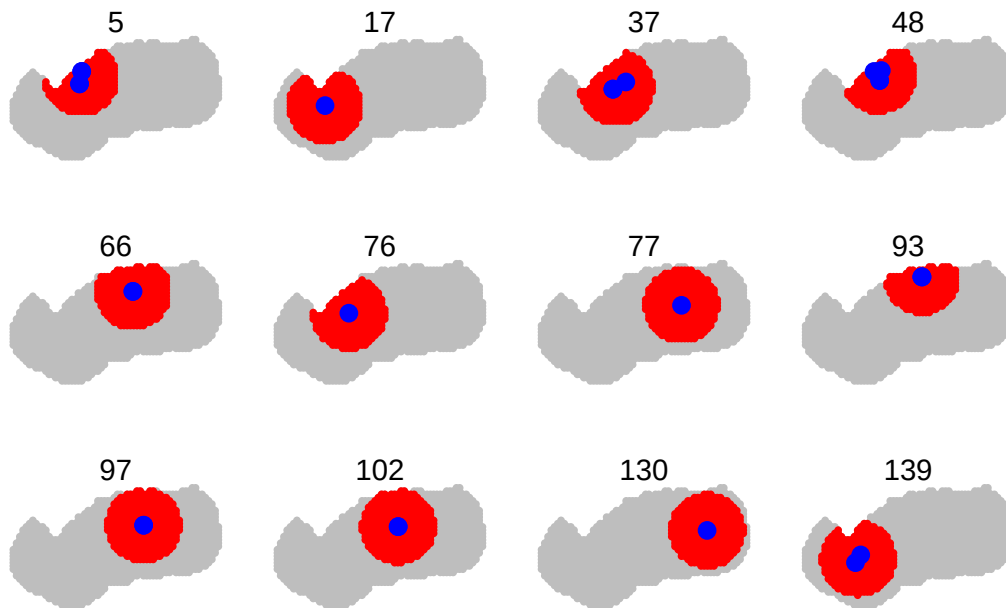


Figure B.1: Sample of individual black bear masks (red) with detection sites overplotted in blue.

```
old <- setNumThreads(2)
# fastproximity default
bbfitfaston <- secr.fit(blackbearCH, mask = msk, trace = FALSE)
bbfitfastonmaxd <- secr.fit(blackbearCH, mask = msk, trace = FALSE,
                           details = list(maxdistance = 6000))

fits <- secrlist(bbfitfaston, bbfitfastonmaxd)

cat("Compare density estimates\n")
collate(fits)[,, 'D']
cat ("\nCompare timing\n")
sapply(fits, '[', 'proctime')
```

Compare density estimates

	estimate	SE.estimate	lcl	ucl
bbfitfaston	0.0084355	0.00081890	0.0069770	0.010199
bbfitfastonmaxd	0.0084421	0.00081855	0.0069841	0.010204

Compare timing

bbfitfaston.elapsed	bbfitfastonmaxd.elapsed
4.12	1.35

B.9 Collapsing detectors

The benefit from collapsing occasions has a spatial analogue: if there are many detectors and they are closely spaced relative to animal movements σ then nearby detectors may be aggregated into new notional detectors located at the centroid. The `reduce.traps` method has an argument ‘span’ explained as follows in the help –

If ‘span’ is specified a clustering of detector sites will be performed with `hclust` and detectors will be assigned to groups with `cutree`. The default algorithm in `hclust` is complete linkage, which tends to yield compact, circular clusters; each will have diameter less than or equal to ‘span’.

B.10 “multi” detector type instead of “proximity”

The type of the detectors is usually determined by the sampling reality. For example, if individuals can physically be detected at several sites on one occasion then the “proximity” detector type is preferred over “multi”. However, if data are very sparse, so that individuals in practice are almost never observed at multiple sites on one occasion, then the detectors may as well be of type “multi”, in the sense that there is no observable difference between the two types of detection process. “multi” models used to fit much more quickly than “proximity” models, and this is still true for elaborate or time-dependent models that cannot use the ‘fastproximity’ option.

B.11 Some models are just slower than others

Detector covariates pose a particular problem. Models with learned responses take slightly longer to fit.

C Spatial data

These notes explain how **secr** uses spatial data. Spatial data are used to

1. locate detectors (`read.traps`, `read.caphist`)
2. map the extent of habitat near detectors (`make.mask`)
3. attach covariates to traps or mask objects (`addCovariates`)
4. delimit regions of interest (`region.N` and other functions)

Some spatial results may be exported, particularly the raster density surfaces generated by `predictDsurface` from a fitted model.

Internally, **secr** uses a very simple concept of space. The locations of detectors (traps), the potential locations of activity centres (habitat mask) and the simulated locations of individuals (popn) are described by Cartesian (x-y) coordinates assumed to be in metres. Distances are Euclidean unless specifically modelled as non-Euclidean (Appendix F). Only relative positions matter for the calculations, so the origin of the coordinates is arbitrary. The map projection ('coordinate reference system' or CRS) is not recorded.

Most spatial computations in **secr** (distances, areas, overlay etc.) use internal R and C++ code. Polygon and transect detectors are represented as dataframes in which each row gives the x- and y-coordinates of a vertex and topology is ignored (holes are not allowed).

The simple approach works fine within limits (discussed later), but issues arise when **secr**

- exchanges spatial data (regions, covariates or predicted density) with other software, or
- uses functions from R spatial packages, especially **sf** and **spsurvey**.

C.1 Spatial data in R

To use spatial data or functions from external sources in **secr** it helps to know a little about the expanding spatial ecosystem in R.

C.1.1 R packages for spatial data

Several widely used packages define classes and methods for spatial data ('Used by' in the following table is the number of CRAN packages from `crandep::get_dep` on 2025-02-10).

Package	Scope	Year	Used by	Citation	Relevant S4 classes
sp	vector	2005–	465	Pebesma & Bivand (2005)	SpatialPolygons, SpatialPolygonsDataFrame, SpatialGridDataFrame, SpatialLinesDataFrame
raster	raster	2010–	339	Hijmans (2023a)	RasterLayer
sf	vector	2016–	936	Pebesma (2018)	sfg, sfc, sf
stars	both	2018–	79	Edzer & Bivand (2023)	stars
terra	both	2020–	398	Hijmans (2023b)	SpatVector, SpatRaster

The reader should already understand the distinction between vector and raster spatial data. There are many resources for learning about spatial analysis in R that may be found by web search on, for example ‘R spatial data’. The introduction by [Claudia Engel](#) covers both **sp** and **sf**.

The capability of **sp** is being replaced by [sf](#) and **raster** is being replaced by [terra](#). The more recent packages tend to be faster. **sf** implements the ‘simple features’ standard.

C.1.2 Geographic vs projected coordinates

QGIS has an excellent [introduction](#) to coordinate reference systems (CRS) for GIS. Coordinate reference systems may be specified in many ways; the most simple is the 4- or 5-digit EPSG code (search for EPSG on the web).

Geographic coordinates (EPSG 4326, ignoring some details) specify a location on the earth’s surface by its latitude and longitude. This is the standard in Google Earth and Geographic Positioning Systems (GPS).

C.2 Spatial data in secr

C.2.1 Input of detector locations

secr uses relative Cartesian coordinates. Detector coordinates from GPS should therefore be projected from geographic coordinates before input to **secr**. Most of the R spatial packages include projection functions. Here is a simple example using `st_transform` from the **sf** package:

```
library(sf)
```

Linking to GEOS 3.11.2, GDAL 3.7.2, PROJ 9.3.0; sf_use_s2() is TRUE

```
# unprojected (geographic) coordinates (decimal degrees)
# longitude before latitude
df <- data.frame(x = c(174.9713, 174.9724, 174.9738),
                 y = c(-41.3469, -41.3475, -41.3466))
# construct sf object
latlon <- st_as_sf(df, coords = 1:2)
# specify initial CRS: WGS84 lat-lon
st_crs(latlon) <- 4326
# project to Cartesian coordinate system, units metres
# EPSG:27200 is the old (pre-2001) NZMG
trps <- st_transform(latlon, crs = 27200)
# print
st_coordinates(latlon)
```

```
      X      Y
[1,] 174.97 -41.347
[2,] 174.97 -41.347
[3,] 174.97 -41.347
```

```
st_coordinates(trps)
```

```
      X      Y
[1,] 2674948 5982573
[2,] 2675038 5982504
[3,] 2675157 5982602
```

C.2.2 Adding spatial covariates to a traps or mask object

SECR models may include covariates for each detector (e.g., trap or searched polygon) in the detection model (parameters g_0 , λ_0 , σ etc.) and for each point on the discretized habitat mask in the density model (parameter D).

Covariates measured at detector locations may be included in the text files read by `read.traps` or `read.caphist`.

Covariates measured at each point on a habitat mask may be included in a file or `data.frame` input to `read.mask`, but this is an uncommon way to establish mask covariates. More commonly, a habitat mask is built using `make.mask` and initially has no covariates,

The function `addCovariates` is a convenient way to attach covariates to a traps or mask object *post hoc*. The function extracts covariate values from the ‘spatialdata’ argument by a spatial query for each point on a mask. Options are

spatialdata	Notes
character	name of ESRI shapefile, excluding ‘.shp’
sp::SpatialPolygonsDataFrame	
sp::SpatialGridDataFrame	
raster::RasterLayer	
secr::mask	covariates of nearest point
secr::traps	covariates of nearest point
terra::SpatRaster	new in 4.5.3
sf::sf	new in 4.5.3

Data sources should use the coordinate reference system of the target detectors and mask (see previous section).

C.2.3 Functions with ‘poly’ or ‘region’ spatial argument

Several **secr** functions use spatial data to define a region of interest (i.e. one or more polygons). All such polygons may be defined as

- 2-column matrix or data.frame of x- and y-coordinates
- SpatialPolygons or SpatialPolygonsDataFrame S4 classes from package **sp**
- SpatRaster S4 class from package **terra**
- sf or sfc S4 classes from package **sf** (POLYGON or MULTIPOLYGON geometries)

Data in these formats are converted to an object of class `sfc` by the documented internal function **boundarytoSF**. The S4 classes allow complex regions with multiple polygons (islands), possibly containing ‘holes’ (lakes).

This applies to the following functions and arguments:

secr function	Argument
bufferContour	poly
deleteMaskPoints	poly
esaPlot	poly
make.mask	poly
make.systematic	region
mask.check	poly
pdotContour	poly
PG	poly
pointsInPolygon	poly*
region.N	region*
sim.popn	poly
subset.popn	poly
trap.builder	region
trap.builder	exclude

* `pointsInPolygon` and `region.N` also accept a habitat mask.

C.2.4 GIS functionality imported from other R packages

Some specialised spatial operations are out-sourced by **secr**:

secr function	Operation	Other-package function	Reference
<code>randomHabitat</code>	simulated habitat	<code>raster::adjacent</code>	Hijmans (2023a)
<code>nedist</code>	non-Euclidean distances	<code>raster::clump</code> <code>gdistance::transition</code> <code>gdistance::geoCorrection</code> <code>gdistance::costDistance</code>	van Etten (2023)
<code>discretize</code>	cell overlap with polygon(s)	<code>sf::st_intersection</code>	Pebesma (2018)
<code>polyarea</code>	area of polygon(s)	<code>sf::st_area</code>	
<code>make.mask</code>	polybuffer mask type	<code>sf::st_buffer</code>	
<code>rbind.caphist</code>	merge polygon detectors	<code>sf::st_union</code>	
<code>trap.builder</code>	SRS sample	<code>sf::st_sample</code>	
<code>trap.builder</code>	GRTS sample (<code>spsurvey >= 5.3.0</code>)	<code>spsurvey::grts</code>	Dumelle et al. (2024)

C.2.5 Exporting raster data for use in other packages

A mask or predicted density surface (`Dsurface`) generated in **secr** may be used or plotted as a raster layer in another R package. **secr** provides `rast` and `raster` methods for **secr** mask and `Dsurface` objects, based on the respective generic functions exported by **terra** and **raster**. These return `SpatRaster` and `RasterLayer` objects respectively. For example,

```
library(secr)
summary(possummask)
```

```
Object class      mask
Mask type         trapbuffer
Number of points  5120
Spacing m         20
Cell area ha      0.04
Total area ha     204.8
x-range m         2697463 2699583
y-range m         6077080 6078580
Bounding box
      x      y
1 2697453 6077070
```



```
2 2699593 6077070
4 2699593 6078590
3 2697453 6078590
```

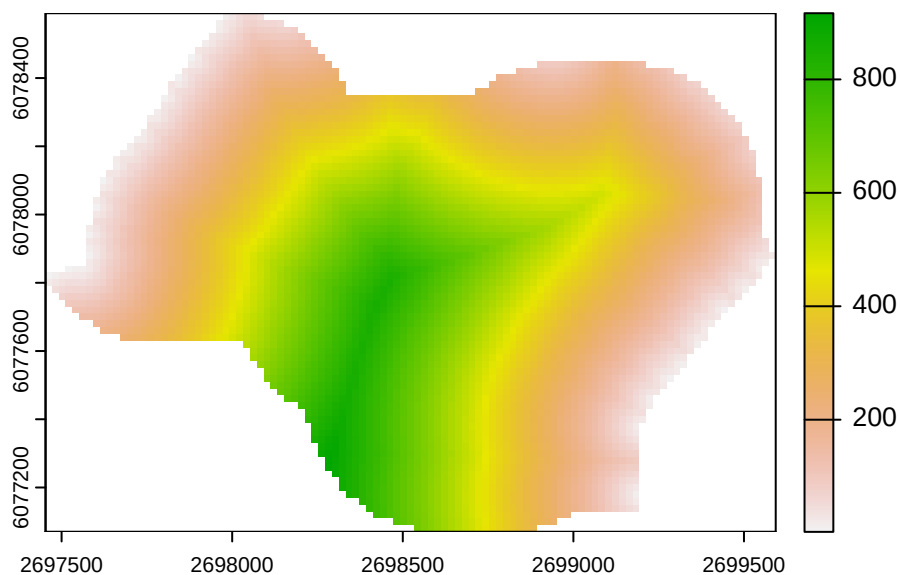
Summary of covariates

```
d.to.shore
Min.   : 2.24
1st Qu.:200.11
Median :370.80
Mean   :389.24
3rd Qu.:560.82
Max.   :916.69
```

```
# make SpatRaster object from mask covariate
r <- rast(possummask, covariate = 'd.to.shore')
print(r)
```

```
class       : SpatRaster
dimensions  : 76, 107, 1  (nrow, ncol, nlyr)
resolution  : 20, 20  (x, y)
extent      : 2697453, 2699593, 6077070, 6078590  (xmin, xmax, ymin, ymax)
coord. ref. :
source(s)   : memory
name        : tmp
min value    : 2.2361
max value    : 916.6924
```

```
terra::plot(r)
```



C.3 Limits of the Cartesian model in secr

C.3.1 Distances computed in large studies

Distances on the curved surface of the earth are not well represented by straight-line Euclidean distances when the study area is very large, as happens with large carnivores such as grizzly bears and wolverines. That has led some authors to use more rigorous (great-circle) distances. This is not possible in **secr** because there is no record of the projected coordinate reference system used for the detectors and habitat mask.

D Area and transect search

The ‘polygon’ detector type is used in **secr** for count data from searches of one or more areas (polygons). Transect detectors are the linear equivalent of polygons; as the theory and implementation are very similar we mostly refer to polygon detectors and only briefly mention transects. The relevant theory is in Chapter 4. The method may be used with individually identifiable cues (e.g., faeces) as well as for direct observations of individuals.

Cells of a polygon capthist contain the number of detections per animal per polygon per occasion, supplemented by the x-y coordinates of each detection).

Polygons may be independent (detector type ‘polygon’) or exclusive (detector type ‘polygonX’). Exclusivity is a particular type of dependence in which an animal may be detected at no more than one polygon on each occasion, resulting in binary data (i.e. polygons function more like multi-catch traps than ‘count’ detectors). Transect detectors also may be independent (‘transect’) or exclusive (‘transectX’).

D.1 Parameterisation

The detection model is fundamentally different for polygon detectors and detectors at a point (“single”, “multi”, “proximity”, “count”):

- For point detectors, the detection function directly models the probability of detection or the expected number of detections. All that matters is the distance between the animal’s centre and the detector.
- For polygon detectors, these quantities (probability or expected number) depend also on the geometrical relationships and the integration in Eq. 4.1. The detection function serves only to define the *potential* detections if the search area was unbounded (blue contours in Fig. 4.1).

We use a parameterisation that separates two aspects of detection – the expected number of cues from an individual (λ_c) and their spatial distribution given the animal’s location ($h(\mathbf{u}|\mathbf{x})$ normalised by dividing by $H(\mathbf{x})$ (Eq. 4.1). The parameters of $h()$ are those of a typical detection function in **secr** (e.g., λ_0, σ), except that the factor λ_0 cancels out of the normalised expression. The expected number of cues, given an unbounded search area, is determined by a different parameter here labelled λ_c .

There are complications:

1. Rather than designate a new ‘real’ parameter `lambdac`, **secr** grabs the redundant intercept of the detection function (`lambda0`) and uses that for λ_c . Bear this in mind when reading output from polygon or transect models.
2. If each animal can be detected at most once per detector per occasion (as with exclusive detector types ‘polygonX’ and ‘transectX’) then instead of $\lambda(\mathbf{x})$ we require a probability of detection between 0 and 1, say $g(\mathbf{x})$. In **secr** the probability of detection is derived from the cumulative hazard using $g(\mathbf{x}) = 1 - \exp(-\lambda(\mathbf{x}))$. The horned lizard dataset of Royle & Young (2008) has detector type ‘polygonX’ and their parameter ‘ p ’ was equivalent to $1 - \exp(-\lambda_c)$ ($0 < p \leq 1$). For the same scenario and parameter Efford (2011) used p_∞ .
3. Unrelated to (2), detection functions in **secr** may model either the probability of detection (HN, HR, EX etc.) or the cumulative hazard of detection (HHN, HHR, HEX etc.) (see `?detectfn` for a list). Although probability and cumulative hazard are mostly interchangeable for point detectors, it’s not so simple for polygon and transect detectors. The integration always uses the hazard form for $h(\mathbf{u}|\mathbf{x})$ (**secr** $\geq 3.0.0$)¹, and only hazard-based detection functions are allowed (HHN, HHR, HEX, HAN, HCG, HVP). The default function is HHN.

D.2 Example data: flat-tailed horned lizards

Royle & Young (2008) reported a Bayesian analysis of data from repeated searches for flat-tailed horned lizards (*Phrynosoma mcallii*) on a 9-ha square plot in Arizona, USA. Their dataset is included in **secr** as `hornedlizardCH` and will be used for demonstration. See ‘`?hornedlizard`’ for more details.

The lizards were free to move across the boundary of the plot and often buried themselves when approached. Half of the 134 different lizards were seen only once in 14 searches over 17 days. Fig. 2 shows the distribution of detections within the quadrat; lines connect successive detections of the individuals that were recaptured.

```
par(mar=c(1,1,2,1))
plot(hornedlizardCH, tracks = TRUE, varycol = FALSE, lab1cap =
      TRUE, laboffset = 8, border = 10, title = '')
```

¹The logic here is that hazards are additive whereas probabilities are not.

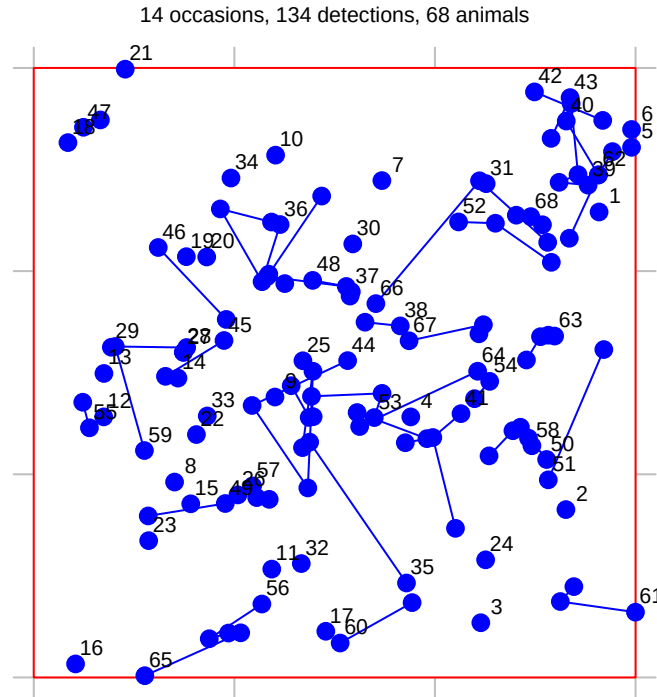


Figure D.1: Locations of horned lizards on a 9-ha plot in Arizona (Royle & Young, 2008). Grid lines are 100 m apart.

D.3 Data input

Input of data for polygon and transect detectors is described in [secre-datainput.pdf](#). It is little different to input of other data for **secre**. The key function is `read.caphist`, which reads text files containing the polygon or transect coordinates² and the capture records. Capture data should be in the ‘XY’ format of Density (one row per record with fields in the order Session, AnimalID, Occasion, X, Y). Capture records are automatically associated with polygons on the basis of X and Y (coordinates outside any polygon give an error). Transect data are also entered as X and Y coordinates and automatically associated with transect lines.

D.4 Fitting

The function `secre.fit` is used to fit polygon or transect models by maximum likelihood, exactly as for other detectors. Any model fitting requires a habitat mask – a representation of the region around the detectors possibly occupied by the detected animals (aka the ‘area of integration’ or ‘state space’). It’s simplest to use a simple buffer around the detectors,

²For constraints on the shape of polygon detectors see [Polygon shape](#)

specified via the ‘buffer’ argument of `secr.fit`³. For the horned lizard dataset it is safe to use the default buffer width (100 m) and the default detection function (circular bivariate normal). We use `trace = FALSE` to suppress intermediate output that would be untidy here.

```
FTHL.fit <- secr.fit(hornedlizardCH, buffer = 80, trace = FALSE)
predict(FTHL.fit)
```

		link estimate	SE.estimate	lcl	ucl
D	log	8.01307	1.061701	6.18740	10.37742
lambda0	log	0.13171	0.015128	0.10524	0.16485
sigma	log	18.50490	1.199388	16.29950	21.00871

The estimated density is 8.01 / ha, somewhat less than the value given by Royle & Young (2008); see Efford (2011) for an explanation, also Marques et al. (2011) and Dorazio (2013). The parameter labelled ‘lambda0’ (i.e. λ_c) is equivalent to p in Royle & Young (2008) (using $\hat{p} \approx 1 - \exp(-\hat{\lambda}_c)$).

`FTHL.fit` is an object of class ‘secr’. We would use the ‘plot’ method to graph the fitted detection function :

```
plot(FTHL.fit, xv = 0:70, ylab = 'p')
```

D.5 Cue data

By ‘cue’ in this context we mean a discrete sign identifiable to an individual animal by means such as microsatellite DNA. Faeces and passive hair samples may be cues. Animals may produce more than one cue per occasion. The number of cues in a specific polygon then has a discrete distribution such as Poisson, binomial or negative binomial.

A cue dataset is not readily available, so we simulate some cue data to demonstrate the analysis. The text file ‘polygonexample1.txt’ contains the boundary coordinates.

```
datadir <- system.file("extdata", package = "secr")
file1 <- paste0(datadir, '/polygonexample1.txt')
example1 <- read.traps(file = file1, detector = 'polygon')
polygonCH <- sim.capthist(example1, popn = list(D = 1,
  buffer = 200), detectfn = 'HHN', detectpar = list(
  lambda0 = 5, sigma = 50), noccasions = 1, seed = 123)
```

³Alternatively, one can construct a mask with `make.mask` and provide that in the ‘mask’ argument of `secr.fit`. Note that `make.mask` defaults to `type = 'rectangular'`; see Transect search for an example in which points are dropped if they are within the rectangle but far from detectors (the default in `secr.fit`)

```
par(mar = c(1,2,3,2))
plot(polygonCH, tracks = TRUE, varycol = FALSE, lab1cap = TRUE,
     laboffset = 15, title = paste("Simulated 'polygon' data",
                                   "D = 1, lambda0 = 5, sigma = 50"))
```

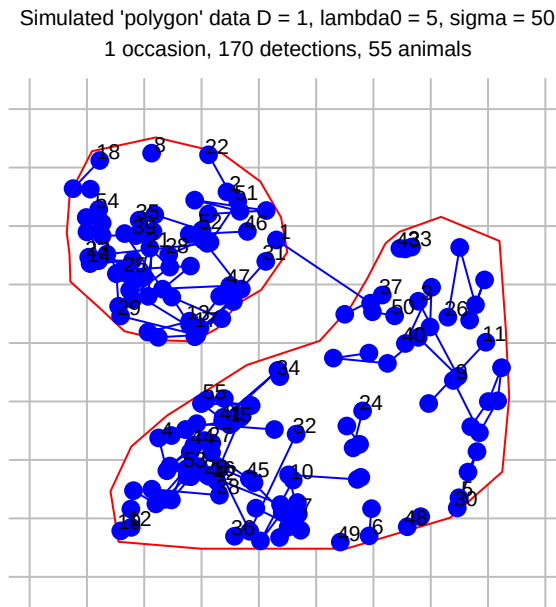


Figure D.2: Simulated cue data from a single search of two irregular polygons.

Our simulated sampling was a single search (`noccasions = 1`), and the intercept of the detection function (`lambda0 = 5`) is the expected number of cues that would be found per animal if the search was unbounded. The plot (Fig. D.2) is slightly misleading because, although ‘`tracks = TRUE`’ serves to link cues from the same animal, the cues are not ordered in time.

To fit the model by maximum likelihood we use `secr.fit` as before:

```
cuesim.fit <- secr.fit(polygonCH, buffer = 200, trace = FALSE)
predict(cuesim.fit)
```

	link	estimate	SE.estimate	lcl	ucl
D	log	1.1035	0.15366	0.84099	1.4478
lambda0	log	4.3764	0.41885	3.62938	5.2771
sigma	log	49.4464	2.43133	44.90613	54.4457

D.6 Discretizing polygon data

An alternative way to handle polygon capthist data is to convert it to a raster representation i.e. to replace each polygon with a set of point detectors, each located at the centre of a square pixel. Point detectors ('multi', 'proximity', 'count' etc.) tend to be more robust and models often fit faster. They also allow habitat attributes to be associated with detectors at the scale of a pixel rather than the whole polygon. The `secr` function `discretize` performs the necessary conversion in a single step. Selection of an appropriate pixel size (`spacing`) is up to the user. There is a tradeoff between faster execution (larger pixels are better) and controlling artefacts from the discretization, which can be checked by comparing estimates with different levels of `spacing`.

Taking our example from before,

```
discreteCH <- discretize (polygonCH, spacing = 20)
par(mar = c(1,2,3,2))
plot(discreteCH, varycol = FALSE, tracks = TRUE)
```

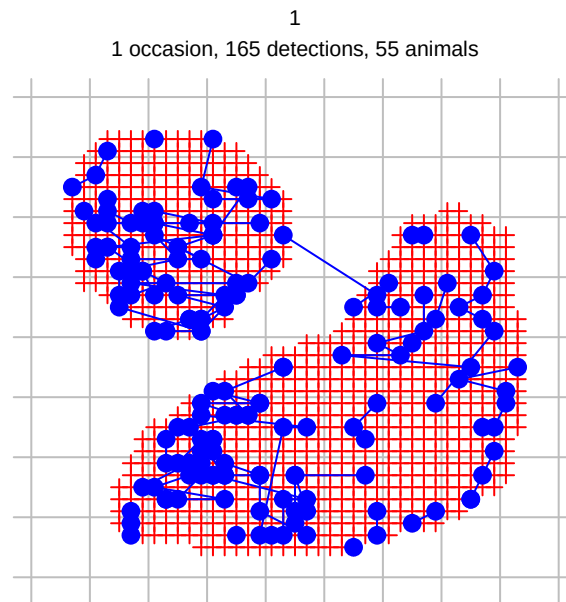


Figure D.3: Discretized area search data

```
discrete.fit <- secr.fit(discreteCH, buffer = 200, detectfn =  
  'HHN', trace = FALSE)  
predict(discrete.fit)
```

	link	estimate	SE.estimate	lcl	ucl
D	log	1.09535	0.152767	0.834461	1.43779
lambda0	log	0.11205	0.014492	0.087048	0.14422
sigma	log	49.83843	2.674000	44.867027	55.36067

Post-hoc discretization is also considered by Russell et al. (2012), Milleret et al. (2018) and Paterson et al. (2019).

D.7 Transect search

Transect data, as understood here, include the positions from which individuals are detected along a linear route through 2-dimensional habitat. They *do not* include distances from the route to the location of the individual, at least, not yet. A route may be searched multiple times, and a dataset may include multiple routes, but neither of these is necessary. Searches of linear habitat such as river banks require a different approach - see the package [seclinear](#).

We simulate some data for an imaginary wiggly transect.

```
x <- seq(0, 4*pi, length = 20)
transect <- make.transect(x = x*100, y = sin(x)*300,
  exclusive = FALSE)
summary(transect)
```

Object class	traps
Detector type	transect
Number vertices	20
Number transects	1
Total length	2756.1 m
x-range	0 1256.6 m
y-range	-298.98 298.98 m

```
transectCH <- sim.capthist(transect, popn = list(D = 2,
  buffer = 300), detectfn = 'HHN', detectpar = list(
  lambda0 = 1.0, sigma = 50), binomN = 0, seed = 123)
```

By setting `exclusive = FALSE` we signal that there may be more than one detection per animal per occasion on this single transect (i.e. this is a ‘transect’ detector rather than ‘transectX’).

Constructing a habitat mask explicitly with `make.mask` (rather than relying on ‘buffer’ in `secl.fit`) allows us to specify the point spacing and discard outlying points (Fig. D.4).

```
transectmask <- make.mask(transect, type = 'trapbuffer', buffer = 300, spacing = 20)
par(mar = c(3,1,3,1))
plot(transectmask, border = 0)
plot(transect, add = TRUE, detpar = list(lwd = 2))
plot(transectCH, tracks = TRUE, add = TRUE, title = '')
```

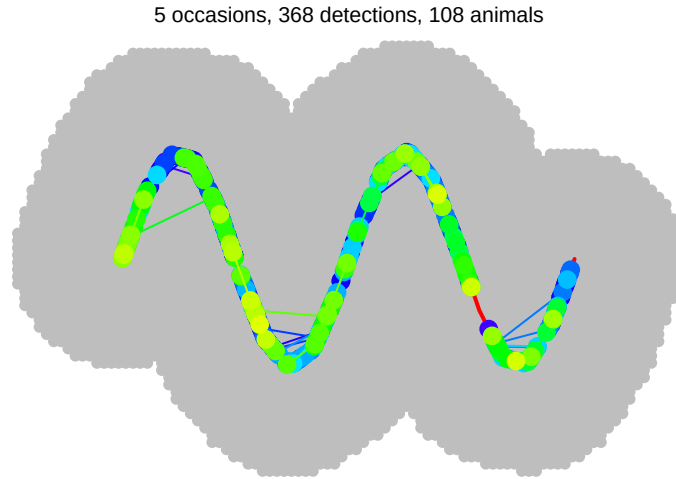


Figure D.4: Habitat mask (grey dots) and simulated transect data from five searches of a 2.8-km transect. Colours differ between individuals, but are not unique.

Model fitting uses `secr.fit` as before. We specify the distribution of the number of detections per individual per occasion as Poisson (`binomN = 0`), although this also happens to be the default. Setting `method = 'Nelder-Mead'` is slightly more likely to yield valid estimates of standard errors than using the default method (see [Technical notes](#)).

```
transect.fit <- secr.fit(transectCH, mask = transectmask,
  binomN = 0, method = 'Nelder-Mead', trace = FALSE)
```

Occasional 'ier' error codes may be ignored (see [Technical notes](#)). The estimates are close to the true values except for `sigma`, which may be positively biased.

```
predict (transect.fit)
```

	link	estimate	SE.estimate	lcl	ucl
D	log	1.8527	0.196674	1.50552	2.2798
lambda0	log	1.0415	0.080433	0.89539	1.2114
sigma	log	53.6290	2.313387	49.28317	58.3581

Another way to analyse transect data is to discretize it. We divide the transect into 25-m segments and then change the detector type. In the resulting capthist object the transect has been replaced by a series of proximity detectors, each at the midpoint of a segment.

```
snippedCH <- snip(transectCH, by = 25)
snippedCH <- reduce(snippedCH, outputdetector = 'proximity')
```

The same may be achieved with `newCH <- discretize(transectCH, spacing = 25)`. We can fit a model using the same mask as before. The result differs in the scaling of the `lambda0` parameter, but in other respects is similar to that from the transect model.

```
snipped.fit <- secr.fit(snippedCH, mask = transectmask,
  detectfn = 'HHN', trace = FALSE)
predict(snipped.fit)
```

		link estimate	SE.estimate	lcl	ucl
D	log	1.84217	0.195625	1.49689	2.26708
lambda0	log	0.19312	0.018034	0.16089	0.23182
sigma	log	53.67901	2.321128	49.31908	58.42437

D.8 More on polygons

The implementation in **secr** allows any number of disjunct polygons or non-intersecting transects.

Polygons may be irregularly shaped, but there are some limitations in the default implementation. Polygons may not be concave in an east-west direction, in the sense that there are more than two intersections with a vertical line. Sometimes east-west concavity may be fixed by rotating the polygon and its associated data points (see function **rotate**). Polygons should not contain holes, and the polygons used on any one occasion should not overlap.

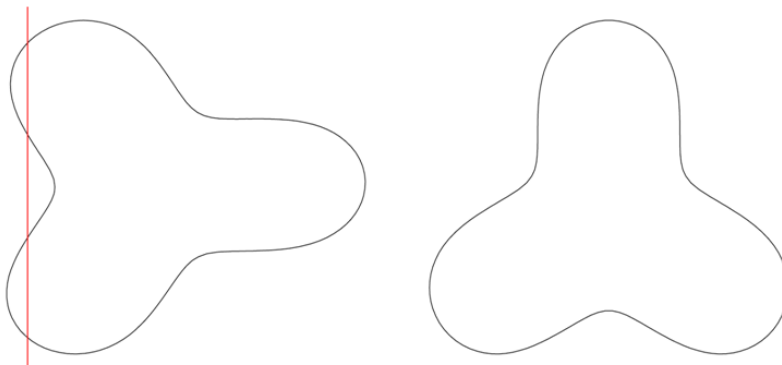


Figure D.5: The polygon on the left is not allowed because its boundary is intersected by a vertical line at more than two points.

D.8.1 Solutions for non-conforming polygons

1. Break into parts

One solution to ‘east-west concavity’ is to break the offending polygon into two or more parts. For this you need to know which vertices belong in which part, but that is (usually) easily determined from a plot. In this real example we recognise vertices 11 and 23 as critical, and split the polygon there. Note the need to include the clip vertices in both polygons, and to maintain the order of vertices. Both `example2` and `newpoly` are traps objects.

```
file2 <- paste0(datadir, '/polygonexample2.txt')
example2 <- read.traps(file = file2, detector = 'polygon')
par(mfrow = c(1,2), mar = c(2,1,1,1))
plot(example2)
text(example2$x, example2$y, 1:nrow(example2), cex = 0.7)
newpoly <- make.poly (list(p1 = example2[11:23,],
  p2 = example2[c(1:11, 23:27),]))
```

No errors found :-)

```
plot(newpoly, label = TRUE)
```

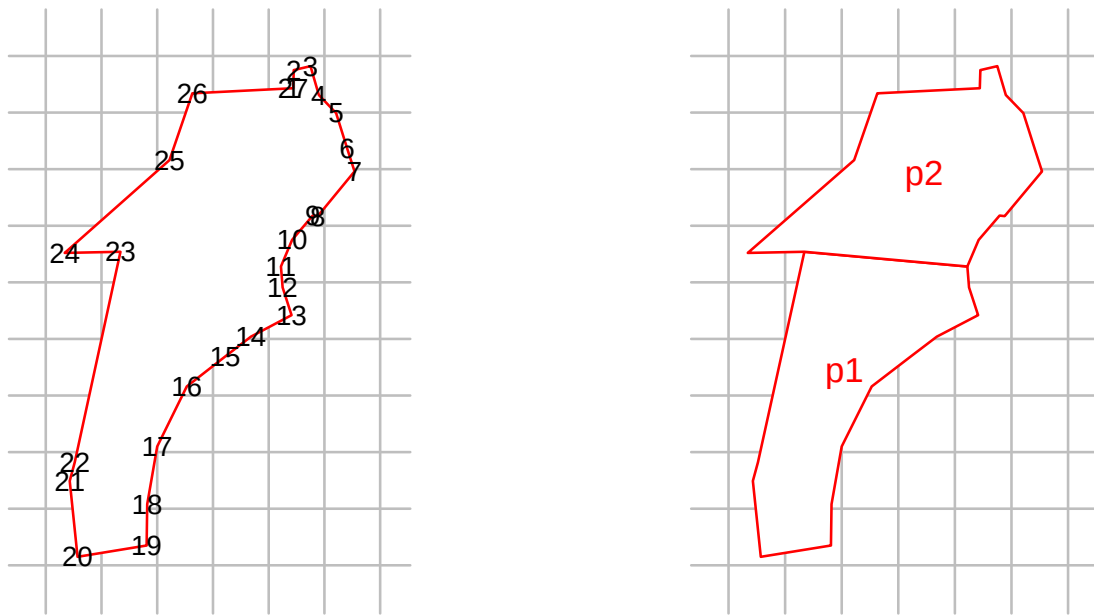


Figure D.6: Splitting a non-conforming polygon

Attributes such as covariates and usage must be rebuilt by hand.

2. Pointwise testing

Another solution is to evaluate whether each point chosen dynamically by the integration code lies inside the polygon of interest. This is inevitably slower than the default algorithm that assumes all points between the lower and upper intersections lie within the polygon. Select the slower, more general option by setting `details = list(convexpolygon = FALSE)`.

D.9 Technical notes

Fitting models for polygon detectors with `secr.fit` requires the hazard function to be integrated in two-dimensions many times. In `secr` $\geq$ 4.4 this is done with repeated one-dimensional Gaussian quadrature using the C++ function ‘integrate’ in RcppNumerical (Qiu et al., 2023).

Polygon and transect SECR models seem to be prone to numerical problems in estimating the information matrix (negative Hessian), which flow on into poor variance estimates and missing values for the standard errors of ‘real’ parameters. At the time of writing these seem to be overcome by overriding the default maximization method (Newton-Raphson in ‘nlm’) and using, for example, “method = ‘BFGS’”. Another solution, perhaps more reliable, is to compute the information matrix independently by setting ‘details = list(hessian = ‘fdhess’)’ in the call to `secr.fit`. Yet another approach is to apply `secr.refit` with “method = ‘none’” to a previously fitted model to compute the variances.

The algorithm for finding a starting point in parameter space for the numerical maximization is not entirely reliable; it may be necessary to specify the ‘start’ argument of `secr.fit` manually, remembering that the values should be on the link scale (default log for D, lambda0 and sigma).

Data for polygons and transects are unlike those from detectors such as traps in several respects:

- The association between vertices in a ‘traps’ object and polygons or transects resides in an attribute ‘polyID’ that is out of sight, but may be retrieved with the `polyID` or `transectID` functions. If the attribute is NULL, all vertices are assumed to belong to one polygon or transect.
- The x-y coordinates for each detection are stored in the attribute ‘detectedXY’ of a capthist object. To retrieve these coordinates use the function `xy`. Detections are ordered by occasion, animal, and detector (i.e., polyID).
- `subset` or `split` applied to a polygon or transect ‘traps’ object operate at the level of whole polygons or transects, not vertices (rows).
- `usage` also applies to whole polygons or transects. The option of specifying varying usage by occasion is not fully tested for these detector types.
- The interpretation of detection functions and their parameters is subtly different; the detection function must be integrated over 1-D or 2-D rather than yielding a probability directly (see Efford (2011)).

E Mark-resight models

E.1 How is mark-resight different?

In capture–recapture, all newly detected unmarked animals become marked and are distinguishable in future. Some field protocols also involve ‘resighting’ occasions on which previously marked animals are identified but newly detected animals are not marked. We call these ‘mark–resight’ data. McClintock & White (2012) provide an excellent summary of non-spatial mark–resight methods. The extension of spatially explicit capture–recapture models for mark–resight data was begun by Sollmann et al. (2013), and spatial mark–resight models were applied by Rich et al. (2014) and Rutledge et al. (2015). Whittington et al. (2018) describe a major Bayesian application.

E.2 Distribution of marked animals

The historical development of mark–resight models has stressed the resighting component of mark–resight data; the sampling process leading to marking is not modelled. Modelling of the sighting data then requires some assumption about the distribution of the marked animals. Most commonly, marked animals are assumed to be a random sample from a defined population, and that may be achievable in some situations. An important case is when some animals carry natural marks and others don’t, as in the puma study of Rich et al. (2014).

However, when a spatially distributed population is sampled with localized detectors such as traps, animals living near the traps have a higher chance of becoming marked, and the marked animals are not representative of a well-defined population. The difficulty may be overcome by spatially modelling both the marking and resighting processes. This requires that data on detection locations were collected during the marking phase, and that the spatial distribution of marking effort was recorded. Matechou et al. (2013) analysed non-spatial capture–recapture–resighting data jointly with counts of unmarked individuals, and the core capture–mark–resight models in **secr** are their spatial equivalent (see also Whittington et al. (2018)).

Rather than modelling the marking process we may take the traditional path, i.e. assume a certain spatial distribution for the marked individuals. Typically, the distribution is assumed to be uniform over a known area (e.g., Sollmann et al. (2013), Rich et al. (2014)). The number of marked individuals remaining at the time of resighting may be known or [unknown](#). We use ‘capture–mark–resight’ for models that include the marking process and ‘sighting-only’ for models in which the spatial distribution of marks is assumed.

E.3 Overview of implementation in secr

The implementation in **secr** follows Efford & Hunter (2018). The models discard some spatial information in the unmarked sightings – information that is used in the models of Chandler & Royle (2013) and Sollmann et al. (2013). This results in some (probably small) loss of precision, and requires an adjustment for overdispersion to ensure confidence intervals have good coverage properties.

Following Efford & Hunter (2018),

- the population is assumed to follow a (possibly inhomogeneous) Poisson distribution in space
- detection histories of marked animals are modelled as usual in SECR
- counts of unmarked sightings are modelled with a Poisson distribution
- overdispersion in the counts (relative to a Poisson distribution) is estimated by simulation and used in a pseudo-likelihood estimate. The adjustment has minimal effect on the estimates themselves, but improves coverage of confidence intervals

Unidentified sightings of marked animals are treated as independent of identified sightings. This is an approximation, but its effect on estimates is believed to be negligible.

The core models are listed in Table E.1. These may be customized in various ways, particularly by specifying the assumed distribution of marked animals for sighting-only models. Each model may be fitted with or without a parameter for incomplete identification of marked animals when they are resighted (**pID**).

Table E.1: Classification of mark–resight models available in **secr**. ‘Capture–mark–resight’ models include the marking process; ‘sighting-only’ models rest on prior knowledge or assumptions regarding where animals have been marked. For sighting-only data some detection histories may be all-zero.

Data	Pre-marking ¹	Code
Capture–mark–resight	none	NA
Sighting-only	Known n_0	<code>details = list(knownmarks = TRUE)</code>
	Unknown n_0	<code>details = list(knownmarks = FALSE)</code>

1. n_0 is the number of marked individuals in the study area at the time of the sighting surveys.

Mark–resight data are represented in **secr** by a **capthist** data object, with minor extensions (Table E.2). We describe the structure for a single-session **capthist** object, which records detections of each marked animal over S occasions at any of K detectors. Detectors on sighting occasions may be of the usual binary proximity (‘proximity’) or count proximity

(‘count’) types¹. Detectors on marking occasions may be any combination of these types and multi-catch traps. Area search (‘polygon’) and transect search (‘transect’) types are not yet supported for either phase when there are sighting occasions: [discretization](#) is recommended.

Warning

Not all detector types and options in **secr** will work with mark–resight data. See [Limitations](#) for a list of known constraints (some will be mentioned along the way). See the [step-by-step guide](#) for a quick introduction.

Table E.2: Special attributes of data objects for mark–resight analysis in **secr**.

Object	Attribute	Description
traps	markocc	Occasions may be marking (1), sighting (0) or unresolved (-1)
capthist	Tu	Counts of unmarked animals
capthist	Tm	Counts of marked animals not identified
capthist	Tn	Counts of sightings with unknown mark status (not modelled)
mask	marking ¹	Distribution of pre-marked animals (sighting-only data ; optional)

1. **marking** is the name of a special mask covariate, accessed like other covariates.

E.3.1 Marking and sighting occasions

Sampling occasions (intervals) are either marking occasions or sighting occasions. On marking occasions any recaptured animals are also recorded². On sighting occasions newly caught animals are released unmarked. The body of the capthist object has one row for each individual detected on at least one marking occasion, and one column for each occasion, whether marking or sighting.

The ‘traps’ object, a required attribute of any capthist object, has its own optional attribute **markocc** to distinguish marking occasions (1) from sighting occasions (0) (e.g., c(1,0,0,0,0) for marking on one occasion followed by four resighting occasions). A capthist object is recognised as a mark–resight dataset if its **traps** has a **markocc** attribute with at least one sighting occasion (i.e. **!all(markocc)**). Marking occasions and sighting occasions may be interspersed.

¹‘Count’ detectors most closely approximate sampling with replacement (McClintock and White 2012); sampling strictly without replacement implies detection at no more than one site per occasion (detector type ‘multi’) that has yet to be implemented for mark–resight data, and may prove difficult.

²There may also be marking occasions on which recaptures are ignored, but this possibility has yet to be modelled.

E.3.2 Sighting-only data

A sighting-only dataset has a `markocc` attribute with no marking occasions. Two scenarios are possible: either the capthist object includes a sighting history for each marked animal (including all-zero histories for any not re-sighted), or it includes sighting histories only for the re-sighted animals. In the latter case the number of marked animals at the time of sampling is unknown, and the fitted model must take account of this (see [Number of marks unknown](#)).

E.3.3 Sightings of unmarked animals

In addition to the marking and sighting events of known individuals recorded in the body of the capthist object, there will usually be sightings of unmarked animals. These data take the form of a matrix with the number of detections of unmarked animals at each detector (row) on each sampling occasion (column). A single searched polygon or transect is one detector and hence contributes a single row. Columns corresponding to marking occasions are all-zero. This matrix is stored as attribute `Tu` of the capthist object.

Instead of providing `Tu` as a matrix, the counts may be summed over occasions (`Tu` is a vector of detector-specific counts), or provided as the grand total (`Tu` is a single integer). These alternatives have definite limitations:

- occasion-specific or detector-specific models cannot be fitted
- plot, summary and verify are less informative
- cannot subset by occasion or detector
- AIC should not be used compare models differing in summarisation
- cannot mix `markocc` -1 and 0

E.3.4 Unidentified sightings of marked animals

An observer may be able to determine that an animal is marked, but be unable to positively identify it as a particular individual. The number of sightings of marked but unidentified individuals is stored as capthist attribute `Tm`, which uses the same formats as `Tu`.

E.3.5 Sightings of unknown status

Sightings for which the mark status could not be determined are a further category of sighting on sighting occasions. For example, the identifying mark may not be visible in a photograph because of the orientation the animal. The number of sightings with unknown mark status is generally not used in the models (but see [unresolved sightings](#)). However, it is good practice to account for these observations. They may be stored as capthist attribute `Tn` and will appear in summaries.

E.3.6 Unresolved sightings

Sometimes the sighting method does not allow marked animals to be distinguished from unmarked animals. Sighting occasions of this type are coded with ‘-1’ in the `markocc` vector. On these occasions the counts of all sightings (‘unresolved sightings’) stored in the appropriate column of `Tn`, the corresponding columns of `Tu` and `Tm` are all-zero, and no sightings of marked animals appear in the `capthist` object. Models comprising one or more marking occasions and only unresolved sightings (e.g., `markocc <- c(1, -1, -1, -1, -1)`) may be fitted in `secr`, but there are constraints (see [Sighting without attention to marking](#)).

E.4 Data preparation

Data on marked animals may be formatted and read as usual with `read.capthist` ([secr-datainput.pdf](#)). Each identified sighting of a marked individuals appears as a row in the ‘capture’ file, like any other detection. The `markocc` attribute may be set as an argument of `read.capthist` or assigned later.

Sighting data for unmarked animals (`Tu`) are provided separately as a $K \times S$ matrix, where K is the number of detectors and S is the total number of occasions (including marking occasions). Elements in the matrix are either binary (0/1) for proximity detectors, or whole numbers (0, 1, 2,...) for count, polygon and transect detectors. Sightings of marked animals that are not identified to individual (`Tm`) are optionally provided in a separate $K \times S$ matrix. Usually you will read these data from separate text files or spreadsheets. Sightings with unknown mark status (`Tn`) may also be provided, but will be ignored in analyses except for occasions with code -1 in `markocc`.

The function `addSightings` may be used to merge these matrices of sighting data with an existing mark-resight `capthist` object, i.e. to set its attributes `Tu` and `Tm`. There are also custom extraction and replacement functions for sighting-related attributes (`markocc`, `Tu` and `Tm`). `addSightings` also allows input from text files in which the first column is a session identifier (see [Step-by-step guide](#) for an example).

Let’s assume you have prepared the files `MRCHcapt.txt`, `MRCHtrap.txt`, `Tu.txt` and `Tm.txt`. [Simulated examples](#) can be found in the `extdata` folder of the `secr` distribution (version 4.4.3 and above). This code can be used to build a mark-resight `capthist` object:

```
library(secr)
olddir <- setwd(system.file("extdata", package = "secr"))
MRCH <- read.capthist("MRCHcapt.txt", "MRCHtrap.txt", detector =
  c("multi", rep("proximity",4)), markocc = c(1,0,0,0,0))
MRCH <- addSightings(MRCH, "Tu.txt", "Tm.txt")
session(MRCH) <- "Simulated mark-resight data"
setwd(olddir)
```

E.4.1 Data summary

The `summary` method for capthist objects recognises mark–resight data and provides a summary of the `markocc`, `Tu`, and `Tm` attributes.

```
summary(MRCH)
```

```
Object class      capthist
Detector type     multi, proximity (4)
Detector number   36
Average spacing   20 m
x-range           0 100 m
y-range           0 100 m
```

```
Marking occasions
```

```
 1 2 3 4 5
1 0 0 0 0
```

```
Counts by occasion
```

	1	2	3	4	5	Total
n	70	15	16	19	17	137
u	70	0	0	0	0	70
f	28	24	13	3	2	70
M(t+1)	70	70	70	70	70	70
losses	0	0	0	0	0	0
detections	70	17	18	28	23	156
detectors visited	31	14	15	18	13	91
detectors used	36	36	36	36	36	180

```
Sightings by occasion
```

	1	2	3	4	5	Total
ID	0	17	18	28	23	86
Not ID	0	8	5	11	7	31
Unmarked	0	7	9	6	9	31
Uncertain	0	0	0	0	0	0
Total	0	32	32	45	39	148

The main table of counts is calculated differently for sighting-only and capture–mark–resight data. When `markocc` includes marking occasions the counts refer to marking, recaptures and sightings of marked animals. This means that `u` (number of new individuals) is always zero on sighting occasions, and `M(t+1)` increases only on marking occasions. When `markocc` records that the data are sighting-only, the main table of counts summarises identified sightings of previously marked animals, accumulating `u` and `M(t+1)` as if all occasions were marking occasions.

On sighting occasions the row ‘detections’ in Counts by occasion corresponds to the number of sightings of marked and identified individuals (‘ID’) in Sightings by occasion. Other detections in the sightings table are additional to the main table.

The default plot of a mark–resight capthist object shows only marked and identified individuals. To make a separate plot of the sightings of unmarked individuals, either use the same function with `type = "sightings"` or use `sightingPlot`:

```
par(mar = c(1,3,3,3), mfrow = c(1,2), pty = 's')
# petal plot with key
plot(MRCH, type = "sightings", border = 20)
occasionKey(MRCH, cex = 0.7, rad = 5, px = 0.96, py = 0.88,
            xpd = TRUE)
# bubble plot
sightingPlot(MRCH, "Tu", mean = FALSE, border = 20, fill = TRUE,
             col = "blue", legend = c(1,2,3), px = 1.0, py = c(0.85,0.72),
             xpd = TRUE)
```

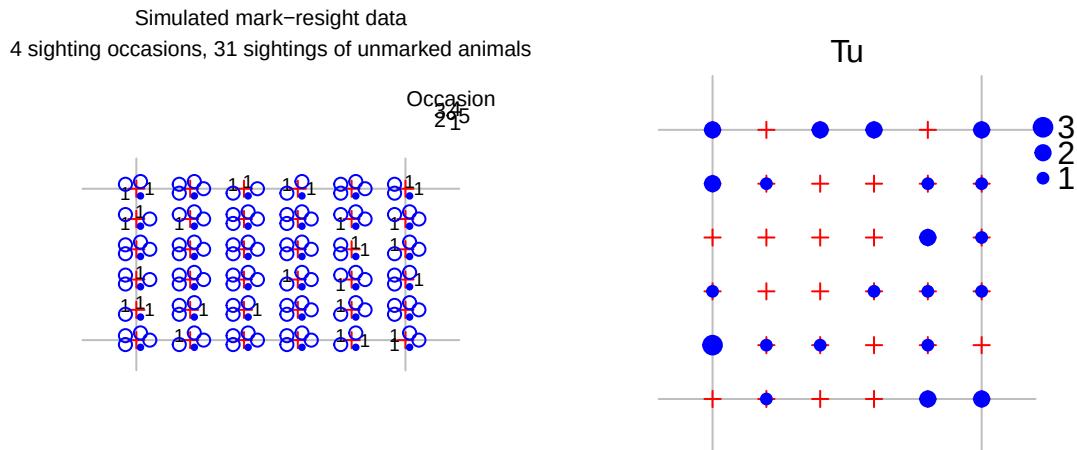


Figure E.1: Two ways to plot sightings of unmarked animals. Petals in the lefthand plot indicate the number of sightings, except that marking occasions are shown as a solid dot. `mean = FALSE` causes `sightingPlot` to use the total over all occasions rather than the mean. Other settings are used here only to customise the layout and colour of the plots.

E.4.2 Data checks

The `verify` method for capthist objects checks that mark–resight data satisfy

- the length of `markocc` matches the number of sampling occasions S
- animals are not marked on sighting occasions, or sighted on marking occasions
- the dimensions of matrices with counts of sighted animals (T_u , T_m) match the rest of the capthist object

- sightings are made only when detector usage is > 0
- counts are whole numbers.

Checks are performed by default when a model is fitted with `secl.fit`, or when `verify` is called independently.

E.4.3 Different layouts for marking and sighting

Detector layouts are likely to differ between marking and sighting occasions. This can be accommodated within a single, merged detector layout using detector- and occasion-specific usage codes. See [here](#) for a function to create a single-session mark-resight traps object from two components.

E.4.4 Simulating mark-resight data

Mark-resight data may be generated with function `sim.resight`. In this example, one grid of detectors is used for both marking and for resighting, but the detection rate is higher on the initial (marking) occasion.

```
grid <- make.grid(detector = c("multi", rep("proximity",4)))
markocc(grid) <- c(1, 0, 0, 0, 0)
g0mr <- c(0.3, 0.1, 0.1, 0.1, 0.1)
simMRCH <- sim.resight(grid, detectpar=list(g0 = g0mr, sigma = 25),
  popn = list(D = 30), pID = 0.7)
```

If all elements of `markocc` are zero then a separate mechanism is used to pre-mark a fraction `pmark` of individuals selected at random from the population. The resulting sighting-only data may include all-zero sighting histories for marked animals. Use the argument `markingmask` to simulate marking of animals with centres in a certain subregion.

E.4.5 Input of sighting-only data

For sighting-only data we postulate that the marked animals are a random sample of individuals from a known spatial distribution, possibly a specified region. The body of the `capthist` object then comprises sightings of known individuals, with other sightings in attributes `Tu` and `Tm`. Marked animals that were known to be present but not resighted are included as ‘all-zero’ detection histories in the body of the `capthist` object. These detection histories are input by including a record in the detection file for each marked-but-unsighted individual, distinguished by 0 (zero) in the ‘occasion’ field, and with an arbitrary (but not missing) detector ID or coordinates in the next field(s).

E.5 Model fitting

Properly prepared, a `capthist` object includes all the data for fitting a mark-resight model³. It is only necessary to call `secr.fit`. Here is an example in which we use a coarse mask (`nx = 32` instead of the default `nx = 64`) to speed things up. The predictor ‘ts’ and the new ‘real’ parameter `pID` are explained in following sections.

```
mask <- make.mask(traps(MRCH), nx = 32, buffer = 100)
fit0 <- secr.fit(MRCH, model = list(g0 ~ ts), mask = mask,
  trace = FALSE)
predict(fit0, newdata = data.frame(ts = factor(c("M","S"))))
```

```
$`ts = M`
      link estimate SE.estimate      lcl      ucl
D      log 25.87864    3.410206 20.01037 33.46784
g0     logit 0.37635    0.053074 0.27923 0.48454
sigma   log 25.30921    1.972904 21.72832 29.48024
pID     logit 0.67424    0.054842 0.55923 0.77150

$`ts = S`
      link estimate SE.estimate      lcl      ucl
D      log 25.878639    3.410206 20.01037 33.46784
g0     logit 0.093003    0.012526 0.07119 0.12063
sigma   log 25.309209    1.972904 21.72832 29.48024
pID     logit 0.674240    0.054842 0.55923 0.77150
```

This example relies on the built-in `autoini` function to provide starting parameter values for the numerical maximization. However, the automatic values may not work with complex mark-resight models. It is not worth continuing if the first likelihood evaluation fails, as indicated by `LogLik NaN` when `trace = TRUE`. Then you should experiment with the `start` argument until you find values that work.

E.5.1 Adjustment for overdispersion

The default confidence intervals have poor coverage when the data include counts in `Tu` or `Tm`. This is due to overdispersion of the summed counts, caused by a quirk of the model specification: covariation is ignored in the multivariate Poisson model used to describe the counts. A simple protocol fixes the problem:

1. Fit a model assuming no overdispersion,
2. Estimate the overdispersion by simulating at the initial estimates, and
3. Re-fit the model (or at least re-estimate the variance-covariance matrix) using an overdispersion-adjusted pseudo-likelihood.

³A slight exception is the optional `marking` mask covariate used for sighting-only data.

Steps (2) and (3) may be performed together in one call to `secr.fit`. In the example below, the first line repeats the model specification, while the second line provides the previously fitted model as a starting point and calls for `nsim` simulations to estimate overdispersion ($\hat{c}$ in MARK parlance).

```
fit1 <- secr.fit(MRCH, model = list(g0~ts), mask = mask,
  trace = FALSE, start = fit0, details = list(nsim = 10000))
```

sims completed

```
predict(fit1, newdata = data.frame(ts = factor(c("M","S"))))
```

```
$`ts = M`
      link estimate SE.estimate      lcl      ucl
D      log 24.18511    3.393631 18.39459 31.79846
g0     logit 0.44323    0.076187 0.30298 0.59316
sigma   log 25.54846    2.035289 21.86060 29.85847
pID     logit 0.66553    0.064967 0.52897 0.77904

$`ts = S`
      link estimate SE.estimate      lcl      ucl
D      log 24.18511    3.393631 18.39458 31.79846
g0     logit 0.094573   0.013525 0.071187 0.12461
sigma   log 25.54846    2.035289 21.86059 29.85847
pID     logit 0.66553    0.064967 0.52897 0.77904
```

For this dataset the adjustment had a small effect on the confidence intervals (0% increase in interval length). The estimates of overdispersion are saved in the `details$chat` component of the fitted model, and we can see that they are close to 1.0. Note that the adjustment is strictly for overdispersion in the unmarked sightings, and the effect of that overdispersion is ‘diluted’ by other components of the likelihood.

```
fit1$details$chat[1:2]
```

```
      Tu      Tm
1.9242 1.7436
```

A shortcut is to specify `method = "none"` at Step 3 to block re-maximization of the pseudolikelihood: then only the variance-covariance matrix, likelihood and related parameters will be re-computed. Pre-determined values of $\hat{c}$ may be provided as input in the `chat` component of `details`; if `nsim = 0` the input `chat` will be used without further simulation (example in [Step-by-step guide](#) Step 8).

Stochasticity in the estimate of overdispersion causes parameter estimates to vary. The problem is minimized by running many simulations (say, `nsim = 10000`), which has relatively little effect on total execution time.

CAVEAT: the multi-threaded simulation code does not allow for potential non-independence of random number streams across threads. If this is a concern then set `ncores = 1` to block multi-threading. See also `?secreNG`.

E.5.2 Different parameter values on marking and sighting occasions

Marking and sighting occasions share parameters by default. This may make sense for the spatial scale parameter ($\sigma \sim 1$), but it is unlikely to hold for the baseline rate (intercept) g_0 or λ_0 . *Density estimates can be very biased if the difference is ignored*⁴. A new canned predictor ‘ts’ is introduced to distinguish marking and sighting occasions. For example $g_0 \sim ts$ will fit two levels of g_0 , one for marking occasions and one for sighting occasions. The same can be achieved with $g_0 \sim tcov$, `timecov = factor(2-markocc)`.

To see the values associated with each level of ‘ts’ (i.e., marking and sighting occasions) we specify the `newdata` argument of `predict.secr`, as in [Adjustment for overdispersion](#).

E.5.3 Proportion identified

If some sightings are of marked animals that cannot be identified to individual then a further ‘real’ parameter is required for the proportion identified. This is called `pID` in `secr`. `pID` is estimated by default, whether or not the `capthist` object has attribute `Tm`, because failure to identify marked animals affects the re-detection probability of marked individuals. For data with a single initial marking occasion, a model estimating `pID` is exactly equivalent to a learned response model ($g_0 \sim b$) in the absence of `Tm` (same density estimate, same likelihood).

`pID` may be fixed at an arbitrary value, just like other real parameters. For example, a call to `secr.fit` with `fixed = list(pID = 1.0)` implies that every sighted individual that was already marked could be identified.

Only a subset of the usual predictors is appropriate for `pID`. Use of predictors other than `session`, `Session`, or the names of any session or trap covariate will raise a warning.

E.5.4 Specifying distribution of pre-marked animals (sighting-only models)

By default, animals in a sighting-only dataset are assumed to have been marked throughout the habitat mask. However, uniform marking across a subregion of the habitat mask, or any other known distribution, may be specified by including a mask covariate named ‘marking’. For example,

⁴This is not an issue for sighting-only models as the marked fraction is not modelled.


```
mask <- make.mask(grid)
d <- distancetotrap(mask, grid)
covariates(mask) <- data.frame(marking = d < 60)
```

If the mask is used in `secr.fit` with a sighting-only dataset, the fitted model will ‘understand’ that animals centred within 60 m of the grid had a uniform probability of marking, and no animals from beyond this limit were marked. Note that the area is supposed to contain the *centres* of all marked animals; this is not the same as a searched polygon. If the covariate is missing then all animals in the mask are assumed to have had an equal chance of becoming marked. The ‘marking’ covariate is actually more general, in that it can represent any known or assumed probability distribution for the locations of marked animals within the mask: values are automatically normalized by dividing by their sum. Be warned that assumptions about the extent of marking directly affect the estimated density.

It is possible in principle that the marked animals are distributed over an area larger than the habitat mask. The software does not directly allow for this. An easy solution is to increase the size of the habitat mask, but take care to control its coarseness (`spacing`), and be aware of the effect on estimates of N , and on the sampling variance if `distribution = "binomial"`.

E.5.5 Number of marks unknown (sighting-only models)

Sighting-only datasets may entail the further complication that the number of marked animals in the population at the time of resighting is unknown (Model 3). This can happen if enough time has passed for some marked animals to have died or emigrated. It also arises if individuation relies on unique natural marks, but these are missing from some individuals (Rich et al. 2014).

The resighting data do not include any all-zero detection histories, as the only way to know for certain that a marked animal is still present is to sight it at least once. The theory nevertheless allows a spatial model to be fitted⁵.

Data preparation is exactly as for sighting-only models with known number of marks, except that there are no all-zero histories. The unknown-marks model is fitted by setting the `secr.fit` argument `details = list(knownmarks = FALSE)`, (possibly along with other `details` components).

E.5.6 Covariates, groups and finite mixtures

Conditional-likelihood models are generally not useful for mark-resight analyses. Individual covariates therefore should not be used in mark-resight models in `secr`. Other covariates (at the level of session, occasion, or detector) should work in any of the models.

⁵The number of marked individuals may be estimated as a derived parameter if required.

The mark–resight models in **secr** do not currently allow groups (g). Finite mixtures, including ‘hcov’ models, are only partly implemented.

E.5.7 Discarding unmarked sightings

Sighting data can be highly problematic because of difficulty in reading marks and determining when consecutive sightings are independent. A conservative approach is to discard the counts of unmarked sightings (Tu) and unidentified marked animals (Tm), while retaining sighting data on marked animals. This works for capture–mark–resight data (marking occasions included), but not for all-sighting data (which rely on unmarked sightings to estimate detection parameters). The parameter pID is confounded with the sighting-phase g0 so we fix it at 1.0. The likelihood does not require adjustment for overdispersion of unmarked sightings.

```
Tu(MRCH) <- NULL
Tm(MRCH) <- NULL
mask <- make.mask(traps(MRCH), nx = 32, buffer = 100)
fit2 <- secr.fit(MRCH, model = list(g0~ts), fixed = list(pID = 1.0),
  mask = mask, trace = FALSE)
predict(fit2, newdata = data.frame(ts = factor(c("M","S"))))
```

```
$`ts = M`
      link estimate SE.estimate      lcl      ucl
D      log  18.1896      2.96615 13.241289 24.98724
g0     logit   0.9446      0.15065  0.056989  0.99979
sigma   log  26.8962      2.32807 22.706549 31.85897

$`ts = S`
      link estimate SE.estimate      lcl      ucl
D      log 18.189647      2.9661482 13.241289 24.987239
g0     logit  0.063549      0.0088513  0.048255  0.083266
sigma   log 26.896233      2.3280668 22.706549 31.858973
```

Confidence interval length has increased by -12% over the full estimates (we did throw out a lot of data), but we expect the result to be more robust.

E.5.8 Comparing models

Fitted models may be compared by AIC or AICc. Models can be compared when they describe the same data. In practice this means

1. Do not compare models with and without either sighting attribute (Tu, Tm), and
2. Do not compare sighting-only models with different pre-marking assumptions (the **marking** covariate of the mask).

E.6 Limitations

These limitations apply when fitting mark–resight models in **secr**:

- only ‘multi’, ‘proximity’ and ‘count’ detector types are allowed
- groups (g) are not allowed
- [hybrid mixtures](#) with known class membership (‘hcov’) have not been implemented for mark-resight data
- some **secr** functions do not yet handle mark–resight data or models. These are mostly flagged with a Warning on their documentation page.

If the sighting phase is an area or transect search then it may be possible to analyse the data by rasterizing the areas or transects (function `discretize`).

E.6.1 Warning

The implementation of spatially explicit mark–resight models is still somewhat limited. It is intended that mark–resight models work across other capabilities of **secr**, particularly

- data may span multiple sessions ([multi-session](#) capthist objects can contain mark–resight data)
- mark–resight may be augmented with telemetry data (`addTelemetry`), but joint telemetry models have not been tested and should be used with care
- `fixiContour()` and related functions (probability density plots of centres of detected animals)
- finite mixture (h2) models are allowed for detection parameters, including `pID`
- `modelAverage()` and `collate()`
- linear habitat masks are allowed (as in the package **secrlinear**)

However, these uses have not been tested much, where they have been tested at all..

It is unclear what sample size should be used for sample-size-adjusted AIC (AICc). This is one more reason to use `AIC(..., criterion = "AIC")`. Currently **secr** uses the number of detection histories of marked individuals, as for other models.

E.6.2 Pitfalls

The likelihood for the sighting-only model with unknown number of marks (Model 3) has a boundary corresponding to the density of detected marks in the marking mask (true density cannot be less than this). This is ordinarily not a problem (estimated density will usually be larger). However, for multi-session data with a common density parameter it is quite possible for the number of marks in a particular session to exceed the threshold. The log-likelihood function then returns ‘NA’; although maximization may proceed, variance estimation is likely to fail. A possible (but slow) workaround is to combine the multiple sessions in a single-session capthist (you’re on your own here - there is no function for this).

E.7 Step-by-step guide

This guide should help you get started on a simple mark–resight analysis.

1. Decide where your study fits in Table 1 and determine the `markocc` vector. Do you have data for the marking process (capture–mark–resight data), or will this be assumed (sighting-only data)? For capture–mark–resight data, `markocc` will have ‘1’ for modelled marking occasions and ‘0’ for sighting occasions (e.g., `markocc(traps) <- c(1,0,0,0,0)` for one marking occasion and four sighting occasions). For sighting-only data, `markocc` is ‘0’ for every occasion (omit the quotation marks). If you have sighting-only data, is the number of marked animals known or unknown?
2. Prepare a capture file and detector layout file as usual ([secre-datainput.pdf](#)). Sightings of marked animals are included as if they were recaptures. If your data are sighting-only and the number of marked animals is known then include a capture record with occasion = 0 for any marked animal that was never resighted. If your detector type is ‘count’ (sampling with replacement; repeat observations possible at each detector) then repeat data rows in the capture file as necessary.
3. Prepare separate text files for sightings of unmarked animals (Tu) and unidentified sightings of marked animals (Tm). Each row has a session identifier followed by the number of sightings for one detector on each occasion (always zero in columns corresponding to marking occasions). The session identifier is used to split the file when the data span multiple sessions; it should be constant for a single-session capthist. The files will be read with `read.table`, so if necessary consult the help for that function. The start of a file for 1 marking occasion and 4 sighting occasions might look like this:

```
S1 0 0 0 0 0
S1 0 1 0 2 0
S1 0 0 1 1 1
S1 0 0 0 1 0
S1 0 1 0 0 1
S1 0 0 2 0 0
...
```

4. Input data

```
library(secr)
datadir <- system.file("extdata", package = "secr") # or choose your own
olddir <- setwd(datadir)
CH <- read.caphist("MRCHcapt.txt", "MRCHtrap.txt", detector =
  c("multi", rep("proximity",4)), markocc = c(1,0,0,0,0),
  verify = FALSE)
CH <- addSightings(CH, "Tu.txt", "Tm.txt")
```

No errors found :-)

```
setwd(olddir) # return to original folder
```

5. Review data

```
summary(CH)
```

```
Object class      capthist
Detector type     multi, proximity (4)
Detector number   36
Average spacing   20 m
x-range           0 100 m
y-range           0 100 m
```

Marking occasions

```
1 2 3 4 5
1 0 0 0 0
```

Counts by occasion

	1	2	3	4	5	Total
n	70	15	16	19	17	137
u	70	0	0	0	0	70
f	28	24	13	3	2	70
M(t+1)	70	70	70	70	70	70
losses	0	0	0	0	0	0
detections	70	17	18	28	23	156
detectors visited	31	14	15	18	13	91
detectors used	36	36	36	36	36	180

Sightings by occasion

	1	2	3	4	5	Total
ID	0	17	18	28	23	86
Not ID	0	8	5	11	7	31
Unmarked	0	7	9	6	9	31
Uncertain	0	0	0	0	0	0
Total	0	32	32	45	39	148

RPSV with `CC = TRUE` gives a crude estimate of the spatial scale parameter `sigma`.

```
RPSV(CH, CC = TRUE)
```

[1] 20.234

6. Design an appropriate habitat mask, representing the region from which animals are potentially detected (sighted). Most simply this is got by buffering around the detectors, for example applying a 100-m buffer:

```
mask <- make.mask(traps(CH), nx = 32, buffer = 100)
```

7. Fit a null model and adjust for overdispersion

```
fit0 <- secr.fit(CH, mask = mask, trace = FALSE)
fit0 <- secr.fit(CH, mask = mask, trace = FALSE, start = fit0,
  details = list(nsim = 10000))
```

sims completed

```
predict(fit0)
```

	link	estimate	SE.estimate	lcl	ucl
D	log	17.14768	2.633788	12.71238	23.13043
g0	logit	0.21939	0.034158	0.15974	0.29353
sigma	log	24.50798	1.810621	21.20837	28.32094
pID	logit	0.28498	0.043918	0.20713	0.37814

8. Consider other detection models

- model with different marking and sighting rates

```
fitts <- secr.fit(CH, model = g0 ~ ts, mask = mask, trace = FALSE,
  details = list(chat = fit0$details$chat))
predict(fitts, newdata = data.frame(ts = c("M","S")))
```

```
$`ts = M`
```

	link	estimate	SE.estimate	lcl	ucl
D	log	22.23749	3.375444	16.54308	29.89201
g0	logit	0.55319	0.122097	0.31982	0.76526
sigma	log	25.89570	2.114989	22.07103	30.38314
pID	logit	0.65171	0.083724	0.47593	0.79405

```
$`ts = S`
```

	link	estimate	SE.estimate	lcl	ucl
D	log	22.237488	3.375444	16.543078	29.89201
g0	logit	0.097514	0.015854	0.070553	0.13330
sigma	log	25.895698	2.114989	22.071032	30.38314
pID	logit	0.651713	0.083724	0.475926	0.79405

```
AIC(fit0, fitts)[,-c(2,6)]
```

				model	npar	logLik	AIC	dAIC	AICwt
fitts	D~1	g0~ts	sigma~1	pID~1	5	-521.42	1052.8	0.000	1
fit0	D~1	g0~1	sigma~1	pID~1	4	-552.32	1112.6	59.809	0

Better model fit was achieved by estimating different detection rates on marking and sighting occasions, and the estimates from the null model should be discarded. We now reveal that the data were simulated with $D = 30$ animals/ha, $g0(\text{marking}) = 0.3$, $g0(\text{sighting}) = 0.1$, $\sigma = 25$ m and $pID = 0.7$: confidence intervals for the estimates from model ‘fitts’ comfortably cover these values.

We have used the initial null-model estimates of $\hat{c}$ to adjust the later model. It may be better to use the $\hat{c}$ of the larger (more general) model. The AIC values compared are strictly quasi-AIC values as they use the pseudolikelihood.

9. Analyses for other data types

- marked animals all identified on resighting

In this case we would want to block estimation of the parameter `pID` by fixing it at 1.0:

```
fit <- secr.fit(CH, mask = mask, fixed = list(pID = 1),
  trace = FALSE)
fit <- secr.fit(CH, mask = mask, fixed = list(pID = 1),
  trace = FALSE, start = fit, details = list(nsim = 10000))
predict(fit)
```

- sighting-only data with unknown number of marks (assume previous marking uniform throughout mask)

```
fit <- secr.fit(CH, mask = mask, trace = FALSE,
  details = list(knownmarks = FALSE))
fit <- secr.fit(CH, mask = mask, trace = FALSE,
  details = list(knownmarks = FALSE, nsim = 10000))
predict(fit)
```

E.8 Miscellaneous

E.8.1 Sighting without attention to marking

When sighting occasions never distinguish between marked and unmarked animals, the parameter `pID` is redundant and cannot be estimated. If no action is taken it will appear in the fitted model as the starting value (default 0.7) with zero variance. It is preferable to suppress this by fixing it to some arbitrary value (e.g., `fixed = list(pID = 1.0)`).

The contribution of unresolved sightings to the estimates is likely to be small.

1. Estimation of the spatial scale parameter (σ) rests entirely on recaptures of marked animals within and between any marking occasions. The unresolved counts carry a little spatial information (Chandler and Royle 2013), but this is discarded in the **secr** implementation and cannot contribute to estimating σ (see also [Adjustment for overdispersion](#)).
2. It will usually be necessary to fit a distinct level of the intercept parameter λ_0 on the sighting occasions (perhaps using the predictor **ts**).

Nevertheless, it may be useful to model unresolved counts in a joint analysis of a larger dataset.

E.8.2 Combining marking and sighting detector layouts

Here is a function to create a single-session mark-resight traps object from two components. ‘trapsM’ and ‘trapsR’ are **secr** traps objects for marking and resighting occasions respectively. ‘markocc’ is an integer vector distinguishing marking (1) and sighting (-1, 0) occasions. The value returned is a combined traps object with usage attribute based on **markocc**. Detectors shared between the marking and sighting layouts are duplicated in the result; this may slow down model fitting, but it is otherwise not a problem.

```
trapmerge <- function (trapsM, trapsR, markocc){
  if (!is.null(usage(trapsM)) | !is.null(usage(trapsR)))
    warning("discarding existing usage data")
  KM <- nrow(trapsM)
  KR <- nrow(trapsR)
  S <- length(markocc)
  notmarking <- markocc<1
  newdetector <- c(detector(trapsM)[1],
                   detector(trapsR)[1])[notmarking+1]
  detector(trapsM) <- detector(trapsR)[1]
  usage(trapsM) <- matrix(as.integer(!notmarking), byrow = TRUE,
                          nrow = KM, ncol = S)
  usage(trapsR) <- matrix(as.integer(notmarking), byrow = TRUE,
                          nrow = KR, ncol = S)
  trps <- rbind(trapsM, trapsR)
  # Note: nrow(trps) == KM + KR
  detector(trps) <- newdetector
  markocc(trps) <- markocc
  trps
}
```

This demonstration uses only 9 marking points and 25 re-sighting points.


```

gridM <- make.grid(nx = 3, ny = 3, detector = "multi")
gridR <- make.grid(nx = 5, ny = 5, detector = "proximity")
combined <- trapmerge(gridM, gridR, c(1,1,0,0,0))
detector(combined)

```

```
[1] "multi"      "multi"      "proximity" "proximity" "proximity"
```

```

# show usage for first 16 detectors of 34 in the combined layout
usage(combined)[1:16,]

```

```

      1 2 3 4 5
1  1 1 0 0 0
2  1 1 0 0 0
3  1 1 0 0 0
4  1 1 0 0 0
5  1 1 0 0 0
6  1 1 0 0 0
7  1 1 0 0 0
8  1 1 0 0 0
9  1 1 0 0 0
10 0 0 1 1 1
11 0 0 1 1 1
12 0 0 1 1 1
13 0 0 1 1 1
14 0 0 1 1 1
15 0 0 1 1 1
16 0 0 1 1 1

```

E.8.3 Simulation code

This code was used to simulate data for the initial demonstration.

```

library(secr)
grid <- make.grid(detector = c("multi", rep("proximity",4)))
markocc(grid) <- c(1, 0, 0, 0, 0)
g0mr <- c(0.3, 0.1, 0.1, 0.1, 0.1)
MRCH <- sim.resight(grid, detectpar=list(g0 = g0mr, sigma = 25),
                    popn = list(D = 30), pID = 0.7, seed = 123)
# write to files in working directory
Tu.char <- paste ("S1", apply (Tu(MRCH), 1, paste, collapse = " "))
Tm.char <- paste ("S1", apply (Tm(MRCH), 1, paste, collapse = " "))
write.capthist(MRCH, filestem = "MRCH")
write.table(Tu.char, file = "Tu.txt", col.names = F, row.names = F,

```

```

    quote = FALSE)
write.table(Tm.char, file = "Tm.txt", col.names = F, row.names = F,
    quote = FALSE)

```

E.8.4 Sighting-only example

```

library(secr)
grid <- make.grid(detector = 'proximity')
markocc(grid) <- c(0, 0, 0, 0, 0)
MRCH5 <- sim.resight(grid, detectpar=list(g0 = 0.3, sigma = 25),
    unsighted = TRUE, popn = list(D = 30), pID = 1.0, seed = 123)
Tm(MRCH5) <- NULL
summary(MRCH5)

```

```

Object class      capthist
Detector type     proximity
Detector number   36
Average spacing   20 m
x-range           0 100 m
y-range           0 100 m

```

Marking occasions

```

 1 2 3 4 5
0 0 0 0 0

```

Counts by occasion

	1	2	3	4	5	Total
n	26	27	25	23	31	132
u	26	6	4	4	3	43
f	8	11	5	8	11	43
M(t+1)	26	32	36	40	43	43
losses	0	0	0	0	0	0
detections	48	57	49	52	62	268
detectors visited	25	28	27	25	34	139
detectors used	36	36	36	36	36	180

Empty histories : 75

Sightings by occasion

	1	2	3	4	5	Total
ID	48	57	49	52	62	268
Not ID	0	0	0	0	0	0
Unmarked	30	30	28	29	34	151

Uncertain	0	0	0	0	0	0
Total	78	87	77	81	96	419

```
fit0 <- secr.fit(MRCH5, fixed = list(pID = 1.0), trace = FALSE,
  details = list(knownmarks = TRUE))
fit1 <- secr.fit(MRCH5, fixed = list(pID = 1.0), start = fit0,
  details = list(nsim = 2000, knownmarks = TRUE),
  trace = FALSE)
```

sims completed

```
MRCH6 <- sim.resight(grid, detectpar=list(g0 = 0.3, sigma = 25),
  unsighted = FALSE, popn = list(D = 30), pID = 1.0, seed = 123)
Tm(MRCH6) <- NULL
summary(MRCH6)
```

```
Object class      capthist
Detector type     proximity
Detector number   36
Average spacing   20 m
x-range           0 100 m
y-range           0 100 m
```

Marking occasions

1	2	3	4	5
0	0	0	0	0

Counts by occasion

	1	2	3	4	5	Total
n	26	27	25	23	31	132
u	26	6	4	4	3	43
f	8	11	5	8	11	43
M(t+1)	26	32	36	40	43	43
losses	0	0	0	0	0	0
detections	48	57	49	52	62	268
detectors visited	25	28	27	25	34	139
detectors used	36	36	36	36	36	180

Sightings by occasion

	1	2	3	4	5	Total
ID	48	57	49	52	62	268
Not ID	0	0	0	0	0	0
Unmarked	30	30	28	29	34	151
Uncertain	0	0	0	0	0	0
Total	78	87	77	81	96	419

```
fit2 <- secr.fit(MRCH6, fixed = list(pID = 1.0), trace = FALSE,  
                 details = list(knownmarks = FALSE))
```

F Non-Euclidean distances

Spatially explicit capture–recapture (SECR) entails a distance-dependent observation model: the expected number of detections (λ) or the probability of detection (g) declines with increasing distance between a detector and the home-range centre of a focal animal. ‘Distance’ here usually, and by default, means the Euclidean distance. The observation model can be customised by replacing the Euclidean distance with one that ‘warps’ space in some ecologically meaningful way. There are innumerable ways to do this. One is the non-Euclidean ‘ecological distance’ envisioned by Royle, Chandler, Gazenski, et al. (2013). Redefining distance is a way to model spatial variation in the size of home ranges, and hence the spatial scale of movement σ ; Efford et al. (2016) used this to model inverse covariation between density and home range size. Distances measured along a linear habitat network such as a river system are also non-Euclidean (see package **secrlinear**).

This document shows how to define and use non-Euclidean distances in **secr**. Code is included for the non-Euclidean SECR simulation of Sutherland et al. (2015).

F.1 Basics

The Euclidean distance between points (x_1, y_1) and (x_2, y_2) is given by $d = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$. Non-Euclidean distances are defined in **secr** by setting the ‘userdist’ component of the ‘details’ argument of **secr.fit**. The options are (i) to provide a static (pre-computed) $K \times M$ matrix containing the distances between the K detectors and each of the M mask points, or (ii) to provide a function that computes the distances dynamically. A static distance matrix can allow for barriers to movement. Providing a function is more flexible and allows the estimation of a parameter for the distance model, but evaluating the function for each likelihood slows down model fitting.

F.2 Static userdist

A pre-computed non-Euclidean distance matrix may incorporate constraints on movement, particularly mapped barriers to movement, and this is the most obvious reason to employ a static userdist. The function **nedist** builds a suitable matrix.

As an example, take the 1996 DNA survey of the grizzly bear population in the Central Selkirk mountains of British Columbia by Mowat & Strobeck (2000). Their study area was partly bounded by lakes and reservoirs that we assume are rarely crossed by bears. To treat

the lakes as barriers in a SECR model we need a matrix of hair snag – mask point distances for the terrestrial (non-Euclidean) distance between each pair of points.

We start with the hair snag locations `CStraps` and a `SpatialPolygonsDataFrame` object `BLKS_BC` representing the large lakes (Fig. F.1). Buffering 30 km around the detectors gives a naive mask (Fig. F.1 b); we use a 2-km pixel size and reject points centred in a lake. The shortest dry path from many points on the naive mask to the nearest detector is much longer than the straight line distance.

```
Csmask2000 <- make.mask (CStraps, buffer = 30000,
  type = "trapbuffer", spacing = 2000, poly = BLKS_BC,
  poly.habitat = FALSE, keep.poly = FALSE)
```

```
par(mfrow = c(1,2), mar = c(1,4,1,4), cex = 1.6, col = "black")
plot(CStraps, gridl = FALSE, border = 30000)
selectedlakes <- c(1730, 2513, 2514, 2769, 2686, 2749)
plot(BLKS_BC[selectedlakes,], col = "blue", add = TRUE)
text(1546000, 510000, "a.", xpd = TRUE, col = "black")

plot(CStraps, gridl = FALSE, border = 30000, hidetr = TRUE)
plot(Csmask2000, dots = FALSE, add = TRUE)
plot(CStraps, add = TRUE)
par(col = "black", cex = 1.6)
text(1537000, 510000, "b.", xpd = TRUE)
```

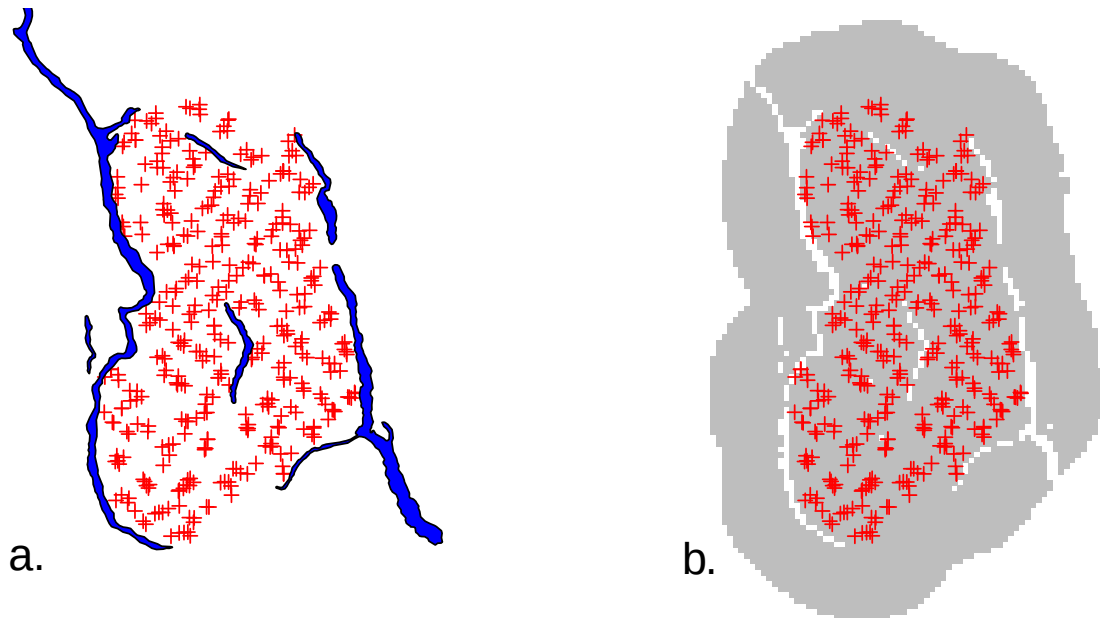


Figure F.1: (a) Central Selkirk grizzly bear hair snag locations, (b) Mask using naive 30-km buffer around hair snags.

We next calculate the matrix of all dry detector–mask distances by calling `nedist` that in turn uses functions from the R package `gdistance` (van Etten, 2023). Missing pixels in the mask represent barriers to movement when their combined width exceeds a threshold determined by the adjacency rule in `gdistance`.

What do we mean by an adjacency rule? `gdistance` finds least-cost distances through a graph formed by joining ‘adjacent’ pixels. Adjacent pixels are defined by the argument ‘directions’, which may be 4 (rook’s case), 8 (queen’s case) or 16 (knight and one-cell queen moves) as in the `raster` function `adjacent`. The default in `nedist` is ‘directions = 16’ because that gives the best approximation to Euclidean distances when there are no barriers. The knight’s moves are $\sqrt{5} \approx 2.24 \times$ cell width (‘spacing’), so the width of a polygon intended to map a barrier should be at least $2.24 \times$ cell width.

Some of the BC lakes are narrow and less than 4.48 km wide. To ensure these act as barriers we could simply reduce the spacing of our mask, but that would slow down model fitting. The alternative is to retain the 2-km mask for model fitting and to define a finer (0.5-km) mask purely for the purpose of computing distances¹:

```
CSmask500 <- make.mask (CStraps, buffer = 30000, type =
  "trapbuffer", spacing = 500, poly = BLKS_BC, poly.habitat =
  FALSE, keep.poly = FALSE)
userd <- nedist(CStraps, CSmask2000, CSmask500)
```

The first argument of `nedist` provides the rows of the distance matrix and the second argument the columns; the third (if present) defines an alternative mask on which to base the calculations. To verify the computation, map the distance from a chosen detector i to every point in a mask. Here is a short function to do that (example in Fig. F.2 a).

```
# map non-Euclidean distance from detector i
# or arbitrary point on existing plot (i = NA)
dmap <- function (traps, mask, userd, i = 1, ...) {
  if (is.na(i)) i <- nearesttrap(unlist(locator(1)), traps)
  covariates(mask) <- data.frame(d = userd[i,])
  covariates(mask)$d[!is.finite(covariates(mask)$d)] <- NA
  plot(mask, covariate = "d", ...)
  points(traps[i,], pch = 3, col = "red")
}
```

At this point we could simply use ‘userd’ as our userdist matrix. However, `CSmask2000` now includes a lot of points that are further than 30 km from any detector. It is better to drop these points and the associated columns of ‘userd’ (Fig. F.2 b):

¹Mixing 2-km and 0.5-km cells carries a slight penalty: the centres of a few 2-km cells (<1%) do not lie in valid 0.5-km cells; these become inaccessible (infinite distance from all detectors) and are silently dropped in a later step.

```

OK <- apply(userd, 2, min) < 30000
CSmask2000b <- subset(CSmask2000, OK)
userdx <- userd[,OK]

par(mfrow = c(1,2), mar = c(1,4,1,4), cex = 1.1, col = "black")

dmap(CSstraps, CSmask2000, userd, dots = FALSE, scale = 0.001,
     title = "km")
par(cex = 1.6)
text(1537000, 510000, "a.", xpd=T, col="black")

plot(CSstraps, gridl = FALSE, border = 30000, hidetr = TRUE)
plot(CSmask2000b, dots = FALSE, add = TRUE)
plot(CSstraps, add = TRUE); par(col = "black", cex = 1.6)
text(1537000, 510000, "b.", xpd = TRUE, col = "black")

```

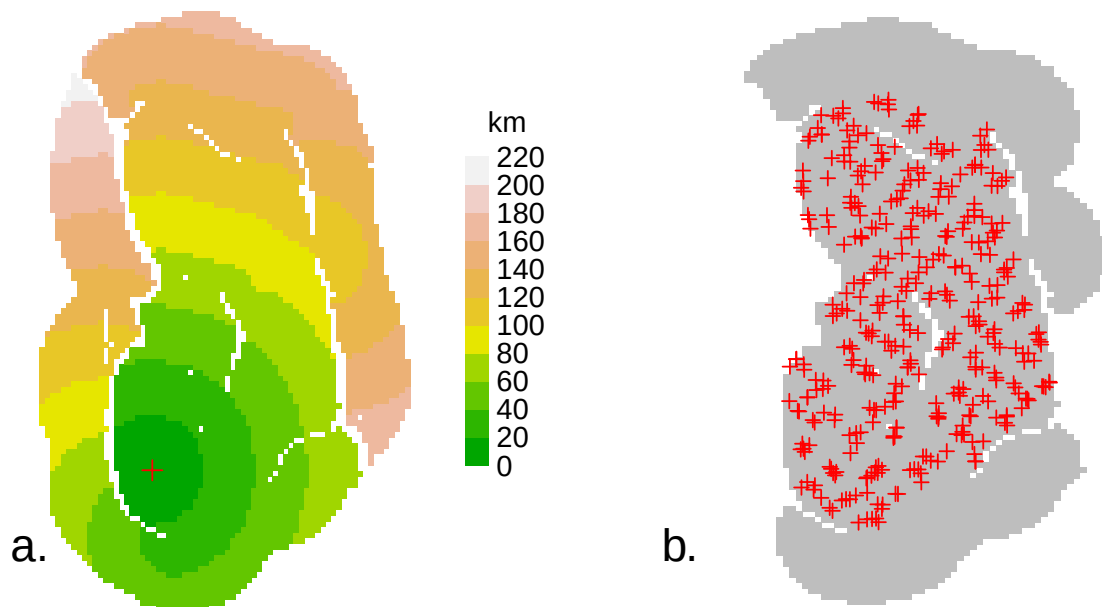


Figure F.2: (a) Central Selkirk dry-path distances from an arbitrary point (+), and (b) Efficient mask rejecting dry-path distances >30km.

Finally, we can fit a model using the non-Euclidean distance matrix:

```

CSa <- secr.fit(CS_sexcov_all, mask = CSmask2000b,
               details = list(userdist = userdx))
predict(CSa)

```

For completeness, note that Euclidean distances may also be pre-calculated, using the function `edist`. By default, `secr.fit` uses that function internally, and there is usually little

speed improvement when the calculation is done separately.

F.3 Dynamic userdist

For dynamic non-Euclidean distances a function is passed to `secr.fit` in the ‘details’ argument ‘userdist’, rather than a matrix. The userdist function should take three arguments. The first two are simply 2-column matrices with the coordinates of the detectors and animal locations (mask points) respectively. The third is a habitat mask (this may be the same as `xy2`). The function has this form:

```
mydistfn <- function (xy1, xy2, mask) {  
  if (missing(xy1)) return(charactervector)  
  ...  
  distmat # return nrow(xy1) x nrow(xy2) matrix  
}
```

Computation of the distances is entirely under the control of the user – here we indicate that by ‘...’. The calculations may use cell-specific values of two ‘real’ parameters ‘D’ and ‘noneuc’ that as needed are passed by `secr.fit` as covariates of the mask. ‘D’ is the usual cell-specific expected density in animals per hectare. ‘noneuc’ is a special cell-specific ‘real’ parameter used only here: it means whatever the user wants it to mean.

Whether ‘noneuc’, ‘D’ or other mask covariates are needed by `mydistfn` is indicated by the character vector returned by `mydistfn` when it is called with no arguments. Thus, `charactervector` may be either a zero-length character vector or a vector of one or more parameter names (“noneuc”, “D”, `c(“noneuc”, “D”)`).

‘noneuc’ has its own link scale (default ‘log’) on which it may be modelled as a linear function of any of the predictors available for density (`x`, `y`, `x2`, `y2`, `xy`, `session`, `Session`, `g`, or any mask covariate – see Chapter 11. It may also, in principle, be modelled using regression splines (Borchers & Kidney, 2014), but this is untested. When the model is fitted by `secr.fit`, the beta parameters for the ‘noneuc’ sub-model are estimated along with all the others. To make `noneuc` available to `userdist`, ensure that it appears in the ‘model’ argument. Use the formula `noneuc ~ 1` if `noneuc` is constant.

The function may compute least-cost paths via intervening mask cells using the powerful **igraph** package (Csardi & Nepusz, 2006). This is most easily accessed with package **gdistance**, which in turn uses the `RasterLayer` S4 object class from the package **raster**. To facilitate this, **secr** includes code to treat the ‘mask’ S3 class as a virtual S4 class, and provides a method for the function ‘`raster`’ to convert a mask to a `RasterLayer`.

If the function generates any bad distances (negative, infinite or missing) these will be replaced by `1e10`, with a warning.

F.4 Examples

We use annotated examples to show how the `userdist` function may be used to define different models. For illustration we use the Orongorongo Valley brushtail possum dataset from February 1996 ([OVpossumCH](#)). The data are captures of possums over 5 nights in single-catch traps at 30-m spacing. We start by extracting the data, defining a habitat mask, and fitting a null model:

```
datadir <- system.file("extdata", package = "secr")
ovforest <- sf::st_read(paste0(datadir, "/OVforest.shp"),
  quiet = TRUE)
ovposs <- OVpossumCH[[1]] # select February 1996
ovmask <- make.mask(traps(ovposs), buffer = 120, type = "trapbuffer",
  poly = ovforest[1:2,], spacing = 7.5, keep.poly = FALSE)
# for plotting only
leftbank <- read.table(paste0(datadir, "/leftbank.txt"))[21:195,]

fit0 <- secr.fit(ovposs, mask = ovmask, detectfn = "HHN", trace = FALSE)
```

Warning: multi-catch likelihood used for single-catch traps

The warning is routine: we will suppress it in later examples. The distance functions below are not specific to a particular study: each may be applied to other datasets.

F.4.1 Scale of movement σ depends on location of home-range centre

In this simple case we use the non-Euclidean distance function to model continuous spatial variation in σ . This cannot be done directly in `secr` because sigma is treated as part of the detection model, which does not allow for continuous spatial variation in its parameters. Instead we model spatial variation in 'noneuc' as a stand-in for 'sigma'

```
fn1 <- function(xy1, xy2, mask) {
  if (missing(xy1)) return("noneuc")
  sig <- covariates(mask)$noneuc # sigma(x,y) at mask points
  sig <- matrix(sig, byrow = TRUE, nrow = nrow(xy1), ncol = nrow(xy2))
  euc <- edist(xy1, xy2)
  euc / sig
}
fit1 <- secr.fit(ovposs, mask = ovmask, detectfn = "HHN",
  details = list(userdist = fn1), model = noneuc ~ x + y +
  x2 + y2 + xy, fixed = list(sigma = 1), trace = FALSE)
```

```
predict(fit1)
```

	link	estimate	SE.estimate	lcl	ucl
D	log	14.68433	1.0914749	12.696149	16.98385
lambda0	log	0.10852	0.0096263	0.091235	0.12908
noneuc	log	25.92426	1.3019335	23.495538	28.60404

We can take the values of noneuc directly from the mask covariates because we know xy2 and mask are the same points. We may sometimes want to use fn1 in context where this does not hold, e.g., when simulating data.

```
fn1a <- function (xy1, xy2, mask) {  
  if(missing(xy1)) return("noneuc")  
  xy1 <- addCovariates(xy1, mask)  
  sig <- covariates(xy1)$noneuc # sigma(x,y) at detectors  
  sig <- matrix(sig, nrow = nrow(xy1), ncol = nrow(xy2))  
  euc <- edist(xy1, xy2)  
  euc / sig  
}  
fit1a <- secr.fit(ovposs, mask = ovmask, detectfn = "HHN", trace =  
  FALSE, details = list(userdist = fn1a), model = noneuc ~  
    x + y + x2 + y2 + xy, fixed = list(sigma = 1))
```

```
predict(fit1a)
```

	link	estimate	SE.estimate	lcl	ucl
D	log	14.46195	1.0296894	12.580498	16.62477
lambda0	log	0.10758	0.0095253	0.090471	0.12792
noneuc	log	26.12539	1.4198773	23.487424	29.05965

We can verify the use of ‘noneuc’ in fn1 by using it to re-fit the null model:

```
fit0a <- secr.fit(ovposs, mask = ovmask, detectfn = "HHN",  
  details = list(userdist = fn1), model = noneuc ~ 1,  
  fixed = list(sigma = 1), trace = FALSE)
```

```
predict(fit0)
```

	link	estimate	SE.estimate	lcl	ucl
D	log	14.38019	1.0030934	12.544717	16.48422
lambda0	log	0.10148	0.0089476	0.085403	0.12058
sigma	log	27.37646	0.9729684	25.534941	29.35079

```
predict(fit0a)
```

	link	estimate	SE.estimate	lcl	ucl
D	log	14.38019	1.0030934	12.544717	16.48422
lambda0	log	0.10148	0.0089477	0.085403	0.12058
noneuc	log	27.37646	0.9729687	25.534941	29.35079

Here, fitting noneuc as a constant while holding sigma fixed is exactly the same as fitting sigma alone.

F.4.2 Scale of movement σ depends on locations of both home-range centre and detector

Hypothetically, detections at $xy1$ of an animal centred at $xy2$ may depend on both locations (this may also be seen as an approximation to the following case of continuous variation along the path between $xy1$ and $xy2$). To model this we need to retrieve the value of noneuc for both locations. Within `fn2` we use `addCovariates` to extract the covariates of the mask (and hence noneuc) for each point in $xy1$ and $xy2$. The call to `secre.fit` is identical except that it uses `fn2` instead of `fn1`:

```
fn2 <- function (xy1, xy2, mask) {  
  if (missing(xy1)) return("noneuc")  
  xy1 <- addCovariates(xy1, mask)  
  xy2 <- addCovariates(xy2, mask)  
  sig1 <- as.numeric(covariates(xy1)$noneuc) # sigma at detector  
  sig2 <- as.numeric(covariates(xy2)$noneuc) # sigma at mask pt  
  euc <- edist(xy1, xy2)  
  sig <- outer(sig1, sig2, FUN = function(s1, s2) (s1 + s2)/2)  
  euc / sig  
}  
fit2 <- secre.fit(ovposs, mask = ovmask, detectfn = "HHN",  
  details = list(userdist = fn2), model = noneuc ~ x + y +  
  x2 + y2 + xy, fixed = list(sigma = 1), trace = FALSE)
```

```
predict(fit2)
```

	link	estimate	SE.estimate	lcl	ucl
D	log	14.54655	1.0580295	12.616250	16.77219
lambda0	log	0.10781	0.0095493	0.090662	0.12821
noneuc	log	26.02326	1.3510080	23.507204	28.80862

Tip

The value of `noneuc` reported by `predict.secr` is the predicted value at the centroid of the mask, because the model uses standardised mask coordinates.

F.4.3 Continuously varying σ using `gdistance`

A more elegant but slower approach is to find the least-cost path across the network of cells between `xy1` and `xy2`, using `noneuc` (i.e. σ) as the cell-specific cost weighting (large cell-specific σ equates with greater ‘conductance’, the inverse of friction or cost). For this we use functions from the package **`gdistance`**, which in turn uses **`igraph`**.

```
fn3 <- function (xy1, xy2, mask) {
  if (missing(xy1)) return("noneuc")
  # warp distances to be \propto \int_{\text{along path}} \sigma(x,y) dp
  # where p is path distance
  if (!require(gdistance))
    stop ("install package gdistance to use this function")
  # make raster from mask
  Sraster <- raster(mask, "noneuc")
  # Assume animals can traverse gaps: bridge gaps using mean
  Sraster[is.na(Sraster[])] <- mean(Sraster[], na.rm = TRUE)
  # TransitionLayer
  tr <- transition(Sraster, transitionFunction = mean,
                  directions = 16)
  tr <- geoCorrection(tr, type = "c", multpl = FALSE)
  # costDistance
  costDistance(tr, as.matrix(xy1), as.matrix(xy2))
}
fit3 <- secr.fit(ovposs, mask = ovmask, detectfn = "HHN",
  details = list(userdist = fn3), model = noneuc ~ x + y +
  x2 + y2 + xy, fixed = list(sigma = 1), trace = FALSE)
```

```
predict(fit3)
```

	link	estimate	SE.estimate	lcl	ucl
D	log	14.4063	1.0500171	12.490932	16.6154
lambda0	log	0.1076	0.0095041	0.090528	0.1279
noneuc	log	26.4338	1.3643666	23.892156	29.2459

The **`gdistance`** function `costDistance` uses a `TransitionLayer` object that essentially describes the connections between cells in a `RasterLayer`. In `transition` adjacent cells are assigned a positive value for ‘conductance’ and all other cells a zero value. Adjacency is

defined by the `directions` argument as 4 (rook's case), 8 (queen's case), 16 (knight and one-cell queen moves) and possibly other values (see `?adjacent` in **gdistance**). Values < 16 can considerably distort distances even if conductance is homogeneous. **geoCorrection** is needed to allow for the greater separation ($\times\sqrt{2}$) of cell centres measured along diagonals.

In **ovmask** there are two forest blocks separated by a shingle stream bed and low scrub that is easily crossed by possums but does not count as 'habitat'. Habitat gaps are assumed in **secr** to be traversible. The opposite is assumed by **gdistance**. To coerce **gdistance** to behave like **secr** we here temporarily fill in the gaps.

The argument `'transitionFunction'` determines how the conductance values of adjacent cells are combined to weight travel between them. Here we simply average them, but any other single-valued function of 2 inputs can be used.

Integrating along the path (fn3) takes about 3.6 times as long as the approximation (fn2) and gives quite similar results.

F.4.4 Density-dependent σ

A more interesting variation makes sigma a function of the cell-specific density, which may vary independently across space (Efford et al., 2016). Specifically, $\sigma(x, y) = k/\sqrt{D(x, y)}$, where k is the fitted parameter (noneuc).

```
fn4 <- function (xy1, xy2, mask) {
  if(missing(xy1)) return(c("D", "noneuc"))
  if (!require(gdistance))
    stop ("install package gdistance to use this function")
  # make raster from mask
  D <- covariates(mask)$D
  k <- covariates(mask)$noneuc
  Sraster <- raster(mask, values = k / D^0.5)
  # Assume animals can traverse gaps: bridge gaps using mean
  Sraster[is.na(Sraster[])] <- mean(Sraster[], na.rm = TRUE)
  # TransitionLayer
  tr <- transition(Sraster, transitionFunction = mean,
                  directions = 16)
  tr <- geoCorrection(tr, type = "c", multpl = FALSE)
  # costDistance
  costDistance(tr, as.matrix(xy1), as.matrix(xy2))
}

fit4 <- secr.fit(ovposs, mask = ovmask, detectfn = "HHN", trace =
  FALSE, details = list(userdist = fn4), fixed = list(sigma = 1),
  model = list(noneuc ~ 1, D ~ x + y + x2 + y2 + xy))

predict(fit4)
```

	link	estimate	SE.estimate	lcl	ucl
D	log	15.45493	1.6388618	12.561941	19.01417
lambda0	log	0.10669	0.0094078	0.089787	0.12678
noneuc	log	103.22831	5.0335255	93.824863	113.57420

```
# or using regression splines with same df
fit4a <- secr.fit(ovposs, mask = ovmask, detectfn = "HHN",
  details = list(userdist = fn4), fixed = list(sigma = 1),
  model = list(noneuc~1, D ~ s(x,y, k = 6)), trace = FALSE)
```

```
predict(fit4a)
```

	link	estimate	SE.estimate	lcl	ucl
D	log	15.76414	1.7685370	12.661194	19.62754
lambda0	log	0.10681	0.0094167	0.089887	0.12691
noneuc	log	103.08276	5.0094306	93.722800	113.37748

```
par(mar = c(1,4,1,6), cex = 1.4)
plot(predictDsurface(fit4a))
plot(traps(ovposs), add = TRUE)
lines(leftbank)
```

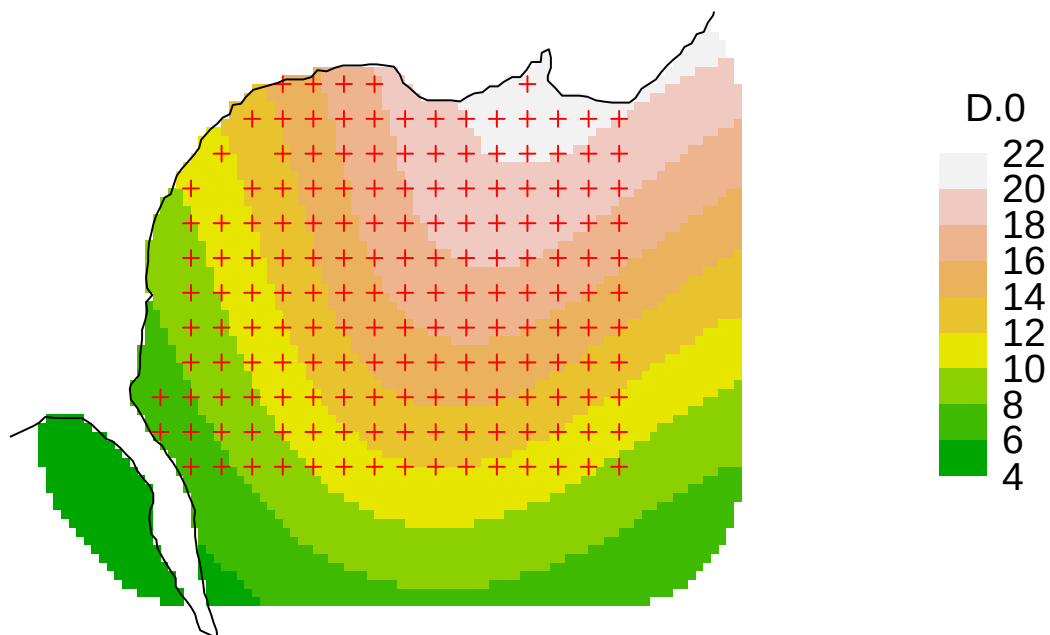


Figure F.3: Surface 4a.

F.4.5 Habitat model for connectivity

Yet another possibility, in the spirit of Royle, Chandler, Gazenski, et al. (2013), is to model conductance as a function of habitat covariates. As usual in **secr** these are stored as one or more mask covariates. It is easy to add a covariate for forest type (*Nothofagus*-dominant ‘beech’ vs ‘nonbeech’) to our mask:

```
ovmask <- addCovariates(ovmask, ovforest[1:2,])
fit5 <- secr.fit(ovposs, mask = ovmask, detectfn = "HHN",
  details = list(userdist = fn2), model = list(D ~ forest,
    noneuc ~ forest), fixed = list(sigma = 1), trace = FALSE)
```

```
predict(fit5, newdata =
  data.frame(forest = c("beech", "nonbeech")))
```

```
$`forest = beech`
      link estimate SE.estimate      lcl      ucl
D      log  9.42514   2.6531452  5.48569 16.19366
lambda0 log  0.10111   0.0089337  0.08506  0.12019
noneuc  log 29.55770   3.4060703 23.59968 37.01990
```

```
$`forest = nonbeech`
      link estimate SE.estimate      lcl      ucl
D      log 15.67451   1.2679364 13.37985 18.36271
lambda0 log  0.10111   0.0089337  0.08506  0.12019
noneuc  log 27.23476   1.0319165 25.28619 29.33349
```

Note that we have re-used the `userdist` function `fn2`, and allowed both density and `noneuc` (`sigma`) to vary by forest type. Strictly, we should have identified “forest” as a required covariate in the (re)definition of `fn2`, but this is obviously not critical.

A full analysis should also consider models with variation in `lambda0`. There is no simple way in **secr** to model continuous spatial variation in `lambda0` as a function of home-range location (cf `sigma` in Example 1 above). However, variation in `lambda0` at the point of detection may be modelled with detector-level covariates (Section 10.2.1).

F.5 And the winner is...

Now that we have a bunch of fitted models, let’s see which does the best:

```
fits <- secrlist(fit0, fit0a, fit1, fit1a, fit2, fit3, fit4,
  fit4a, fit5)
AIC(fits)[, -c(2,4,6)]
```


	model	npar	AIC	dAIC	AICwt
fit4a	D~s(x, y, k = 6) lambda0~1 noneuc~1	8	3098.1	0.000	0.4549
fit4	D~x + y + x2 + y2 + xy lambda0~1 noneuc~1	8	3098.5	0.382	0.3758
fit1	D~1 lambda0~1 noneuc~x + y + x2 + y2 + xy	8	3101.8	3.704	0.0714
fit3	D~1 lambda0~1 noneuc~x + y + x2 + y2 + xy	8	3102.4	4.304	0.0529
fit2	D~1 lambda0~1 noneuc~x + y + x2 + y2 + xy	8	3103.5	5.411	0.0304
fit1a	D~1 lambda0~1 noneuc~x + y + x2 + y2 + xy	8	3105.0	6.862	0.0147
fit0	D~1 lambda0~1 sigma~1	3	3118.1	19.979	0.0000
fit0a	D~1 lambda0~1 noneuc~1	3	3118.1	19.979	0.0000
fit5	D~forest lambda0~1 noneuc~forest	5	3118.4	20.280	0.0000

...the model with a quadratic or spline trend in density and density-dependent sigma.

F.6 Notes

The ‘real’ parameter for spatial scale (σ) is lurking in the background as part of the detection model. User-defined non-Euclidean distances are used in the detection function just like ordinary Euclidean distances. This means in practice that they are (almost) always divided by σ . Formally: the distance d_{ij} between an animal i and a detector j appears in all commonly used detection functions as the ratio $r_{ij} = d_{ij}/\sigma$ (e.g., halfnormal $\lambda = \lambda_0 \exp(-0.5r_{ij}^2)$ and exponential $\lambda = \lambda_0 \exp(-r_{ij})$).

What if we want non-Euclidean distances, but do not want to estimate noneuc? This is a perfectly reasonable request if sigma is constant across space and the distance computation is determined entirely by the habitat geometry, with no need for an additional parameter. If ‘noneuc’ is not included in the character vector returned by your userdist function when it is called with no arguments then noneuc is not modelled at all. This is the default in **secrelinear**.

Providing a suitable initial value for ‘noneuc’ can be a problem. The argument ‘start’ of **secre.fit** may be a named, and possibly incomplete, list of real parameter values, so a call such as this is valid:

```
secre.fit (captdata, model = noneuc~1, details = list(userdist = fn2),
          trace = FALSE, start = list(noneuc = 25), fixed = list(sigma = 1))
```

```
secre.fit(capthist = captdata, model = noneuc ~ 1, start = list(noneuc = 25),
          fixed = list(sigma = 1), details = list(userdist = fn2),
          trace = FALSE)
```

secre 5.2.0, 20:25:03 13 Feb 2025

```
Detector type      single
Detector number    100
```

```

Average spacing      30 m
x-range              365 635 m
y-range              365 635 m

N animals           : 76
N detections         : 235
N occasions          : 5
Mask area            : 21.227 ha

Model                : D~1 g0~1 noneuc~1
User distances       : dynamic (function)
Fixed (real)         : sigma = 1
Detection fn         : halfnormal
Distribution          : poisson
N parameters         : 3
Log likelihood       : -759.03
AIC                  : 1524.1
AICc                 : 1524.4

```

```

Beta parameters (coefficients)
      beta  SE.beta    lcl    ucl
D      1.70107 0.117615  1.4705  1.93159
g0     -0.97849 0.136239 -1.2455 -0.71147
noneuc  3.37983 0.044415  3.2928  3.46688

```

```

Variance-covariance matrix of beta parameters
      D      g0      noneuc
D      0.01383322  0.00015579 -0.00099075
g0      0.00015579  0.01856106 -0.00334338
noneuc -0.00099075 -0.00334338  0.00197272

```

```

Fitted (real) parameters evaluated at base levels of covariates
      link estimate SE.estimate    lcl    ucl
D      log  5.47980    0.646741  4.35162  6.90047
g0     logit 0.27319    0.027051  0.22348  0.32927
noneuc  log 29.36583    1.304938 26.91757 32.03677

```

We have ignored the parameter λ_0 . This is almost certainly a mistake, as large variation in σ without compensatory or normalising variation in λ_0 is biologically implausible and can lead to improbable results Efford (2014).

It is intended that non-Euclidean distances should work with all relevant functions in **secr**.

You may be tempted to model ‘noneuc’ as a function of group - after all, D~g is permitted, right? Unfortunately, this will not work. There is only one pre-computed distance matrix, not one matrix per group.

F.7 Simulation after Sutherland et al. (2015)

Sutherland et al. (2015) simulated SECR data from a population of animals whose movement was channelled to varying extents along a dendritic network (river system). Their model treated the habitat as 2-dimensional and shrank distances for pixels close to water and expanded them for pixels further away. The authors kindly provided data for the network map and detector layout which we use here to emulate their simulations in **secr**. We assume an existing `SpatialLinesDataFrame` `sample.water` for the network, and a matrix of x-y coordinates for detector locations `gridTrapsXY`. `rivers` is a version of `sample.water` clipped to the habitat mask and used only for plotting.

```
# use package secrlinear to create a discretised version of the
# network as a handy way to get distance to water
# loading secrlinear also loads secr
library(secrlinear)
library(gdistance)

swlinearmask <- read.linearmask(data = sample.water, spacing = 100)

# generate secr traps object from detector locations
tr <- data.frame(gridTrapsXY*1000) # convert to metres
names(tr) <- c("x","y")
tr <- read.traps(data=tr, detector = "count")

# generate 2-D habitat mask
sw2Dmask <- make.mask(tr, buffer = 3950, spacing = 100)
d2w <- distancetotrap(sw2Dmask, swlinearmask)
covariates(sw2Dmask) <- data.frame(d2w = d2w/1000) # km to water
```

Warning: attribute variables are assumed to be spatially constant throughout all geometries

```
par(mar = c(1,6,1,6))
plot(sw2Dmask, covariate = "d2w", dots = FALSE)
plot(tr, add = TRUE)
plot(rivers, add = TRUE, col = "blue")
```

The distance function requires a value of the friction parameter ‘noneuc’ for each mask pixel. Distances are approximated using **gdistance** functions as before, except that we interpret the distance-to-water scale as ‘friction’ and invert that for **gdistance**.

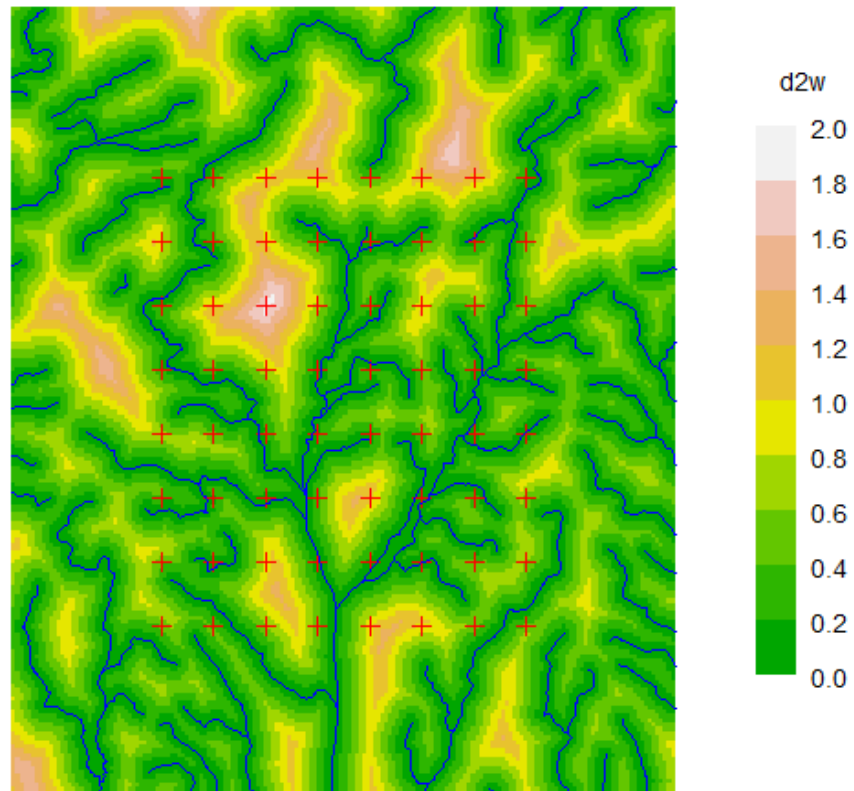


Figure F.4: Shaded plot of distance to water (d2w in km) with detector sites (red crosses) and rivers superimposed. Detector spacing 1.5 km N-S.

```
dfn <- function (xy1, xy2, mask) {
  if (missing(xy1)) return("noneuc")
  require(gdistance)
  Sraster <- raster(mask, "noneuc")
  # conductance is inverse of friction
  trans <- transition(Sraster, transitionFunction =
    function(x) 1/mean(x), directions = 16)
  trans <- geoCorrection(trans)
  costDistance(trans, as.matrix(xy1), as.matrix(xy2))
}
```

The Royle, Chandler, Gazenski, et al. (2013) and Sutherland et al. (2015) models use an (α_0, α_1) parameterisation instead of (λ_0, σ) . Their α_2 translates directly to a coefficient in the **secr** model, as we'll see. We consider just one realisation of one scenario (the package **secrdesign** Efford (2025b) manages replicated simulations of multiple scenarios).

```
# parameter values from Sutherland et al. 2014
alpha0 <- -1 # implies lambda0 = invlogit(-1) = 0.2689414
sigma <- 1400
alpha1 <- 1 / (2 * sigma^2)
alpha2 <- 5 # just one scenario from the range 0..10
K <- 10 # sampling over 10 occasions, collapsed to 1 occ
```

Now we are ready to build a simulated dataset.

```
# simulate fixed population of 200 animals in masked area
pop <- sim.popn (D = 200/nrow(sw2Dmask), core = tr,
  buffer = 3950, Ndist = "fixed")
# to simulate non-Euclidean detection we attach a mask with
# the pixel-specific friction to the simulated popn object
covariates(sw2Dmask)$noneuc <- exp(alpha2 *
  covariates(sw2Dmask)$d2w)
attr(pop, "mask") <- sw2Dmask
# simulate detections, specifying non-Euclidean distance function
CH <- sim.capthist(tr, pop = pop, userdist = dfn,
  nooccasions = 1, binomN = K, detectpar = list(lambda0 =
    invlogit(alpha0), sigma = sigma), detectfn = "HHN")
```

```
summary(CH, moves = TRUE)
```

Object class	capthist
Detector type	count
Detector number	64
Average spacing	1385.7 m

```
x-range      1698699 1708399 m
y-range      2387891 2398391 m
```

Counts by occasion

	1	Total
n	37	37
u	37	37
f	37	37
M(t+1)	37	37
losses	0	0
detections	109	109
detectors visited	33	33
detectors used	64	64

Number of movements per animal

	0	1	2	3
	16	13	7	1

Distance moved, excluding zero (m)

Min.	1st Qu.	Median	Mean	3rd Qu.	Max.
1386	1386	1443	1552	1500	3151

Individual covariates

```
sex
F:21
M:16
```

Model fitting is simple, but the default starting value for noneuc is not suitable and is overridden:

```
fitne1 <- secr.fit (CH, mask = sw2Dmask, detectfn = "HHN",
  binomN = 10, model = noneuc ~ d2w -1, details = list(
  userdist = dfn), start = list(D = 0.005, lambda0 = 0.3,
  sigma = 1000, noneuc = 100), trace = FALSE)
```

The warning from nlm indicates a potential problem, but the standard errors and confidence limits below look plausible (they could be checked by running `secr.refit` with `method = "none"`). Fitting is slow (4 minutes on an aging PC). This is partly because the mask is large (32384 pixels) in order to maintain resolution in relation to the stream network.

```
coef(fitne1)
```

	beta	SE.beta	lcl	ucl
D	-5.3042	0.181714	-5.6603	-4.94800

```
lambda0    -1.1386 0.169830 -1.4715 -0.80577
sigma       7.1491 0.091395  6.9699  7.32821
noneuc.d2w  4.5961 0.503034  3.6102  5.58205
```

```
predict(fitne1)
```

	link	estimate	SE.estimate	lcl	ucl
D	log	4.9709e-03	9.1079e-04	3.4814e-03	7.0976e-03
lambda0	log	3.2026e-01	5.4784e-02	2.2958e-01	4.4674e-01
sigma	log	1.2729e+03	1.1658e+02	1.0642e+03	1.5226e+03
noneuc	log	7.5395e+00	1.6876e+00	4.8881e+00	1.1629e+01

```
region.N(fitne1)
```

	estimate	SE.estimate	lcl	ucl	n
E.N	160.98	29.495	112.74	229.85	37
R.N	160.98	26.627	118.77	224.97	37

The coefficient noneuc.d2w corresponds to α_2 . Estimates of predicted ('real') parameters D and lambda0, and the coefficient noneuc.d2w, and are comfortably close to the true values, and all true values are covered by the 95% CI.

We fit the 'noneuc' (friction) parameter through the origin (zero intercept; -1 in formula). The predicted value of 'noneuc' relates to the covariate value for the first pixel in the mask ($d2w = 1.133$ km), but in this zero-intercept model the meaning of 'noneuc' itself is obscure. In effect, the parameter α_1 (or sigma) serves as the intercept; the same model may be fitted by fixing sigma (`fixed = list(sigma = 1)`) and estimating an intercept for noneuc (`model = noneuc ~ d2w`). In this case, 'noneuc' may be interpreted as the site-specific sigma (see also examples in the main text).

It is interesting to plot the predicted detection probability under the simulated model. For plotting we add the pdot value as an extra covariate of the mask. Note that pdot here uses the 'noneuc' value previously added as a covariate to `sw2Dmask`.

```
covariates(sw2Dmask)$predicted.pdot <- pdot(sw2Dmask, tr,
      nooccasions = 1, binomN = 10, detectfn = "HHN", detectpar =
      list(lambda0 = invlogit(-1), sigma = sigma), userdist = dfn)
```

```
par(mar = c(1,6,1,6))
plot(sw2Dmask, covariate = "predicted.pdot", dots = FALSE)
plot(tr, add = TRUE)
plot(rivers, add = TRUE, col = "blue")
```

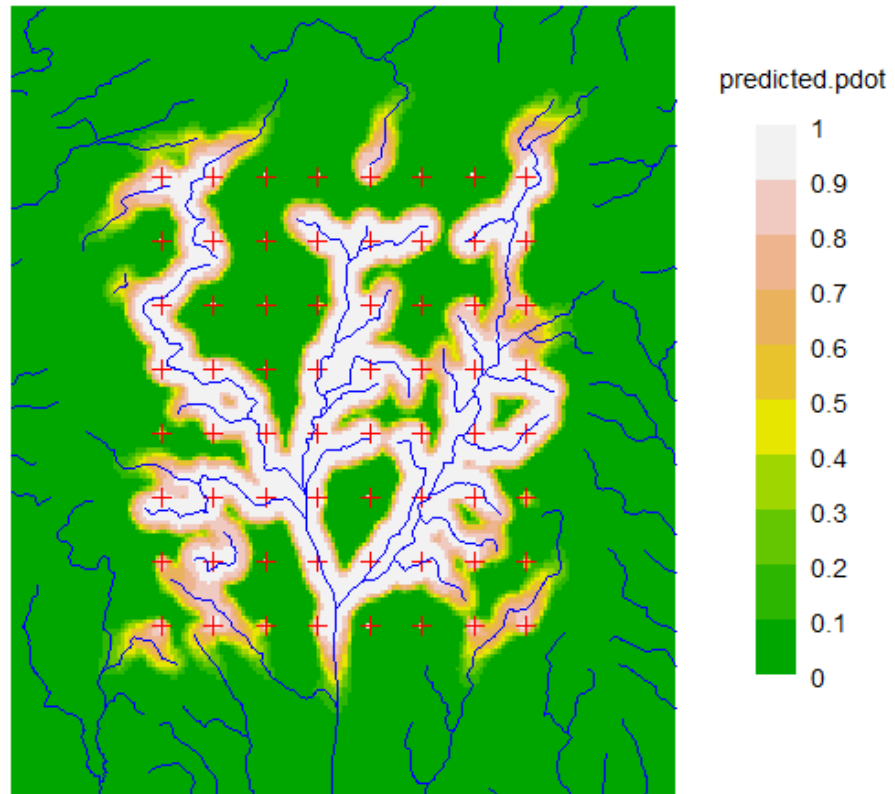


Figure F.5: Shaded plot of probability animal is detected at least once. Animals living within the detector array and away from a river (about half the population within the array) stand very little chance of being detected because the model confines them to a small home range and λ_0 is constant.

G Telemetry

In some capture–recapture studies there are additional data from radiotelemetry of a sample of animals. Telemetry fixes provide an unbiased sample of animal activity, unlike detections at fixed points (traps, cameras or hair snares) or from searching circumscribed areas. Telemetry data therefore provide a direct estimate of the spatial scale of activity, which is represented in spatially explicit capture–recapture (SECR) by the parameter σ . Telemetry data also reduce uncertainty regarding the location of animals’ centres relative to detectors, so detection histories of telemetered animals may improve estimates of other parameters. These are *potential* benefits – we do not consider whether they are realised *in practice*.

Previous examples are Sollmann et al. (2013), Whittington et al. (2018), Linden et al. (2018). Combining data types has been considered generally by Gopalaswamy et al. (2012), Chandler et al. (2021), Ruprecht et al. (2021), Margenau et al. (2022) and others.

This chapter explains and demonstrates the use of telemetry data in the R package **secr**, particularly the use of the ‘telemetry’ detector type.

Warning

Telemetry is used in **secr** to augment SECR analyses, particularly the estimation of population density. **secr** is not intended for the detailed analysis of telemetry data *per se*. There is no provision in the data structure for recording the time of each fix, except as each relates to a discrete sampling occasion. Nor is there provision for associating behavioural or environmental covariates with each fix.

G.1 Example

We start with a concrete example based on a simulated dataset.

```
# Code to simulate capthist objects (trCH, teCH)

# detectors
te <- make.telemetry()
tr <- make.grid(detector = "multi", nx = 8, ny = 8)

# spatial population
set.seed(123)
pop4 <- sim.popn(tr, D = 5, buffer = 100, seed = 567)
```

```
# select 12 telemetered individuals from larger population
pop4C <- subset(pop4, sample.int(nrow(pop4), 12))

# renumber = FALSE (keep original animalID) needed for matching
trCH <- sim.caphist(tr, popn = pop4, renumber = FALSE,
  detectfn = "HHN", detectpar = list(lambda0 = 0.1,
    sigma = 25), seed = 123)
session(trCH) <- 'Trapping'
teCH <- sim.caphist(te, popn = pop4C, renumber = FALSE,
  detectfn = "HHN", detectpar = list(lambda0 = 1, sigma = 25),
  nooccasions = 10, seed = 345)
session(teCH) <- 'Telemetry'
```

The first step is to combine the caphist objects `trCH` (trapping data) and `teCH` (telemetry fixes).

```
combinedCH <- addTelemetry(trCH, teCH)
```

Generating plots is straightforward. By default, `plot.caphist` displays the captures and ignores telemetry fixes. The plot type “telemetry” displays the fixes and distinguishes those of animals that also appear in the (unplotted) capture data of the combined object.

```
par(mfrow = c(1,2), mar = c(2,2,3,2))
plot(traps(trCH), border = 150, bty = 'o') # base plot
plot(combinedCH, title = 'Trapping', tracks = TRUE, rad = 4, add = TRUE)
plot(traps(trCH), border = 150, bty = 'o') # base plot
plot(combinedCH, title = 'Telemetry', type = 'telemetry',
  tracks = TRUE, add = TRUE)
```

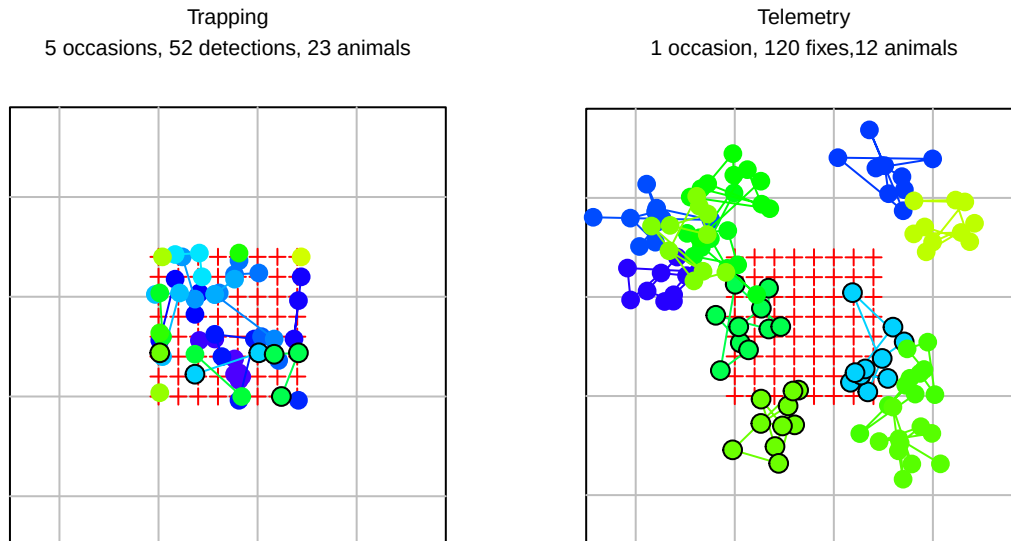


Figure G.1: Simulated trapping and telemetry data. The 5-day trapping study (left) yielded 29 recaptures. Telemetry data were obtained on 10 occasions for each of 12 animals (right). Points for animals that were both trapped and telemetered are ringed in black. Colours distinguish individuals.

Next we fit trapping-only and joint trapping-and-telemetry models:

```
mask <- make.mask(traps(trCH), buffer = 100, type = 'trapbuffer',
  nx = 32)
args <- list(mask = mask, detectfn = 'HHN', trace = FALSE)
fits <- list.secr.fit(list(trCH, combinedCH), constant = args,
  names = c('tr','combined'))
collate(fits)[1,,]
```

, , D

	estimate	SE.estimate	lcl	ucl
tr	5.9057	1.3312	3.8175	9.1363
combined	5.7070	1.2801	3.6967	8.8104

, , lambda0

	estimate	SE.estimate	lcl	ucl
tr	0.118276	0.029911	0.072604	0.19268
combined	0.073111	0.015752	0.048157	0.11100

, , sigma

	estimate	SE.estimate	lcl	ucl
--	----------	-------------	-----	-----

tr	22.027	2.4031	17.798	27.262
combined	26.498	1.1295	24.375	28.806

Here, telemetry data greatly improves the precision of the estimated scale of movement σ , but the effect on estimates of the other two parameters is small.

G.2 Standalone telemetry data

Telemetry data are stored in modified **secr** capthist objects. A capthist object may comprise telemetry fixes only ('standalone telemetry') or telemetry fixes in association with capture–recapture data (composite telemetry and capture–recapture). This section describes standalone telemetry data. A composite capthist is formed by combining a telemetry-only object and a standard capthist object with `addTelemetry`, as described in Section [G.3](#).

G.2.1 The 'traps' object for telemetry data

Telemetry data differ from all other SECR data in that the detection process does not constrain where animals are detected (telemetry provides a spatially unbiased sample of each animal's activity). That is clearly not the case for area-search and point detectors, which inevitably constrain where animals are detected. Nevertheless, for compatibility with the rest of **secr**, we associate telemetry data with a notional detector located at a point. The 'traps' object for telemetry data comprises the coordinates of this point plus the usual attributes of a 'traps' object in **secr** (detector type, usage, etc.). Remember that the point is only a 'notional' detector - it is never visited.

The function `make.telemetry` generates a suitable object:

```
te <- make.telemetry()
summary(te)
```

```
Object class      traps
Detector type     telemetry
Telemetry type    independent
```

```
str(te)
```

```
Classes 'traps' and 'data.frame':  1 obs. of  2 variables:
 $ x: num 0
 $ y: num 0
 - attr(*, "detector")= chr "telemetry"
 - attr(*, "telemetrytype")= chr "independent"
```

The attribute `telemetrytype` is always 'independent' for standalone telemetry data; other possible values for composite data are described later.

G.2.2 Detector type

Every ‘traps’ object has an associated detector type (attribute `detector`, commonly ‘multi’ or ‘proximity’). This may be a vector with a different value for each occasion. The detector type for telemetry data is ‘telemetry’. In a standalone telemetry capthist, all elements of `detector` are ‘telemetry’.

G.2.3 But where are the data?

As for any other detector type, the body of a telemetry capthist is a 3-D array whose elements are the number of detections for each combination of animal, occasion and detector. Coordinates are stored separately in the ‘telemetryxy’ attribute. Use one of these functions to reveal the telemetry component of a capthist object `CH`:

1. `str(CH)`
2. `summary(CH)`
3. `plot(CH, type = 'telemetry')`
4. `telemetryxy(CH)`

The attribute ‘telemetryxy’ is a list with one component for each animal. The fixes of each animal are sorted in chronological order.

G.2.4 Data input

Data for a telemetry-only object should be read with function `read.telemetry`, a simplified version of `read.capthist`. The input format for telemetry fixes follows the ‘XY’ format for captures, with one line per fix (see [seccr-datainput.pdf](#)).

The first few lines of a text file containing telemetry data collected on 5 occasions might look like this –

```
1 10 1 -83.3 -20.04
1 10 2 -57.91 -4.77
1 10 3 -112.96 -7.51
1 10 4 -77.71 -75.79
1 10 5 -85.81 -42.45
1 101 1 143.06 170.48
1 101 2 99.22 145.49
etc.
```

The first column is a session code, the next an animal identifier (‘10’, ‘101’), the third an occasion number (1..nooccasions) and the last two are the x and y coordinates. GPS coordinates should be projected (i.e. not latitude and longitude), and in metres if possible.

A file named ‘telemetrydemo.txt’ may be read with

```
CHt <- read.telemetry(file = "data/telemetrydemo.txt")
```

No errors found :-)

```
head(CHt)
```

```
Session = 1  
, , 1
```

```
      1 2 3 4 5  
4  1 1 1 1 1  
9  1 1 1 1 1  
10 1 1 1 1 1  
11 1 1 1 1 1  
12 1 1 1 1 1  
14 1 1 1 1 1
```

```
telemetryxy(CHt)[['10']] # coordinates of first animal
```

```
      x      y  
1 -83.30 -20.04  
2 -57.91  -4.77  
3 -112.96 -7.51  
4 -77.71 -75.79  
5 -85.81 -42.45
```

Input may be from a text file (named in argument ‘file’) or dataframe (argument ‘data’). The body of the resulting capthist object merely tallies the number of detections per animal per session and occasion. The fixes for one session are stored separately in an attribute that is a list of dataframes, one per animal. Use `telemetryxy(CHt)` to retrieve this list.

The summary of a telemetry-only capthist is quirky:

```
summary(CHt)
```

```
Object class      capthist  
Detector type     telemetry (5)  
Telemetry type    independent  
  
Counts by occasion  
      1  2  3  4  5 Total  
n      79 79 79 79 79 395
```

u	79	0	0	0	0	79
f	0	0	0	0	79	79
M(t+1)	79	79	79	79	79	79
losses	0	0	0	0	0	0
detections	79	79	79	79	79	395
detectors visited	0	0	0	0	0	0
detectors used	0	0	0	0	0	0

Empty histories : 79
 79 telemetered animals, 0 detected
 5-5 locations per animal, mean = 5, sd = 0

Individual covariates
 V6

Min. :1.00
 1st Qu.:1.00
 Median :2.00
 Mean :1.52
 3rd Qu.:2.00
 Max. :2.00

Even though each fix is counted as a ‘detection’ in the body of the final capthist object, none of the telemetered animals is considered to have been ‘detected’ in a conventional SECR sense. The telemetry-only capthist object includes a trivial traps object with a single point. The telemetry type of the traps for a telemetry-only capthist defaults to ‘independent’.

G.3 Combining telemetry and capture–recapture

For the purpose of density estimation and modelling, standalone telemetry data are added to an existing spatial capture–recapture (capthist) data object with the function `addTelemetry`. The relationship between the telemetry and capture–recapture samples is determined by the `type` argument (default ‘concurrent’). We first explain the possible telemetry types.

G.3.1 Types of telemetry data

`secr` distinguishes three types of telemetry data – independent, dependent and concurrent – that differ in how they relate to other SECR samples (capture–recapture data). Each type corresponds to a particular probability model.

1. Independent telemetry

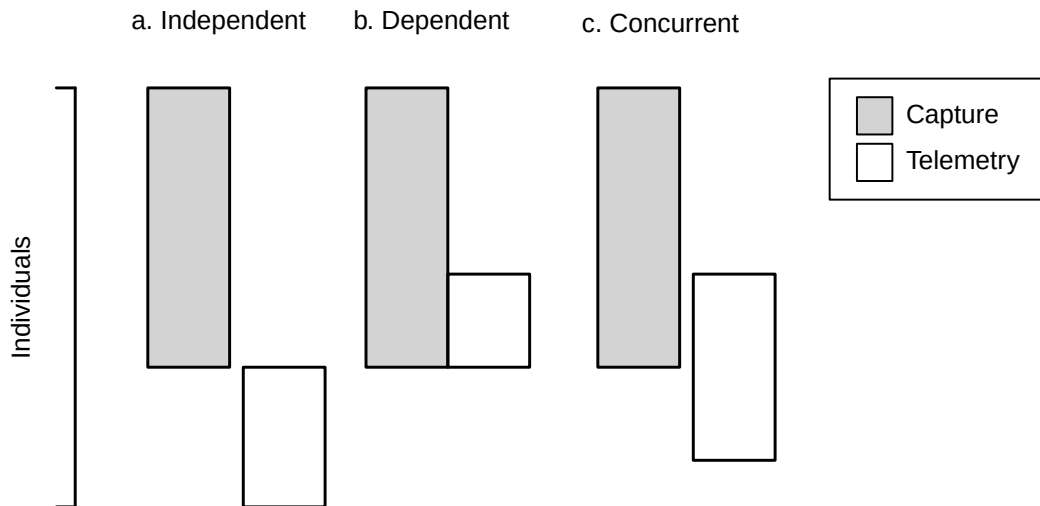


Figure G.2: Schematic relationship of capture–recapture data to three types of telemetry data. Vertical overlap indicates individuals that appear in both datasets.

Independent telemetry data have no particular relationship to spatial capture–recapture data except that they may be modelled using a shared value of the spatial-scale parameter σ , and possibly other spatial parameters. Telemetered animals do not have detection histories.

2. Dependent telemetry

Dependent telemetry data relate to a sample of animals detected during the capture–recapture study: an animal must be caught in that study to become telemetered, and no animal is telemetered and not otherwise detected (i.e. no detection history is all-zero).

3. Concurrent telemetry

Concurrent telemetry data are obtained for a sample of animals from the same regional population as the capture–recapture study. Telemetered animals appear stochastically in the capture–recapture sample with probability related to their location. Detection histories of some animals may be all-zero, and these are modelled. Whether the capture–recapture phase precedes or follows telemetry is not material.

G.3.2 What exactly does `addTelemetry` do?

The `addTelemetry` function forms a composite capthist object. Its usage follows -

```
addTelemetry (detectionCH, telemetryCH, type = c("concurrent", "dependent",
  "independent"), collapse telemetry = TRUE, verify = TRUE)
```

Capture–recapture data in the argument ‘`detectionCH`’ form the basis for `addTelemetry`. The base capthist is modified in these ways -

- For all telemetry types `addTelemetry` extends the capture–recapture ‘traps’ object by adding a single (notional) detector location (duplicating the first).
- By default, the ‘detector’ attribute is extended by a single sampling occasion with type ‘telemetry’; all telemetry data are associated with this occasion, regardless of how many occasions there were in the telemetry input. If `collapsetelemetry = FALSE` distinct telemetry occasions are retained.
- The ‘usage’ attribute is set to zero for the notional telemetry detector on capture occasions and for capture detectors on telemetry occasions. Other usage data from ‘detectionCH’ is retained.
- All-zero detection histories are generated for the ‘concurrent’ data type.
- The coordinates of telemetry fixes are transferred from `telemetryCH` as the attribute ‘telemetryxy’ of the output.
- If the data are independent then the labels of telemetered animals are prefixed by ‘T’ to reduce the chance of identity conflicts with animals in ‘detectionCH’.
- By default, `addtelemetry` calls `verify.caphist` to check its output.

G.3.3 Composite data, different sessions

We use `addTelemetry` to combine telemetry data and capture–recapture data from the same session, or possibly for each of several sessions when the `detectionCH` and `telemetryCH` are parallel (equal-length) multi-session objects. It is also feasible to concatenate telemetry and capture–recapture data as separate sessions of a multi-session object with `MS.caphist`. The effect is similar to a single-session composite `caphist` with `telemetrytype` ‘independent’, because `secr` treats sessions as independent (i.e. individual histories do not span session boundaries). See the next section for an example.

G.4 Model fitting

G.4.1 Standalone telemetry data

We can estimate σ for a half-normal circular home-range model directly:

```
RPSV (CHt, CC = TRUE)
```

```
[1] 24.868
```

Note the `CC` argument (named for Calhoun & Casby (1958)) that is required to scale the result correctly.

More laboriously:

```
fit0 <- secr.fit(CHt, buffer = 300, detectfn = 'HHN',
                 trace = FALSE)
predict(fit0)
```

```
      link estimate SE.estimate      lcl      ucl
sigma log    24.867      0.69957 23.533 26.277
```

The detection function (argument `detectfn`) must be either hazard half-normal (14, ‘HHN’) or hazard exponential (16, ‘HEX’)¹. The default detection function for a dataset with any telemetry component is ‘HHN’. For telemetry-only data the likelihood is conditional on the number of observations, so the argument `CL` is set internally to `TRUE`. A large `buffer` value here brings $\hat{\sigma}$ from `secr.fit` closer to $\hat{\sigma}$ from `RPSV`. See [below](#) for more on the `buffer` argument.

See [Technical notes](#) for potential numerical problems.

G.4.2 Composite telemetry and capture–recapture data

Fitting a model to composite data should raise no further problems: `secr.fit` receives all the information it requires in the composite capthist input. The likelihood is a straightforward extension of the usual SECR likelihood, with some subtle differences in the case of dependent or concurrent telemetry.

The use of detection functions expressed in terms of the hazard provides a more natural link between the model for the activity distribution and the model for detection probability. When a hazard function is used `secr.fit` automatically flips the default model for the first detection parameter from ‘ $g_0 \sim 1$ ’ to ‘ $\lambda_0 \sim 1$ ’.

Our introductory [example](#) fitted a model to single-session composite data. We can compare the results when the telemetry and trapping data are in separate sessions:

```
msCH <- MS.capthist(trCH, teCH)
fit.ms <- secr.fit(msCH, mask=mask, detectfn = 'HHN',
                  trace = FALSE)
predict(fit.ms)
```

```
$`session = trCH`
      link estimate SE.estimate      lcl      ucl
D      log  5.68802   1.251718  3.714104  8.7110
lambda0 log  0.10628   0.021189  0.072182  0.1565
sigma    log 23.63780   1.043956 21.678653 25.7740
```

¹This constraint arises from the need internally to normalise the probability density function for each telemetry fix. The normalising constant for these functions is $1/(2\pi\sigma^2)$, whereas for most other possible values of `detectfn` it is hard to compute or the function does not correspond to a probability density.

```
$`session` = teCH`
      link estimate SE.estimate      lcl      ucl
sigma log    23.638         1.044 21.679 25.774
```

Note: If the order of `teCH` and `trCH` had been reversed in `msCH` we would need to use `details=list(autoini=2)` to base parameter starting values on the trapping data, or provide start values manually.

G.4.3 Habitat mask for telemetry data

The centres of both detected and telemetry-only animals are assumed to lie on the habitat mask. Ensure the mask is large enough to encompass telemetry-only animals. A conservative approach is to buffer around the individual telemetry centroids. Using `teCH` from before:

```
centroids <- data.frame(t(sapply(telemetryxy(teCH),
  apply, 2, mean)))
mask1 <- make.mask(centroids, buffer = 100, type = 'trapbuffer')
```

For composite telemetry and capture–recapture, buffering should include the detector sites:

```
tmpxy <- rbind(centroids, data.frame(traps(trCH)))
mask2 <- make.mask(tmpxy, buffer = 100, type = 'trapbuffer')
```

```
par(mfrow = c(1,2))
plot(mask1)
plot(teCH, add = TRUE, title = 'Telemetry only')
plot(mask2)
plot(traps(trCH), add = TRUE)
plot(teCH, add = TRUE, title = 'Telemetry and detector sites')
```

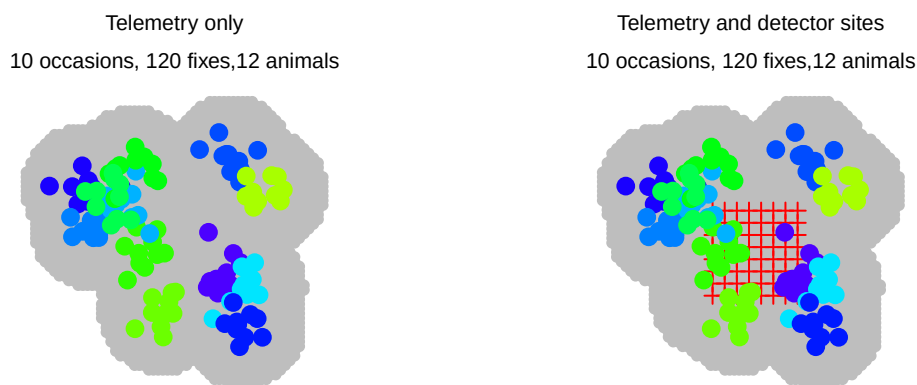


Figure G.3: Habitat masks prepared by buffering around telemetry sites (left) or both telemetry and detector sites (right).

The mask generated automatically by `secr.fit` buffers around both detector sites and telemetry fixes, as shown here. The ‘buffer’ argument in `make.mask` can be problematic when used with a standalone telemetry traps object because the notional detector location is an arbitrary point - it is better to use the centroid coordinates as input.

G.5 Simulation

Simulation of joint capture–recapture and telemetry data is a 2-step operation in **secr**, with the steps depending on the type of telemetry sampling. Here is an example of each type.

We choose to fix the number of observations per animal at 25 using the `exactN` argument of `sim.caphist`. The same effect can be achieved by increasing the number of occasions to 25 and setting `exactN = 1`.

G.5.1 Independent telemetry

For independent data there is no specified connection between the populations sampled, so we separately generate telemetry and capture–recapture datasets and stick them together.

```
# detectors
te <- make.telemetry()
tr <- make.grid(nx = 8, ny = 8, detector = "proximity")
pop1 <- sim.popn(tr, D = 10, buffer = 200)
pop2 <- sim.popn(core = tr, buffer = 200, Nbuffer = 20,
                 Ndist = 'fixed')
trCH <- sim.caphist(tr, popn = pop1, detectfn = "HHN",
                  detectpar = list(lambda0 = 0.1, sigma = 25))
teCH <- sim.caphist(te, popn = pop2, detectfn = "HHN",
                  detectpar = list(sigma = 25), nooccasions = 1, exactN = 25)
CHI <- addTelemetry(trCH, teCH, type = 'independent')
```

No errors found :-)

```
session(CHI) <- 'Independent'
summary(CHI)
```

Object class	capthist
Detector type	proximity (5), telemetry
Telemetry type	independent
Detector number	64
Average spacing	20 m
x-range	0 140 m
y-range	0 140 m

Usage range by occasion

	1	2	3	4	5	6
min	0	0	0	0	0	0
max	1	1	1	1	1	1

Counts by occasion

	1	2	3	4	5	6	Total
n	24	16	19	22	17	20	118
u	24	9	7	5	1	20	66
f	35	14	13	4	0	0	66
M(t+1)	24	33	40	45	46	66	66
losses	0	0	0	0	0	0	0
detections	29	19	30	26	19	500	623
detectors visited	23	16	23	23	18	0	103
detectors used	64	64	64	64	64	0	320

Empty histories : 20

20 telemetered animals, 0 detected

25-25 locations per animal, mean = 25, sd = 0

Individual covariates

sex
F:34
M:32

G.5.2 Dependent telemetry

For dependent data the telemetry sample is drawn from animals caught during the capture–recapture phase. This example uses the previously constructed ‘traps’ objects (`tr` and `te`). The original numbering of animals must be conserved (`renumber = FALSE`).

```
pop3 <- sim.popn(tr, D = 10, buffer = 200)
trCH <- sim.caphist(tr, popn = pop3, detectfn = "HHN",
  detectpar = list(lambda0 = 0.1, sigma = 25), renumber =
  FALSE, savepopn = TRUE)

## select trapped animals from saved popn
pop3D <- subset(attr(trCH, 'popn'), rownames(trCH))
## sample 12 detected animals for telemetry
pop3Dt <- subset(pop3D, sample.int(nrow(pop3D), 12))
## simulate telemetry
teCHD <- sim.caphist(te, popn = pop3Dt, renumber = FALSE,
  detectfn = "HHN", detectpar = list(sigma = 25),
```

```
noccasions = 1, exactN = 25)
CHD <- addTelemetry(trCH, teCHD, type = 'dependent')
```

No errors found :-)

```
session(CHD) <- 'Dependent'
summary(CHD)
```

```
Object class      capthist
Detector type     proximity (5), telemetry
Telemetry type    dependent
Detector number   64
Average spacing   20 m
x-range           0 140 m
y-range           0 140 m
```

```
Usage range by occasion
  1 2 3 4 5 6
min 0 0 0 0 0 0
max 1 1 1 1 1 1
```

```
Counts by occasion
      1  2  3  4  5  6 Total
n      20 22 22 10 26 12  112
u      20 12  8  1  4  0   45
f      12 14  9  5  5  0   45
M(t+1) 20 32 40 41 45 45   45
losses  0  0  0  0  0  0    0
detections 33 31 29 11 36 300  440
detectors visited 25 25 26 10 26  0  112
detectors used   64 64 64 64 64  0  320
12 telemetered animals, 12 detected
25-25 locations per animal, mean = 25, sd = 0
```

```
Individual covariates
sex
F:20
M:25
```

G.5.3 Concurrent telemetry

For concurrent telemetry a sample of animals is taken from the regional population without reference to whether or not each animal was detected in the capture-recapture phase. The

original numbering of animals must be conserved (`renumber = FALSE`), as for dependent telemetry.

```
set.seed(567)
pop4 <- sim.popn(tr, D = 10, buffer = 200)
# select 15 individuals at random from larger population
pop4C <- subset(pop4, sample.int(nrow(pop4), 15))
# original animalID (renumber = FALSE) are needed for matching
trCH <- sim.caphist(tr, popn = pop4, renumber = FALSE,
  detectfn = "HHN", detectpar = list(lambda0 = 0.1,
  sigma = 25))
teCHC <- sim.caphist(te, popn = pop4C, renumber = FALSE,
  detectfn = "HHN", detectpar = list(sigma = 25), noccasions =
  1, exactN = 25)
CHC <- addTelemetry(trCH, teCHC, type = 'concurrent')
```

No errors found :-)

```
session(CHC) <- 'Concurrent'
summary(CHC)
```

```
Object class      caphist
Detector type     proximity (5), telemetry
Telemetry type    concurrent
Detector number   64
Average spacing   20 m
x-range           0 140 m
y-range           0 140 m
```

Usage range by occasion

```
      1 2 3 4 5 6
min 0 0 0 0 0 0
max 1 1 1 1 1 1
```

Counts by occasion

	1	2	3	4	5	6	Total
n	16	17	21	17	16	15	102
u	16	12	7	5	3	11	54
f	28	11	9	5	1	0	54
M(t+1)	16	28	35	40	43	54	54
losses	0	0	0	0	0	0	0
detections	25	26	27	26	24	375	503
detectors visited	23	20	22	22	19	0	106
detectors used	64	64	64	64	64	0	320

```
Empty histories : 11
15 telemetered animals, 4 detected
25-25 locations per animal, mean = 25, sd = 0
```

Individual covariates

```
sex
F:24
M:30
```

G.5.4 Plotting to compare simulated data

```
par(mfrow = c(1,3), mar = c(2,2,4,2), xpd = TRUE)
plot(traps(CHI), border = 200, gridlines = FALSE, bty = 'o')
plot(CHI, type = 'telemetry', tracks = TRUE, add = TRUE)

plot(traps(CHD), border = 200, gridlines = FALSE, bty = 'o')
plot(CHD, type = 'telemetry', tracks = TRUE, add = TRUE)

plot(traps(CHC), border = 200, gridlines = FALSE, bty = 'o')
plot(CHC, type = 'telemetry', tracks = TRUE, add = TRUE)
```

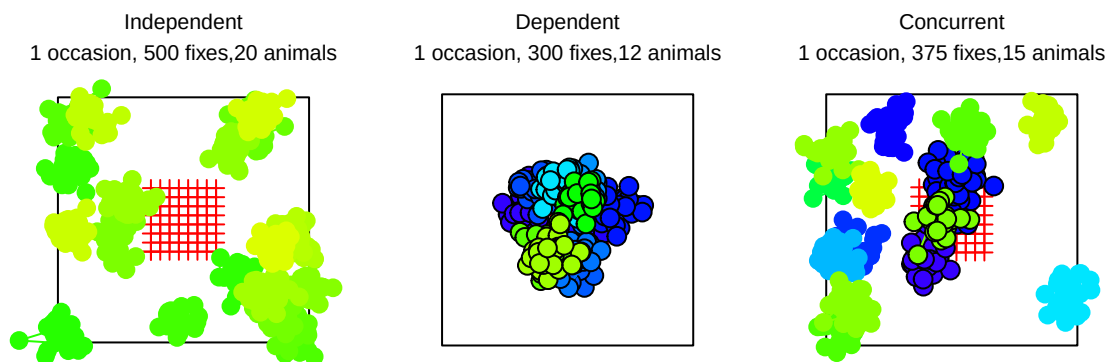


Figure G.4: Simulated telemetry data in relation to a capture-recapture grid (red crosses). Independently telemetered individuals are not recognised if they are caught on the grid. Dependent telemetry is restricted to animals caught on the grid. Individuals telemetered concurrently may or may not be caught, but are recognised when they are.

G.6 Technical notes

G.6.1 Assumption of common σ

Joint analysis of telemetry and capture–recapture data usually relies on the assumption that the same value of the parameter σ applies in both sampling processes. This does not hold when

- telemetry fixes have large measurement error that inflates σ , or
- the tendency of an animal to interact with a detector after encountering it varies systematically with distance from the home-range centre, or
- activity is not stationary and the telemetry and capture–recapture data relate to different time intervals.

The assumption may be avoided altogether by modelling distinct values of σ on trapping and telemetry occasions. This is readily achieved using the automatic predictor `tt` in the formula for sigma, as in `secr.fit(CH, detectfn = 'HHN', model = sigma~tt, ...)`. The model then has one level of sigma for non-telemetry occasions (`tt = 'nontelemetry'`) and another for telemetry occasions (`tt = 'telemetry'`). However, this sacrifices much of the benefit from a joint analysis when the telemetry data are dependent or concurrent, and all benefit for independent telemetry data.

G.6.2 Numerical problems

Fitting joint telemetry and SECR models can be difficult - the usual computations in `secr` may fail to return a likelihood. The problem is often due to a near-zero value in a component of the telemetry likelihood. This occurs particularly in large datasets. The problem may be fixed by scaling the offending values by an arbitrary large number given in the details argument 'telemetryscale'. The required magnitude for 'telemetryscale' may be found by experimentation (try 1e3, 1e6, 1e9, 1e12 etc.). This ad hoc solution must be applied consistently if models are to be compared by AIC.

```
te <- make.telemetry()
teCH2 <- sim.capthist(te, popn = list(D = 2, buffer = 200),
  detectfn = "HHN", exactN = 100, detectpar = list(sigma = 25),
  nooccasions = 1)
mask <- make.mask(traps(teCH2), buffer = 300, type =
  'trapbuffer')
# fails
fit1 <- secr.fit(teCH2, mask = mask, detectfn = 'HHN', CL = TRUE,
  trace = FALSE, details = list(telemetryscale = 1))
predict(fit1)
```

```

      link estimate SE.estimate lcl ucl
sigma log      NA          NA NA NA

```

```

# succeeds
fit1000 <- secr.fit(teCH2, mask = mask, detectfn = 'HHN',
  CL = TRUE, trace = FALSE, details = list(telemetryscale =
    1e3))
predict(fit1000)

```

```

      link estimate SE.estimate    lcl    ucl
sigma log   25.044    0.20636 24.643 25.452

```

Numerical problems may also be caused by inappropriate starting values, poor model specification, or an unknown bug. It may help to use the longer-tailed detection function ‘HEX’ instead of ‘HHN’.

G.6.3 Learned response

Learned responses (b, bk) are not expected in telemetry data. However, they may make sense for the ‘SECR’ occasions of a composite dataset (combined stationary detectors and telemetry). There is no way to avoid a global learned response (b) from propagating to the telemetry occasions (i.e. modelling different telemetry sigmas for animals detected or not detected in the pre-telemetry phase). A site-specific learned response, however, cannot propagate to the telemetry phase if there is a single telemetry ‘occasion’ because (i) no animal is detected at the notional telemetry detector in the pre-telemetry phase, and (ii) there is no opportunity for learning within the telemetry phase if all detections are on one occasion.

G.7 Limitations

Some important functions of **secr** have yet to be updated to work with telemetry data. These are listed in Section [G.8](#). Other limitations are described here.

G.7.1 Incompatible with area search

Telemetry data may not be combined with area-search (polygon) data except as independent data in distinct sessions. This is because the polygon data types presently implemented in **secr** must be constant across a session.

If the ‘independent data, distinct-session’ solution is inadequate you might try rasterizing the search area (function **discretize**).

G.7.2 Incompatible with mark-resight

Telemetry data may not be combined with mark-resight except possibly in distinct sessions (this has not been tested).

G.7.3 Incompatible with hybrid heterogeneity model

The `secl.fit` code for hybrid heterogeneity models (`hcov`) has yet to be updated.

G.7.4 Non-Euclidean distance

Non-Euclidean distance methods cannot be used with telemetry data at present (a very large distance matrix would be required).

G.8 Telemetry-related functions.

Table G.1: Functions specifically for telemetry data.

Function	Purpose
<code>addTelemetry</code>	combine capture-recapture and telemetry data in new capthist
<code>make.telemetry</code>	build a traps object for standalone telemetry data
<code>read.telemetry</code>	input telemetry fixes from text file or dataframe
<code>telemetered</code>	determine which animals in a capthist object have telemetry data
<code>telemetrytype</code>	extract or replace the ‘telemetrytype’ attribute of a traps object
<code>telemetryxy</code>	extract or replace telemetry coordinates from capthist
<code>xy2CH</code>	make a standalone telemetry capthist from a composite capthist

Table G.2: Telemetry-ready general functions. Functions marked with an asterisk (*) use the telemetry coordinates if the capthist is telemetry-only, otherwise the detection sites.

Function	Purpose
<code>derived</code>	Horvitz-Thompson-like density estimate
<code>join</code>	combine sessions of multi-session capthist object
<code>make.capthist</code>	build capthist object
<code>moves</code>	sequential movements*
<code>MS.capthist</code>	form multi-session capthist from separate sessions
<code>plot.capthist</code>	plotting (type = ‘telemetry’)
<code>rbind.capthist</code>	concatenate rows of capthist
<code>RPSV</code> , <code>MMDM</code> , <code>ARL</code> , <code>dbar</code>	indices of home-range size*
<code>secl.fit</code>	model fitting

Function	Purpose
<code>sim.caphist</code>	generate caphist data
<code>subset.caphist</code>	select subset of animals, occasions or detectors
<code>summary.caphist</code>	summary
<code>verify.caphist</code>	perform integrity checks
<code>verify.traps</code>	perform integrity checks

Table G.3: General functions not ready for telemetry.

Function	Purpose
<code>reduce.caphist</code>	change detector type or collapse occasions
<code>sim.secr</code>	parametric bootstrap fitted model
<code>simulate</code>	simulate from fitted model
<code>secr.test</code>	another parametric bootstrap
<code>fxTotal</code>	
<code>fxiContour</code>	

H Parameterizations

At the heart of SECR there is usually a set of three primary model parameters: one for population density (D) and two for the detection function. The detection function is commonly parameterized in terms of its intercept (the probability g_0 or cumulative hazard λ_0 of detection for a detector at the centre of the home range) and a spatial scale parameter σ . Although this parameterization is simple and uncontroversial, it is not inevitable. Sometimes the biology leads us to expect a structural relationship between primary parameters. The relationship may be ‘hard-wired’ into the model by replacing a primary parameter with a function of other parameter(s). This often makes for a more parsimonious model, and model comparisons may be used to evaluate the hypothesized relationship. Here we outline some parameterization options in **secr**.

H.1 Theory

The idea is to replace a primary detection parameter with a function of other parameter(s) in the SECR model. This may allow constraints to be applied more meaningfully. Specifically, it may make sense to consider a function of the parameters to be constant, even when one of the primary parameters varies. The new parameter also may be modelled as a function of covariates etc.

H.1.1 λ_0 and σ

One published example concerns compensatory heterogeneity of detection parameters (Efford and Mowat 2014). Combinations of λ_0 and σ yield the same effective sampling area a when the cumulative hazard of detection ($\lambda(d)$)¹ is a linear function of home-range utilisation. Variation in home-range size then has no effect on estimates of density. It would be useful to allow σ to vary while holding a constant, but this has some fishhooks because computation of λ_0 from a and σ is not straightforward. A simple alternative is to substitute $a_0 = 2\pi\lambda_0\sigma^2$; Efford & Mowat (2014) called a_0 the ‘single-detector sampling area’. If the sampling regime is constant, holding a_0 constant is almost equivalent to holding a constant (but see [Limitations](#)). Fig. [H.1](#) illustrates the relationship for 3 levels of a_0 .

¹The cumulative hazard $\lambda(d)$ and probability $g(d)$ formulations are largely interchangeable because $g(d) = 1 - \exp(-\lambda(d))$.

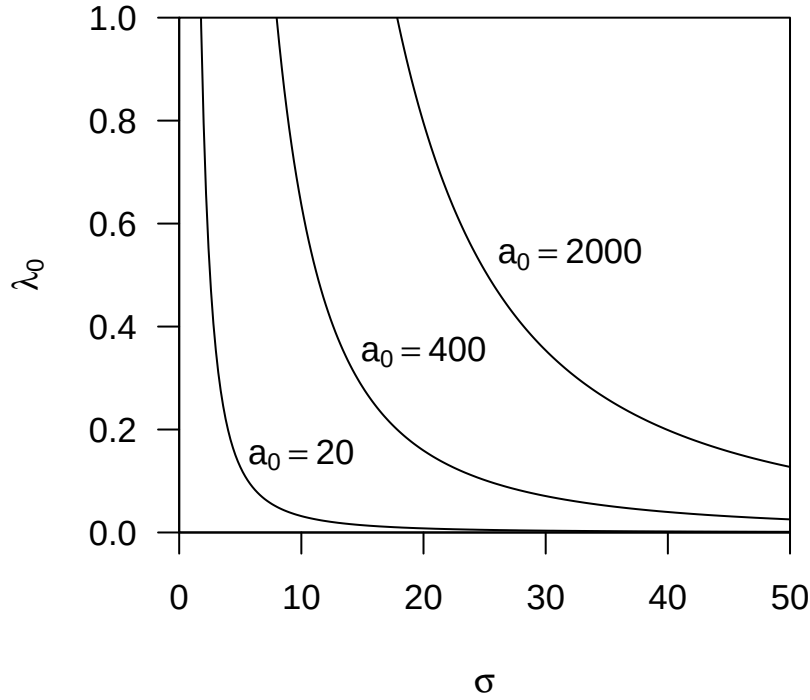


Figure H.1: Structural relationship between parameters λ_0 and σ expressed by holding a_0 constant in $\lambda_0 = a_0/(2\pi\sigma^2)$.

H.1.2 σ and D

Another biologically interesting structural relationship is that between population density and home-range size (Efford et al., 2016). If home ranges have a definite edge and partition all available space then an inverse-square relationship is expected $D = (k/r)^2$ or $r = k/\sqrt{D}$, where r is a linear measure of home-range size (e.g., grid cell width) and k is a constant of proportionality. In reality, the home-range model that underlies SECR detection functions does not require a hard edge, so the language of ‘partitioning’ and ‘elastic discs’ (Huxley, 1934) does not quite fit. However, the inverse-square relationship is empirically useful, and we conjecture that it may also arise from simple models for constant overlap of home ranges when density varies – a topic for future research. For use in SECR we equate r with the spatial scale of detection σ , and predict concave-up relationships as in Fig. H.2.

The relationship may be modified by adding a constant c to represent the lower asymptote of sigma as density increases ($\sigma = k/\sqrt{D} + c$; by default $c = 0$ in **secr**).

It is possible, intuitively, that once a population becomes very sparse there is no further effect of density on home-range size. Alternatively, very low density may reflect sparseness of resources, requiring the few individuals present to exploit very large home ranges even if they seldom meet. If density is no longer related to σ at low density, even indirectly, then the steep increase in σ modelled on the left of Fig. H.3 will ‘level off’ at some value of σ . We don’t know of any empirical example of this hypothetical phenomenon, and do not provide a model.

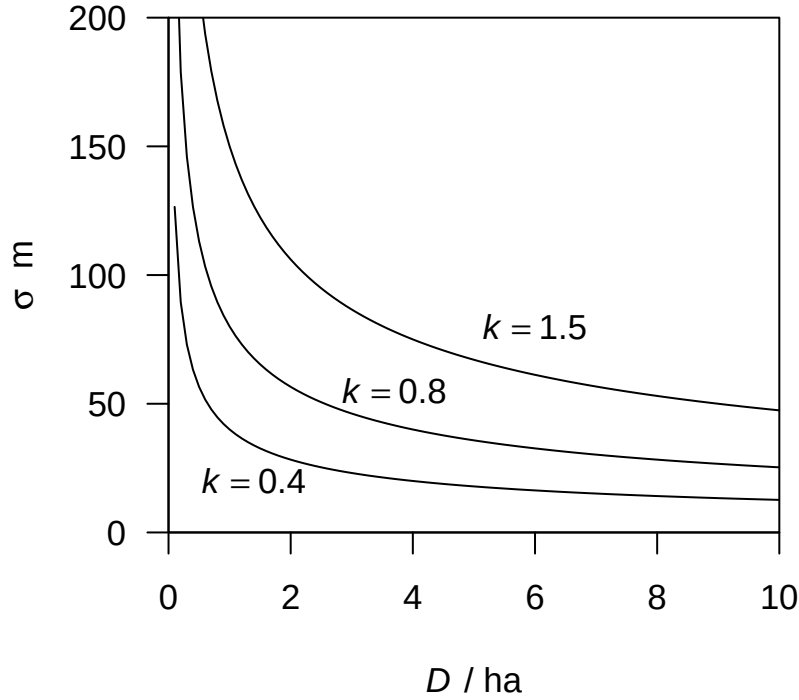


Figure H.2: Structural relationship between parameters D and σ expressed by holding k constant in $\sigma = 100k/\sqrt{D}$. The factor of 100 adjusts for the inconsistent default units of σ and D in **secr** (metres and hectares).

We use ‘primary parameter’ to mean one of (D, λ_0, σ) ² For each relationship there is a primary parameter considered the ‘driver’ that varies for external reasons (e.g., between times, sexes etc.), and a dependent parameter that varies in response to the driver, moderated by a ‘surrogate’ parameter that may be constant or under external control. The surrogate parameter appears in the model in place of the dependent parameter. Using the surrogate parameterization is exactly equivalent to the default parameterization if the driver parameter(s) (σ and λ_0 for a_0 ³, D for k ⁴) are constant.

H.2 Implementation

Parameterizations in **secr** are indicated by an integer code (Table H.1). The internal implementation of the parameterizations (3)–(5) is straightforward. At each evaluation of the likelihood function:

1. The values of the driver and surrogate parameters are determined
2. Each dependent parameter is computed from the relevant driver and surrogate parameters

²**secr** names D , λ_0 or σ .

³**secr** name a_0

⁴**secr** uses $\text{sigmak} = 100k$

3. Values of the now-complete set of primary parameters are passed to the standard code for evaluating the likelihood.

Table H.1: Parameterization codes.

Code	Description	Driver	Surrogate(s)	Dependent
0	Default	—	—	—
3	Single-detector sampling area	σ	a_0	$\lambda_0 = a_0/(2\pi\sigma^2)$
4	Density-dependent home range	D	k, c	$\sigma = 100k/\sqrt{D} + c$
5	3 & 4 combined	D, σ	k, c, a_0	$\sigma = 100k/\sqrt{D} + c,$ $\lambda_0 = a_0/(2\pi\sigma^2)$

The transformation is performed independently for each level of the surrogate parameters that appears in the model. For example, if the model includes a learned response $\mathbf{a0} \sim \mathbf{b}$, there are two levels of a_0 (for naive and experienced animals) that translate to two levels of λ_0 . For parameterization (4), $\sigma = 100k/\sqrt{D}$. The factor of 100 is an adjustment for differing units (areas are expressed in hectares in **secr**, and 1 hectare = 10 000 m²). For parameterization (5), σ is first computed from D , and then λ_0 is computed from σ .

H.2.1 Interface

Users choose between parameterizations either

- explicitly, by setting the ‘param’ component of the **secr.fit** argument ‘details’, or
- implicitly, by including a parameterization-specific parameter name in the **secr.fit** model.

Implicit selection causes the value of `details$param` to be set automatically (with a warning).

The main parameterization options are listed in Table 1 (other specialised options are listed in Section [H.5](#)).

The constant c in the relationship $\sigma = k/\sqrt{D} + c$ is set to zero and not estimated unless ‘c’ appears explicitly in the model. For example, `model = list(sigmak ~ 1)` fixes $c = 0$, whereas `model = list(sigmak ~ 1, c ~ 1)` causes c to be estimated. The usefulness of this model has yet to be proven! By default an identity link is used for ‘c’, which permits negative values; negative ‘c’ implies that for some densities (most likely densities outside the range of the data) a negative sigma is predicted. If you’re uncomfortable with this and require ‘c’ to be positive then set `link = list(c = 'log')` in **secr.fit** and specify a positive starting value for it in **start** (using the vector form for that argument of **secr.fit**).

Initial values may be a problem as the scales for a_0 and σ are not intuitive. Assuming automatic initial values can be computed for a half-normal detection function with parameters g_0 and σ , the default initial value for a_0 is $2\pi g_0 \sigma^2 / 10000$, and for k , $\sigma \sqrt{D}$. If the usual automatic procedure (see `?autoini`) fails then *ad hoc* and less reliable starting values are used. In case of trouble, it is suggested that you first fit a very simple (or null) model using the desired parameterization, and then use this to provide starting values for a more complex model. Here is an example (actually a trivial one for which the default starting values would have been OK; some warnings are suppressed):

```
fit0 <- secr.fit(captdata, model = a0~1, detectfn = 'HHN',
               trace = FALSE)
fitbk <- secr.fit(captdata, model = a0~bk, detectfn = 'HHN',
               trace = FALSE, start = fit0)
```

H.2.2 Models for surrogate parameters a_0 and σ

The surrogate parameters a_0 and σ are treated as if they are full ‘real’ parameters, so they appear in the output from `predict.secr`, and may be modelled like any other ‘real’ parameter. For example, `model = sigma ~ h2` is valid.

Do not confuse this with the modelling of primary ‘real’ parameters as functions of covariates, or built-in effects such as a learned response.

H.3 Example

Among the datasets included with `secr`, only `ovenCH` provides a useful temporal sequence - 5 years of data from mistnetting of ovenbirds (*Seiurus aurocapilla*) at Patuxent Research Refuge, Maryland. A full model for annually varying density and detection parameters may be fitted with

```
msh <- make.mask(traps(ovenCH[[1]]), buffer = 300, nx = 25)
oven0509b <- secr.fit(ovenCH, model = list(D ~ session,
      sigma ~ session, lambda0 ~ session + bk), mask = msh,
      detectfn = 'HHN', trace = FALSE)
```

This has 16 parameters and takes some time to fit.

We hypothesize that home-range (territory) size varied inversely with density, and model this by fixing the parameter k . Efford & Mowat (2014) reported for this dataset that λ_0 did not compensate for within-year, between-individual variation in σ , but it is nevertheless possible that variation between years was compensatory, and we model this by fixing a_0 . For good measure, we also allow for site-specific net shyness by modelling a_0 with the builtin effect ‘bk’:

```
oven0509bs <- secr.fit(ovenCH, model = list(D ~ session, sigmak ~ 1,
      a0 ~ bk), mask = msk, detectfn = 'HHN', trace = FALSE)
```

Warning: Using parameterization details\$param = 5

The effect of including both sigmak and a0 in the model is to force parameterization (5). The model estimates a different density in each year, as in the previous model. Annual variation in D drives annual variation in σ through the relation $\sigma_y = k/\sqrt{D_y}$ where k ($= \text{sigmak}/100$) is a parameter to be estimated and the subscript y indicates year. The detection function ‘HHN’ is the hazard-half-normal which has parameters σ and λ_0 . We already have year-specific σ_y , and this drives annual variation in λ_0 : $\lambda_{0y} = a_{0X}/(2\pi\sigma_y^2)$ where a_{0X} takes one of two different values depending on whether the bird in question has been caught previously in this net.

This is a behaviourally plausible and fairly complex model, but it uses just 8 parameters compared to 16 in a full annual model with net shyness. It may be compared by AIC with the full model (the model structure differs but the data are the same). Although the new model has somewhat higher deviance (1858.5 vs 1851.6), the reduced number of parameters results in a substantially lower AIC ($\Delta\text{AIC} = 9.1$).

In Fig. H.3 we illustrate the results by overplotting the fitted curve for σ_y on a scatter plot of the separate annual estimates from the full model. A longer run of years was analysed by Efford et al. (2016).

H.4 Limitations

Using a_0 as a surrogate for a is unreliable if the distribution or intensity of sampling varies. This is because a_0 depends only on the parameter values, whereas a depends also on the detector layout and duration of sampling. For example, if a different size of trapping grid is used in each session, a will vary even if the detection parameters, and hence a_0 , stay the same. This is also true (a varies, a_0 constant) if the same trapping grid is operated for differing number of occasions in each session. It is a that really matters, and constant a_0 is not a sensible null model in these scenarios.

parameterizations (4) and (5) make sense only if density D is in the model; an attempt to use these when maximizing only the conditional likelihood ($\text{CL} = \text{TRUE}$) will cause an error.

H.5 Other notes

Detection functions 0–3 and 5–8 (‘HN’, ‘HR’, ‘EX’, ‘CHN’, ‘WEX’, ‘ANN’, ‘CLN’, ‘CG’) describe the probability of detection $g(d)$ and use g_0 as the intercept instead of λ_0 . Can parameterizations (3) and (5) also be used with these detection functions? Yes, but the

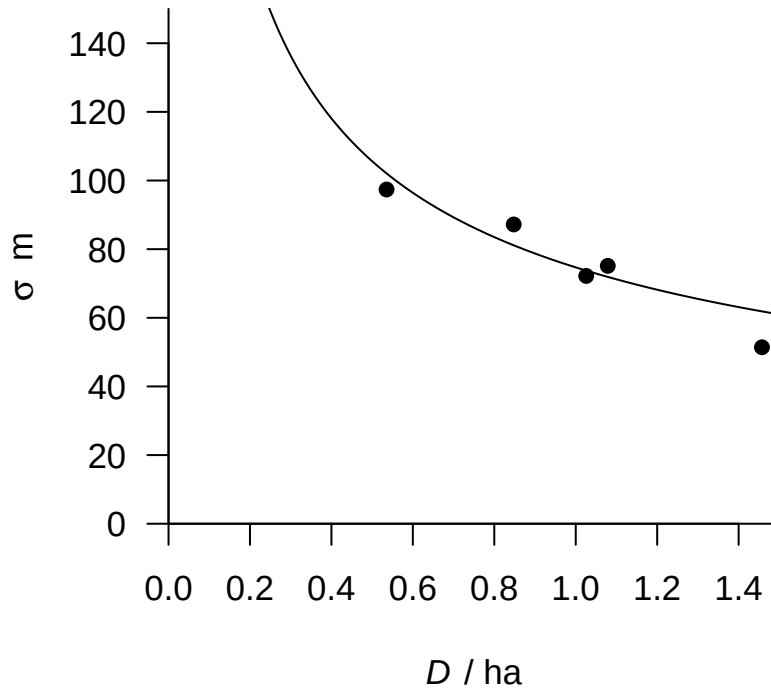


Figure H.3: Fitted structural relationship between parameters D and σ (curve; $\hat{k} = \text{r_round}(\text{predict}(\text{oven0509bs})[[1]][\text{'sigmak'}, \text{'estimate'}]/100, 3)$) and separate annual estimates (ovenbirds mistnetted on Patuxent Research Refuge 2005–2009).

user must take responsibility for the interpretation, which is less clear than for detection functions based on the cumulative hazard (14–19, or ‘HHN’, ‘HHR’, ‘HEX’, ‘HAN’, ‘HCG’, ‘HVP’). The primary parameter is computed as $g_0 = 1 - \exp(-a_0/(2\pi\sigma^2))$.

In a sense, the choice between detection functions ‘HN’ and ‘HHN’, ‘EX’ and ‘HEX’ etc. is between two parameterizations, one with half-normal hazard $\lambda(d)$ and one with half-normal probability $g(d)$, always with the relationship $g(d) = 1 - \exp(-\lambda(d))$ (using d for the distance between home-range centre and detector). It may have been clearer if this had been programmed originally as a switch between ‘hazard’ and ‘probability’ parameterizations, but this would now require significant changes to the code and is not a priority.

If a detection function is specified that requires a third parameter (e.g., z in the case of the hazard-rate function ‘HR’) then this is carried along untouched.

It is possible that home range size, and hence σ , varies in a spatially continuous way with density. The sigmak parameterization does not work when density varies spatially within one population because of the way models of state variables (D) and detection variables (g_0 , λ_0 , σ) are separated within **secr**. Non-Euclidean distance methods allow a workaround as described in Appendix F and Efford et al. (2016).

Some parameterization options (Table H.2) were not included in Table H.1 because they are not intended for general use and their implementation may be incomplete (e.g., not allowing covariates). Although parameterizations (2) and (6) promise a ‘pure’ implementation in terms of the effective sampling area a rather than the surrogate a_0 , this option has not been implemented and tested as extensively as that for a_0 (parameterization 3). The transformation to determine λ_0 or g_0 requires numerical root finding, which is slow. Also, assuming constant a does not make sense when either the detector array or the number of sampling occasions varies, as both of these must affect a . Use at your own risk!

Table H.2: Additional parameterizations.

Code	Description	Driver	Surrogate(s)	Dependent
2	Effective sampling area	σ	a	λ_0 such that $a = \int p(\mathbf{x} \sigma, \lambda_0) d\mathbf{x}$
6	2 & 4 combined	D	k, c, a	$\sigma = 100k/\sqrt{D} + c, \lambda_0$ as above

H.6 Abundance

We use density D as the primary parameter for abundance; the number of activity centres $N(A)$ is secondary as it is contingent on delineation of an area A .

Appendix J considers an alternative parameterization of multi-session density in terms of the initial density D_1 and session-to-session trend λ_t .

I Density revisited

I.1 User-provided model functions

Some density models cannot be coded in the generalized linear model form of the model argument. To alleviate this problem, a model may be specified as an R function that is passed to `secr.fit`, specifically as the component `'userDfn'` of the list argument `'details'`. We document this feature here, although you may never use it.

The `userDfn` function must follow some rules.

- It should accept four arguments, the first a vector of parameter values or a character value (below), and the second a `'mask'` object, a data frame of x and y coordinates for points at which density must be predicted.

Argument	Description
<code>Dbeta</code>	coefficients of density model, or one of <code>c('name', 'parameters')</code>
<code>mask</code>	habitat mask object
<code>ngroup</code>	number of groups
<code>nsession</code>	number of sessions

- When called with `Dbeta = "name"`, the function should return a character string to identify the density model in output. (This should not depend on the values of other arguments).
- When called with `Dbeta = "parameters"`, the function should return a character vector naming each parameter. (When used this way, the call always includes the `mask` argument, so information regarding the model may be retrieved from any attributes of `mask` that have been set by the user).
- Otherwise, the function should return a numeric array with `dim = c(nmask, ngroup, nsession)` where `nmask` is the number of points (rows in `mask`). Each element in the array is the predicted density (natural scale, in animals / hectare) for each point, group and session. This is simpler than it sounds, as usually there will be a single session and single group.

The coefficients form the density part of the full vector of beta coefficients used by the likelihood maximization function (`nlm` or `optim`). Ideally, the first one should correspond to an intercept or overall density, as this is what appears in the output of `predict.secr`.

If transformation of density to the ‘link’ scale is required then it should be hard-coded in `userDfn`.

Covariates are available to user-provided functions, but within the function they must be extracted ‘manually’ (e.g., `covariates(mask)$habclass` rather than just ‘habclass’). To pass other arguments (e.g., a basis for splines), add `attribute(s)` to the mask.

It will usually be necessary to specify starting values for optimisation manually with the `start` argument of `secr.fit`.

If the parameter values in `Dbeta` are invalid the function should return an array of all zero values.

Here is a ‘null’ `userDfn` that emulates $D \sim 1$ with log link

```
userDfn0 <- function (Dbeta, mask, ngroup, nsession) {
  if (Dbeta[1] == "name") return ("0")
  if (Dbeta[1] == "parameters") return ("intercept")
  D <- exp(Dbeta[1]) # constant for all points
  tempD <- array(D, dim = c(nrow(mask), ngroup, nsession))
  return(tempD)
}
```

We can compare the result using `userDfn0` to a fit of the same model using the ‘model’ argument. Note how the model description combines ‘user.’ and the name ‘0’.

```
model.0 <- secr.fit(captdata, model = D ~ 1, trace = FALSE)
userDfn.0 <- secr.fit(captdata, details = list(userDfn = userDfn0), trace = FALSE)
AIC(model.0, userDfn.0)[,-c(2,5,6)]
```

		model	npar	logLik	dAIC	AICwt
model.0	D~1 g0~1 sigma~1		3	-759.03	0	0.5
userDfn.0	D~userD.0 g0~1 sigma~1		3	-759.03	0	0.5

```
predict(model.0)
```

	link	estimate	SE.estimate	lcl	ucl
D	log	5.47980	0.646741	4.35162	6.90047
g0	logit	0.27319	0.027051	0.22348	0.32927
sigma	log	29.36585	1.304940	26.91759	32.03680

```
predict(userDfn.0)
```

	link	estimate	SE.estimate	lcl	ucl
D	log	5.47980	0.646741	4.35162	6.90047
g0	logit	0.27319	0.027051	0.22348	0.32927
sigma	log	29.36583	1.304938	26.91757	32.03677

Not very exciting, maybe, but reassuring!

I.2 Sigmoid density function

Now let's try a more complex example. First create a test dataset with an east-west density step (this could be done more precisely with `sim.popn + sim.caphist`):

```
set.seed(123)
ch <- subset(captdata, centroids(captdata)[,1]>500 | runif(76) > 0.75)
plot(ch)
# also make a mask and assign the x coordinate to covariate 'X'
msk <- make.mask(traps(ch), buffer = 100, type = 'trapbuffer')
covariates(msk)$X <- msk$x
```

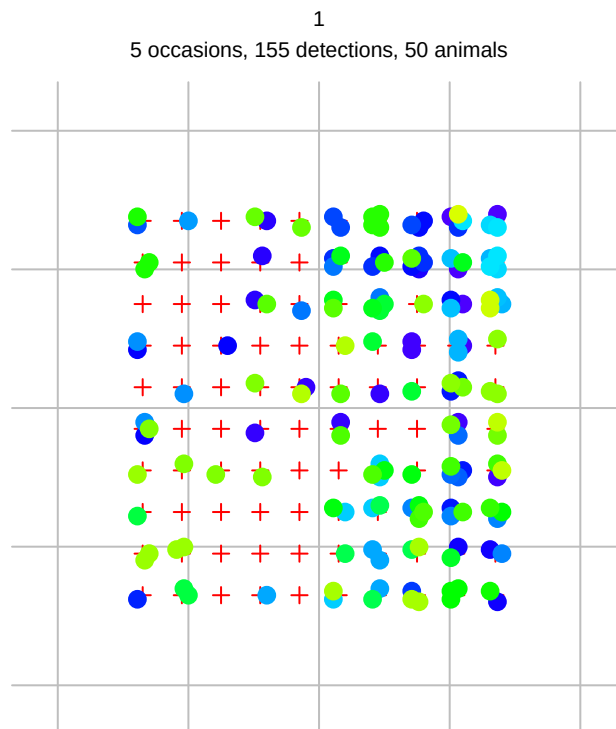


Figure I.1: Test data.

Now define a sigmoid function of covariate X:

```
sigmoidfn <- function (Dbeta, mask, ngroup, nsession) {
  scale <- 7.5 # arbitrary 'width' of step
  if (Dbeta[1] == "name") return ("sig")
  if (Dbeta[1] == "parameters") return (c("D1", "threshold", "D2"))
  X2 <- (covariates(mask)$X - Dbeta[2]) / scale
  D <- Dbeta[1] + 1 / (1+exp(-X2)) * (Dbeta[3] - Dbeta[1])
  tempD <- array(D, dim = c(nrow(mask), ngroup, nsession))
  return(tempD)
}
```

Fit null model and sigmoid model:

```
fit.0 <- secr.fit(ch, mask = msk, link = list(D = "identity"), trace = FALSE)
fit.sigmoid <- secr.fit(ch, mask = msk, details = list(userDfn = sigmoidfn),
  start = c(2.7, 500, 5.8, -1.2117, 3.4260),
  link = list(D = "identity"), trace = FALSE)
coef(fit.0)
```

	beta	SE.beta	lcl	ucl
D	3.6137	0.524474	2.5857	4.64164
g0	-1.0875	0.161739	-1.4045	-0.77051
sigma	3.3953	0.055649	3.2863	3.50439

```
coef(fit.sigmoid)
```

	beta	SE.beta	lcl	ucl
D.D1	1.6839	0.513850	0.67678	2.69104
D.threshold	514.2646	16.652893	481.62551	546.90365
D.D2	5.8810	1.047466	3.82802	7.93401
g0	-1.0860	0.161699	-1.40297	-0.76912
sigma	3.3931	0.055487	3.28439	3.50190

```
AIC(fit.0, fit.sigmoid)[,-c(2,5,6)]
```

	model	npar	logLik	dAIC	AICwt
fit.sigmoid	D~userD.sig g0~1 sigma~1	5	-520.64	0.000	1
fit.0	D~1 g0~1 sigma~1	3	-528.11	10.941	0

The sigmoid model has improved fit, but there is a lot of uncertainty in the two density levels. The average of the fitted levels D1 and D2 (3.7825) is not far from the fitted homogeneous level (3.6137).


```

beta <- coef(fit.sigmoid)[1:3,'beta']
X2 <- (300:700 - beta[2]) / 15
D <- beta[1] + 1 / (1+exp(-X2)) * (beta[3] - beta[1])
plot (300:700, D, type = 'l', xlab = 'X', ylab = 'Density',
      ylim = c(0,7))
abline(v = beta[2], lty = 2)
abline(h = coef(fit.0)[1,1], lty = 1, col = 'blue')
rug(unique(traps(ch)$x), col = 'red')
text(400, 2.2, 'D1')
text(620, 6.4, 'D2')

```

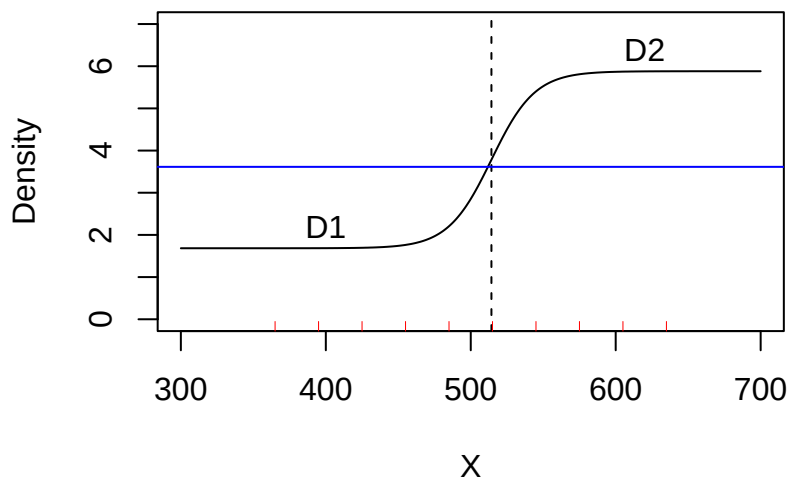


Figure I.2: Sigmoid function fitted to test data.

I.3 More on link functions

The density model for V covariates with an identity link is defined as

$$D(\mathbf{x}|\phi) = \max[\beta_0 + \sum_{v=1}^V \beta_v c_v(\mathbf{x}), 0].$$

From **secr** 4.5.0 there is a scaled identity link ‘i1000’ that multiplies each real parameter value by 1000. Then `secr.fit(..., link = list(D = 'i1000'))` is a fast alternative to specifying `typsize` for low absolute density.

Going further, you can even define your own *ad hoc* link function. To do this, provide the following functions in your workspace (your name ‘xxx’ combined with standard prefixes) and use your name to specify the link:

Name	Purpose	Example
xxx	transform to link scale	i100 <- function(x) x * 100
invxxx	transform from link scale	invi100 <- function(x) x / 100
se.invxxx	transform SE from link scale	se.invi100 <- function (beta, sebeta) sebeta / 100
se.xxx	transform SE to link scale	se.i100 <- function (real, sereal) sereal * 100

Following this example, you would call `secr.fit(..., link = list(D = 'i100'))`. To see the internal transformations for the standard link functions, type `secr:::transform`, `secr:::untransform`, `secr:::se.untransform` or `secr:::se.transform`.

J Trend revisited

This section describes methods specifically for population trend, defined as change in density between sessions and measured by the finite rate of increase $\lambda_t = D_{t+1}/D_t$. The flexible methods described here allow the direct estimation of λ_t , possibly including covariate effects.

J.1 ‘Dlambda’ parameterization

We parameterize the density model in terms of the initial density D_1 and the finite rates of increase λ_t for the remaining sessions (λ_1 refers to the density increase between Session 1 and Session 2, etc.). Reparameterization of the density model is achieved internally in `secr.fit` by manipulating the density design matrix to provide a new array of mask-cell- and session-specific densities at each evaluation of the full likelihood. This happens when the details argument ‘Dlambda’ is set to TRUE. The density model (D~) and the fitted coefficients take on a new meaning determined by the internal function `Dfn2`. More explanation is given in Section J.4.

Now, fitting the ovenbird model with D~1 results in two density parameters (density in session 1, constant finite rate of increase across remaining sessions):

```
msk <- make.mask(traps(ovenCH[[1]]), buffer = 300, nx = 32)
fit1 <- secr.fit(ovenCH, model = D~1, mask = msk, trace =
  FALSE, details = list(Dlambda = TRUE))
coef(fit1)
```

	beta	SE.beta	lcl	ucl
D.D1	0.032027	0.191324	-0.34296	0.407016
D.D2	-0.063858	0.070151	-0.20135	0.073636
g0	-3.561921	0.150654	-3.85720	-3.266646
sigma	4.363969	0.081052	4.20511	4.522828

Density-relevant beta parameters have names starting with ‘D.’¹. The first is the log initial density; others relate to the λ parameters.

¹Their indices are listed in component ‘D’ of the ‘parindx’ component of the fitted model (e.g. `fit1$parindx$D`), but you are unlikely to need this.

To make the most of the reparameterization we need the special prediction function `predictDlambda` to extract the lambda estimates (the simple `predict` method does not work).

```
predictDlambda (fit1)
```

	estimate	SE.estimate	lcl	ucl
D1	1.03255	0.199373	0.70967	1.5023
lambda1	0.93814	0.065893	0.81762	1.0764
lambda2	0.93814	0.065893	0.81762	1.0764
lambda3	0.93814	0.065893	0.81762	1.0764
lambda4	0.93814	0.065893	0.81762	1.0764

This is an advance on the earlier approach using `sdif` contrasts, as we have constrained λ to a constant.

J.2 Covariate and other trend models

The method allows many covariate models for λ . We can fit a time trend in λ using:

```
fit2 <- secr.fit(ovenCH, model = D~Session, mask = msk,
  trace = FALSE, details = list(Dlambda = TRUE))
predictDlambda (fit2)
```

	estimate	SE.estimate	lcl	ucl
D1	0.90131	0.212016	0.57192	1.4204
lambda1	1.14738	0.221236	0.78900	1.6686
lambda2	0.99977	0.092474	0.83433	1.1980
lambda3	0.87115	0.085988	0.71826	1.0566
lambda4	0.75908	0.153416	0.51283	1.1236

Session-specific λ (lower-case ‘session’) provide a direct comparison with the original analysis:

```
fit3 <- secr.fit(ovenCH, model = D~session, mask = msk,
  trace = FALSE, details = list(Dlambda = TRUE))
predictDlambda (fit3)
```

	estimate	SE.estimate	lcl	ucl
D1	0.92026	0.22763	0.57080	1.4837
lambda1	1.04690	0.33132	0.57137	1.9182
lambda2	1.18182	0.34987	0.66963	2.0858
lambda3	0.73077	0.22622	0.40391	1.3221
lambda4	0.84210	0.29542	0.43189	1.6419

The ovenbird population appeared to increase in density for two years and then decline for two years, but the effects are far from significant.

Model selection procedures apply as usual:

```
AIC(fit1, fit2, fit3)[,-c(2,5,6)]
```

	model	npar	logLik	dAIC	AICwt
fit1	D~1 g0~1 sigma~1	4	-930.73	0.000	0.5692
fit2	D~Session g0~1 sigma~1	5	-930.07	0.680	0.4051
fit3	D~session g0~1 sigma~1	8	-929.82	6.192	0.0257

Session covariates are readily applied. The covariate for the second session predicts $\lambda_1 = D_2/D_1$, for the third session predicts $\lambda_2 = D_3/D_2$, etc. The covariate for the first session is discarded (remember D_1 is constant). This all may be confusing, but you can work it out, and it saves extra coding.

```
covs <- data.frame(acov = c(0,2,1,1,2)) # fabricated covariate
fit4 <- secr.fit(ovenCH, model = D~acov, mask = msk,
  trace = FALSE, details = list(Dlambda = TRUE),
  sessioncov = covs)
predictDlambda(fit4)
```

	estimate	SE.estimate	lcl	ucl
D1	1.02804	0.21244	0.68857	1.5349
lambda1	0.95011	0.21202	0.61678	1.4636
lambda2	0.93029	0.14528	0.68627	1.2611
lambda3	0.93029	0.14528	0.68627	1.2611
lambda4	0.95011	0.21202	0.61678	1.4636

J.3 Fixing coefficients

Another possibility is to fit the model with fixed trend (the second beta coefficient corresponds to lambda, before).

```
fit5 <- secr.fit(ovenCH, model = D~1, mask = msk, trace = FALSE,
  details = list(Dlambda = TRUE, fixedbeta =
    c(NA, log(0.9), NA, NA)))
predictDlambda(fit5)
```

	estimate	SE.estimate	lcl	ucl
D1	1.1154	0.15381	0.85231	1.4597
lambda1	0.9000	0.00000	0.90000	0.9000
lambda2	0.9000	0.00000	0.90000	0.9000
lambda3	0.9000	0.00000	0.90000	0.9000
lambda4	0.9000	0.00000	0.90000	0.9000

J.4 Technical notes and tips

Dfn2 performs some tricky manipulations. You can see the code by typing `secr:::Dfn2`. A column is pre-pended to the density design matrix specifically to model the initial density; this takes the value one in Session 1 and is otherwise zero. Other columns in the design matrix are set to zero for the first session. Session-specific density on the link (log) scale is computed as the cumulative sum across sessions of the initial log density and the modelled log-lambda values.

Note –

- The model allows detector locations and habitat masks to vary between sessions.
- The coding of **Dfn2** relies on a log link function for density.
- **Dlambda** is ignored for single-session data and conditional-likelihood (CL) models.
- The method is not (yet) suitable for group models.
- The default start values for **D** in `secr.fit` work well: all lambda are initially 1.0 ($\log(\lambda_t) = 0$ for all t).
- If session covariates are used in any model, `AICcompatible()` expects the argument ‘sessioncov’ to be included in all models.

Tip

D for session 1 is constant over space. It is not possible in the present version of **secr** to model simultaneous spatial variation in density or λ , and using **Dlambda** with a density model that includes spatial covariates will cause an error.

K Expected counts

It can be useful to predict various count statistics from a fitted model or from hypothetical parameter values or from a design that has been altered without changing parameter values. Here we repeat and extend formulae from Efford & Boulanger (2019). See **secrdesign** and [secrdesign-Enrm.pdf](#) for an implementation.

For convenience we formulate the detection process in terms of hazard $\lambda(d_k(\mathbf{x}))$ rather than probability $g(d_k(\mathbf{x}))$, but the two are interchangeable¹. The overall detection rate on occasion s of an AC at location $\mathbf{x}$ is

$$\Lambda_s(\mathbf{x}) = \sum_K \lambda(d_k(\mathbf{x})), \quad (\text{K.1})$$

and aggregating over occasions gives $\Lambda(\mathbf{x}) = \sum_s \Lambda_s(\mathbf{x})$.

If all potential detections are recorded then $\Lambda_s(\mathbf{x})$ is the expected total number of detections on one occasion for an animal centred at $\mathbf{x}$. Only Poisson proximity detectors are assumed to act like this. Other detector types collect binary data (e.g. binary proximity detectors record only whether an individual appeared at least once or not at all at a detector on a certain occasion). Nevertheless, $\Lambda_s(\mathbf{x})$ is useful for predicting the outcome for binary detector types as shown later. Single-catch traps are a special case for which there are not closed-form expressions for $E(n)$ and $E(r)$.

K.1 Number of individuals n

The expected number of individuals detected at least once is

$$E(n) = \int [1 - \exp\{-\Lambda(\mathbf{x})\}] \times D(\mathbf{x}) d\mathbf{x}.$$

This is the same for all detector types in which individuals are detected independently of each other ('multi', 'binary proximity' or 'Poisson proximity'). Integration is over all locations in the plane from which an individual might be detected. The region of integration is represented in practice by a discretized 'habitat mask', and integration is performed by summing over cells.

K.2 Number of detections C

The total number of detections C depends on the detector type, as follows.

¹ $\lambda(d_k(\mathbf{x})) = -\log[1 - g(d_k(\mathbf{x}))]$.

K.2.1 Detector type ‘Poisson proximity’

This is the simplest case –

$$E(C) = \int \Lambda(\mathbf{x}) \times D(\mathbf{x}) d\mathbf{x}.$$

K.2.2 Detector type ‘multi-catch trap’

Data from ‘multi’ detectors are binary at the level of each animal $\times$ occasion, with Bernoulli probability $p_s = 1 - \exp\{-\Lambda_s(\mathbf{x})\}$. This leads to the overall number of detections –

$$E(C) = \int \sum_s p_s(\mathbf{x}) \times D(\mathbf{x}) d\mathbf{x}.$$

K.2.3 Detector type ‘binary proximity’

Data from binary proximity detectors are binary at the level of each animal $\times$ detector $\times$ occasion, with Bernoulli probability $p_{ks}(\mathbf{x}) = 1 - \exp\{-\lambda(d_k(\mathbf{x}))\}$. This leads to the overall number of detections –

$$E(C) = \int \sum_s \sum_k p_{ks}(\mathbf{x}) \times D(\mathbf{x}) d\mathbf{x}.$$

K.3 Number of recaptures r

For all detector types the expected number of recaptures is simply $E(r) = E(C) - E(n)$.

K.4 Number of movements m

A movement is a recapture (redetection) at a site other than the previous one. Movements are a subset of recaptures. We calculate the expected number of movements by considering each recapture event in turn and calculating the conditional probability that it is at the same site as before. This is a sum of squared detector-wise conditional probabilities.

Conditional on detection somewhere, the probability of detection in detector k is $q_k(\mathbf{x}) = \lambda(d_k(\mathbf{x})) / \sum_k \lambda(d_k(\mathbf{x}))$. For clarity in the following detector-specific expressions we use $a(\mathbf{x}) = 1 - \exp\{-\Lambda(\mathbf{x})\}$ and $b(\mathbf{x}) = 1 - \sum_k q_k(\mathbf{x})^2$.

K.4.1 Detector type ‘Poisson proximity’

$$E(m) = \int \{\Lambda(\mathbf{x}) - a(\mathbf{x})\} \times b(\mathbf{x}) \times D(\mathbf{x}) d\mathbf{x}.$$

K.4.2 Detector type ‘multi-catch trap’

$$E(m) = \int \left\{ \sum_s p_s(\mathbf{x}) - a(\mathbf{x}) \right\} \times b(\mathbf{x}) \times D(\mathbf{x}) d\mathbf{x}.$$

K.4.3 Detector type ‘binary proximity’

$$E(m) = \int \left\{ \sum_s \sum_k p_{ks}(\mathbf{x}) - a(\mathbf{x}) \right\} \times b(\mathbf{x}) \times D(\mathbf{x}) d\mathbf{x}.$$

K.4.4 Caveat

If an animal may be detected more than once on one occasion (as with ‘proximity’ and ‘count’ detector types) and time of detection is not recorded within each occasion (the norm in **secr**) then the temporal sequence of detections is not fully observed. The number of observed (apparent) movements is then less than or equal to the true number. Results from the **moves** function in **secr** are also not to be trusted: they effectively assume any repeat detections at the same site precede other redetections rather than being interspersed in time. Precise formulae are not available for the expected number of observed movements among proximity and count detectors. There should be little discrepancy between observed and true numbers when detections are sparse. The predicted number of movements is close to the apparent number in simulations (see later section; this deserves further investigation).

K.5 Individuals detected at two or more detectors n_2

This count is related to the optimization criterion Q_{p_m} of G. Dupont et al. (2021). The value is simply the total count $E(n)$ minus the number detected at only one detector $E(n_1)$. For independent detectors (proximity detectors of any sort) the calculation follows from G. Dupont et al. (2021): setting $p_0(\mathbf{x}) = \exp(-S\Lambda(\mathbf{x}))$ and $p_k(\mathbf{x}) = \exp\{-S\lambda[d_k(\mathbf{x})]\}$,

$$E(n_1) = \int p_0(\mathbf{x}) \sum_k \frac{p_k(\mathbf{x})}{1 - p_k(\mathbf{x})} \times D(\mathbf{x}) d\mathbf{x}.$$

Then $E(n_2) = E(n) - E(n_1)$.

The calculation of $E(n_1)$ is more messy for non-independent detectors, specifically multi-catch traps. Using $p_{ks}(\mathbf{x}) = [1 - \exp(-\Lambda(\mathbf{x}))] \lambda(d_k(\mathbf{x}))/\Lambda(\mathbf{x})$ for the probability an individual at $\mathbf{x}$ is caught at k on a particular occasion, and $p_{ks}^*(\mathbf{x}) = [1 - \exp(-\Lambda(\mathbf{x}))] (1 - \lambda(d_k(\mathbf{x}))/\Lambda(\mathbf{x}))$ for the probability it is caught elsewhere:

$$E(n_1) = \int \sum_k (1 - [1 - p_{ks}(\mathbf{x})]^S) [1 - p_{ks}^*(\mathbf{x})]^{S-1} \times D(\mathbf{x}) d\mathbf{x}.$$

K.6 Single-catch traps

All the preceding calculations assume independence among animals. If traps can catch only one animal at a time then animals effectively compete for access (the first arrival is most likely to be caught). This depresses the realised hazard of detection $\lambda(d_k(\mathbf{x}); \theta)$; the effect increases with density. No closed-form expressions exist for this case. The computed $E(n)$, $E(r)$ and $E(m)$ for multi-catch traps (detector ‘multi’) will exceed the true values for the single-catch traps (detector ‘single’) *given the same detection parameters*. That final caveat is significant because a pilot value of $\hat{\lambda}_0$ from fitting a multi-catch model to single-catch data will be an underestimate (Efford, Borchers, et al., 2009).

L Datasets

These datasets are included in **secr**. See each linked help page for details. Code for model fitting is in [secr-version4.pdf](#). The CAPTURE software and datasets are available from a [USGS archive](#).

Table L.1: Datasets included in **secr**

Dataset	Description
blackbear	<i>Ursus americanus</i> hair snag DNA data from Tennessee (Laufenberg et al., 2013; Settlage et al., 2008)
deermouse	<i>Peromyscus maniculatus</i> live-trapping data of V. H. Reid published as a CAPTURE example by Otis et al. (1978)
hornedlizard	Repeated searches of a quadrat in Arizona for flat-tailed horned lizards <i>Phrynosoma mcallii</i> (Royle & Young, 2008)
housemouse	<i>Mus musculus</i> live-trapping data of H. N. Coulombe published as a CAPTURE example by Otis et al. (1978)
ovenbird	Multi-year mist-netting study of ovenbirds <i>Seiurus aurocapilla</i> at a site in Maryland, USA (Dawson & Efford, 2009)
ovensong	Acoustic detections of ovenbirds (Dawson & Efford, 2009)
OVpossum	Brush-tail possum <i>Trichosurus vulpecula</i> live trapping in the Orongorongo Valley, Wellington, New Zealand 1996–1997 (Efford & Cowan, 2004)
possum	Brush-tail possum <i>Trichosurus vulpecula</i> live trapping at Waitarere, North Island, New Zealand April 2002 (Efford et al., 2005)
captdata	Simulated data and some fitted models (secrdemo.0, secrdemo.CL)
skink	Multi-session lizard (<i>Oligosoma infrapunctatum</i> and <i>O. lineoocellatum</i>) pitfall trapping data from Lake Station, Upper Buller Valley, South Island, New Zealand (Efford et al. in prep)
stoatDNA	Stoat <i>Mustela erminea</i> hair tube DNA data from Matakita Valley, South Island, New Zealand (Efford, Borchers, et al., 2009)

References

- Akima, H., & Gebhardt, A. (2022). *akima: Interpolation of irregularly and regularly spaced data*. <https://CRAN.R-project.org/package=akima>
- Allaire, J., Francois, R., Ushey, K., Vandenbrouck, G., Geelnard, M., & Intel. (2023). *RcppParallel: Parallel programming tools for 'rcpp'*. <https://CRAN.R-project.org/package=RcppParallel>
- Augustine, B. C., Royle, J. A., Kelly, M. J., Satter, C. B., Alonso, R. S., Boydston, E. E., & Crooks, K. R. (2018). Spatial capture–recapture with partial identity: An application to camera traps. *The Annals of Applied Statistics*, 12(1). <https://doi.org/10.1214/17-aos1091>
- Augustine, B. C., Royle, J. A., Linden, D. W., & Fuller, A. K. (2020). Spatial proximity moderates genotype uncertainty in genetic tagging studies. *Proceedings of the National Academy of Sciences*, 117(30), 17903–17912. <https://doi.org/10.1073/pnas.2000247117>
- Baddeley, A., Rubak, E., & Turner, R. (2015). *Spatial point patterns*. Chapman; Hall/CRC. <https://doi.org/10.1201/b19708>
- Bischof, R., Dupont, P., Milleret, C., Chipperfield, J., & Royle, J. A. (2020). Consequences of ignoring group association in spatial capture–recapture analysis. *Wildlife Biology*, 1–10. <https://doi.org/10.2981/wlb.00649>
- Borchers, D. L., Buckland, S. T., & Zucchini, W. (2002). Estimating animal abundance. In *Statistics for Biology and Health*. Springer London. <https://doi.org/10.1007/978-1-4471-3708-5>
- Borchers, D. L., & Efford, M. G. (2007). *Supplements to biometrics paper*. <https://www.otago.ac.nz/density>
- Borchers, D. L., & Efford, M. G. (2008). Spatially explicit maximum likelihood methods for capture–recapture studies. *Biometrics*, 64, 377–385. <https://doi.org/10.1111/j.1541-0420.2007.00927.x>
- Borchers, D. L., & Fewster, R. (2016). Spatial capture–recapture models. *Statistical Science*, 31(2). <https://doi.org/10.1214/16-sts557>
- Borchers, D. L., & Kidney, D. (2014). *Flexible density surface estimation for spatially explicit capture–recapture surveys*. University of St Andrews. https://research-repository.st-andrews.ac.uk/bitstream/handle/10023/9147/secrgam_techrep.pdf?sequence=1
- Borchers, D. L., Stevenson, B. C., Kidney, D., Thomas, L., & Marques, T. A. (2015). A unifying model for capture–recapture and distance sampling surveys of wildlife populations. *Journal of the American Statistical Association*, 110(509), 195–204. <https://doi.org/10.1080/01621459.2014.893884>
- Boutin, S. (1984). Home range size and methods of estimating snowshoe hare densities. *Acta Zoologica Fennica*, 171, 275–278.
- Boyce, M. S., Vernier, P. R., Nielsen, S. E., & Schmiegelow, F. K. A. (2002). Evaluating resource selection functions. *Ecological Modelling*, 157, 281–300.

- Brooks, C., S. P., & Morgan, B. J. T. (2000). Bayesian animal survival estimation. *Statistical Science*, 15, 357–376.
- Brooks, S. P., Morgan, B. J. T., Ridout, M. S., & Pack, S. E. (1997). Finite mixture models for proportions. *Biometrics*, 53(3), 1097. <https://doi.org/10.2307/2533567>
- Buckland, S. T., Anderson, D. R., Burnham, K. P., Laake, J. L., Borchers, D. L., & Thomas, L. (2001). *Introduction to distance sampling*. Oxford University Press. <https://doi.org/10.1093/oso/9780198506492.001.0001>
- Buckland, S. T., Burnham, K. P., & Augustin, N. H. (1997). Model selection: An integral part of inference. *Biometrics*, 53(2), 603. <https://doi.org/10.2307/2533961>
- Buckland, S. T., Rexstad, E. A., Marques, T. A., & Oedekoven, C. S. (2015). *Distance sampling: Methods and applications*. Springer.
- Burnham, K. P., & Anderson, D. R. (2002). *Model selection and multimodel inference: A practical information-theoretic approach*. Springer.
- Burnham, K. P., Anderson, D. R., White, G. C., Brownie, C., & Pollock, K. H. (1987). *Design and analysis methods for fish survival experiments based on release-recapture*. American Fisheries Society.
- Calhoun, J. B., & Casby, J. U. (1958). Calculation of home range and density of small mammals. *Public Health Monograph*, 55.
- Chandler, R. B., Crawford, D. A., Garrison, E. P., Miller, K. V., & Cherry, M. J. (2021). Modeling abundance, distribution, movement and space use with camera and telemetry data. *Ecology*, 103(10). <https://doi.org/10.1002/ecy.3583>
- Chandler, R. B., & Royle, J. A. (2013). Spatially explicit models for inference about density in unmarked or partially marked populations. *Annals of Applied Statistics*, 7, 936–954.
- Chandler, R., & Hepinstall-Cymerman, J. (2016). Estimating the spatial scales of landscape effects on abundance. *Landscape Ecology*, 31, 1383–1394. <https://doi.org/10.1007/s10980-016-0380-z>
- Chao, A. (1989). Estimating population size for sparse data in capture-recapture experiments. *Biometrics*, 45(2), 427. <https://doi.org/10.2307/2531487>
- Choo, Y. R., Sutherland, C., & Johnston, A. (2024). A monte carlo resampling framework for implementing goodness-of-fit tests in spatial capture-recapture models. *Methods in Ecology and Evolution*, 15, 1653–1666. <https://doi.org/10.1111/2041-210x.14386>
- Clark, J. D. (2019). Comparing clustered sampling designs for spatially explicit estimation of population density. *Population Ecology*, 61, 93–101. <https://doi.org/10.1002/1438-390X.1011>
- Cochran, W. G. (1977). *Sampling techniques*. (3rd ed.). John Wiley; Sons.
- Cooch, E., & White, G. (Eds.). (2023). *Program MARK: A gentle introduction*. (23rd ed.). <http://www.phidot.org/software/mark/docs/book/>
- Csardi, G., & Nepusz, T. (2006). The igraph software package for complex network research. *InterJournal*, 1695. <http://igraph.org>
- Dawson, D. K., & Efford, M. G. (2009). Bird population density estimated from acoustic signals. *Journal of Applied Ecology*, 46, 1201–1209.
- Després-Einspenner, M., Howe, E. J., Drapeau, P., & Kühl, H. S. (2017). An empirical evaluation of camera trapping and spatially explicit capture-recapture models for estimating chimpanzee density. *American Journal of Primatology*, 79(7). <https://doi.org/10.1002/ajp.22647>
- Dey, S., Moqanaki, E., Milleret, C., Dupont, P., Tourani, M., & Bischof, R. (2023). Mod-

- elling spatially autocorrelated detection probabilities in spatial capture-recapture using random effects. *Ecological Modelling*, 479, 110324. <https://doi.org/10.1016/j.ecolmodel.2023.110324>
- Distiller, G., & Borchers, D. L. (2015). A spatially explicit capture–recapture estimator for single-catch traps. *Ecology and Evolution*, 5(21), 5075–5087. <https://doi.org/10.1002/ece3.1748>
- Doherty, P. F., White, G. C., & Burnham, K. P. (2010). Comparison of model building and selection strategies. *Journal of Ornithology*, 152(S2), 317–323. <https://doi.org/10.1007/s10336-010-0598-5>
- Dorazio, R. M. (2013). Bayes and empirical bayes estimators of abundance and density from spatial capture-recapture data. *PLoS ONE*, 8, e84017.
- Dumelle, M., Kincaid, T. M., Olsen, A. R., & Weber, M. H. (2024). Spsurvey: Spatial sampling design and analysis. In *R package version 5.5.1*. <https://CRAN.R-project.org/package=spsurvey>
- Dumelle, M., Kincaid, T., Olsen, A. R., & Weber, M. (2023). Spsurvey: Spatial sampling design and analysis in r. *Journal of Statistical Software*, 105. <https://doi.org/10.18637/jss.v105.i03>
- Dunn, J. E., & Gipson, P. S. (1977). Analysis of radio telemetry data in studies of home range. *Biometrics*, 33(1), 85. <https://doi.org/10.2307/2529305>
- Dupont, G., Royle, J. A., Nawaz, M. A., & Sutherland, C. (2021). Optimal sampling design for spatial capture–recapture. *Ecology*, 102, e03262. <https://doi.org/10.1002/ecy.3262>
- Dupont, P., Milleret, C., Gimenez, O., & Bischof, R. (2019). Population closure and the bias-precision trade-off in spatial capture–recapture. *Methods in Ecology and Evolution*, 10(5), 661–672. <https://doi.org/10.1111/2041-210x.13158>
- Durbach, I., Borchers, D., Sutherland, C., & Sharma, K. (2021). Fast, flexible alternatives to regular grid designs for spatial capture-recapture. *Methods in Ecology and Evolution*, 12(298-310), 2041–2210. <https://doi.org/10.1111/2041-210X.13517>
- Durbach, I., Chopara, R., Borchers, D. L., Phillip, R., Sharma, K., & Stevenson, B. C. (2024). That’s not the mona lisa! How to interpret spatial capture-recapture density surface estimates. *Biometrics*, 80. <https://doi.org/10.1093/biomtc/ujad020>
- Eddelbuettel, D., Francois, R., Allaire, J., Ushey, K., Kou, Q., Russell, N., Ucar, I., Bates, D., & Chambers, J. (2023). *Rcpp: Seamless r and c++ integration*. <https://CRAN.R-project.org/package=Rcpp>
- Edzer, E., & Bivand, R. (2023). *Spatial Data Science: With applications in R* (p. 352). Chapman; Hall/CRC. <https://doi.org/10.1201/9780429459016>
- Efford, M. G. (2004). Density estimation in live-trapping studies. *Oikos*, 106, 598–610.
- Efford, M. G. (2011). Estimation of population density by spatially explicit capture-recapture analysis of data from area searches. *Ecology*, 92, 2202–2207.
- Efford, M. G. (2012). *DENSITY 5.0: Software for spatially explicit capture-recapture*. Department of Mathematics; Statistics, University of Otago. <https://www.otago.ac.nz/density>
- Efford, M. G. (2014). Bias from heterogeneous usage of space in spatially explicit capture-recapture analyses. *Methods in Ecology and Evolution*, 5, 599–602.
- Efford, M. G. (2019). Non-circular home ranges and the estimation of population density. *Ecology*, 100(2). <https://doi.org/10.1002/ecy.2580>
- Efford, M. G. (2023). ipsecr: An R package for awkward spatial capture–recapture data.

- Methods in Ecology and Evolution*, 14(5), 1182–1189. <https://doi.org/10.1111/2041-210x.14088>
- Efford, M. G. (2024). *secrlinear: Spatially Explicit Capture-Recapture for Linear Habitats*. <https://CRAN.R-project.org/package=secrlinear>
- Efford, M. G. (2025a). *secr: Spatially explicit capture-recapture models*. <https://doi.org/10.32614/CRAN.package.secr>
- Efford, M. G. (2025b). *secrdesign: Sampling design for spatially explicit capture-recapture*. <https://doi.org/10.32614/CRAN.package.secrdesign>
- Efford, M. G. (2025c). Spatially explicit capture-recapture models for relative density. *BioRxiv*. <https://doi.org/10.1101/2025.01.22.634401>
- Efford, M. G., Borchers, D. L., & Byrom, A. E. (2009). Density estimation by spatially explicit capture-recapture: Likelihood-based methods. In D. L. Thomson, E. G. Cooch, & M. J. Conroy (Eds.), *Modeling demographic processes in marked populations* (pp. 255–269). Springer. https://doi.org/10.1007/978-0-387-78151-8_11
- Efford, M. G., Borchers, D. L., & Mowat, G. (2013). Varying effort in capture-recapture studies. *Methods in Ecology and Evolution*, 4, 629–636.
- Efford, M. G., & Boulanger, J. (2019). Fast evaluation of study designs for spatially explicit capture-recapture. *Methods in Ecology and Evolution*, 10, 1529–1535. <https://doi.org/10.1111/2041-210X.13239>
- Efford, M. G., & Cowan, P. E. (2004). Long-term population trend of *trichosurus vulpecula* in the orongorongo valley, new zealand. In R. L. Goldingay & S. M. Jackson (Eds.), *The biology of australian possums and gliders* (pp. 471–483). Surrey Beatty & Sons.
- Efford, M. G., Dawson, D. K., & Borchers, D. L. (2009). Population density estimated from locations of individuals on a passive detector array. *Ecology*, 90, 2676–2682.
- Efford, M. G., Dawson, D. K., Jhala, Y. V., & Qureshi, Q. (2016). Density-dependent home-range size revealed by spatially explicit capture-recapture. *Ecography*, 39, 676–688.
- Efford, M. G., Dawson, D. K., & Robbins, C. S. (2004). DENSITY: Software for analysing capture-recapture data from passive detector arrays. *Animal Biodiversity and Conservation*, 27, 217–228.
- Efford, M. G., & Fewster, R. M. (2013). Estimating population size by spatially explicit capture-recapture. *Oikos*, 122, 918–928. <https://doi.org/10.1111/j.1600-0706.2012.20440.x>
- Efford, M. G., & Hunter, C. M. (2018). Spatial capture-mark-resight estimation of animal population density. *Biometrics*, 74, 411–420. <https://doi.org/10.1111/biom.12766>
- Efford, M. G., & Mowat, G. (2014). Compensatory heterogeneity in spatially explicit capture-recapture data. *Ecology*, 95, 1341–1348.
- Efford, M. G., & Schofield, M. R. (2020). A spatial open-population capture-recapture model. *Biometrics*, 76, 392–402.
- Efford, M. G., Warburton, B., Coleman, M. C., & Barker, R. J. (2005). A field test of two methods for density estimation. *Wildlife Society Bulletin*, 33, 731–738.
- Ergon, T., & Gardner, B. (2013). Separating mortality and emigration: Modelling space use, dispersal and survival with robust-design spatial capture-recapture data. *Methods in Ecology and Evolution*, 5(12), 1327–1336. <https://doi.org/10.1111/2041-210x.12133>
- Evans, M. A., Kim, H.-M., & O'Brien, T. E. (1996). An application of profile-likelihood based confidence interval to capture: Recapture estimators. *Journal of Agricultural, Biological, and Environmental Statistics*, 1(1), 131. <https://doi.org/10.2307/1400565>

- Fewster, R. M., & Buckland, S. T. (2004). Assessment of distance sampling estimators. In S. T. Buckland, D. R. Anderson, K. P. Burnham, J. L. Laake, D. L. Borchers, & L. Thomas (Eds.), *Advanced distance sampling* (pp. 281–306). Oxford University Press.
- Fieberg, J. (2007). Kernel density estimators of home range: Smoothing and the autocorrelation red herring. *Ecology*, *88*, 1059–1066.
- Fleming, J., Grant, E. H. C., Sterrett, S. C., & Sutherland, C. (2021). Experimental evaluation of spatial capture–recapture study design. *Ecological Applications*, *31*(7). <https://doi.org/10.1002/eap.2419>
- Fletcher, D. (2019). *Model averaging*. Springer.
- Gardner, B., Royle, J. A., & Wegan, M. T. (2009). Hierarchical models for estimating density from DNA mark-recapture studies. *Ecology*, *90*, 1106–1115.
- Gelman, M., A., & Stern, H. (1996). Posterior predictive assessment of model fitness via realized discrepancies. *Statistica Sinica*, *6*, 733–807.
- Gerber, B. D., & Parmenter, R. R. (2015). Spatial capture–recapture model performance with known small-mammal densities. *Ecological Applications*, *25*(3), 695–705. <https://doi.org/10.1890/14-0960.1>
- Gimenez, O., Viallefont, A., Catchpole, E. A., Choquet, R., & Morgan, B. J. T. (2004). Methods for investigating parameter redundancy. *Animal Biodiversity and Conservation*, *27*, 561–572.
- Gopalaswamy, A. M., Royle, J. A., Delampady, M., Nichols, J. D., Karanth, K. U., & Macdonald, D. W. (2012). Density estimation in tiger populations: Combining information for strong inference. *Ecology*, *93*(7), 1741–1751. <https://doi.org/10.1890/11-2110.1>
- Hahsler, M., & Hornik, K. (2007). TSP- infrastructure for the traveling salesperson problem. *Journal of Statistical Software*, *23*(2). <https://doi.org/10.18637/jss.v023.i02>
- Harihar, A., Chanchani, P., Pariwakam, M., Noon, B. R., & Goodrich, J. (2017). Defensible inference: Questioning global trends in tiger populations. *Conservation Letters*, *10*(5), 502–505. <https://doi.org/10.1111/conl.12406>
- Hayes, R. J., & Buckland, S. T. (1983). Radial-distance models for the line-transect method. *Biometrics*, *39*, 29–42.
- Hijmans, R. J. (2023a). Raster: Geographic data analysis and modeling. In *R package version 3.6-26*. <https://CRAN.R-project.org/package=raster>
- Hijmans, R. J. (2023b). *Terra: Spatial data analysis*. <https://CRAN.R-project.org/package=terra>
- Hooten, M. B., Johnson, D. S., McClintock, B. T., & Morales, J. M. (2017). *Animal movement: Statistical models for telemetry data*. CRC Press. <https://doi.org/10.1201/9781315117744>
- Hooten, M. B., Schwob, M. R., Johnson, D. S., & Ivan, J. S. (2024). Geostatistical capture–recapture models. *Spatial Statistics*, *59*, 100817. <https://doi.org/10.1016/j.spasta.2024.100817>
- Huggins, R. M. (1989). On the statistical analysis of capture experiments. *Biometrika*, *76*, 133–140.
- Hurvich, C. M., & Tsai, C.-L. (1989). Regression and time series model selection in small samples. *Biometrika*, *76*(2), 297–307. <https://doi.org/10.1093/biomet/76.2.297>
- Huxley, J. S. (1934). A natural experiment on the territorial instinct. *British Birds*, *27*, 270–277.
- Illian, J., Penttinen, A., Stoyan, H., & Stoyan, D. (2008). *Statistical analysis and modelling*

- of spatial point patterns. Wiley. <https://doi.org/10.1002/9780470725160>
- Ivan, J. S., White, G. C., & Shenk, T. M. (2013). Using auxiliary telemetry information to estimate animal density from capture–recapture data. *Ecology*, 94(4), 809–816. <https://doi.org/10.1890/12-0101.1>
- Jackson, H. B., & Fahrig, L. (2014). Are ecologists conducting research at the optimal scale? *Global Ecology and Biogeography*, 24, 52–63. <https://doi.org/10.1111/geb.12233>
- Jennrich, R. I., & Turner, F. B. (1969). Measurement of non-circular home range. *Journal of Theoretical Biology*, 22, 227–237. [https://doi.org/10.1016/0022-5193\(69\)90002-2](https://doi.org/10.1016/0022-5193(69)90002-2)
- Johansson, Ö., Samelius, G., Wikberg, E., Chapron, G., Mishra, C., & Low, M. (2020). Identification errors in camera-trap studies result in systematic population overestimation. *Scientific Reports*, 10. <https://doi.org/10.1038/s41598-020-63367-z>
- Johnson, D. H. (1980). The comparison of usage and availability measurements for evaluating resource preference. *Ecology*, 61, 65–71.
- Johnson, D. S., Laake, J. L., & Ver Hoef, J. M. (2010). A model-based approach for making ecological inference from distance sampling data. *Biometrics*, 66(1), 310–318. <https://doi.org/10.1111/j.1541-0420.2009.01265.x>
- Johnson, D. S., London, J. M., Lea, M.-A., & Durban, J. W. (2008). Continuous-time correlated random walk model for animal telemetry data. *Ecology*, 89(5), 1208–1215. <https://doi.org/10.1890/07-1032.1>
- Karanth, K. U., & Nichols, J. D. (1998). Estimation of tiger densities in india using photographic captures and recaptures. *Ecology*, 79, 2852–2862.
- Kendall, W. L. (1999). Robustness of closed capture–recapture methods to violations of the closure assumption. *Ecology*, 80(8), 2517–2525. [https://doi.org/10.1890/0012-9658\(1999\)080%5B2517:roccrm%5D2.0.co;2](https://doi.org/10.1890/0012-9658(1999)080%5B2517:roccrm%5D2.0.co;2)
- Kendeigh, S. C. (1944). Measurement of bird populations. *Ecological Monographs*, 14, 67–106. <https://doi.org/10.2307/1961632>
- Kéry, M., & Royle, J. A. (2020). *Applied hierarchical modeling in ecology. Volume 2*. Elsevier Science.
- King, R., McClintock, B. T., Kidney, D., & Borchers, D. (2016). Capture–recapture abundance estimation using a semi-complete data likelihood approach. *The Annals of Applied Statistics*, 10, 264–285. <https://doi.org/10.1214/15-aos890>
- Kodi, A. R., Howard, J., Borchers, D. L., Worthington, H., Alexander, J. S., Lkhagvajav, P., Bayandonoi, G., Ochirjav, M., Erdenebaatar, S., Byambasuren, C., Battulga, N., Johansson, Ö., & Sharma, K. (2024). Ghostbusting - reducing bias due to identification errors in spatial capture-recapture histories. *Methods in Ecology and Evolution*, 15, 1060–1070. <https://doi.org/10.1111/2041-210x.14326>
- Laake, J. L., & Collier, B. A. (2024). Understanding implications of detection heterogeneity in wildlife abundance estimation. *Journal of Wildlife Management*, 88, e22516. <https://doi.org/10.1002/jwmg.22516>
- Lampa, S., Henle, K., Klenke, R., Hoehn, M., & Gruber, B. (2013). How to overcome genotyping errors in non-invasive genetic mark-recapture population size estimation—a review of available methods illustrated by a case study: Genotyping errors in CMR—a review. *The Journal of Wildlife Management*, 77(8), 1490–1511. <https://doi.org/10.1002/jwmg.604>
- Laufenberg, J. S., Van Manen, F. T., & Clark, J. D. (2013). Effects of sampling conditions on DNA-based estimates of american black bear abundance: Sampling for bear population

- studies. *Journal of Wildlife Management*, 77, 1010–1020. <https://doi.org/10.1002/jwmg.534>
- Linden, D. W., Sirén, A. P. K., & Pekins, P. J. (2018). Integrating telemetry data into spatial capture–recapture modifies inferences on multi-scale resource selection. *Ecosphere*, 9(4). <https://doi.org/10.1002/ecs2.2203>
- López-Bao, J. V., Godinho, R., Pacheco, C., Lema, F. J., García, E., Llaneza, L., Palacios, V., & Jiménez, J. (2018). Toward reliable population estimates of wolves by combining spatial capture–recapture models and non-invasive DNA monitoring. *Scientific Reports*, 8(1). <https://doi.org/10.1038/s41598-018-20675-9>
- Lukacs, P. M., & Burnham, K. P. (2005). Estimating population size from DNA-based closed capture–recapture data incorporating genotyping error. *Journal of Wildlife Management*, 69, 396–403.
- Malcolm-White, E., McMahon, C. R., & Cowen, L. L. E. (2020). Complete tag loss in capture–recapture studies affects abundance estimates: An elephant seal case study. *Ecology and Evolution*, 10(5), 2377–2384. <https://doi.org/10.1002/ece3.6052>
- Margenau, L. L. S., Cherry, M. J., Miller, K. V., Garrison, E. P., & Chandler, R. B. (2022). Monitoring partially marked populations using camera and telemetry data. *Ecological Applications*, 32(4). <https://doi.org/10.1002/eap.2553>
- Marques, T. A., Thomas, L., & Royle, J. A. (2011). A hierarchical model for spatial capture–recapture data: comment. *Ecology*, 92, 526–528.
- Matechou, E., Morgan, B. J. T., Pledger, S., Collazo, J. A., & Lyons, J. E. (2013). Integrated analysis of capture–recapture–resighting data and counts of unmarked birds at stopover sites. *Journal of Agricultural, Biological and Environmental Statistics*, 18, 120–135.
- McClintock, B. T., & White, G. C. (2012). From NOREMARK to MARK: Software for estimating demographic parameters using mark–resight methodology. *Journal of Ornithology, Supplement 2*, 152, S641–S650.
- McCrea, R. S., & Morgan, B. J. T. (2011). Multistate mark–recapture model selection using score tests. *Biometrics*, 67, 234–241. <https://doi.org/10.1111/j.1541-0420.2010.01421.x>
- McLaughlin, P., & Bar, H. (2020). A spatial capture–recapture model with attractions between individuals. *Environmetrics*, 32(1). <https://doi.org/10.1002/env.2653>
- McLellan, B. A., Howe, E., Marrotte, R. R., & Northrup, J. M. (2023). Accounting for heterogeneous density and detectability in spatially explicit capture–recapture studies of carnivores. *Ecosphere*, 14(10). <https://doi.org/10.1002/ecs2.4669>
- Milleret, C., Dupont, P., Brøseth, H., Kindberg, J., Royle, J. A., & Bischof, R. (2018). Using partial aggregation in spatial capture recapture. *Methods in Ecology and Evolution*, 9(8), 1896–1907. <https://doi.org/10.1111/2041-210x.13030>
- Mills, L. S., Ciotta, J. C., Lair, K. P., Schwartz, M. K., & Tallmon, D. A. (2000). Estimating animal abundance using noninvasive genetic sampling: Promise and pitfalls. *Ecological Applications*, 10, 283–294.
- Moqanaki, E. M., Milleret, C., Tourani, M., Dupont, P., & Bischof, R. (2021). Consequences of ignoring variable and spatially autocorrelated detection probability in spatial capture–recapture. *Landscape Ecology*, 36(10), 2879–2895. <https://doi.org/10.1007/s10980-021-01283-x>
- Mowat, G., & Strobeck, C. (2000). Estimating population size of grizzly bears using hair capture, DNA profiling, and mark–recapture analysis. *Journal of Wildlife Management*, 64, 183–193.

- Murphy, S. M., Cox, J. J., Augustine, B. C., Hast, J. T., Guthrie, J. M., Wright, J., McDermott, J., Maehr, S. C., & Plaxico, J. H. (2016). Characterizing recolonization by a reintroduced bear population using genetic spatial capture-recapture: Recolonization by reintroduced bear population. *The Journal of Wildlife Management*, 80(8), 1390–1407. <https://doi.org/10.1002/jwmg.21144>
- Murphy, S. M., & Luja, V. H. (2025). Jaguar density estimation in mexico: The conservation importance of considering home range orientation in spatial capture–recapture. *Conservation Science and Practice*. <https://doi.org/10.1111/csp2.13301>
- Noonan, M. J., Tucker, M. A., Fleming, C. H., Akre, T. S., Alberts, S. C., Ali, A. H., Altmann, J., Antunes, P. C., Belant, J. L., Beyer, D., Blaum, N., Böhning-Gaese, K., Cullen, L., Paula, R. C. de, Dekker, J., Drescher-Lehman, J., Farwig, N., Fichtel, C., Fischer, C., ... Calabrese, J. M. (2019). A comprehensive analysis of autocorrelation and bias in home range estimation. *Ecological Monographs*, 89(2). <https://doi.org/10.1002/ecm.1344>
- Noss, A. J., Gardner, B., Maffei, L., Cuéllar, E., Montaña, R., Romero-Muñoz, A., Sollman, R., & O’Connell, A. F. (2012). Comparison of density estimation methods for mammal populations with camera traps in the <scp>k</scp>aa-<scp>i</scp>ya del <scp>g</scp>ran <scp>c</scp>haco landscape. *Animal Conservation*, 15(5), 527–535. <https://doi.org/10.1111/j.1469-1795.2012.00545.x>
- Otis, D. L., Burnham, K. P., White, G. C., & Anderson, D. R. (1978). Statistical inference from capture data on closed animal populations. *Wildlife Monographs*, 62.
- Otis, D. L., & White, G. C. (1999). Autocorrelation of location estimates and the analysis of radiotracking data. *The Journal of Wildlife Management*, 63(3), 1039. <https://doi.org/10.2307/3802819>
- Paetkau, D. (2003). An empirical exploration of data quality in DNA-based population inventories. *Molecular Ecology*, 12(6), 1375–1387. <https://doi.org/10.1046/j.1365-294x.2003.01820.x>
- Palmero, S., Premier, J., Kramer-Schadt, S., Monterroso, P., & Heurich, M. (2023). Sampling variables and their thresholds for the precise estimation of wild felid population density with camera traps and spatial capture–recapture methods. *Mammal Review*, 53(4), 223–237. <https://doi.org/10.1111/mam.12320>
- Parmenter, R., Yates, T., Anderson, D., Burnham, K., Dunnum, J., Franklin, A., Friggens, M., Lubow, B., Miller, M., Olson, G., Parmenter, C., Pollard, J., Rexstad, E., Shenk, T., Stanley, T., & White, G. (2003). Small-mammal density estimation: A field comparison of grid-based vs web-based density estimators. *Ecological Monographs*, 73(1), 1–26.
- Paterson, J. T., Proffitt, K., Jimenez, B., Rotella, J., & Garrott, R. (2019). Simulation-based validation of spatial capture-recapture models: A case study using mountain lions. *PLOS ONE*, 14(4), e0215458. <https://doi.org/10.1371/journal.pone.0215458>
- Pebesma, E. J. (2018). Simple Features for R: Standardized Support for Spatial Vector Data. *The R Journal*, 10(1), 439–446. <https://doi.org/10.32614/RJ-2018-009>
- Pebesma, E. J., & Bivand, R. (2005). Classes and methods for spatial data in R. *R News*, 5(2), 9–13. <https://CRAN.R-project.org/doc/Rnews/>
- Pledger, S. (2000). Unified maximum likelihood estimates for closed capture-recapture models using mixtures. *Biometrics*, 56, 434–442.
- Proffitt, K. M., Goldberg, J. F., Hebblewhite, M., Russell, R., Jimenez, B. S., Robinson, H. S., Pilgrim, K., & Schwartz, M. K. (2015). Integrating resource selection into spatial

- capture-recapture models for large carnivores. *Ecosphere*, 6, 1–15. <https://doi.org/10.1890/es15-00001.1>
- Qiu, Y., Balan, S., Beall, M., Sauder, M., Okazaki, N., & Hahn, T. (2023). *RcppNumerical: 'Rcpp' integration for numerical computing libraries*. <https://CRAN.R-project.org/package=RcppNumerical>
- R Core Team. (2024). *R: A language and environment for statistical computing*. R Foundation for Statistical Computing. <https://www.R-project.org/>
- Reich, B. J., & Gardner, B. (2014). A spatial capture-recapture model for territorial species. *Environmetrics*, 25(8), 630–637. <https://doi.org/10.1002/env.2317>
- Rich, L. N., Kelly, M. J., Sollmann, R., Noss, A. J., Maffei, L., Arispe, R. L., Paviolo, A., De Angelo, C. D., Blanco, D., E., Y., & Di Bitetti, M. S. (2014). Comparing capture-recapture, mark-resight, and spatial mark-resight models for estimating puma densities via camera traps. *Journal of Mammalogy*, 95, 382–391.
- Robertson, B., McDonald, T., Price, C., & Brown, J. (2018). Halton iterative partitioning: Spatially balanced sampling via partitioning. *Environmental and Ecological Statistics*, 25(3), 305–323. <https://doi.org/10.1007/s10651-018-0406-6>
- Royle, J. A., Chandler, R. B., Gazenski, K. D., & Graves, T. A. (2013). Spatial capture-recapture models for jointly estimating population density and landscape connectivity. *Ecology*, 94, 287–294.
- Royle, J. A., Chandler, R. B., Sollmann, R., & Gardner, B. (2014). *Spatial capture–recapture*. Academic Press. <https://doi.org/10.1016/C2012-0-01222-7>
- Royle, J. A., Chandler, R. B., Sun, C. C., & Fuller, A. K. (2013). Integrating resource selection information with spatial capture–recapture. *Methods in Ecology and Evolution*, 4(6), 520–530. <https://doi.org/10.1111/2041-210x.12039>
- Royle, J. A., Fuller, A. K., & Sutherland, C. (2015). Spatial capture–recapture models allowing markovian transience or dispersal. *Population Ecology*, 58(1), 53–62. <https://doi.org/10.1007/s10144-015-0524-z>
- Royle, J. A., & Gardner, B. (2011). Hierarchical spatial capture-recapture models for estimating density from trapping arrays. In A. F. O’Connell, J. D. Nichols, & K. U. Karanth (Eds.), *Camera traps in animal ecology: Methods and analyses* (pp. 163–190). Springer.
- Royle, J. A., Karanth, K. U., Gopalaswamy, A. M., & Kumar, N. S. (2009). Bayesian inference in camera trapping studies for a class of spatial capture–recapture models. *Ecology*, 90(11), 3233–3244. <https://doi.org/10.1890/08-1481.1>
- Royle, J. A., Magoun, A. J., Gardner, B., Valkenburg, P., & Lowell, R. E. (2011). Density estimation in a wolverine population using spatial capture-recapture models. *Journal of Wildlife Management*, 75, 604–611.
- Royle, J. A., Nichols, J. D., Karanth, K. U., & Gopalaswamy, A. M. (2009). A hierarchical model for estimating density in camera-trap studies. *Journal of Applied Ecology*, 46(1), 118–127. <https://doi.org/10.1111/j.1365-2664.2008.01578.x>
- Royle, J. A., & Young, K. V. (2008). A hierarchical model for spatial capture-recapture data. *Ecology*, 89, 2281–2289.
- Ruprecht, J. S., Eriksson, C. E., Forrester, T. D., Clark, D. A., Wisdom, M. J., Rowland, M. M., Johnson, B. K., & Levi, T. (2021). Evaluating and integrating spatial capture–recapture models with data of variable individual identifiability. *Ecological Applications*, 31(7). <https://doi.org/10.1002/eap.2405>

- Russell, R. E., Royle, J. A., Desimone, R., Schwartz, M. K., Edwards, V. L., Pilgrim, K. P., & Mckelvey, K. S. (2012). Estimating abundance of mountain lions from unstructured spatial sampling. *The Journal of Wildlife Management*, 76(8), 1551–1561. <https://doi.org/10.1002/jwmg.412>
- Rutledge, M. E., Sollmann, R., Washburn, B. E., Moorman, C. E., & DePerno, C. S. (2015). Using novel spatial mark-resight techniques to monitor resident canada geese in a suburban environment. *Wildlife Research*, 41, 447–453.
- Seber, G. A. F. (1982). *The estimation of animal abundance and related parameters*. (2nd ed.). Griffin.
- Seber, G. A. F., & Schofield, M. R. (2019). Capture-recapture: Parameter estimation for open animal populations. In *Statistics for Biology and Health*. Springer International Publishing. <https://doi.org/10.1007/978-3-030-18187-1>
- Sethi, S. A., Cook, G. M., Lemons, P., & Wenburg, J. (2014). Guidelines for MSAT and SNP panels that lead to high-quality data for genetic mark-recapture studies. *Canadian Journal of Zoology*, 92(6), 515–526. <https://doi.org/10.1139/cjz-2013-0302>
- Settlage, K. E., Van Manen, F. T., Clark, J. D., & King, T. L. (2008). Challenges of DNA-based mark-recapture studies of american black bears. *Journal of Wildlife Management*, 72, 1035–1042. <https://doi.org/10.2193/2006-472>
- Sollmann, R. (2024). Mt or not mt: Temporal variation in detection probability in spatial capture-recapture and occupancy models. *Peer Community Journal*, 4. <https://doi.org/10.24072/pcjournal.357>
- Sollmann, R., Gardner, B., & Belant, J. L. (2012). How does spatial study design influence density estimates from spatial capture-recapture models? *PLoS ONE*, 7(4), e34575. <https://doi.org/10.1371/journal.pone.0034575>
- Sollmann, R., Gardner, B., Parsons, A. W., Stocking, J. J., McClintock, B. T., Simons, T. R., Pollock, K. H., & O’Connell, A. F. (2013). A spatial mark-resight model augmented with telemetry data. *Ecology*, 94, 553–559.
- Stephens, M. (2000). Dealing with label switching in mixture models. *Journal of the Royal Statistical Society Series B*, 62, 795–809.
- Stevenson, B. C., Fewster, R. M., & Sharma, K. (2021). Spatial correlation structures for detections of individuals in spatial capture-recapture models. *Biometrics*, 78(3), 963–973. <https://doi.org/10.1111/biom.13502>
- Sun, C. C., Fuller, A. K., & Royle, J. A. (2014). Trap configuration and spacing influences parameter estimates in spatial capture-recapture models. *PLoS ONE*, 9(2), e88025. <https://doi.org/10.1371/journal.pone.0088025>
- Sutherland, C., Fuller, A. K., & Royle, J. A. (2015). Modelling non-euclidean movement and landscape connectivity in highly structured ecological networks. *Methods in Ecology and Evolution*, 6, 169–177.
- Sutherland, C., Royle, J. A., & Linden, D. W. (2019). oSCR: A spatial capture-recapture R package for inference about spatial ecological processes. *Ecography*, 42, 1459–1469. <https://doi.org/10.1111/ecog.04551>
- Thompson, S. K. (2012). Sampling. In *Wiley Series in Probability and Statistics*. Wiley. <https://doi.org/10.1002/9781118162934>
- Tobler, M. W., & Powell, G. V. N. (2013). Estimating jaguar densities with camera traps: Problems with current designs and recommendations for future studies. *Biological Conservation*, 159, 109–118.

- Turek, D., Milleret, C., Ergon, T., Brøseth, H., Dupont, P., Bischof, R., & Valpine, P. de. (2021). Efficient estimation of large-scale spatial capture–recapture models. *Ecosphere*, 12(2). <https://doi.org/10.1002/ecs2.3385>
- Twining, J. P., McFarlane, C., O’Meara, D., O’Reilly, C., Reyne, M., Montgomery, W. I., Helyar, S., Tosh, D. G., & Augustine, B. C. (2022). A comparison of density estimation methods for monitoring marked and unmarked animal populations. *Ecosphere*, 13(10). <https://doi.org/10.1002/ecs2.4165>
- van Etten, J. (2023). *Gdistance: Distances and routes on geographical grids*. <https://CRAN.R-project.org/package=gdistance>
- Van Katwyk, K. E. (2014). *Empirical validation of closed population abundance estimates and spatially explicit density estimates using a censused population of north american red squirrels* [Master’s thesis, University of Alberta]. https://redsquirrel.biology.ualberta.ca/wp-content/uploads/sites/32/2014/07/Van-Katwyk_Kristin_Spring-2014.pdf
- van Winkle, W. (1975). Comparison of several probabilistic home-range models. *Journal of Wildlife Management*, 39, 118–123.
- Waits, L. P., & Paetkau, D. (2005). Noninvasive genetic sampling tools for wildlife biologists: A review of applications and recommendations for accurate data collection. *Journal of Wildlife Management*, 69, 1419–1433.
- Walvoort, D. J. J., Brus, D. J., & Gruijter, J. J. de. (2010). An R package for spatial coverage sampling and random sampling from compact geographical strata by k-means. *Computers & Geosciences*, 36, 1261–1267. <https://doi.org/10.1016/j.cageo.2010.04.005>
- Whittington, J., Hebblewhite, M., & Chandler, R. (2018). Generalized spatial mark-resight models with an application to grizzly bears. *Journal of Applied Ecology*, 55, 157–168.
- Whoriskey, K., Martins, E. G., Auger-Méthé, M., Gutowsky, L. F. G., Lennox, R. J., Cooke, S. J., Power, M., & Mills Flemming, J. (2019). Current and emerging statistical techniques for aquatic telemetry data: A guide to analysing spatially discrete animal detections. *Methods in Ecology and Evolution*, 10(7), 935–948. <https://doi.org/10.1111/2041-210x.13188>
- Williams, B. K., Nichols, J. D., & Conroy, M. J. (2002). *Analysis and management of animal populations*. Academic Press.
- Wolters, M. A. (2015). A genetic algorithm for selection of fixed-size subsets with application to design problems. *Journal of Statistical Software, Code Snippets*, 68, 1–18. <https://doi.org/10.18637/jss.v068.c01>
- Wood, S. N. (2006). *Generalized additive models: An introduction with R*. Chapman; Hall/CRC.
- Woodruff, S. P., Johnson, T. R., & Waits, L. P. (2014). Evaluating the interaction of faecal pellet deposition rates and DNA degradation rates to optimize sampling design for DNA-based mark–recapture analysis of sonoran pronghorn. *Molecular Ecology Resources*, 15(4), 843–854. <https://doi.org/10.1111/1755-0998.12362>

Index

- Activity centre, [15](#)
 - estimated location, [54](#)
- Activity centres, [15](#), [36](#)
- Assumptions, [56](#)
 - AC independent, [71](#)
 - AC stationary, [64](#)
 - circular home range, [63](#)
 - detections independent, [71](#)
 - homogeneous detection, [64](#)
 - identification, [59](#)
 - individuals independent, [71](#)
 - population closure, [57](#)
 - uncorrelated locations, [72](#)
- Bayesian methods, [21](#)
- Bias check
 - ‘esaPlot’, [31](#)
 - automatic, [152](#), [208](#)
 - esaPlot, [152](#)
- Black bear, [70](#), [154](#), [221](#), [327](#)
- Brushtail possum, [77](#), [129](#), [154](#), [270](#), [327](#)
- Conditional estimation, [49](#)
- Confidence intervals, [41](#)
- Data entry, [17](#)
- Deer mouse, [121](#), [327](#)
- Density surface, [128](#)
 - not a, [141](#)
 - plotting, [140](#)
 - prediction, [140](#)
 - prior marking, [51](#)
 - relative, [50](#), [143](#)
 - resource selection function, [135](#)
 - scale of effect, [138](#)
- Detection model
 - behavioural response, [121](#)
 - covariates, [119](#)
 - functions, [18](#), [37](#), [111](#)
 - regression splines, [120](#)
 - sub-models, [117](#)
 - time-varying covariates, [120](#)
 - varying effort, [123](#)
- Detectors, [16](#), [38](#)
 - area search, [44](#)
 - binary proximity, [39](#)
 - Binomial proximity, [40](#)
 - multi-catch trap, [40](#)
 - Poisson proximity, [40](#)
 - single-catch trap, [40](#)
 - transect search, [47](#)
- Effective sampling area, [49](#)
- Field validation, [77](#)
- Goodness of fit, [170](#)
- Groups, [48](#)
- Habitat mask, [19](#), [146](#)
 - buffer width, [150](#)
 - covariates, [133](#), [158](#)
 - discretization, [154](#)
 - excluding non-habitat, [156](#)
 - spacing, [154](#)
 - uses, [147](#)
 - vs state space, [147](#)
- Horned lizard, [44](#), [154](#), [232](#), [327](#)
- House mouse, [186](#), [190](#), [192](#), [327](#)
- Individual heterogeneity, [184](#)
- Likelihood
 - area search, [46](#)
 - conditional, [49](#)
 - finite mixture, [52](#)
 - full, [36](#)
 - hybrid mixture, [52](#)

- maximization, [20](#)
 - multi-modal, [188](#)
 - multi-session, [48](#)
- Likelihood ratio test, [169](#)
- Link function, [20](#)
 - cloglog, [20](#), [112](#)
 - density, [20](#), [132](#)
 - detection, [20](#)
 - identity, [20](#), [132](#), [143](#)
 - log, [20](#), [132](#), [143](#)
 - logit, [20](#)
- Mixture models
 - finite, [185](#)
 - hybrid, [190](#)
- Model averaging, [169](#)
- Model selection, [168](#), [170](#)
- Multi-threading, [108](#)
- New Mexico rodents, [78](#)
- Non-Euclidean distances, [265](#)
 - dynamic, [269](#)
 - simulation, [279](#)
 - static, [265](#)
- Notation, [35](#)
- Object classes, [102](#)
- Oligosoma skinks, [209](#), [327](#)
- Ovenbird, [154](#), [169](#), [179](#), [189](#), [309](#), [319](#), [327](#)
- Parameterization, [53](#), [305](#)
 - a0, [305](#)
 - Dlambda, [319](#)
 - sigmak, [306](#)
- Poisson process (2-D), [15](#), [36](#)
- Population closure, [17](#)
- Population size $N(A)$, [160](#)
 - estimation, [51](#), [167](#)
 - fixed, [41](#), [109](#)
- R packages, [10](#)
 - ipsecr, [11](#)
 - openCR, [11](#)
 - RcppParallel, [108](#)
 - secr, [102](#), [105](#)
 - core functions, [104](#)
 - detectors, [104](#)
 - secrdesign, [11](#), [203](#)
 - seclinear, [10](#)
- Red squirrel, [77](#)
- Regression spline
 - density, [135](#)
- Robustness, [56](#)
- Scaling, [140](#)
- Score tests, [170](#)
- secr.fit, [27](#), [106](#)
 - buffer, [106](#)
 - CL, [107](#)
 - details, [108](#)
 - distribution, [109](#)
 - fastproximity, [109](#)
 - fixed, [109](#)
 - maxdistance, [109](#)
 - method, [108](#)
 - model, [107](#)
 - ncores, [108](#)
 - saveprogress, [110](#)
 - start, [106](#)
- secr.refit, [172](#)
- Simulation
 - detection, [201](#)
 - model fitting, [203](#)
 - population, [199](#)
- Snowshoe hare example, [23](#)
- Speed, [217](#)
- Study design, [81](#)
 - algorithmic optimisation, [94](#)
 - area of interest, [81](#)
 - array size, [87](#)
 - clustered designs, [91](#)
 - data requirements, [81](#)
 - genetic algorithm, [94](#)
 - lacework, [92](#)
 - n-r tradeoff, [88](#)
 - optimal spacing, [89](#)
 - pilot parameter values, [84](#)
 - representative sampling, [82](#)
 - sample size, [86](#)
 - scalability, [9](#), [86](#)
 - spatial coverage, [93](#)
 - spatial variation, [97](#)

- statistical power, [83](#)
 - stratified sampling, [98](#)
 - studies, [81](#)
- Traps
 - multi-catch, [27](#), [38](#), [40](#), [325](#)
 - single-catch, [11](#), [16](#), [22](#), [24](#), [27](#), [38](#), [40](#), [71](#), [326](#)
- Trend, [179](#)
 - Dlambda, [319](#)
 - simple, [179](#)
- Troubleshooting, [208](#)
 - buffer bias, [208](#)
 - Covariate factor levels, [215](#)
 - likelihood NA, [208](#)
 - memory, [215](#)
 - nlm code 3, [212](#)
 - nlm code 4, [213](#)
 - variance failed, [210](#)
- Units, [105](#)
- Varying effort, [42](#)